Preistorie, Istorie, Prezent, Viitor

Lenuţa Alboaie

adria@info.uaic.ro

Preistorie|Originile Internetului

- Dorinta de a comunica (si la distanta)
- Primele instrumente: limbajul si vorbirea
- Forma de comunicare: scrisa

  scrierea pe pietre, peretii pesterilor, papirus ☺
- Comunicarea la distanta?
  - Semnale cu foc
  - Codul morse
- Secolul 19: electricitatea si telegraful

Istoric|Originile Internetului

1837 Charles Babbage proiecteaza “primul computer”

1863 Jules Verne:

"photo-telegraphy allowed any writing, signature or illustration to be sent faraway - every house was wired"

1891 – Ada Lovelace – primul program pentru masina analitica a lui Babbage

1936 Alan Turing – descrie masina Turing

1939 HP este fondat (Bill Hewlett, David Packard)

Istoric|Originile Internetului

1961-1972: primele incercari de comunicare prin schimbare de pachete

1961: Kleinrock – propune un model teoretic

1964: Baran – implementare de comunicare intre computere ale armatei americane

1967: ARPAnet este proiectat de Advanced Research Projects Agency

1969: primul nod operational ARPAnet, retea de 4 calculatoare

Istoric|Originile Internetului

1972:

- demonstratie publica a tehnologiilor ARPAnet
- NCP (Network Control Protocol) primul protocol host-host
- Primul program de posta electronica (e-mail)
- Se introduce simbolul @
- ARPAnet are 15 noduri

Istoric|Originile Internetului

1972-1980: Apare conceptul de *Internetworking.* Apar retele aflate in proprietate privata.

1973: DARPA (Defense Advanced Research Projects Agency) – retele interconectate; Robert Metcalf (Hardvard) dezvolta tehnologia Ethernet care permite transferul de date pe cablu coaxial
1974: Cerf si Kahn – propun protocolul de comunicare TCP(Transmission Control Protocol)
1978: suita de protocolae TCP/IP este standardizata via documentele RFC (Request For Comments)
La sfirsitul anilor ‘70: arhitecturi de retea proprietare: DECnet, SNA, XNA
1979: ARPAnet are 200 de noduri

Istoric|Originile Internetului

1980-1990: noi protocoale, creste numarul de retele, Internet

1983: este folosit TCP/IP

1982: este definit SMTP (Simple Mail Transfer Protocol)

1983: apare DNS (translatarea din numele hostului in adresa IP)

1985: apare protocolul FTP(File Transfer Protocol)

1986: coloana vertebrala a **Internetului**

1988: se introduce mecanismul de control al congestiei pentru TCP

Istoric&Prezent|Originile Internetului

1990 ->2000->2017: World Wide Web, noi aplicatii comerciale ale retelelor

Inceputul anilor ‘90:

- **World Wide Web**

Dupa sfirsitul anilor ’90 -> prezent :

- Retele mondiale
- Retele fara fir – *wireless*
- Aplicatii comerciale ale Internetului
  instant messaging, P2P file sharing
  retele sociale, video-streaming,
  cloud-computing,
  …

Prezent

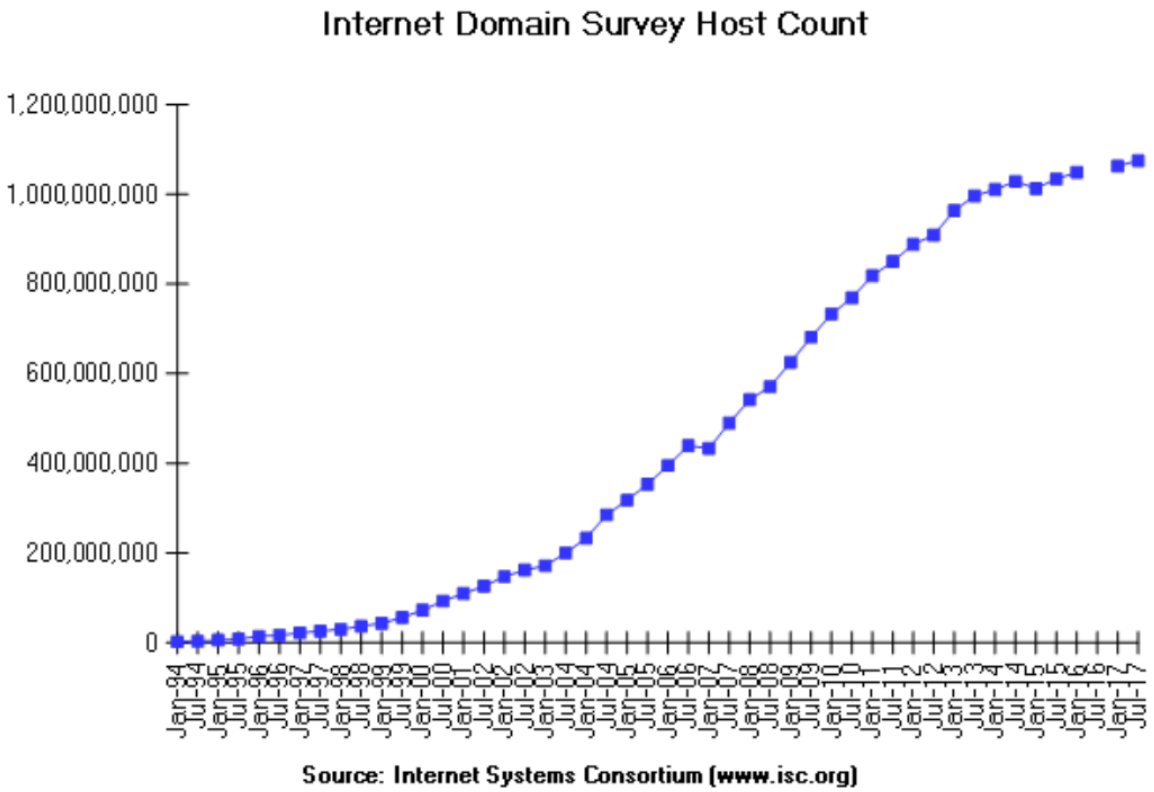


Figura. Cresterea numarului de *host*-uri din Ianuarie 1994 până în Iulie 2017
Sursa: https://www.isc.org/network/survey/

Trecut

WORLD INTERNET USAGE AND POPULATION STATISTICS December 31, 2013					
World Regions	Population (2014 Est.)	Internet Users Dec. 31, 2000	Internet Users Latest Data	Penetration (% Population)	Growth 2000-2014
Africa	1,125,721,038	4,514,400	240,146,482	21.3 %	5,219.6 %
Asia	3,996,408,007	114,304,000	1,265,143,702	31.7 %	1,006.8 %
Europe	825,802,657	105,096,093	566,261,317	68.6 %	438.8 %
Middle East	231,062,860	3,284,800	103,829,614	44.9 %	3,060.9 %
North America	353,860,227	108,096,800	300,287,577	84.9 %	177.8 %
Latin America / Caribbean	612,279,181	18,068,919	302,006,016	49.3 %	1,571.4 %
Oceania / Australia	36,724,649	7,620,480	24,804,226	67.5 %	225.5 %
WORLD TOTAL	7,181,858,619	360,985,492	2,802,478,934	39.0 %	676.3 %

Tabel. **Statistici privind utilizarea internetului în raport cu numărul de persoane aflate în principalele zone geografice ale lumii**

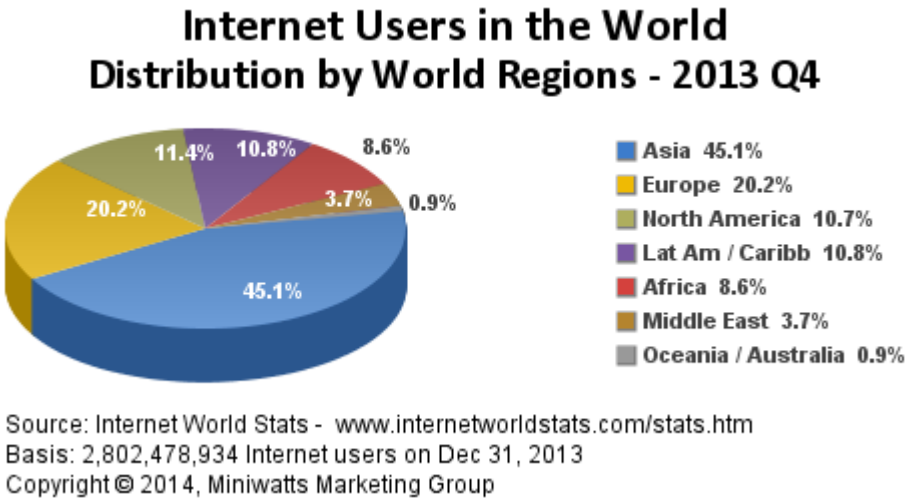


Figura. **Utilizatorii de Internet în lume - împărţirea pe zone geografice**
Decembrie 2013 | Sursa: http://www.internetworldstats.com/

Trecut

WORLD INTERNET USAGE AND POPULATION STATISTICS JUNE 30, 2015 - Mid-Year Update					
World Regions	Population (2015 Est.)	Internet Users Dec. 31, 2000	Internet Users Latest Data	Penetration (% Population)	Growth 2000-2015
Africa	1,158,355,663	4,514,400	313,257,074	27.0 %	6,839.1%
Asia	4,032,466,882	114,304,000	1,563,208,143	38.8 %	1,267.6%
Europe	821,555,904	105,096,093	604,122,380	73.5 %	474.8%
Middle East	236,137,235	3,284,800	115,823,882	49.0 %	3,426.1%
North America	357,172,209	108,096,800	313,862,863	87.9 %	190.4%
Latin America / Caribbean	617,776,105	18,068,919	333,115,908	53.9 %	1,743.6%
Oceania / Australia	37,157,120	7,620,480	27,100,334	72.9 %	255.6%
WORLD TOTAL	7,260,621,118	360,985,492	3,270,490,584	45.0 %	806.0%

Tabel. **Statistici privind utilizarea internetului în raport cu numărul de persoane aflate în principalele zone geografice ale lumii**

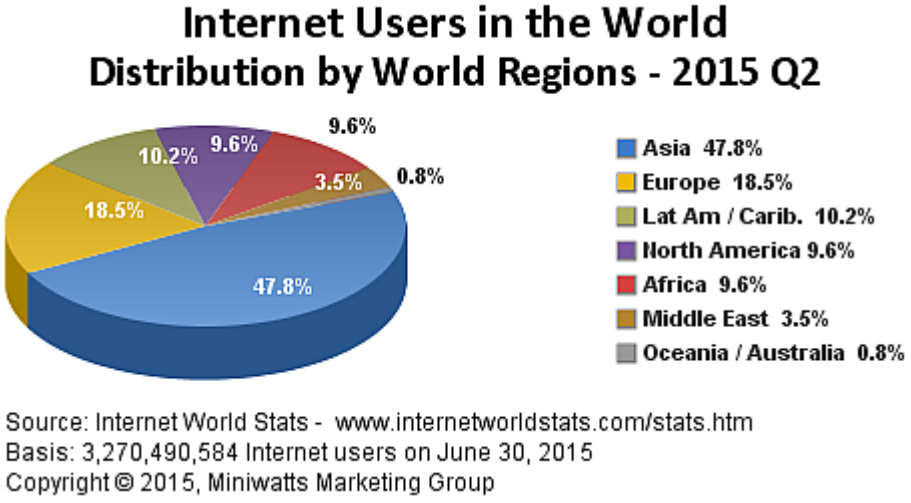


Figura. **Utilizatorii de Internet în lume - împărţirea pe zone geografice**
Iunie 2015 | Sursa: http://www.internetworldstats.com/

Prezent

WORLD INTERNET USAGE AND POPULATION STATISTICS JUNE 30, 2017 - Update					
World Regions	Population (2017 Est.)	Population % of World	Internet Users 30 June 2017	Penetration Rate (% Pop.)	Growth 2000-2017
Africa	1,246,504,865	16.6 %	**388,376,491**	31.2 %	8,503.1%
Asia	4,148,177,672	55.2 %	**1,938,075,631**	46.7 %	1,595.5%
Europe	822,710,362	10.9 %	**659,634,487**	80.2 %	527.6%
Latin America / Caribbean	647,604,645	8.6 %	**404,269,163**	62.4 %	2,137.4%
Middle East	250,327,574	3.3 %	**146,972,123**	58.7 %	4,374.3%
North America	363,224,006	4.8 %	**320,059,368**	88.1 %	196.1%
Oceania / Australia	40,479,846	0.5 %	**28,180,356**	69.6 %	269.8%
WORLD TOTAL	**7,519,028,970**	**100.0 %**	**3,885,567,619**	**51.7 %**	**976.4%**

Tabel. **Statistici privind utilizarea internetului în raport cu numărul de persoane aflate în principalele zone geografice ale lumii**

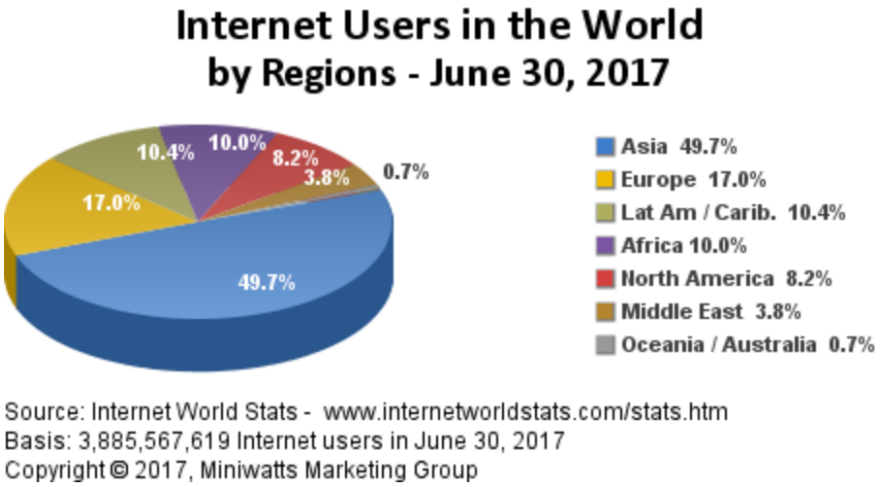
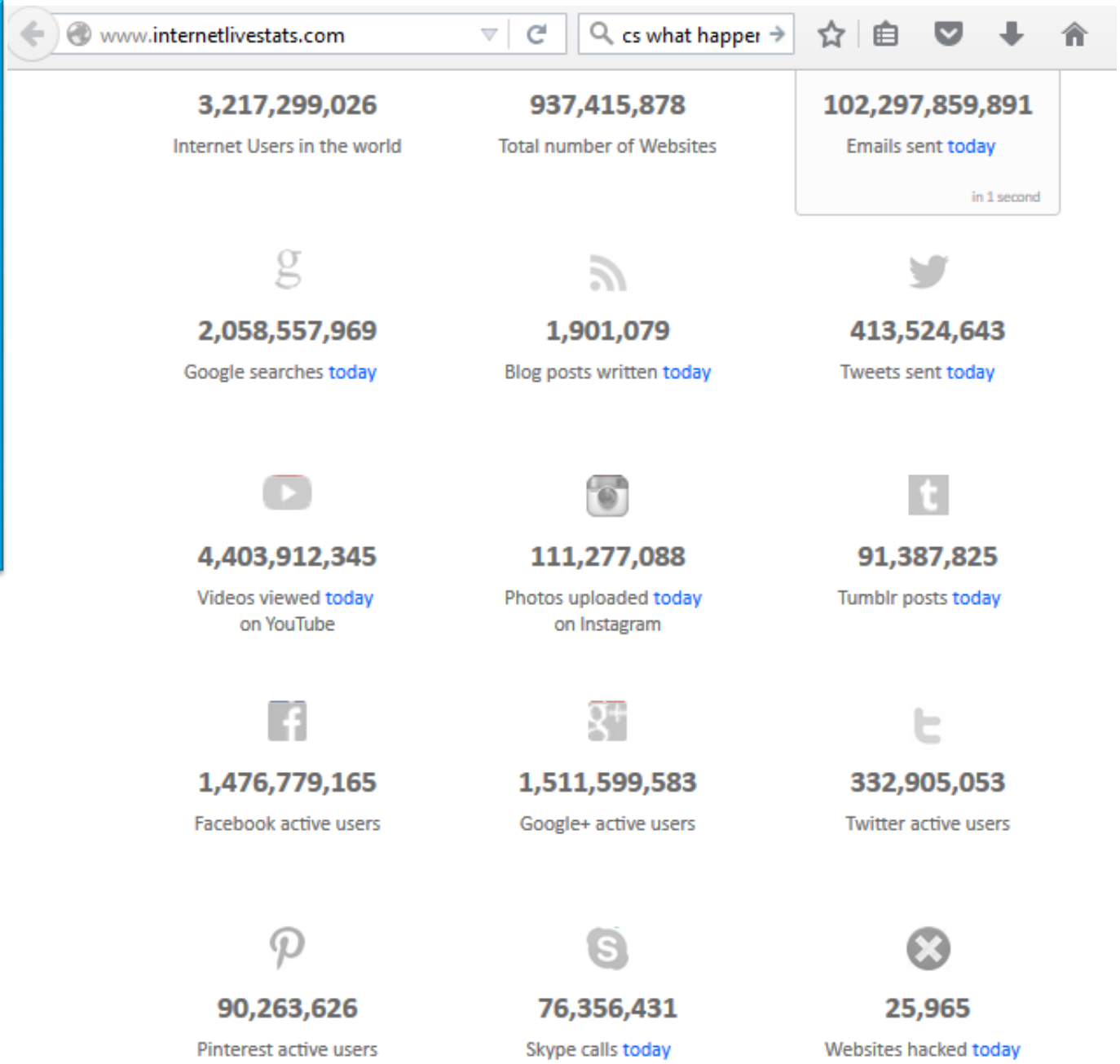
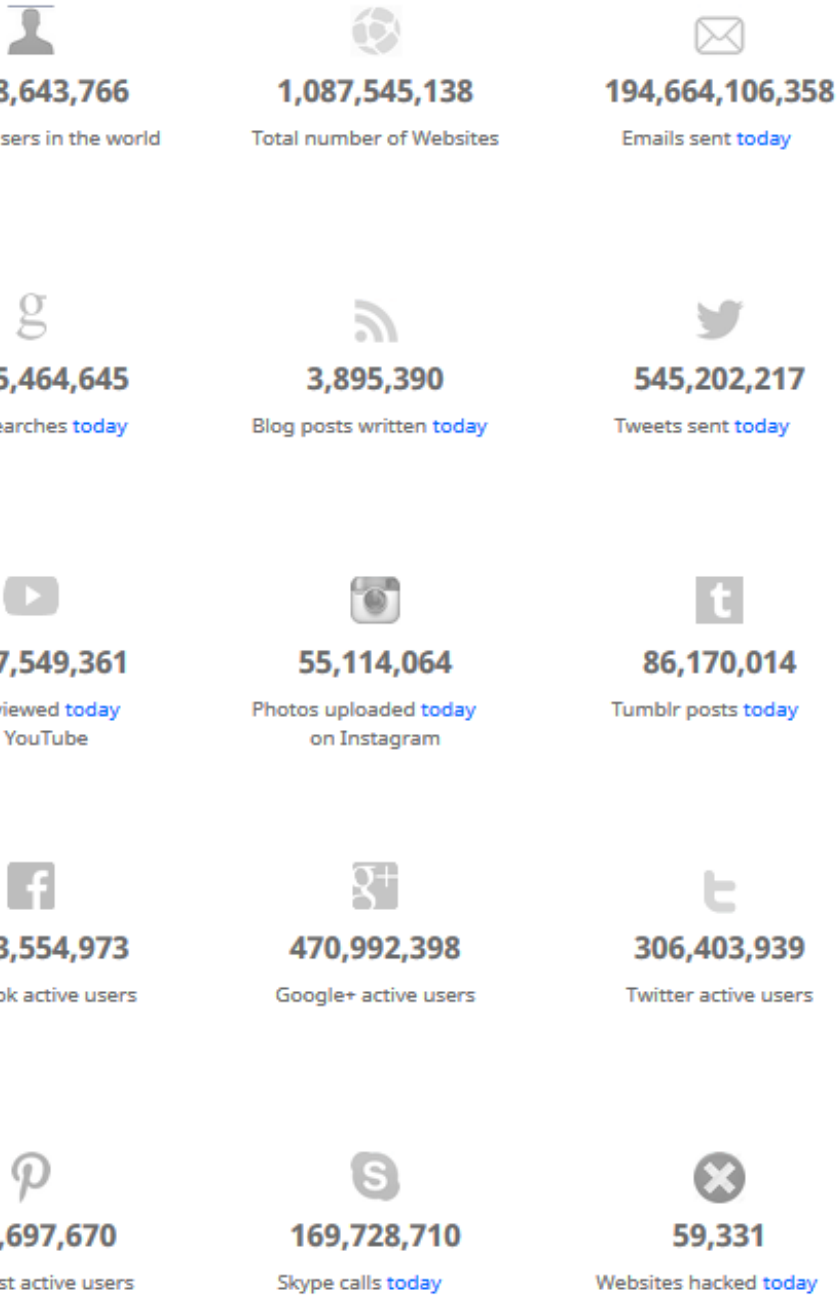


Figura. **Utilizatorii de Internet în lume - împărţirea pe zone geografice**
Iunie 2017 | Sursa: http://www.internetworldstats.com/

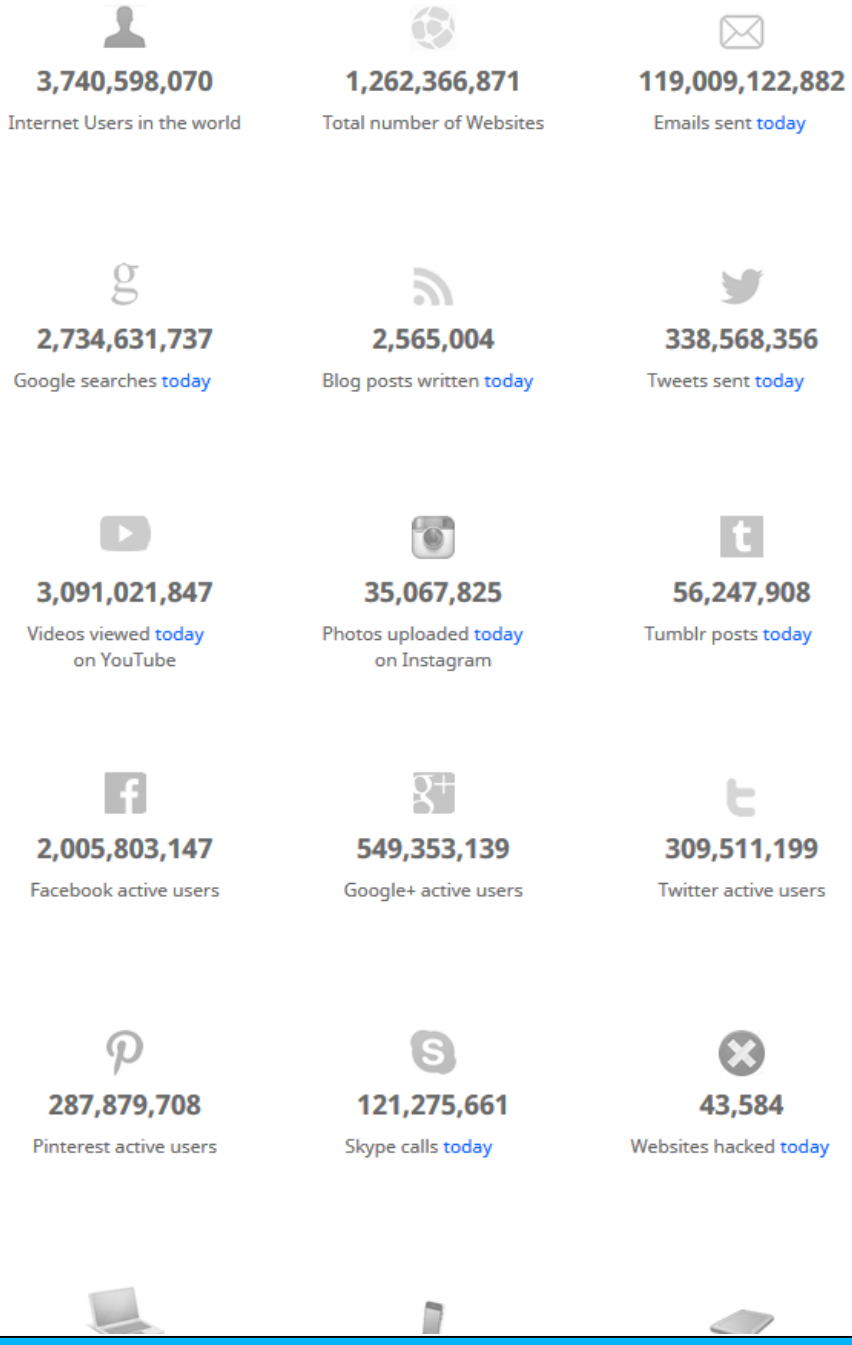
2015


www.internetlivestats.com
cs what happer
3,217,299,026
Internet Users in the world
937,415,878
Total number of Websites
102,297,859,891
Emails sent today
in 1 second
2,058,557,969
Google searches today
1,901,079
Blog posts written today
413,524,643
Tweets sent today
4,403,912,345
Videos viewed today on YouTube
111,277,088
Photos uploaded today on Instagram
91,387,825
Tumblr posts today
1,476,779,165
Facebook active users
1,511,599,583
Google+ active users
332,905,053
Twitter active users
90,263,626
Pinterest active users
76,356,431
Skype calls today
25,965
Websites hacked today

2016


3,468,643,766
Internet Users in the world
1,087,545,138
Total number of Websites
194,664,106,358
Emails sent today
4,185,464,645
Google searches today
3,895,390
Blog posts written today
545,202,217
Tweets sent today
9,797,549,361
Videos viewed today on YouTube
55,114,064
Photos uploaded today on Instagram
86,170,014
Tumblr posts today
1,743,554,973
Facebook active users
470,992,398
Google+ active users
306,403,939
Twitter active users
153,697,670
Pinterest active users
169,728,710
Skype calls today
59,331
Websites hacked today

2017


3,740,598,070
Internet Users in the world
1,262,366,871
Total number of Websites
119,009,122,882
Emails sent today
2,734,631,737
Google searches today
2,565,004
Blog posts written today
338,568,356
Tweets sent today
3,091,021,847
Videos viewed today on YouTube
35,067,825
Photos uploaded today on Instagram
56,247,908
Tumblr posts today
2,005,803,147
Facebook active users
549,353,139
Google+ active users
309,511,199
Twitter active users
287,879,708
Pinterest active users
121,275,661
Skype calls today
43,584
Websites hacked today

Prezent

Cursul de **Reţele de Calculatoare**
– Saptamânile 1-15-

Viitor

Cursul de **Reţele de Calculatoare**
– Saptamâna 16-

Resurse

- Internet Society: http://www.isoc.org
- FNC – Federal Networking Council: http://www.nitrd.gov/fnc/Internet_res.html
- InternetWorld Stats: http://www.internetworldstats.com
- Internet Systems Consortium: http://www.isc.org
- Internet Assigned Numbers Authority (IANA): http://www.iana.org/
- Cisco Systems: http://www.cisco.com
- http://www.nethistory.info
- http://www.caida.org/
- http://news.netcraft.com/archives/web_server_survey.html
- http://www.swivel.com/
- https://www.facebook.com/Grovo/info :)
- http://www.internetlivestats.com/
- http://www.worldometers.info/
- http://www.internetsociety.org/globalinternetreport/?gclid=CN6a2Y2wqMgCFUfkwgodZ8gL9A

Concepte si notiuni de baza

Lenuţa Alboaie

adria@info.uaic.ro

Cuprins

- Concepte
- Definitii
- Necesitatea si utilizarea retelelor de calculatoare
- Clasificare
- Topologii
- Componente
- Protocol
- Modele de arhitecturi de retea

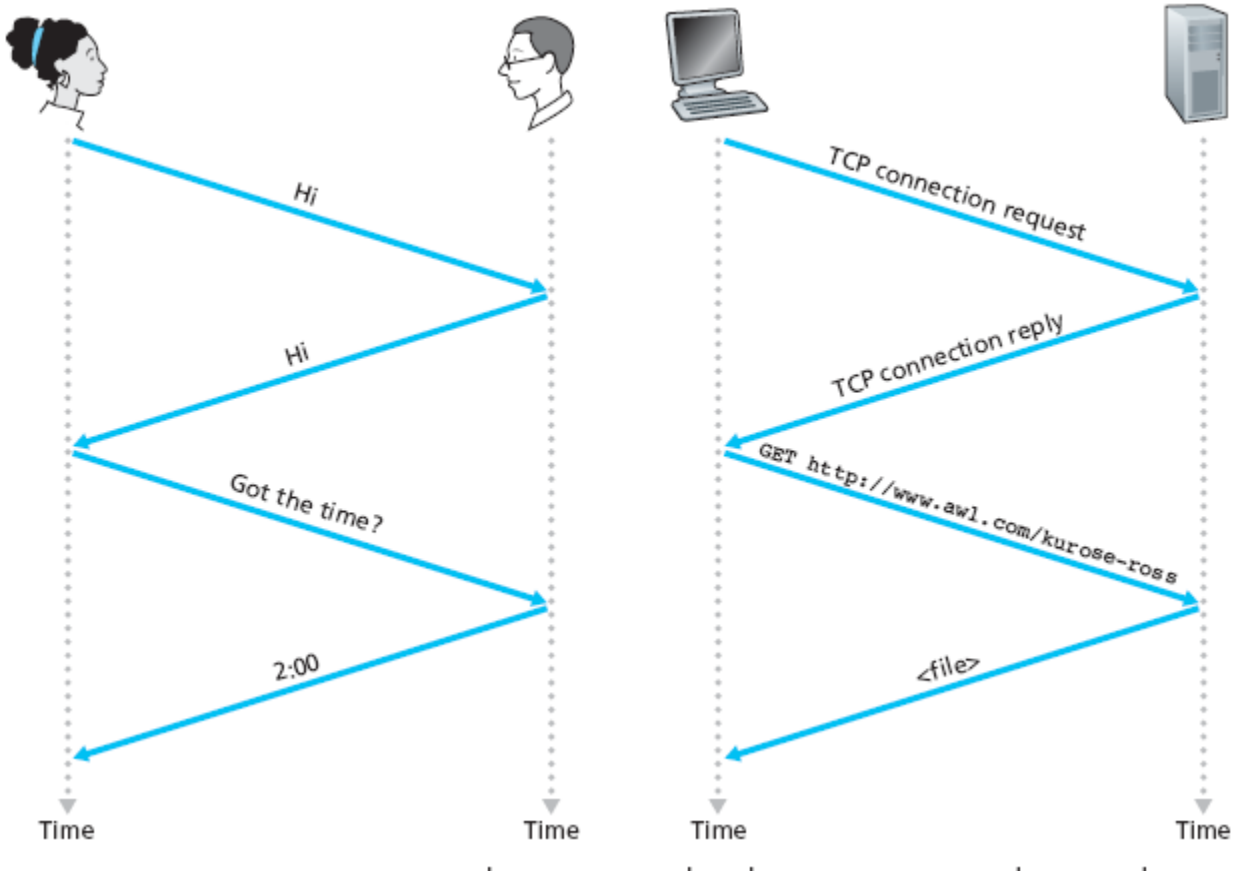
Aspecte Hardware & Software

Concepte

- **Informatie:** orice poate fi reprezentat sub forma de biti
- **Resursa:** termen generic putind reprezenta date, echipamente et. al.
- **Pachet:** modalitate de stocare a datelor
- **Legatura:** conexiune realizata intre membrii retelei
- **Nod:** calculator din retea care are asociata o adresa
- **Protocol:** reguli utilizate pentru realizarea comunicarii
- **Comunicare:** schimb de informatii intre nodurile din retea

Concepte

- **Protocol**



*A **protocol** defines the format and the order of messages exchanged between two or more communicating entities, as well as the actions taken on the transmission and/or receipt of a message or other event.*

Figure. Protocol
[Computer networking : a top-down approach James F. Kurose, Keith W. Ross]

Retea de calculatoare

- **Definitii:**
  - Colectie interconectata de calculatoare autonome
  - O retea poate fi definita in mod recursiv ca doua sau mai multe noduri conectate printr-o legatura fizica, sau prin doua sau mai multe retele conectate de unul sau mai multe noduri
- **Aspecte:**
  - **Hardware:** conecteaza “fizic” computerele intre ele
  - **Software:** Protocoale – specifica serviciile furnizate de retea

Retea de calculatoare

De ce sunt **necesare:**

- Partajarea resurselor (fizice, date)
- Asigurarea fiabilitatii
- Reducerea costurilor
- Impactul in viata reala:
  - Conversatiile electronice
  - Access la informatii la distanta
  - Divertisment interactiv
  - Comert electronic
  - …

Tipuri de retele - clasificare

- Dupa dispunerea spatiala:
  - PAN (Personal Area Network)
  - LAN (Local Area Network)
  - MAN (Metropolitan Area Network)
  - WAN (Wide Area Network)
  - Internet

Tipuri de retele - clasificare

- Dupa dispunerea spatiala:

Distanţa Interprocesor	Procesoare localizate în aceeaşi/acelaşi:	Exemple
1 m	Metru pătrat	PAN
10 m	Cameră	LAN
100 m	Clădire	
1 km	Campus	
10 km	Oraş	MAN
100 km	Ţară	WAN
1000 km	Continent	
10.000 km	Planetă	Internet

Figura. Clasificare dupa dispunerea spatiala
[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et. al.]

Tipuri de retele - clasificare

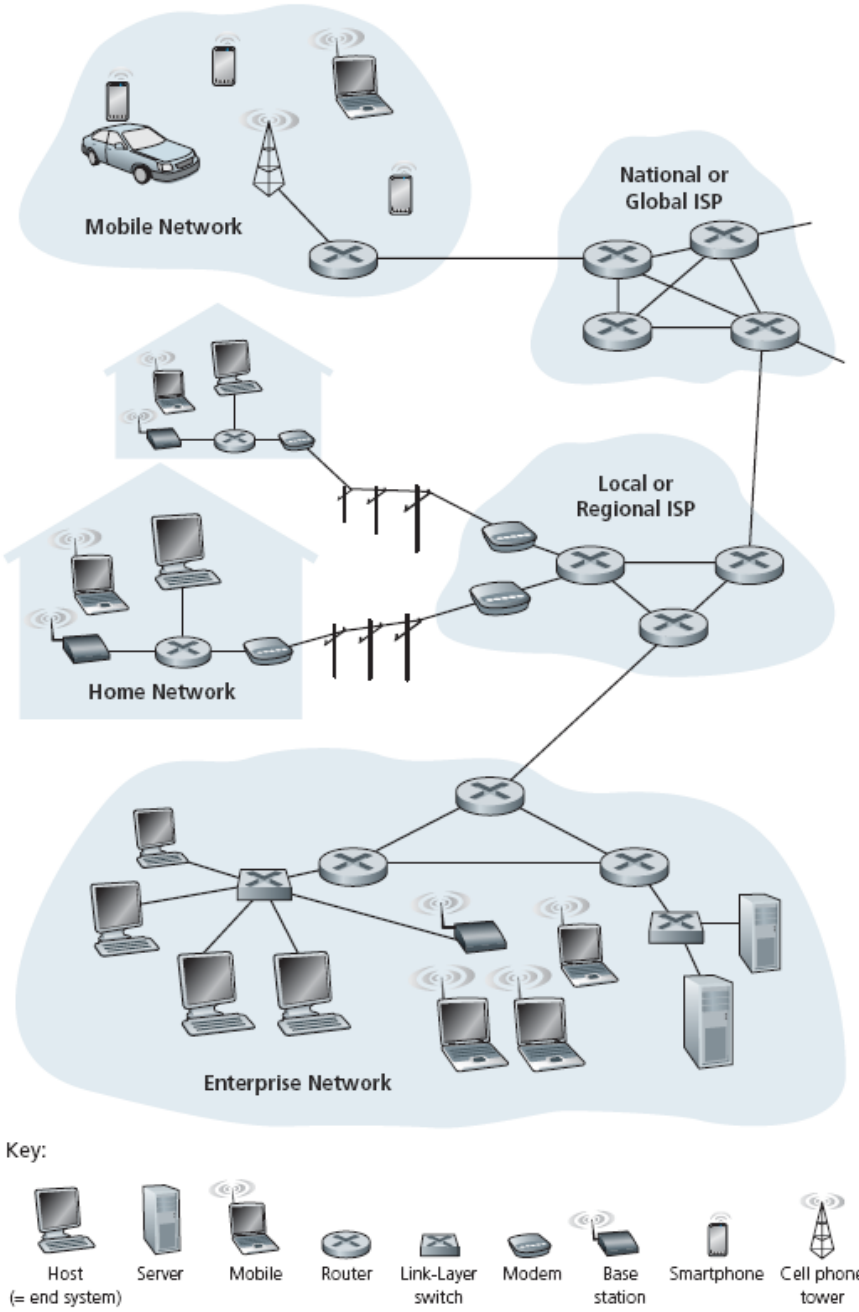


Figura. Internet – elemente componente
[Computer networking : a top-down approach
James F. Kurose, Keith W. Ross]

Tipuri de retele - clasificare

- In functie de tehnologia de transmisie:
  - Retele cu difuzare (un singur canal de comunicare)
    - *broadcast, multicast*
  - Retele *point-to-point*
    - *unicast*

Topologii de retea

Topologie fizica: modul de interconectare a calculatoarelor in retea

Topologie logica: modul in care datele sunt transferate de la un calculator la altul

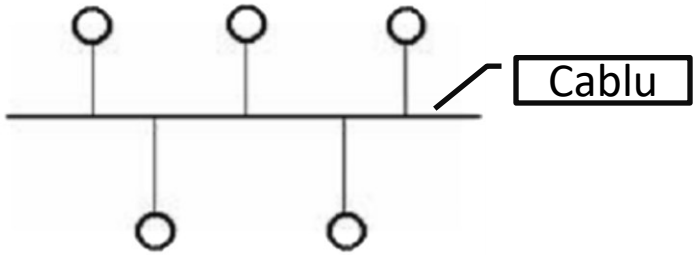
Posibile topologii fizice pentru:

- Retele cu difuzare LAN
  - Magistrala (*bus*)
  - Inel (*ring*)
- Retele *point-to-point*
  - Stea (*star*)
  - Inel (*ring*)
  - Arbore (*tree*)
  - Completa

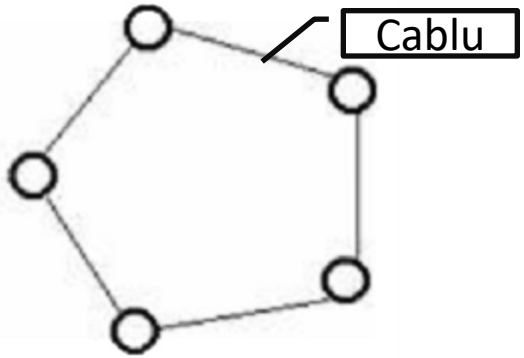
Topologii de retea

Retele cu difuzare - LAN

- *Magistrala (bus)*



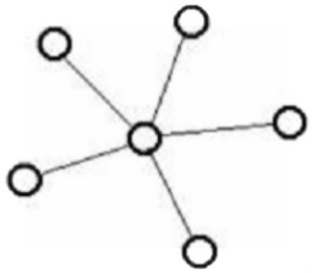
- *Inel (ring)*



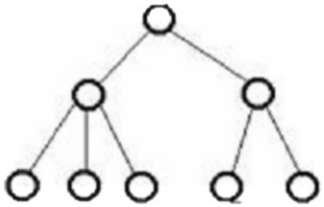
Topologii de retea

Retele *point-to-point*

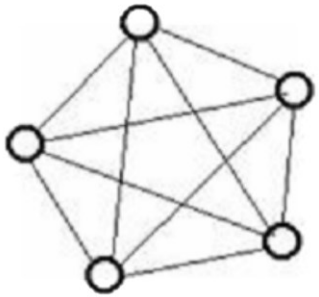
– *Stea (star)*



– *Arbore (tree)*



– *Completa*



Tipuri de retele - clasificare

- In functie de tehnologia hardware (si software) folosita pentru interconectare
  - Retele folosind **medii de transmisie cu fir**
  - Retele folosind **medii de transmisie fara fir** (curs viitor)

Tipuri de retele - clasificare

- In functie de elementele componente:
  - **Omogena:** retea de calculatoare folosind configuratii si protocoale similare
    - Exemplu: O retea folosind Microsoft Windows via TCP/IP
  - **Eterogena:** retea care contine tipuri diferite de calculatoare, sisteme de operare si/sau protocoale diferite.
    - Exemplu: LAN care conecteaza un smart phone cu Android si un computer Apple Machintosh

Componente

Gazda (eng. *Host*) – este un sistem computational conectat la Internet

Hub (*Hub Network*) – dispozitiv (deseori amplificator de semnal) folosit pentru conectarea mai multor dispozitive => *segment de retea (network segment)*

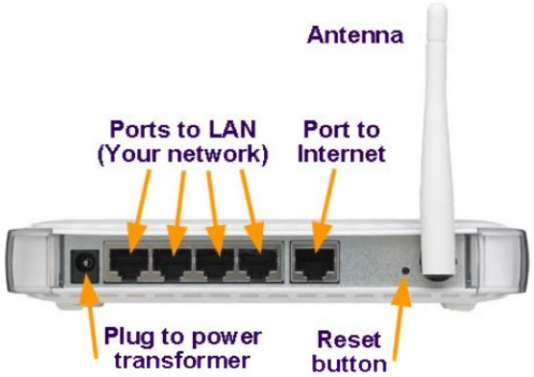
Switch (*Switch*)- dispozitiv care filtreaza si retrimite pachetele in retea

SMC EZ Switch 10/100
SMCGS16-Smart

Leonard-Kleinrock -> IMP (Interface Message Processor) 1969

Componente

- **Ruter** (*Router*) – dispozitiv oferind conectivitatea intre retele individuale, realizind dirijarea pachetelor intre aceste retele



- **Punte** (*Bridge*) – dispozitiv care conecteaza doua sau mai multe segmente intr-o retea
- **Poarta** (*Gateway*) – este punctul de conectare a doua retele incompatibile
- **Repeater -** este un dispozitiv electronic care primeste semnale pe care le retransmite la un nivel mai inalt sau la o putere mai mare, astfel ca semnalul sa poata acoperi zone mari fara degradare a calitatii sale

Rezumat

- Necesitatea si utilizarea retelelor de calculatoare
- Clasificare
- Topologii
- Componente

Intrebari?

Arhitecturi de retea

Lenuta Alboaie

adria@info.uaic.ro

Cuprins

- Retele de calculatoare – organizare
- Modele de arhitecturi de retea (OSI, TCP/IP)
- Modelul TCP/IP
- ISO/OSI versus TCP/IP

Retele de calculatoare – organizare

- Organizarea retelelor de calculatoare – **stiva de nivele**
  - Functionalitate:
    - **Interfata:** asigura comunicarea intre doua nivele consecutive
    - **Serviciu:** furnizeaza functionalitatea unui nivel
  - Rezultat: reducerea complexitatii proiectarii
  - Principiul de comunicare: **ce transmite emitatorul la nivelul n este ceea ce se primeste la destinatar la nivelul n**

- **Protocol** – regulile si conventiile prin care se realizeaza comunicarea

Exemplu: legatura - nivele, protocoale si interfete

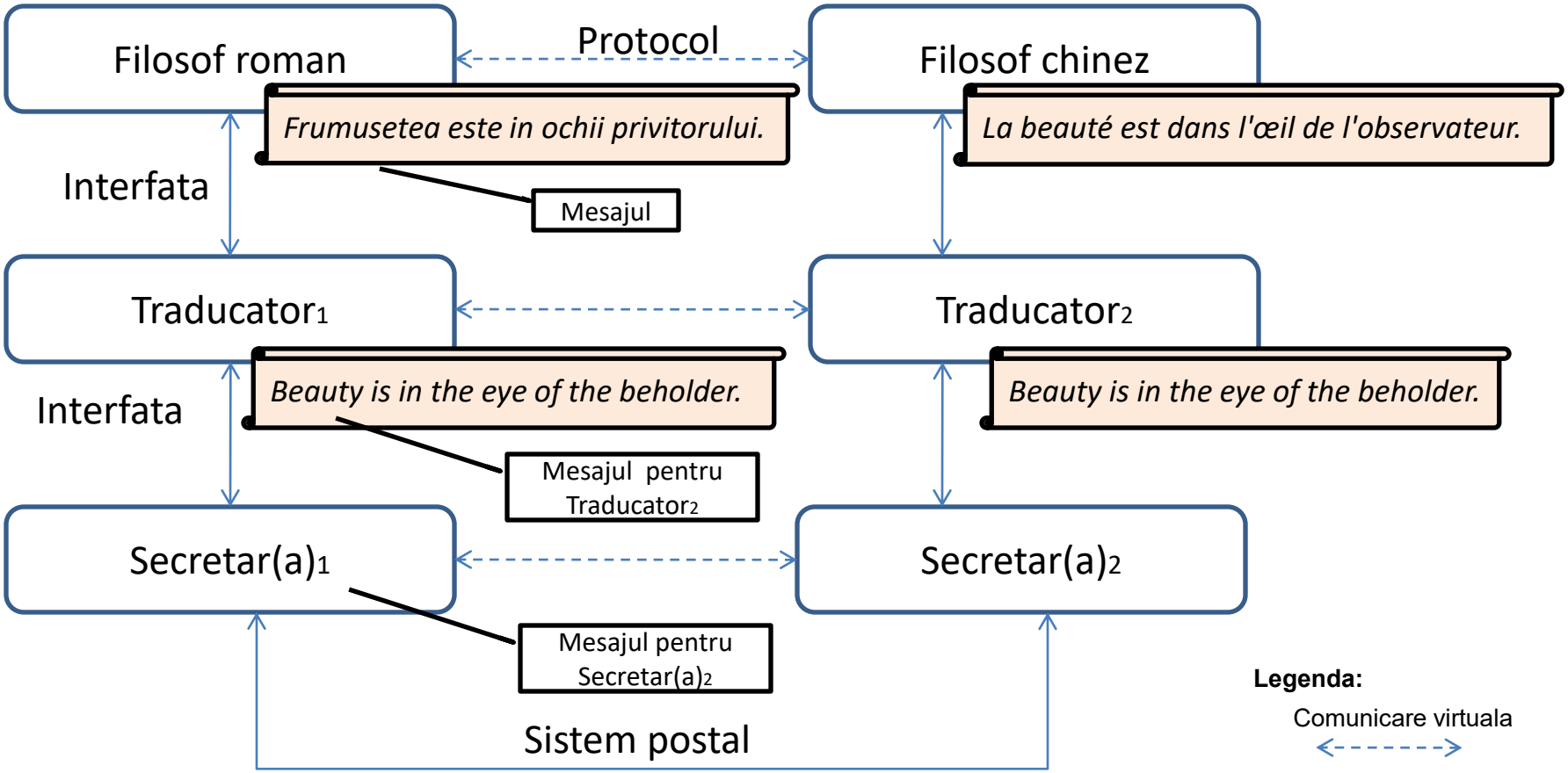
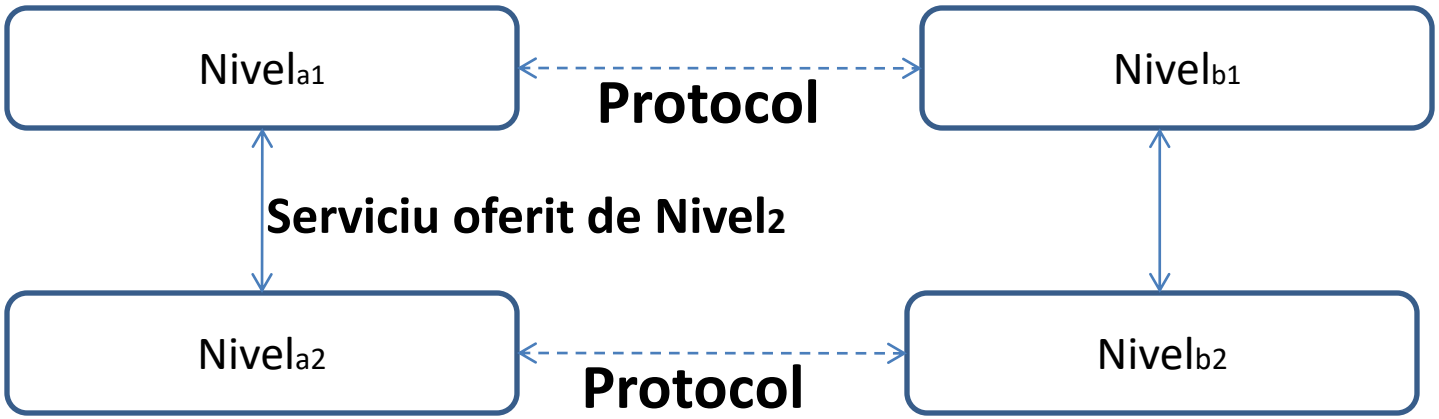


Figura: Arhitectura filosof – traducator - secretara

Aspecte privind proiectarea nivelurilor

- Specificarea serviciului este realizata printr-un set de primitive (operatii) puse la dispozitia celui ce foloseste serviciul
- Serviciu != Protocol



Aspecte privind proiectarea nivelurilor

- Tipuri de servicii
  - **Orientat-conexiune** (eng. *connection-oriented*)
    - Comunicarea necesita stabilirea unei conexiuni
    - Similar serviciului telefonic
  - **Fara conexiune** (eng. *connectionless*)
    - Comunicarea nu necesita stabilirea unei conexiuni
    - Similar serviciului postal

Aspecte privind proiectarea nivelurilor

- **Arhitectura de retea**: multimea de nivele si de protocoale
  - Specificatia unei arhitecturi trebuie sa ofere suficiente informatii pentru ca programele sau echipamentele destinate unui nivel sa indeplineasca protocoalele corespunzatoare
- **Stiva de protocoale**: lista de protocoale (de pe toate nivelele) utilizate de catre un anumit sistem

Aspecte privind proiectarea nivelurilor

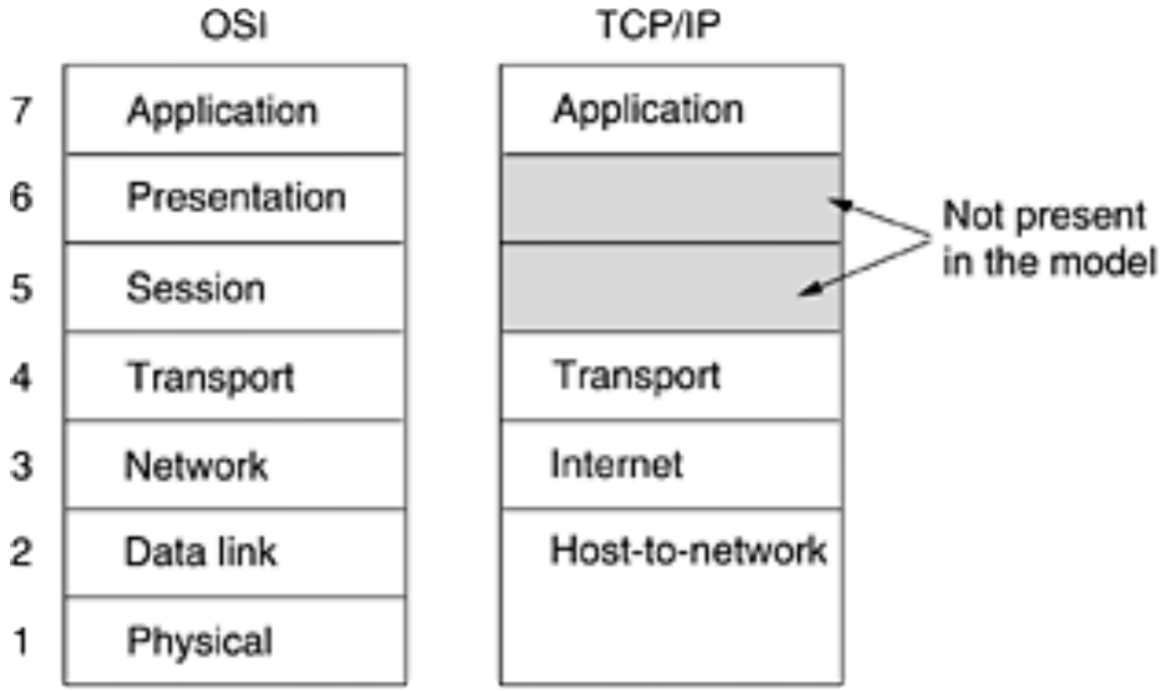
- Fiecare nivel trebuie sa realizeze indentificarea emitatorilor & receptorilor printr-un *mecanism de adresare*
- Identificarea regulilor de transfer a datelor
  - comunicare simplex
    - Exemplu: TV
  - comunicare half-duplex
    - Exemplu: "walkie-talkie"
  - comunicare full-duplex
    - Exemplu: telefon

Aspecte privind proiectarea nivelurilor

- In general canalele de comunicatie nu pastreaza ordinea mesajelor trimise => necesitatea unui protocol ce furnizeaza un mecanism de reconstituire a ordinii corecte a mesajelor
- Exista situatii in care receptorul nu poate face managmentul mesajelor de lungime variabila => trebuie sa existe un mecanism de impartire/asamblare a mesajelor
- Costuri mari in alocarea de conexiuni separate? => multiplexarea – utilizarea aceleiasi conexiuni pentru conversatii independente
- In general exista mai multe cai intre sursa si destinatie => mecanism de rutare
- Circuitele fizice de comunicatii nu sunt perfecte => necesitatea unui mecanism de control al erorilor

Modele de referinta pentru arhitecturi de retea

- ISO/OSI (*International Standard Organization/ Open System Interconnection*)
- TCP/IP (*Transmission Control Protocol/ Internet Protocol*)



[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Arhitectura de retea - Echipamente

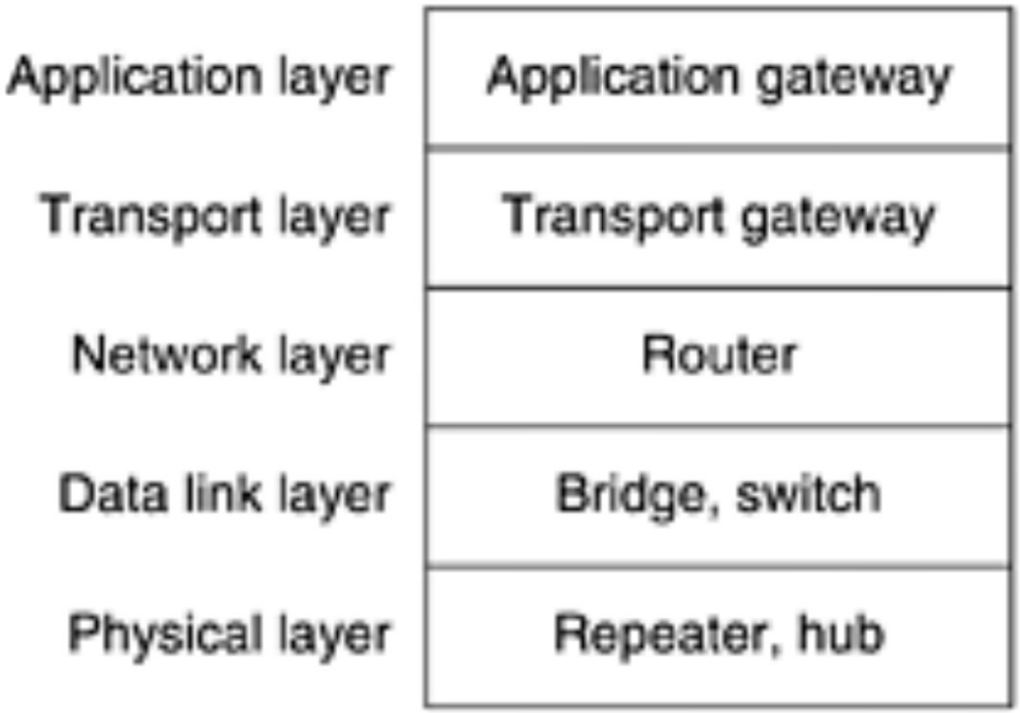


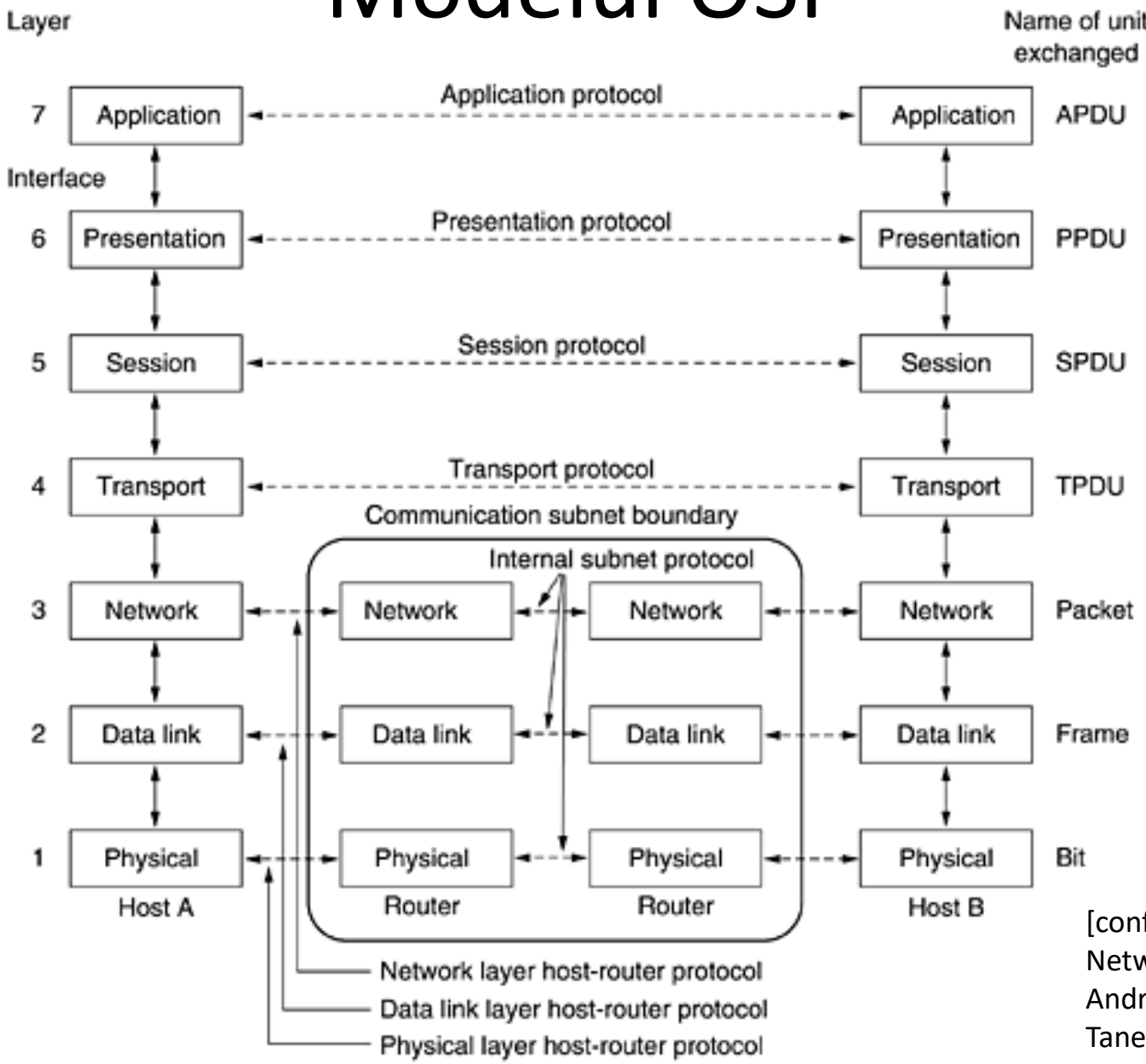
Figura: Dispozitive si nivelele corespunzatoare

Modelul OSI- motivatie

- Necesitatea unui nivel de abstractizare diferit => crearea unui nou nivel
  - Obs. Numarul de niveluri trebuie sa fie optim a.i. acelasi nivel sa aiba functii diferite, dar arhitectura sa fie functionala
- Un nivel are un rol bine definit; functia nivelului trebuie aleasa acordindu-se atentie definirii de protocoale standardizate pe plan international
- Minimizarea fluxului de informatii intre nivele este realizata printr-o buna delimitare a nivelelor

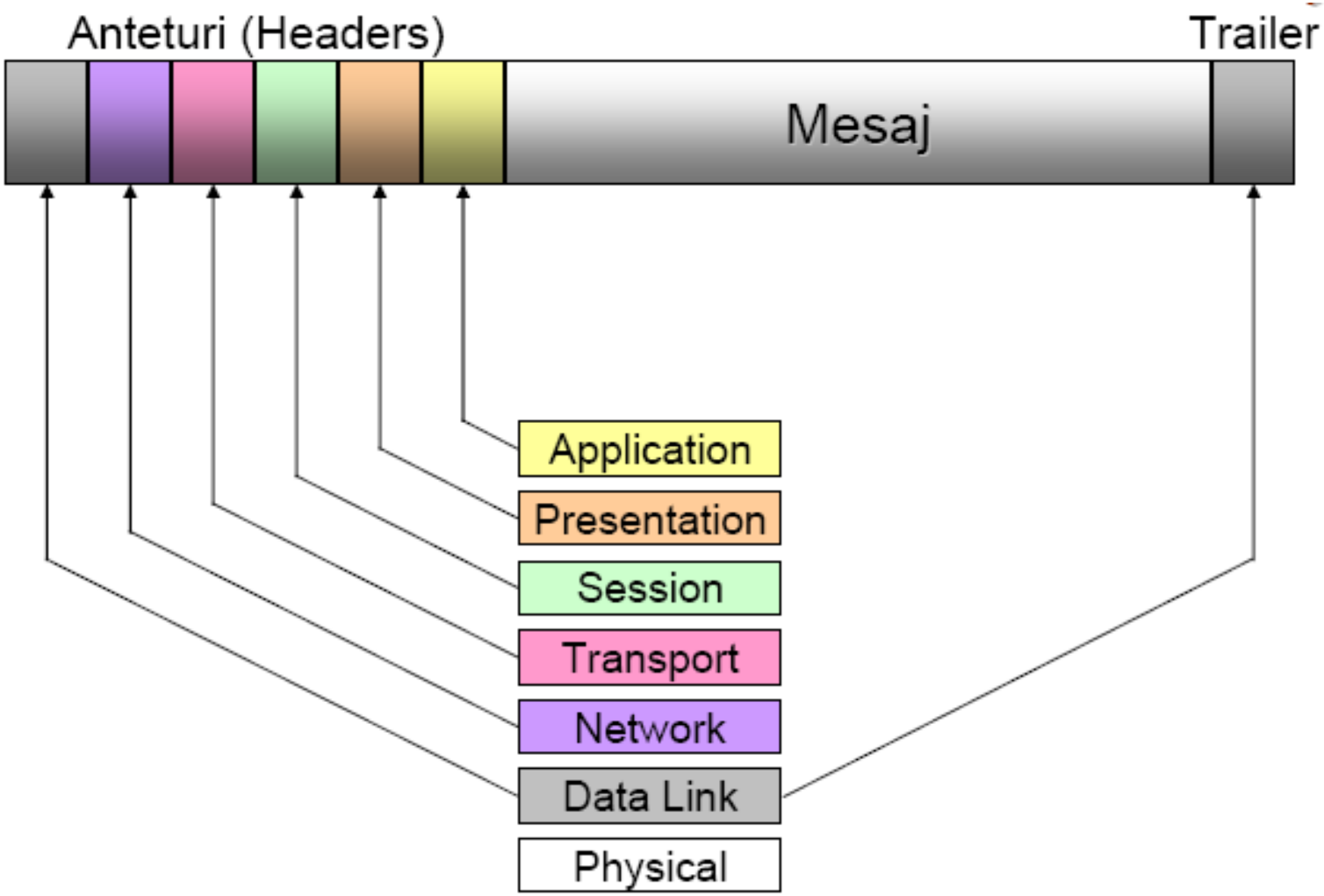
  => nivelele pot fi modificate si implementate in mod independent
- Fiecare nivel ofera un serviciu nivelului superior (folosind servicii de pe nivelurile anterioare)
- Nivelurile "*peer*" al sistemelor diferite comunica via un protocol

Modelul OSI



[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Modelul OSI – structura unui mesaj

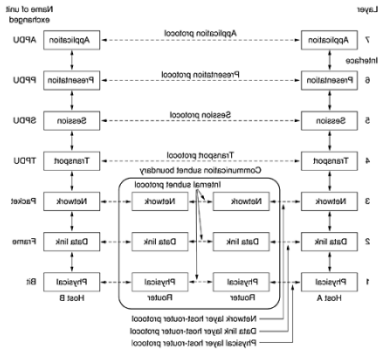


[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Modelul OSI – structura

- Nivelul Fizic
- Nivelul Legaturii de Date
- Nivelul Retea
- Nivelul Transport
- Nivelul Sesiune
- Nivelul Prezentare
- Nivelul Aplicatie

Modelul OSI



- **Nivelul Fizic:** mediu de transmisie a datelor
  - Rol: asigura faptul ca secventa de biti transmisa de la emitator ajunge la receptor
  - Medii de transmisie:
    - Cu fir (cablu torsadat, cablu coaxial, fibre optice)
    - Fara fir (spectru electromagnetic - radio, microunde, infrarosii,…) –> curs viitor

Modelul OSI

- **Nivelul Fizic:**

Transmiterea datelor:

- Analogic (valori continue)
  - Exemplu: vechi sisteme telefonice
- Digital (valori discrete)
  - Exemplu: computerele, …

Conversia datelor din format analogic în format digital si invers

- Modem: date în format digital sunt transmise în format analogic
- Codec (coder/decoder): date în format analogic sunt transmise în format digital

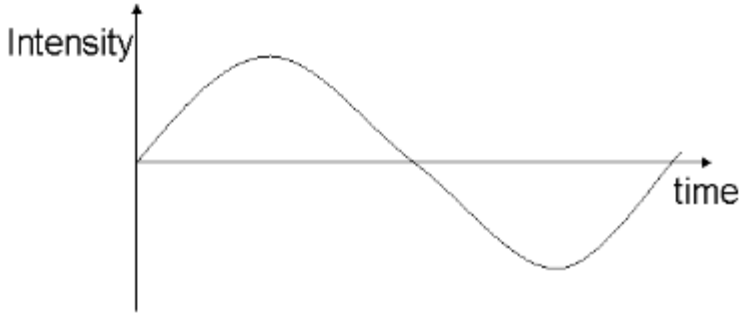


Figura. Semnal Analogic

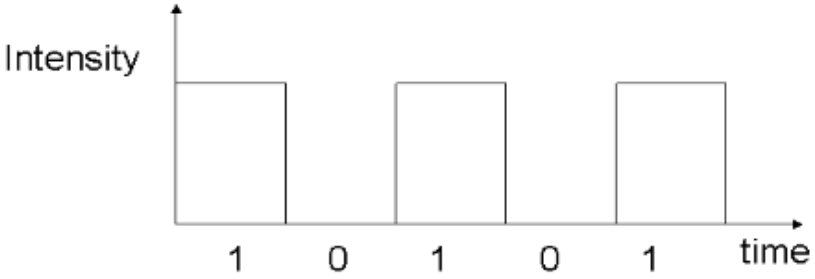
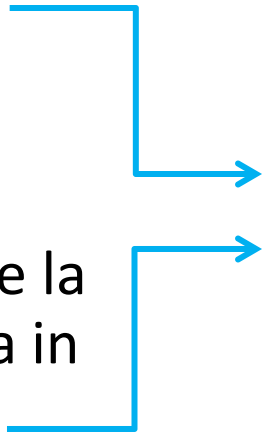


Figura. Semnal Digital

Modelul OSI

- **Nivelul Fizic - aspecte:**
  - **Largimea de banda (*Bandwidth*)**: numarul de biti care pot fi transmisi pe retea intr-o anumita perioada de timp (viteza transfer de date)
    - Se exprima de obicei in *bits/secunda*
  - **Latenta**: reprezinta intervalul de timp maxim necesar unui bit de a se propaga de la o extremitate la alta a retelei si se exprima in unitati de timp
    - RTT(*Round Trip Time*) - Timpul necesar unui bit să traverseze de la un capăt la altul, şi înapoi mediul



Parametrii fundamentali de asigurare a performantei retelei

Modelul OSI

- **Nivelul Fizic – Aspecte**

Modificari suferite de semnale in timpul propagarii in mediile de transmisie:

- **Atenuarea:** pierderea de energie în timpul propagării semnalului printr-un mediu de transmisie

- **Zgomotul:** modificarea semnalului cauzata de factori externi (e.g. fulgere, alte echipamente electronice etc)
  - Diafonia = zgomot provenit din semnal transmis de un mediul de transmisie vecin

- **Distorsiune** (engl. *Distortion*)- este o modificare determinista a semnalului receptionat fata de cel emis

Modelul OSI

- **Nivelul Fizic – Concluzii**

  Ofera servicii de transport, asupra carora putem indentifica o serie de probleme posibile

  – Datele pot fi alterate/distruse din cauza zgomotului
  – Daca destinatia nu poate prelucra datele in ritmul celor emise, o parte se vor pierde
  – Daca un acelasi mediu de transmisie este utilizat de mai multe emitatoare, exista riscul ca pachetele trimise sa se altereze reciproc
  – Este mai putin costisitoare construirea de legaturi logice care sa partajeze aceeeasi legatura fizica, decat crearea de legaturi fizice independente

Un nou nivel?

Modelul OSI

- **Nivelul legatura de date:**
  - Ofera
    - mecanisme de detectie si corectare a erorilor
    - mecansime de reglementare a fluxului de date
    - mecanism de control al accesului la mediu
    - servicii nivelului retea, unitatea de date fiind cadrul (engl. *frame*)

Modelul OSI

- **Nivelul legatura de date:**
  - Datele se incapsuleaza in cadre (*frame*-uri)
  - Analogie: *frame*=plic digital

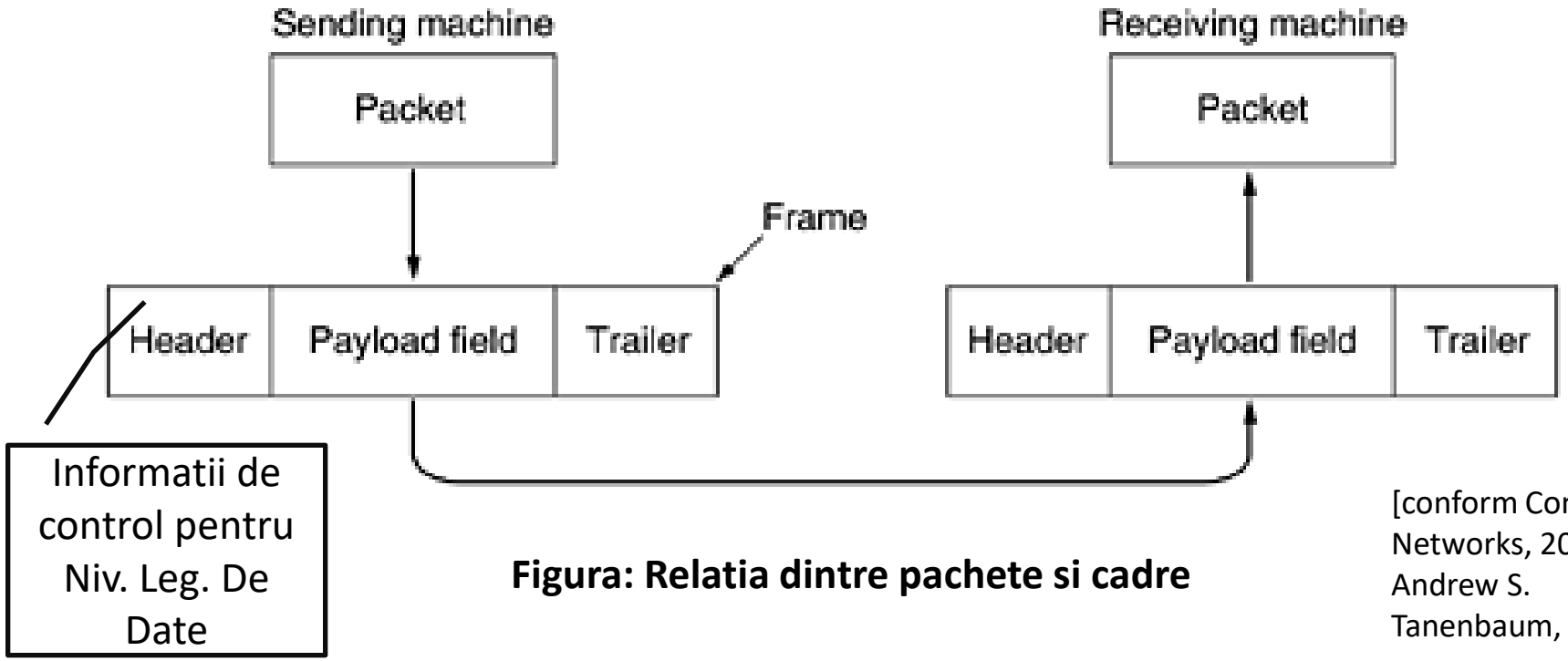


Figura: Relatia dintre pachete si cadre

[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Modelul OSI

- **Nivelul legatura de date:**
  - Ofera servicii nivelului retea
    - Servicii neconfirmate fara conexiune
      - » Emitatorul transmite cadre independente catre destinatar fara sa astepte confirmare
      - » Un cadru pierdut nu este recuperat
    - Servicii confirmate fara conexiune
      - » Se realizeaza confirmarea cadrelor trimise
      - » Transmiterea cadrelor nu se face in ordine
    - Servicii confirmate orientate-conexiune
      - » Inainte de transmiterea datelor se stabileste o conexiune
      - » Cadrele sunt numerotate pentru a se pastra ordinea

Modelul OSI

- **Nivelul legatura de date:**
  - Divizat in doua subniveluri:
    - **Controlul logic al legaturii** – **LLC** (Logical Link Control)
      - Rol: Ofera nivelelor superioare o vedere independenta de mediul de comunicare
    - **Controlul accesului la mediu** – **MAC** (Medium Access Control)
      - Rol: Folosit pentru a determina cine urmeaza sa transmita intr-un canal multi-acces (engl. *multiaccess channel)*

Modelul OSI

- **Nivelul legatura de date:**
- **Controlul accesului la mediu** – **MAC** (Medium Access Control)
  - Contextul problemei: acelasi mediu fizic e folosit de mai multi emitatori (identificati unic printr-o adresa fizica sau adresa MAC) care activeaza simultan, de exemplu:
    - transmisie semi-duplex, intre entitati care utilizeaza acelasi mediu fizic pentru ambele sensuri
    - comunicatia prin unde radio, cand exista statii care emit pe aceeasi lungime de unda (Wireless Ethernet – IEEE 802.11, Bluetooth, etc).

Modelul OSI

- **Nivelul legatura de date:**
- **Controlul accesului la mediu** – **MAC** (Media Access Control)
  - Strategii:
    - Alocare statica
      - » FDM (Frequency Division Multiplexing)
      - » TDM (Time Division Multiplexing)
    - Acceptarea posibilitatii coliziunilor si retransmiterea pachetelor afectate de coliziuni – alocare dinamica

      Coliziune=transmiterea simultana a datelor

      Mecanism general: o statie ce are date de transmis, le transmite imediat; in caz de coliziune va face retransmitere pana la transmitere cu succes

Modelul OSI

- **Nivelul legatura de date:**

  Controlul accesului la mediu – **protocoale:**

  – **ALOHA**
    - Pure ALOHA : “transmite oricind doresti”
    - Slotted ALOHA

  – **CSMA (Carrier Sense - Multiple Access):** protocol cu detectia transmisiei (“canal liber inainte de a transmite?”)
    - *1-persistent CSMA*
    - *…*
    - *p-persistent CSMA*

Modelul OSI

- **Nivelul legatura de date:**
  - Controlul accesului la mediu – **protocoale:**
    - **CSMA** (Carrier Sense - Multiple Access)
      - CSMA/CD *(CSMA with Collision Detection)*
        - » *“canalul e liber in timp ce transmiti?”*
        - » baza pentru Ethernet LAN (IEEE 802.3)
    - **MACA (Multiple Access with Collision Avoidance)**
      - Baza pentru retelele wireless (IEEE 802.11)
    - **MACAW**
      - Imbunatateste MACA

Standard IEEE	Descriere
802	Grupul de standarde pentru reţele LAN şi MAN
802.2	LLC (Logical Link Control)
802.3	Ethernet (Carrier Sense Multiple Access with Collision Detect (CSMA/CD))
802.3u	Fast Ethernet
802.3z	Gigabit Ethernet
802.11 a/b/g/n/ac	Reţele fără fir – wireless (WLAN)
802.15	Wireless PAN (802.15.1 Bluetooth, ...)
802.16	Reţele wireless WAN

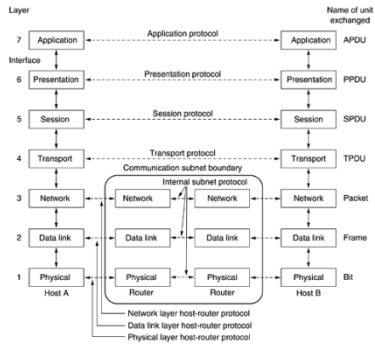
Accesul la mediu – Exemplu de Standarde

Modelul OSI

- **Nivelul legatura de date -** echipamente
  - **punti (engl. *bridges*)**
    - Retransmit frame-urile dintre doua retele (LAN)
    - Nu modifica continutul frame-urilor si pot schimba doar antetele acestora
    - Imbunatatesc siguranta transmiterii si performanta
    - Pot oferi controlul fluxului si congestiei datelor
    - Retransmiterea datelor se realizeaza via rute statice sau folosind un arbore de acoperire

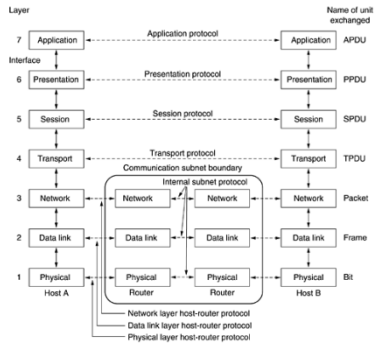
      STP (IEEE 802.1D) – Spanning Tree Protocol
  - **Alte echipamente? (Curs 1)**

Modelul OSI



- **Nivelul retea:**
  - Preia pachetele de la sursa si le transfera catre destinatie
  - Ofera servicii nivelului transport
    - ce fel de servicii?
      - Comunitatea Internet propune:
        - » servicii neorientate conexiune: SEND PACKET, RECEIVE PACKET
        - » Pachetele (numite ***datagrame***) sunt independente si sunt dirijate in mod individual
        - » Serviciile de tip datagrama sunt similare sistemului de posta (obsinuita)

Modelul OSI

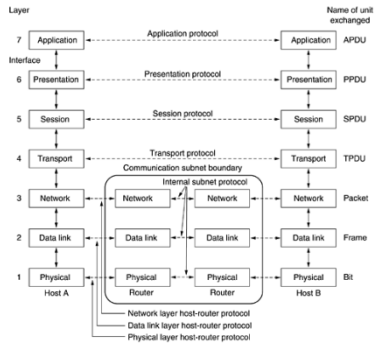


- **Nivelul retea:**
  - Preia pachetele de la sursa si le transfera catre destinatie
  - Ofera servicii nivelului transport
    - ce fel de servicii?
      - Companiile telefonice propun:
        - » Servicii orientate conexiune, sigure
        - » Inainte de transfer se initiaza o negociere pentru stabilirea unei conexiuni (***VC-virtual circuit)***
        - » Serviciile de acest tip sunt similare sistemului telefonic

Modelul OSI

- **Nivelul retea:**
  - Protocoale folosite
    - X.25 (orientat conexiune)
    - IP
  - Probleme
    - Conversii de protocol si adrese
    - Controlul erorilor (flux, congestie)
    - Divizarea si recompunerea pachetelor
    - Securitatea – criptare, *firewall*

Modelul OSI



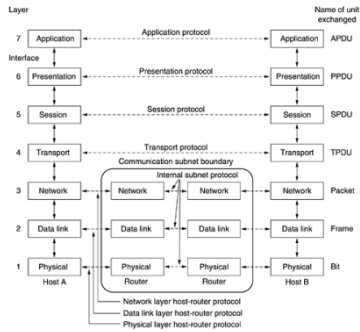
- **Nivelul transport:** ofera siguranta si cost-eficient in transportul datelor de la masina sursa la masina destinatie, independent de reteaua fizica sau retelele in prezent in uz
  - **Servicii:** ofera servicii orientate-conexiune si fara conexiune

Diferente intre nivelul transport si nivelul retea?

Modelul OSI

- **Nivelul transport:**
  - **Primitive:**
    - LISTEN – se blocheaza pina cind un proces incearca sa se conecteze
    - CONNECT – incearca sa stabileasca o conexiune
    - SEND – trimite date
    - RECEIVE – se blocheaza pina se primesc datele
    - DISCONNECT – eliberarea conexiunii
  - **Performanta** – calitatea serviciilor (QoS – Quality of Service): stabilirea/eliberarea conexiunii, rata de eroare, protectia, prioritatea, rezilienta (probabilitatea ca o conexiune sa se inchida din ratiuni interne), duplicarea pachetelor, controlul fluxului

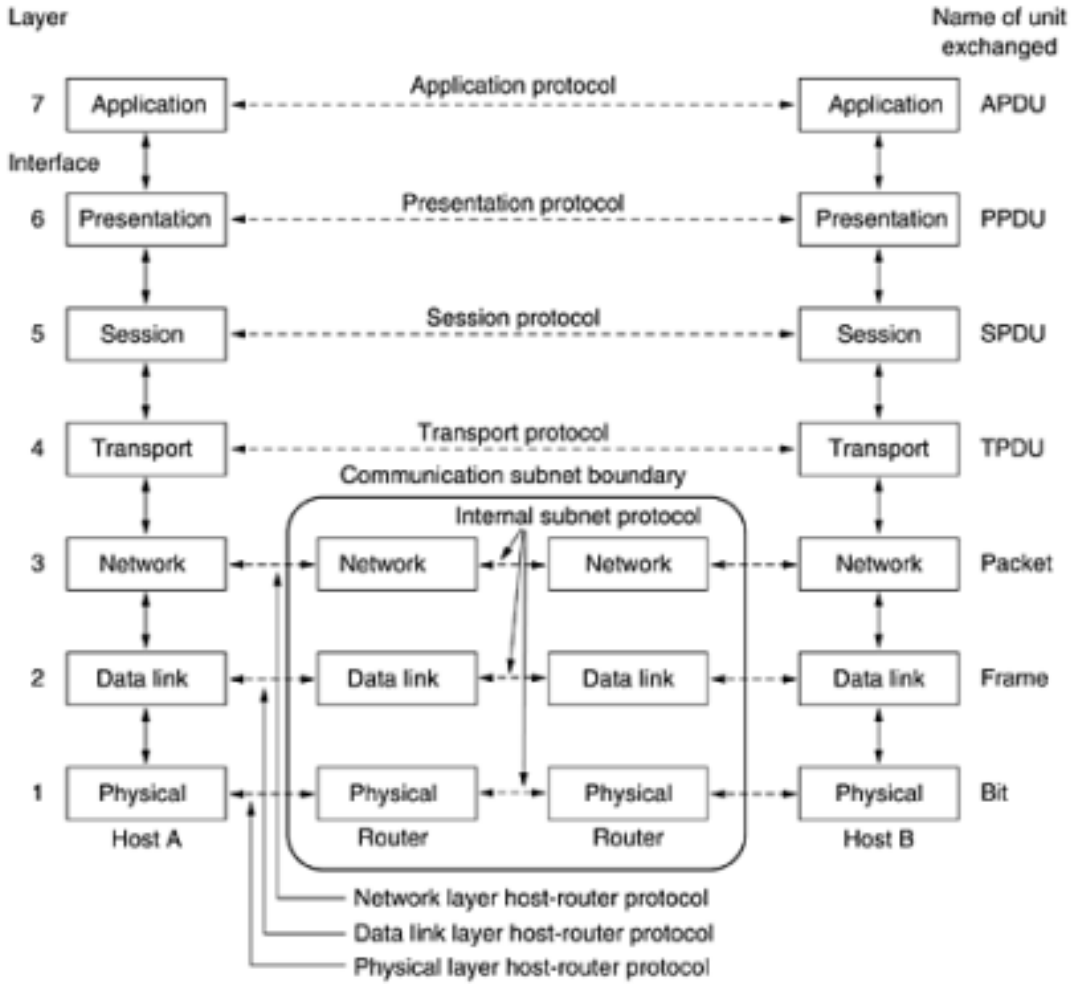
Modelul OSI



- **Nivelul sesiune:** se refera la probleme de stabilire de sesiuni (servicii de control al dialogului, de sincronizare etc.)
- **Nivelul prezentare:** se ocupa de prezentarea datelor, codificindu-le intr-un format standard
  - Pentru a se asigura comunicarea intre calculatoare cu reprezentari diferite, nivelul prezentare asigura conversia reprezentarilor interne a structurilor de date in reprezentare standardizata din retea si invers

Modelul OSI

- **Nivelul aplicatie:** gestioneaza servicii ale retelei: terminal virtual abstract, transfer de fisiere, posta electronica, executia la distanta a aplicatiilor,…



Modelul TCP/IP

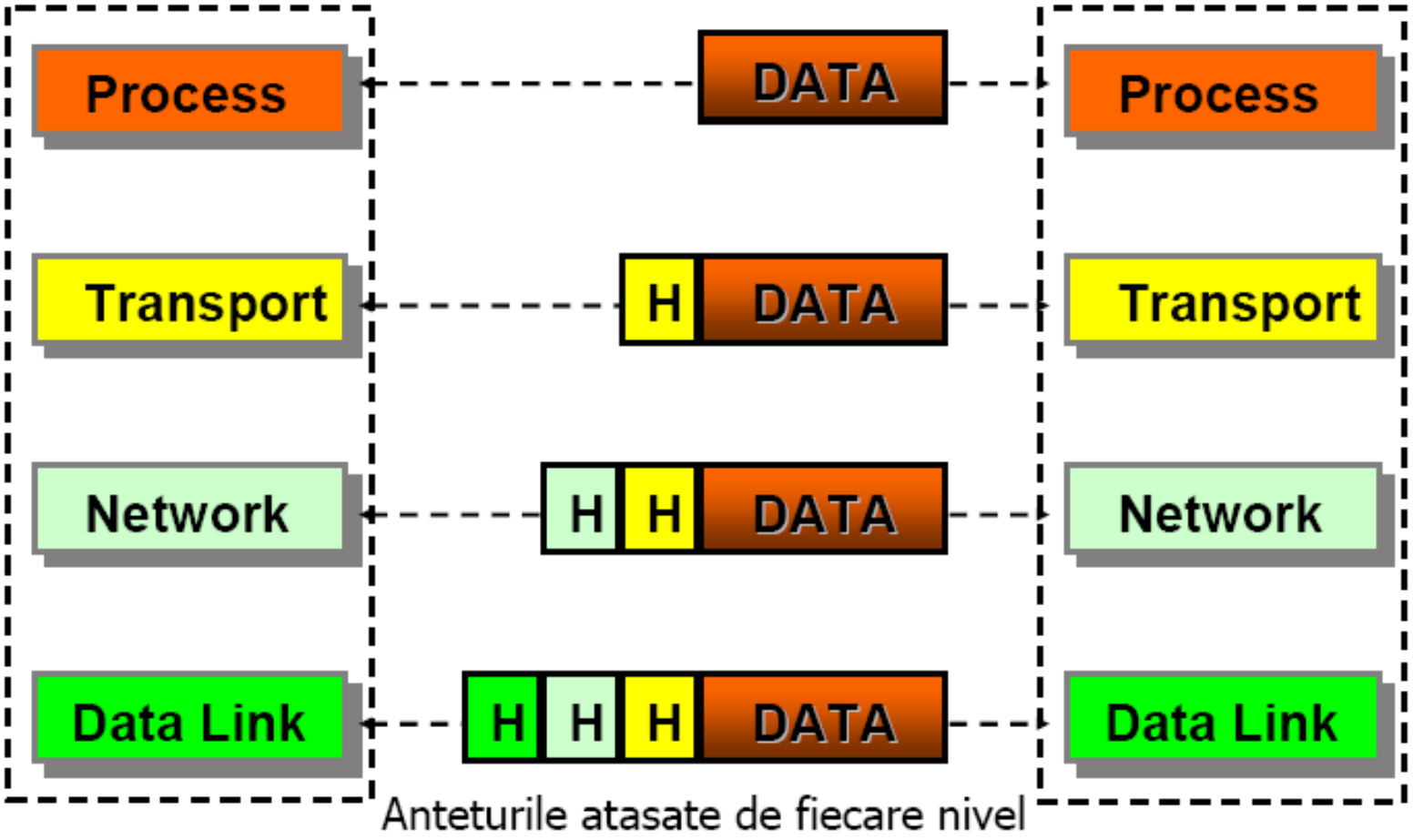
- **Termeni:**
  - sistem terminal (eng. *end-system*) – gazda (eng. *host*)
  - retea (eng. *network*) - ofera suportul pentru transferul de date intre sisteme terminale
  - internet - colectie de retele (interconectate)
  - subretea (eng. *subnetwork*) - componenta din internet
  - sistem intermediar (eng. *intermediate system*) - conecteaza doua subretele

Modelele de referinta: OSI si TCP/IP

TCP/IP Model	TCP/IP - Protocols	OSI Model
Application	FTP, Telnet, HTTP,…	Application
		Presentation
Transport	TCP, UDP, …	Session
		Transport
Internetwork	IP, …	Network
Host to Network	Ethernet, …	Datalink
		Physical

Figura: Imaginea generala a modelelor OSI si TCP/IP

Modelul TCP/IP



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Modelul TCP/IP

- Ofera posibilitatea de a interconecta mai multe tipuri de retele
- Are ca axa nivelurile retea si transport
- Implementat cu succes peste Ethernet (IEEE 802.3) – suportat de multe implementari ale nivelului fizic (cablu coaxial, *twisted pair*, fibra optica)

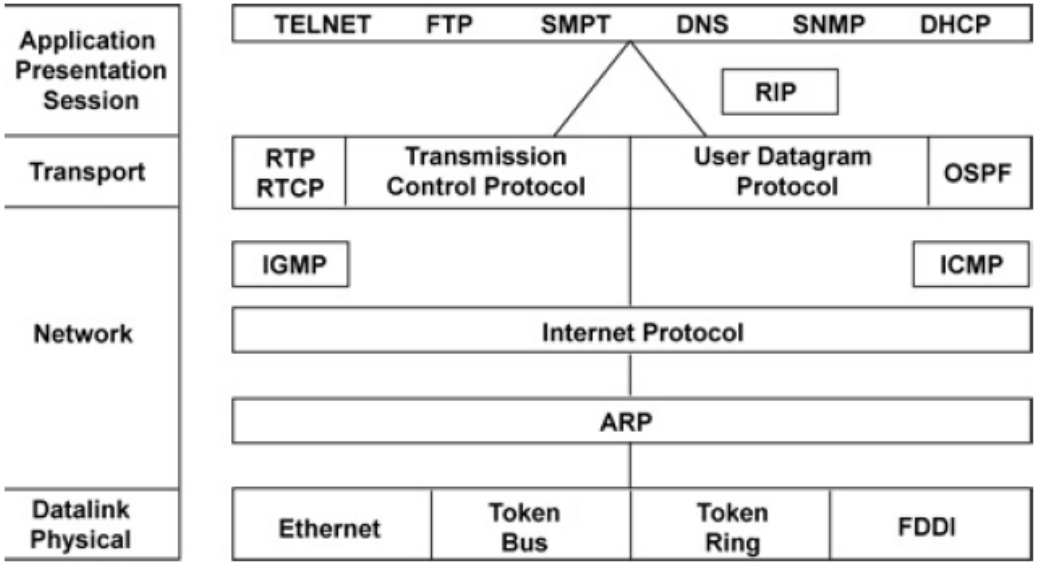


Figura. Modelul TCP/IP - protocoale

Modelul TCP/IP

- **Nivelul "fizic"**
  - Asigura conectarea *host*-ului la retea

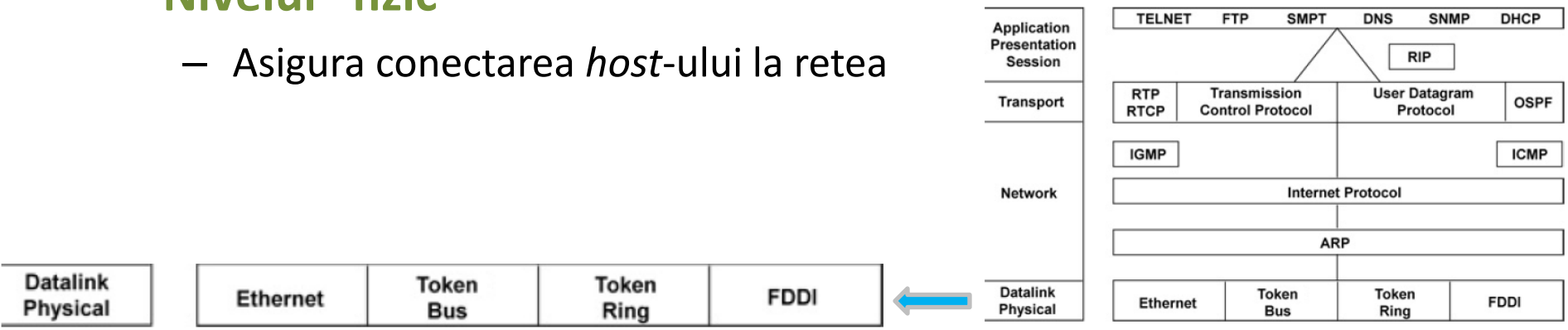


Figura. Modelul TCP/IP

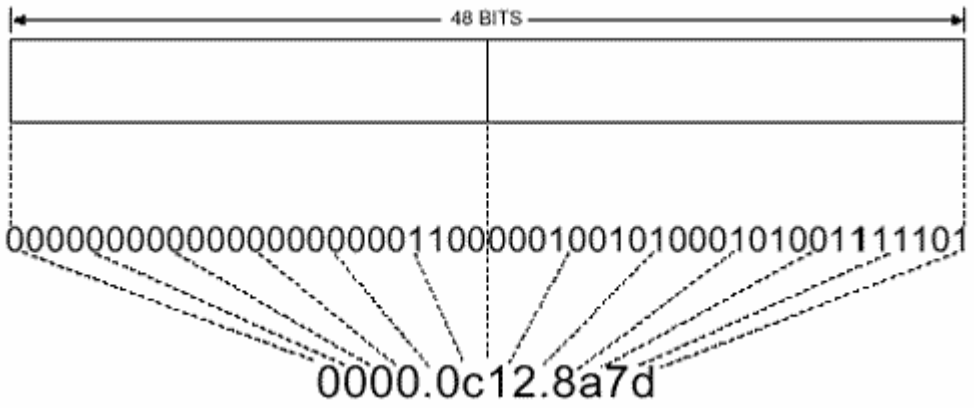
Ethernet

- Ofera acces multiplu (mediu partajat de transmisie) intr-o retea cu difuzare
- Detectia coliziunilor: CSMA/CD (Carrier Sense Multiple Access with Collision Detection)
- Fiecare interfata Ethernet are o adresa unica de 48 biti: *adresa hardware* (MAC) – e.g. C0:B3:44:17:21:17
  - Adresele sunt asignate producatorilor de placi de retea (NIC – Network Interface Card) de o autoritate centrala

Modelul TCP/IP

Ethernet

- Fiecare interfata(placa) de retea are o adresa MAC unica (unele sisteme de operare permit sa fie modificata prin soft)



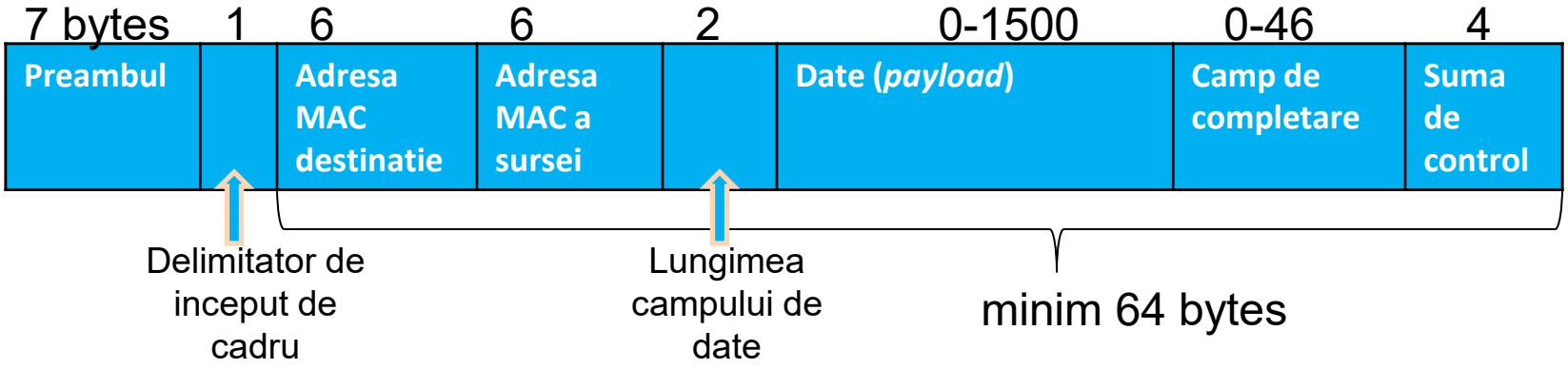
- Primii 24 de biti identifica producatorul

[conform Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Modelul TCP/IP

Ethernet

- Forma unui cadru (*frame*) de date:



- Broadcast: adresa are toti bitii setati pe 1
- Fiecare interfata de retea inspecteaza pentru orice cadru adresa de destinatie
- Daca adresa destinatie nu se potriveste cu adresa hardware sau cea de broadcast, atunci cadrul este ignorat

Modelul TCP/IP

Ethernet – standarde (exemple):

- 10 BASE5: 10 Mbps folosind cablu coaxial gros (Thick Ethernet)- 1980
- 1BASE5: 1 Mbps folosind 2 cabluri UTP (Unshilded Twisted Pair)
- 10BASE-T: 10Mbps folosind 2 perechi UTP – 1990
- 10BASE-FL: 10 Mbps fibra optica cu legatura point-to-point
- 10BASE-FB: 10Mbps backbone cu fibra optica (intre repetoare)
- 100BASE – FX: 100MBps CSMA/CD cu 2 fibre optice, full duplex
- … etc

Modelul TCP/IP

Ethernet versus Fast Ethernet

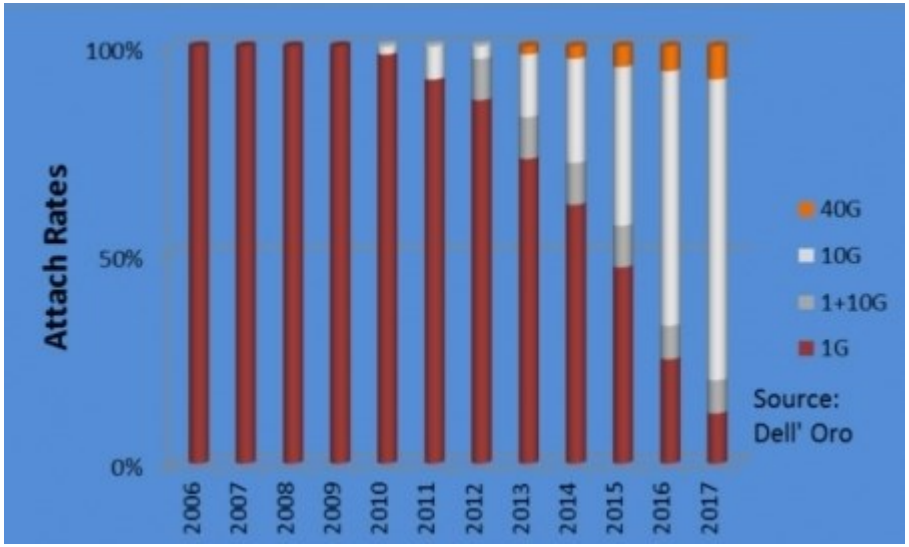
	Ethernet	Fast Ethernet
Viteza	10 Mbiti/s	100 Mbiti/s
Protocolul MAC	CSMA/CD	CSMA/CD
Diametrul retelei	2.5 km	205 m
Topologie	Magistrala, stea	Stea
Tip cablu	Coax, UTP, fibra	UTP, fibra
Standard	802.3	802.3u
Cost	c	2*c

[conform Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Modelul TCP/IP

- **Gigabit Ethernet**
  - Implementari atit pentru cabluri de cupru (802.3ab), cat si pentru fibra optica (802.3z)
  - Diferenta fata de alte implementari Ethernet este la nivel fizic

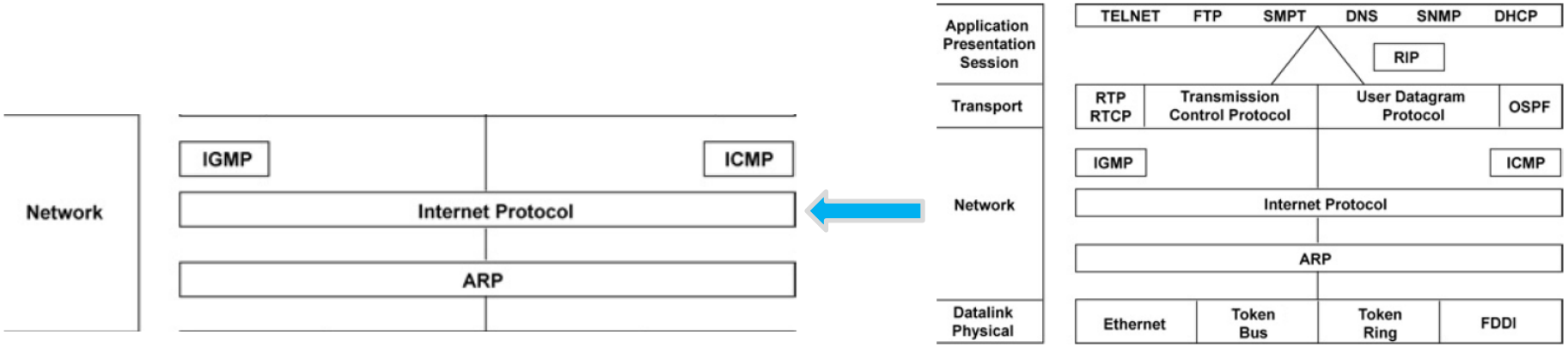
- **10 Gigabit Ethernet**
  - Implementari doar pentru fibra optica (802.3ae)
  - Opereaza la distante de 40km (util pentru MAN si WAN)
  - Formatul cadrelor este similar celui de la celelalte implementari Ethernet



[http://www.networkcomputing.com/networking/will-2014-be-the--year-of-10-gigabit-ethernet/a/d-id/1234640?]

Modelul TCP/IP

- **Nivelul retea**
  - Permite gazdelor sa emita pachete in orice retea; pachetele circula independent pina la destinatie



  - Aspecte principale:
    - Dirijarea pachetelor
    - Evitarea congestiei

Modelul TCP/IP

- **Nivelul retea**
  - Proiectarea nivelului a urmarit atingerea urmatoarelor obiective:
    - Serviciile oferite sunt independente de tehnologia utilizata (e.g. routere)
    - Asigura nivelului transport servicii, care ii permit acestuia sa functioneze in mod independent de numarul, tipul si topologia retelei
    - Furnizeaza un mecanism de adresare unic in LAN-uri si WAN-uri

Modelul TCP/IP

- **Nivelul retea**
  - **IPv4** (vezi curs 3)
  - **IPv6** (vezi curs viitor)
  - Dirijare (*routing*):
    - OSPF(*Open Shortest Path First*) – RFC 1131
    - BGP(*Border Gateway Protocol*) – RFC 1105
  - Multicast:
    - IGMP (*Internet Group Management Protocol*) – RFC 1112, 1054
  - Control:
    - ICMP (*Internet Control Messages Protocol*) - RFC 792,777
    - SNMP (*Simple Network Management Protocol*) – RFC 1157
    - ICMPv6 (vezi curs viitor)

Modelul TCP/IP

- **Nivelul transport**
  - Asigura realizarea comunicarii intre gazda sursa si gazda destinatie
  - Protocoale
    - ***TCP*** (*Transmission Control Protocol*) - RFC 793,761
    - ***UDP*** (*User Datagram Protocol*) – RFC 768
    - **Alte protocoale: SCTP** (*Stream Control Transmission Protocol*) – RFC 4960, 3286 (2960, 3309);**DCCP** (*Datagram Congestion Control Protocol*) – RFC 4340, 4336;

Modelul TCP/IP

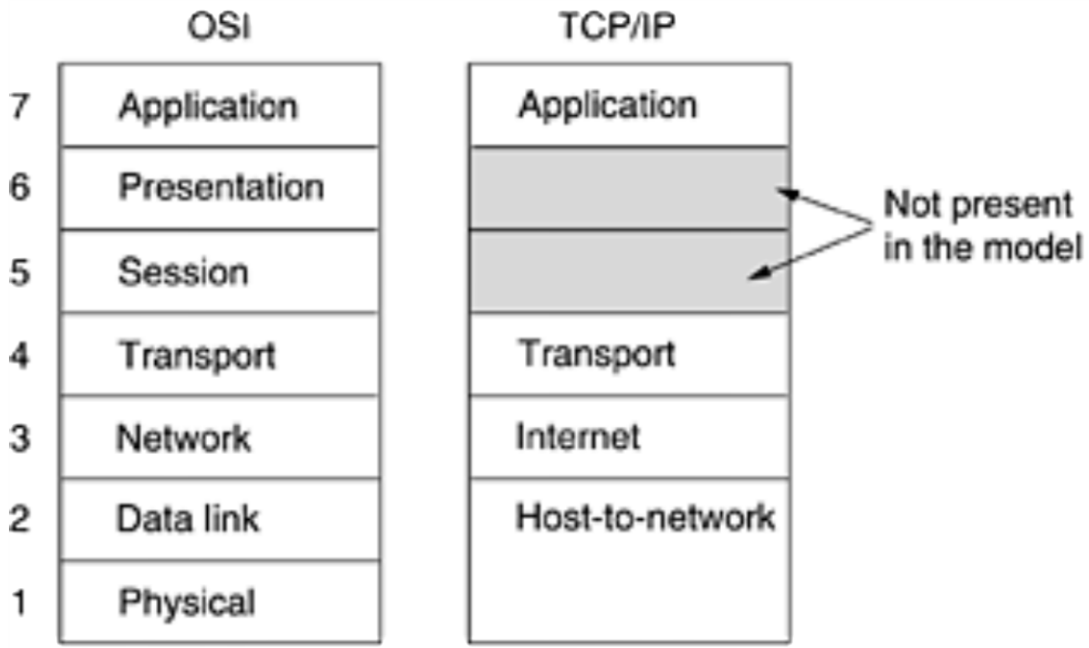
- **Nivelul aplicatie:**
  - Contine protocoale de nivel inalt
  - SMTP (*Simple Mail Transfer Protocol*) – RFC 5321 (821)
  - POP3(*Post Office Protocol*) – RFC 1081
  - TELNET – RFC 854,764
  - FTP (*File Transfer Protocol*) – RFC 454
  - NFS (*Network File System*) – RFC 1095
  - DNS (*Domain Name System*) – RFC 1034,1035
  - HTTP (*HyperText Transfer Protocol*) – RFC 2616
  - RTP (*Real-time Transport Protocol*) – RFC 3550 (1889)
  - SIP (*Session Initiation Protocol*) – RFC 3261
  - …etc

Modelul TCP/IP

- Organizatii implicate in standardizare:
  - ISOC – *Internet Society*
  - IAB – *Internet Architecture Board*
  - IETF – *Internet Engineering Task Force*
  - IRTF – *Internet Research Task Force*
  - InterNIC – *Internet Network Information Center*
  - IANA – *Internet Assigned Number Authority*
- Documentele RFC (*Request For Comments*)
  - Editate de Network Working Group (IETF)
  - RFC 1800 (Internet Official Protocol Standards)
  - Mai multe detalii -> www.ietf.org

OSI versus TCP/IP

- **Asemanari:**
  - Ambele se bazeaza pe o stiva de protocoale
  - Functionalitatile straturilor este oarecum asemanatoare
  - Ambele au nivelul aplicatie ca nivel superior
  - Se bazeaza (direct sau indirect) pe nivelul transport

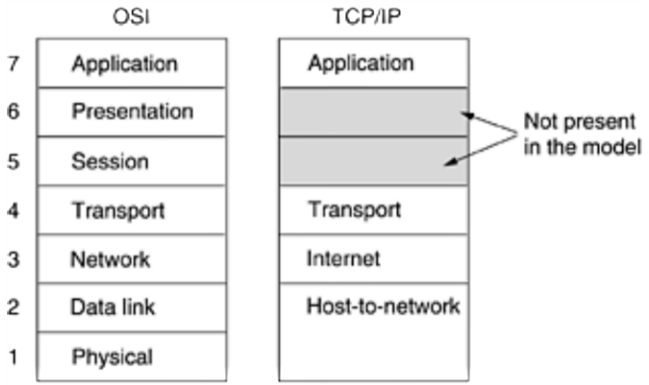


[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

OSI versus TCP/IP

- **Deosebiri:**
  - ISO/OSI este indicat ca model teoretic; TCP/IP este eficient in implementare
  - OSI face explicita distinctia intre serviciu, interfata si protocol; TCP/IP nu
  - ISO/OSI pune la dispozitie protocoale care asigura o comunicare fiabila (detectarea si tratare de erori la fiecare nivel);

    TCP/IP face verificarea comunicarii la nivelul transport
  - OSI suporta ambele tipuri de comunicatii la nivel retea (fara conexiune si orientate conexiune); TCP/IP suporta la nivelul retea comunicatii fara conexiune si la nivelul transport ambele moduri



[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Rezumat

- **Retele de calculatoare – organizare**
- **Modele de arhitecturi de retea (OSI, TCP/IP)**
- **Modelul TCP/IP**
- **ISO/OSI versus TCP/IP**

Intrebari?

Nivelul Retea

Lenuta Alboaie

adria@info.uaic.ro

Cuprins

- Nivelul Retea
  - Protocolul IPv4
    - Problematica
    - Caracterizare
    - Subretele
  - Retele Private
  - ICMP
  - Rezolutia adreselor
  - IPv6 – imagine generala
    - Detalii -> Curs Viitor

Preliminarii

- **Situatia initiala**

  - Inainte de Internet doar nodurile din aceeasi retea puteau comunica intre ele

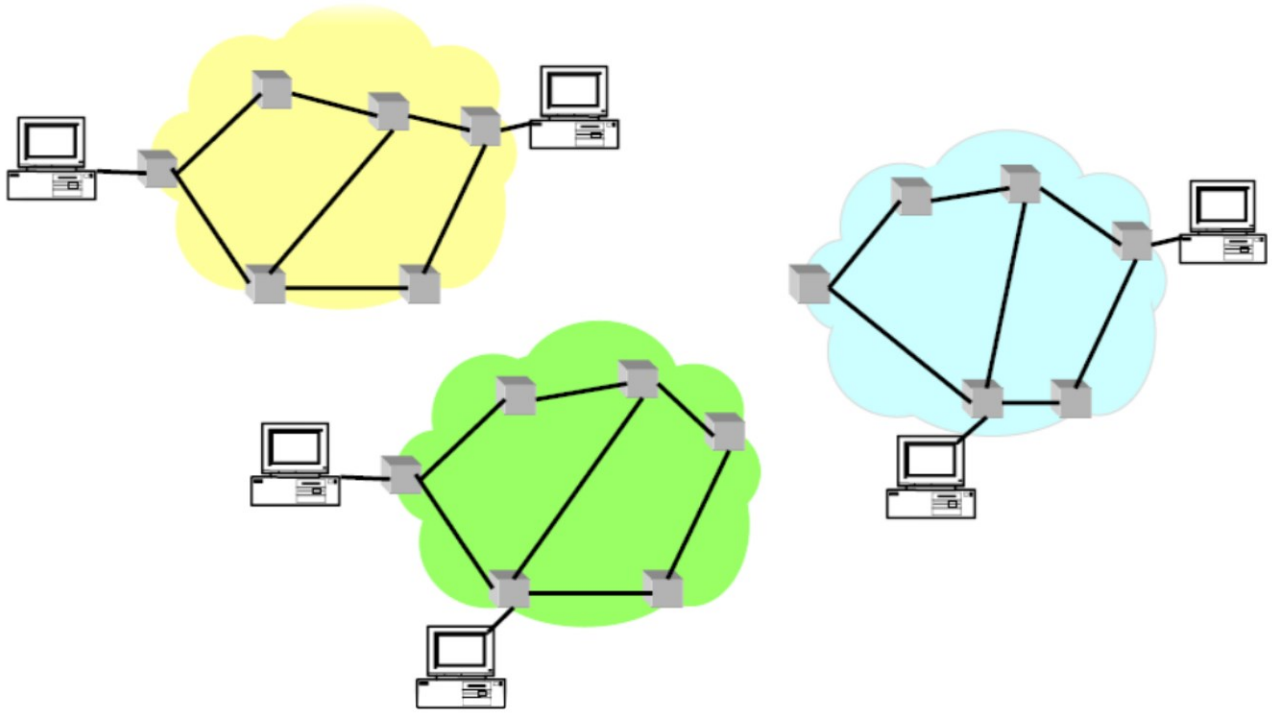
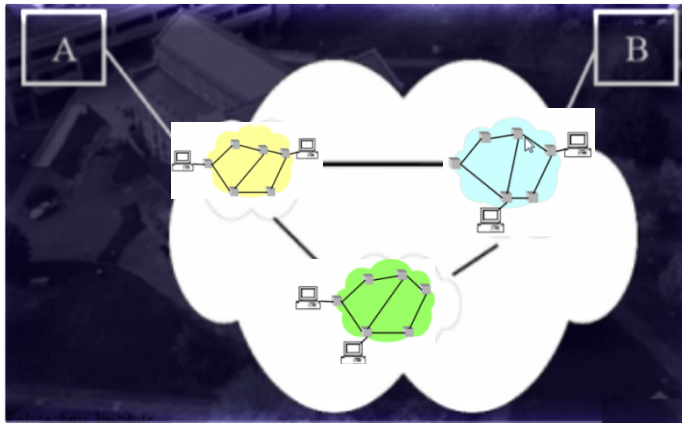


Figura: Retele individuale

Preliminarii

- Probleme
  - Cum se pot transporta pachete intr-un mediu eterogen?

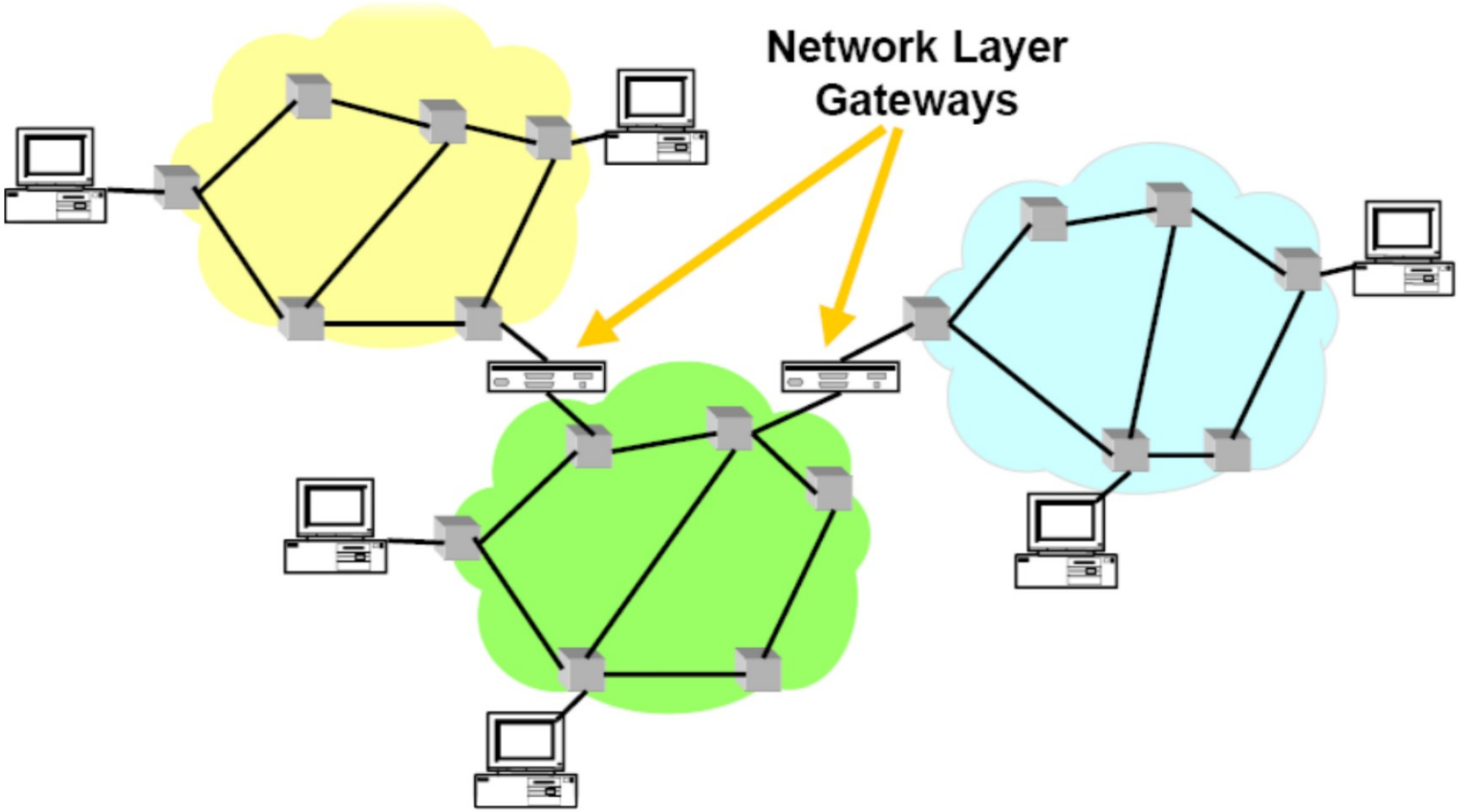
  - **Eterogenitate**
    - La nivelurile inferioare: cum se poate face interconectarea unui numar mare de retele independente?
    - La nivelurile superioare: cum se poate oferi suport pentru o mare varietate de aplicatii?



- **Scalare**: cum s-ar putea suporta un numar mare de noduri si aplicatii intr-un astfel de sistem de retele interconectate?

Solutia

- **IP – Internet Protocol**



Nivelul retea

- Protocolul IP este utilizat de sisteme autonome (*AS – Autonomous Systems*) in vederea interconectarii

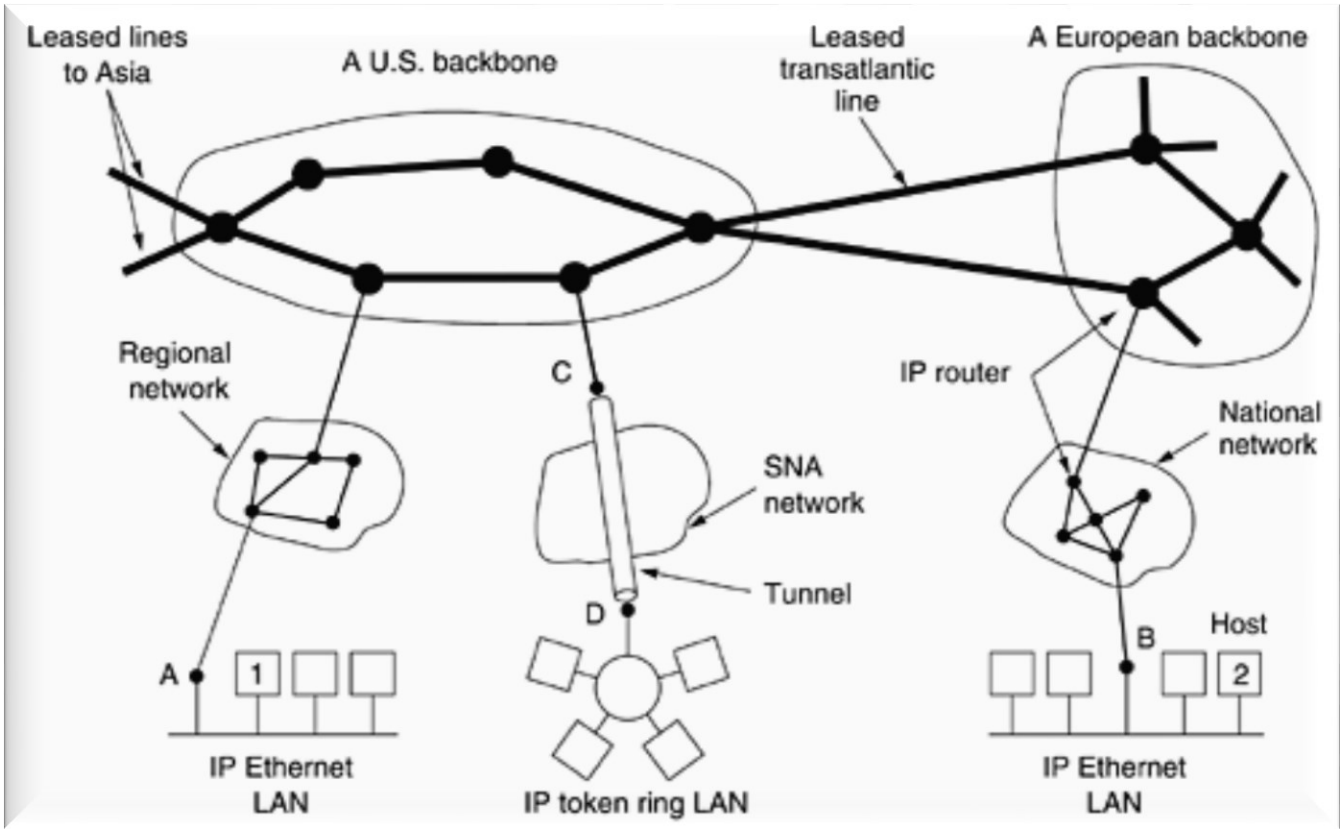


Figura: Internetul - colectie de retele interconectate

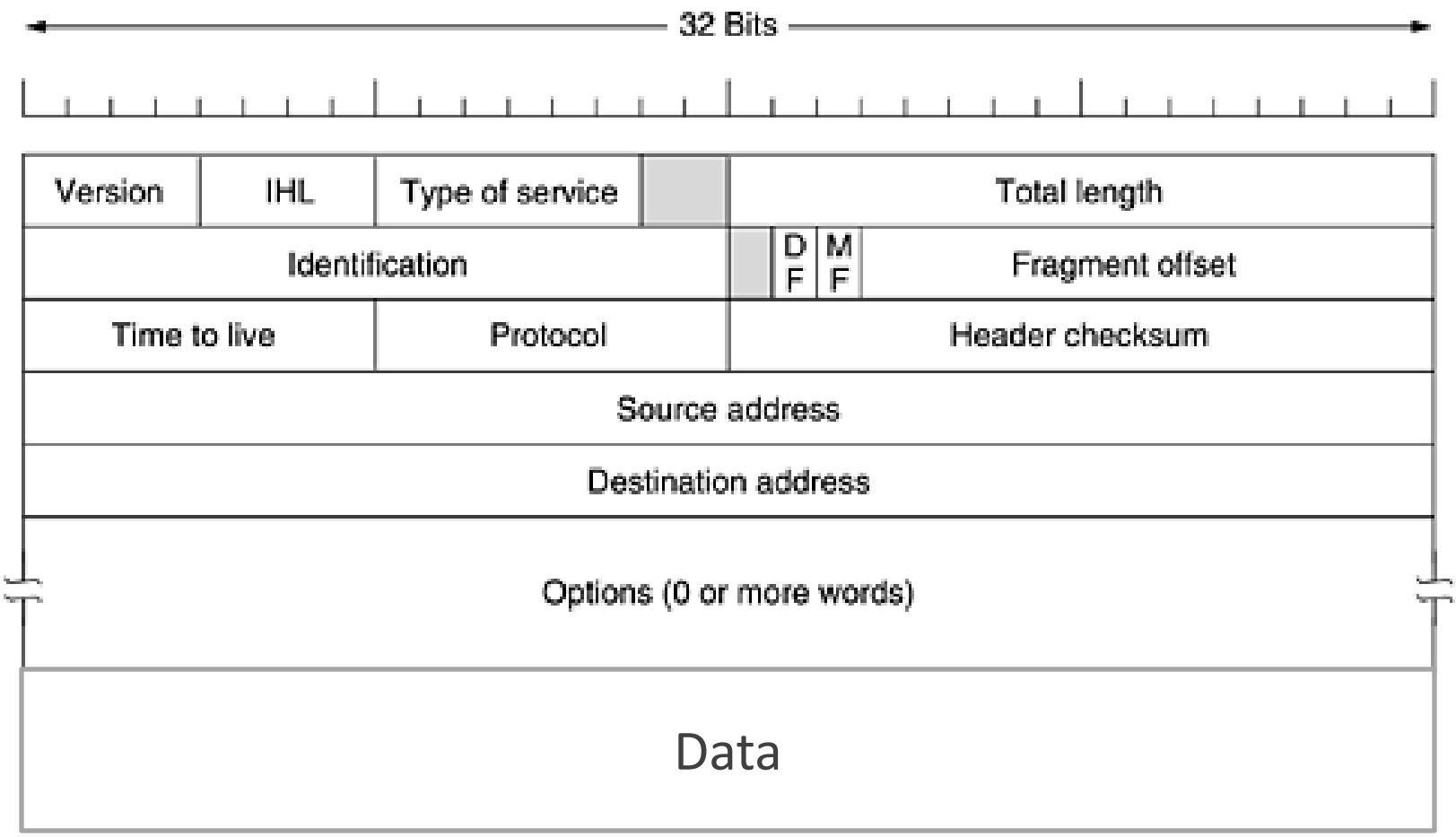
[Computer Networks, 2003 Andrew S. Tanenbaum]

Nivelul Retea

- Rol: ofera servicii neorientate-conexiune pentru a transporta **datagrame** de la sursa la destinatie; sursa si destinatia pot fi in retele diferite
- Fiecare datagrama este independenta de celelalte
- Nu se garanteaza trimiterea corecta a datagramelor (pierdere, multiplicare,…)
- +…Curs viitor

Protocolul IP

- **Datagrama IPv4**



[Computer Networks, 2003
Andrew S. Tanenbaum]

Protocolul IP

- **Datagrama IPv4**
  - Valorile uzuale ale campului *Version* sunt:
    - 4 – protocolul IP (RFC 791)

    (6 pentru protocolul IPv6 (RFC 1883))

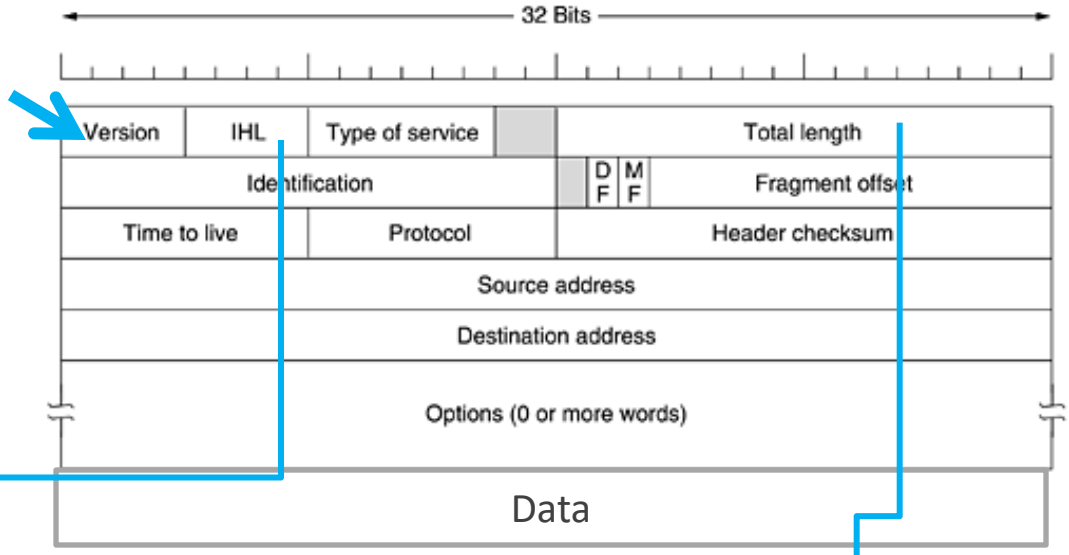


Figura: Datagrama IPv4

Specifica lungimea antetului datagramei

Specifica dimensiunea intregului pachet

Protocolul IP

- **Datagrama IPv4**
  - Campul *Type of service* permite gazdei sa comunice subretelei (e.g. routere) ce tip de serviciu doreste

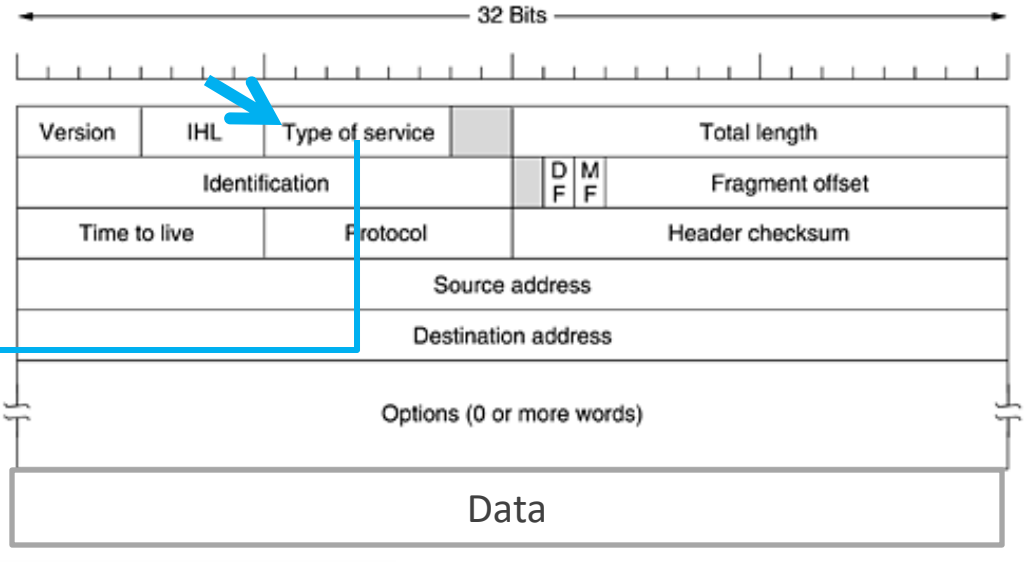


Figura: Datagrama IPv4

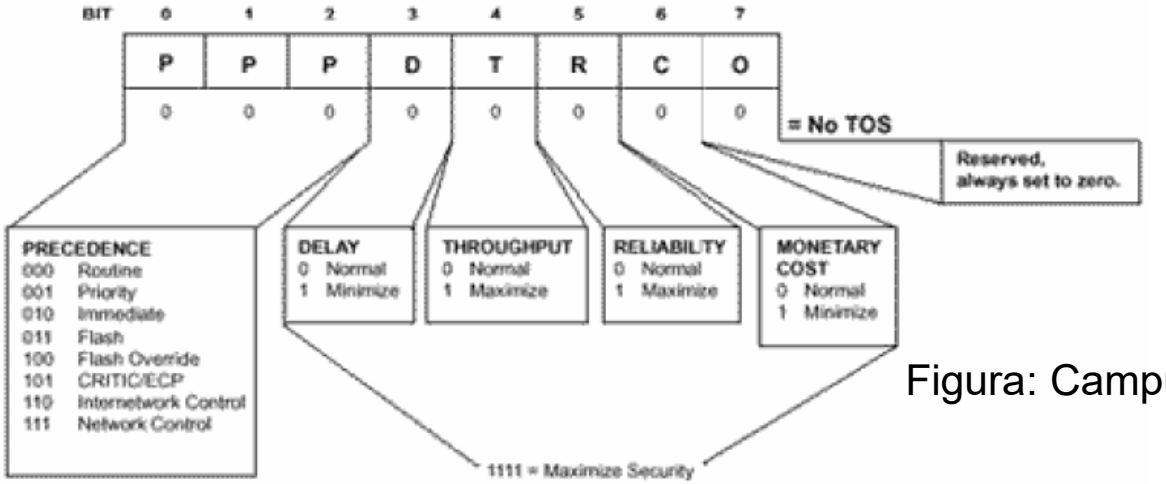
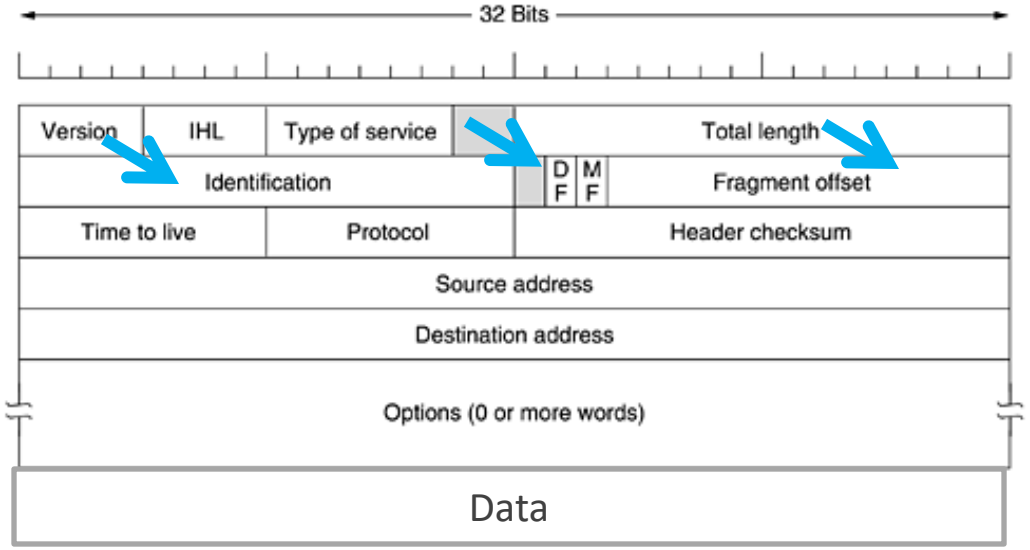


Figura: Campul *Type of Service*

Protocolul IP

- **Datagrama IPv4**
  - Campul *Identification* permite gazdei destinatie sa identifice apartenenta la o datagrama a noului fragment primit
  - Flagurile:
    - *DF (Don't Fragment)* – indica ruterelor sa nu fragmenteze datagrama
    - *MF (More Fragments)* – semnalizeaza ca pachetul este un fragment, urmat de altele; ultimul fragment are acest bit 0
  - Campul *Fragment offset* – locul fragmentului in datagrama

Figura: Datagrama IPv4



Protocolul IP

- **Datagrama IP**
  - Fragmentarea datagramelor:
    - Fiecare fragment (pachet) are aceeasi structura ca datagrama IP
    - Reasamblarea datagramelor se face la destinatar
    - Daca un fragment al unei datagrame e pierdut, acea datagrama e distrusa (se trimite la expeditor un mesaj ICMP – Internet Control Message Protocol)
    - Mecanismul de fragmentare a fost folosit pentru unele atacuri – *firewall piercing* (un fragment “special” e considerat ca fiind parte a unei conexiuni deja stabilite, astfel incat ii va fi permis accesul via *firewall)*

Protocolul IP

- **Datagrama IP**
  - Filtrarea datagramelor:
    - Se realizeaza de un *firewall:* ofera accesul din exterior in reteaua interna, conform unor politici de acces, doar pentru anumite tipuri de pachete (utilizate de anumite protocoale/servicii)
      - Preintimpina o serie de atacuri vizind securitatea
      - Firewall-ul poate fi software sau hardware
      - Firewall-ul poate juca rol de proxy sau de gateway

Protocolul IP

- **Rolul si arhitectura unui proxy:**
  - Acces indirect la alte retele (Internet) pentru gazdele dintr-o retea locala via *proxy*
  - *Proxy*-ul poate fi software sau hardware
  - Rol de poarta (*gateway*), de *firewall* sau de server de *cache*
  - *Proxy*-ul ofera partajarea unei conexiuni Internet
  - Utilizat la imbunatatirea performantei (e.g., *caching*, controlul fluxului), filtrarea cererilor, asigurarea anonimitatii

Protocolul IP

- **Datagrama IPv4**
  - Campul *TTL (Time to Live)* specifica durata de viata a pachetului;

    numarul este decrementat de fiecare *router* prin care trece pachetul

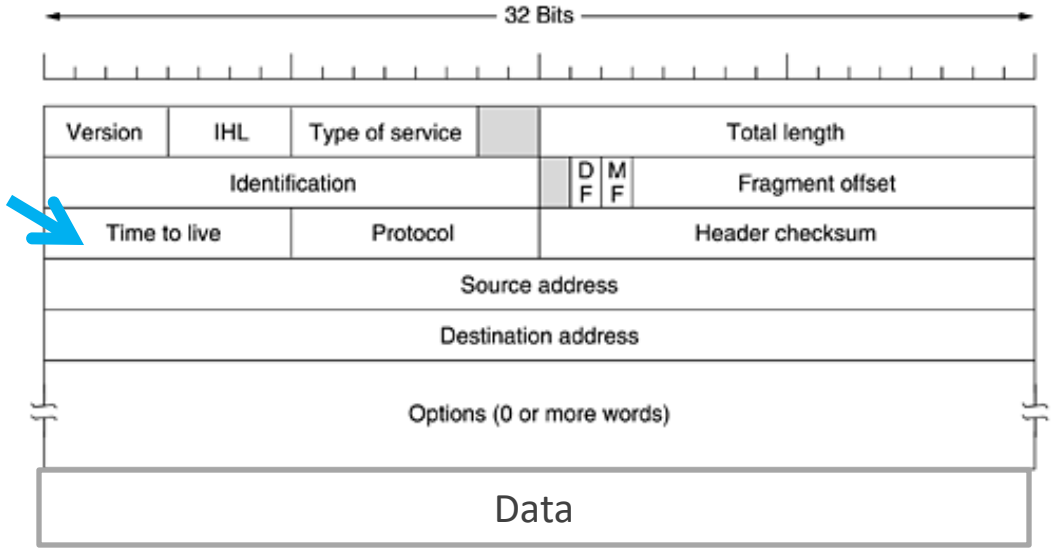
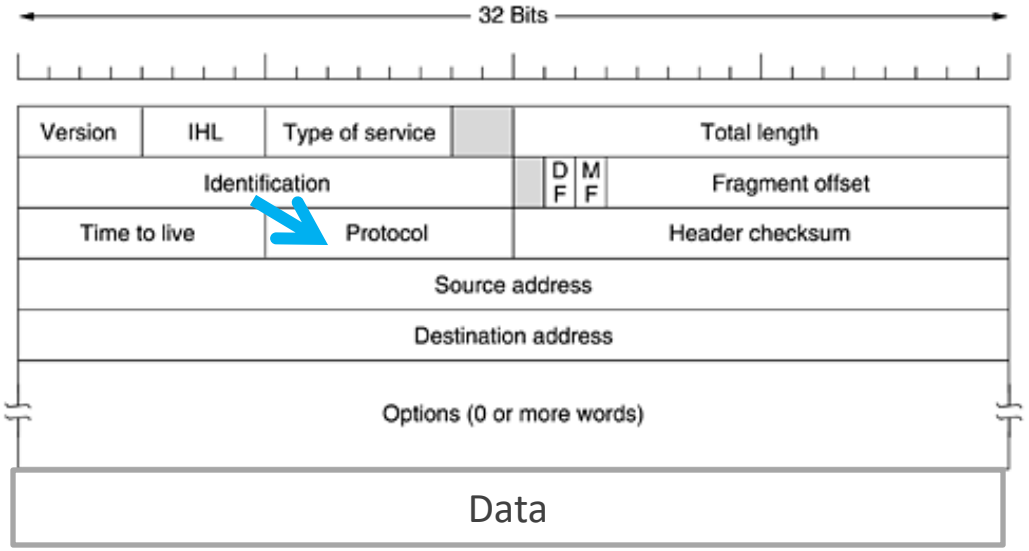


Figura: Datagrama IPv4

Protocolul IP

- **Datagrama IPv4**
  - Campul *Protocol* specifica protocolul (de nivel superior) caruia ii este destinata informatia inclusa in datagrama:

Figura: Datagrama IPv4

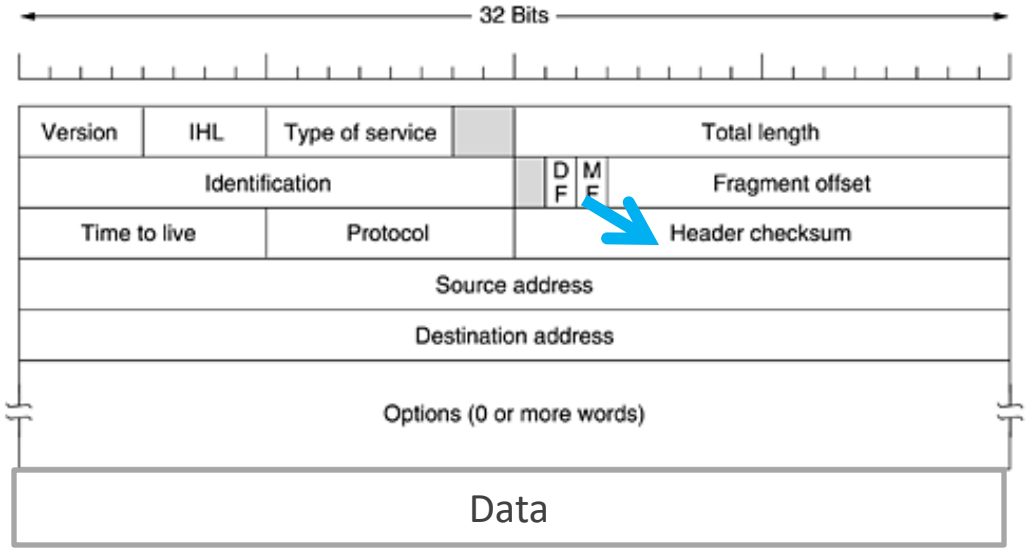


  - 1 ICMP (*Internet Control Message Protocol*)
  - 2 IGMP (*Internet Group Message Protocol*)
  - 6 TCP (*Transmission Control Protocol*)
  - 17 UDP (*User Datagram Protocol*)
  - … etc.(RFC 1700)

Protocolul IP

- **Datagrama IPv4**
  - Campul *Header checksum* folosit pentru detectarea erorilor; daca apare o eroare datagrama este distrusa

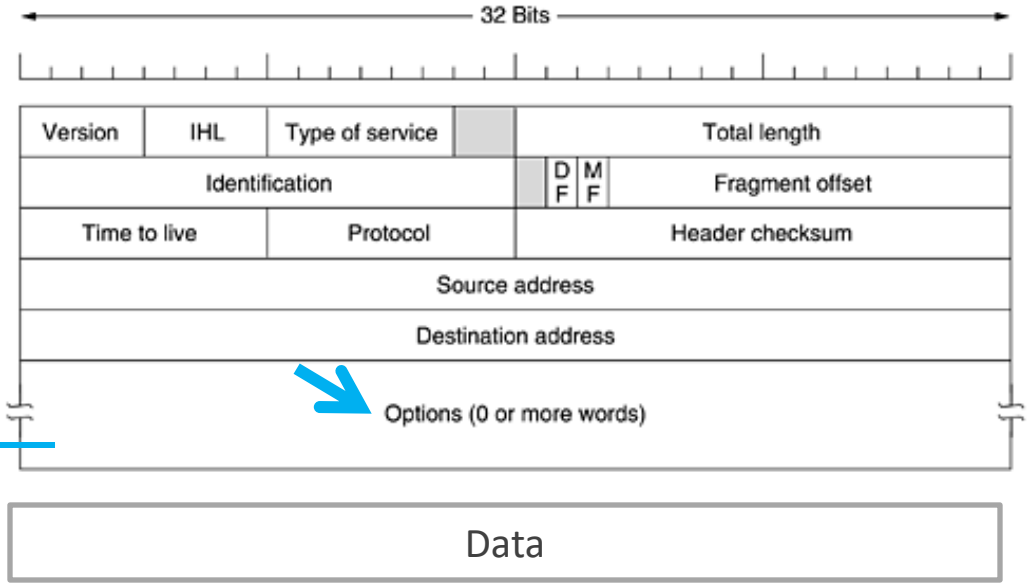
Figura: Datagrama IPv4



Protocolul IP

- **Datagrama IPv4**
  - Campul *Options*

Figura: Datagrama IPv4



Optiune	Descriere
Securitate	Mentioneaza cat de secreta este datagrama
Dirijare stricta pe baza sursei (engl. strict source routing)	Indica calea completa de parcurs
Dirijarea aproximativa pe baza sursei (engl. Loose source routing)	Indica o lista a ruterelor care nu trebuie sarite
Intregistreaza calea (engl. record route)	Face fiecare ruter sa-si adauge adresa IP
Amprenta de timp (engl. timestamp)	Face fiecare ruter sa-si adauge adresa si o amprenta de timp

Protocolul IP

- **Datagrama IPv4**
  - Campul *Source adress* si *Destination adress* indica adresa sursei si destinatiei

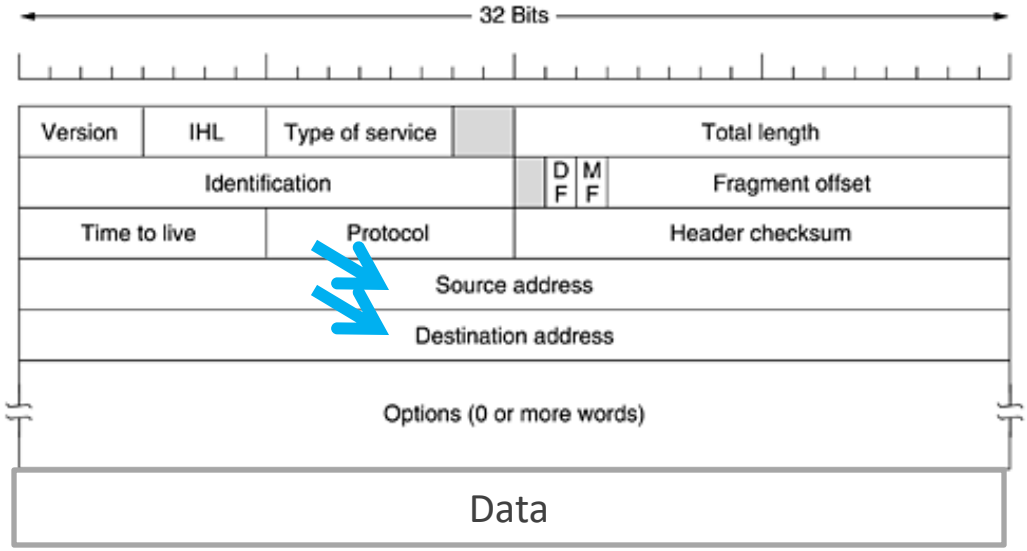


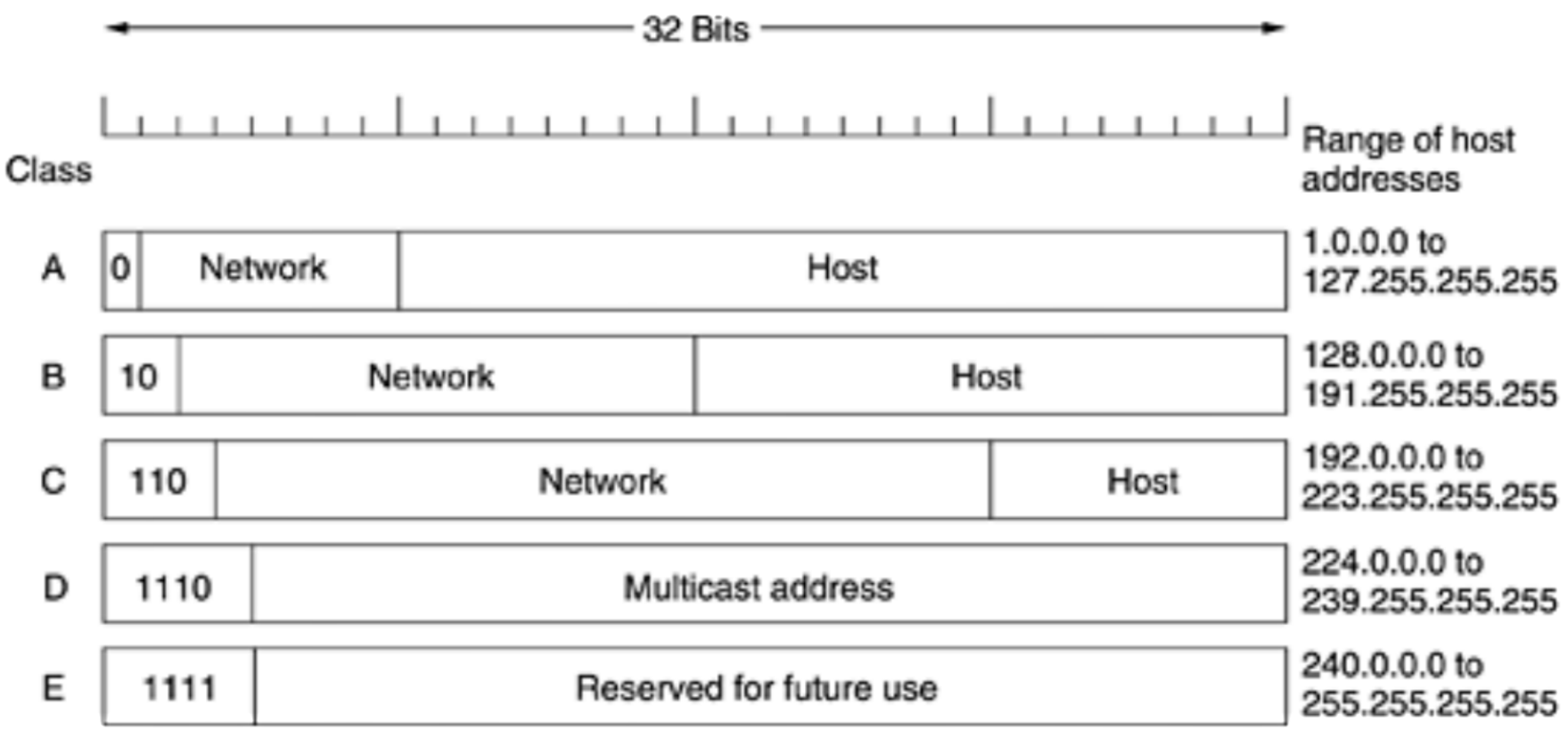
Figura: Datagrama IPv4

Protocolul IP

- **Adrese IPv4**
  - Fiecare adresa IP include un identificator de retea (NetID) si un identificator de gazda (HostID)
  - Fiecare interfata de retea are o adresa IPv4 unica
  - O adresa IPv4 are lungimea de 32 biti
  - Initial (RFC 791) exista impartirea in clase de adrese: A,B,C,D,E

Protocolul IP

- **Adrese IPv4**



[Computer Networks, 2003 Andrew S. Tanenbaum]

Protocolul IP

- **Adrese IPv4**
  - Clasa A: 128 retele posibile, 2^{24} gazde/retea
  - Clasa B: 2^{14} retele posibile, 2^{16} gazde/retea
  - Clasa C: peste 2 milioane de retele, 255 gazde/retea
  - Identificatorul de retea (NetID) este asignat de o autoritate centrala (NIC – *Network Information Center*)
  - Identificatorul de gazda (HostID) este asignat local de administratorul retelei
  - Exemplu: 85.122.23.145 – Clasa A (conventie de notatie in zecimal)

    0101 0101 0111 1010 0001 0111 1001 0001
  - Pentru IPv6 se recomanda reprezentarea hexadecimal

Protocolul IP

- **Adrese IPv4**
  - O interfata (placa) de retea are asignata o unica adresa IP
  - O gazda poate avea mai multe placi de retea, deci mai multe adrese IP
  - Gazdele unei aceleiasi retele vor avea acelasi identificator de retea (acelasi NetID)
  - Adresele de *broadcast* au HostID cu toti bitii 1
  - Adresa IP care are HostID cu toti bitii 0 se numeste adresa retelei – refera intreaga retea
    - Exemplu: adresa 85.122.23.0 (adresa *network* a masinilor 85.122.23.145 si 85.122.23.1
  - 127.0.0.1 – adresa de *loopback* (*localhost*)

Protocolul IP

- **Adrese IPv4**
  - Din spatiul de adrese ce pot fi alocate efectiv sunt rezervate urmatoarele (RFC 1918):
    - 0.0.0.0 – 0.255.255.255
    - 10.0.0.0 – 10.255.255.255 (adrese private)
    - 127.0.0.0 – 127.255.255.255 (pentru *loopback*)
    - 172.16.0.0 - 172.31.255.255 (adrese private)
    - 192.168.0.0 - 192.168.255.255 (adrese private)
  - Adrese private: adrese care nu sunt accesibile spre exterior (Internetul “real”), ci doar in intranetul organizatiei

Retele private

- Aspecte:
  - Cresterea exponentiala a numarului de gazde
  - Nu toate masinile gazda ofera resurse accesibile de pe Internet
- Solutia: NAT (*Network Address Translation*) – RFC 3022, 4008
  - Se reutilizeaza adresele private (RFC 1918)
  - Se bazeaza pe inlocuirea adresei private cu adresa IP neprivata (*IP masquerading*)

Retele private

- Functionalitate:

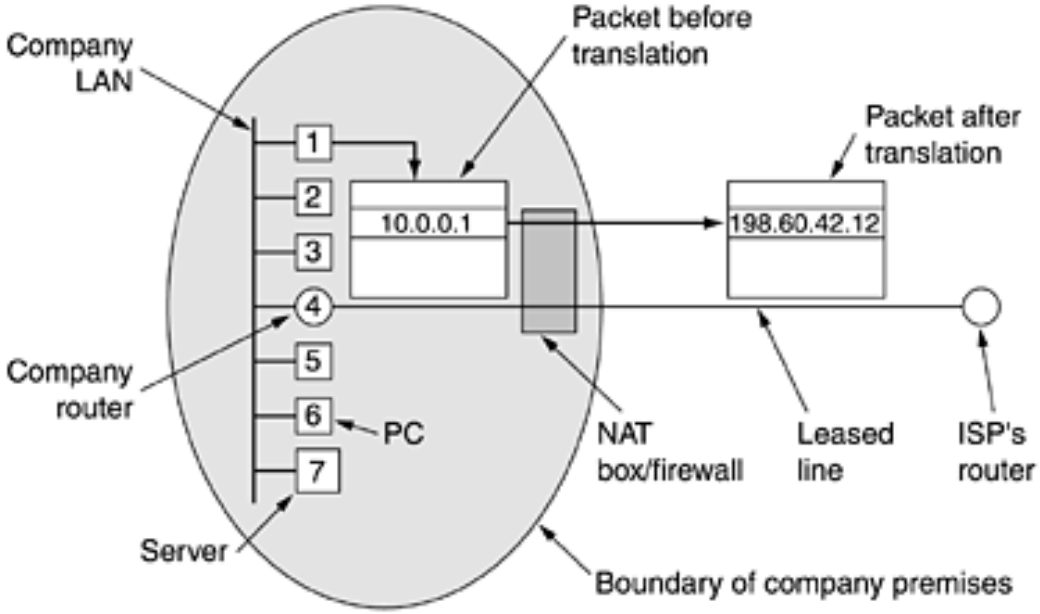


Figura: Functionare NAT

- Ruterele in mod normal ignora datagramele continind adrese private => pot fi folosite adrese IP private in cadrul intranet-ului organizatiei
- Accesul spre exterior (Internetul “real”) se realizeaza via o poarta (mediating gateway) ce rescrie adresele IP sursa/destinatie

[Computer Networks, 2003 Andrew S. Tanenbaum]

Protocolul IP

- **Subretele folosind masti de retea**
  - A aparut ca solutie pentru problema epuizarii spatiului de adrese IP
  - Simplifica rutarea
  - Subretelele nu pot fi detectate ca subretele din exterior

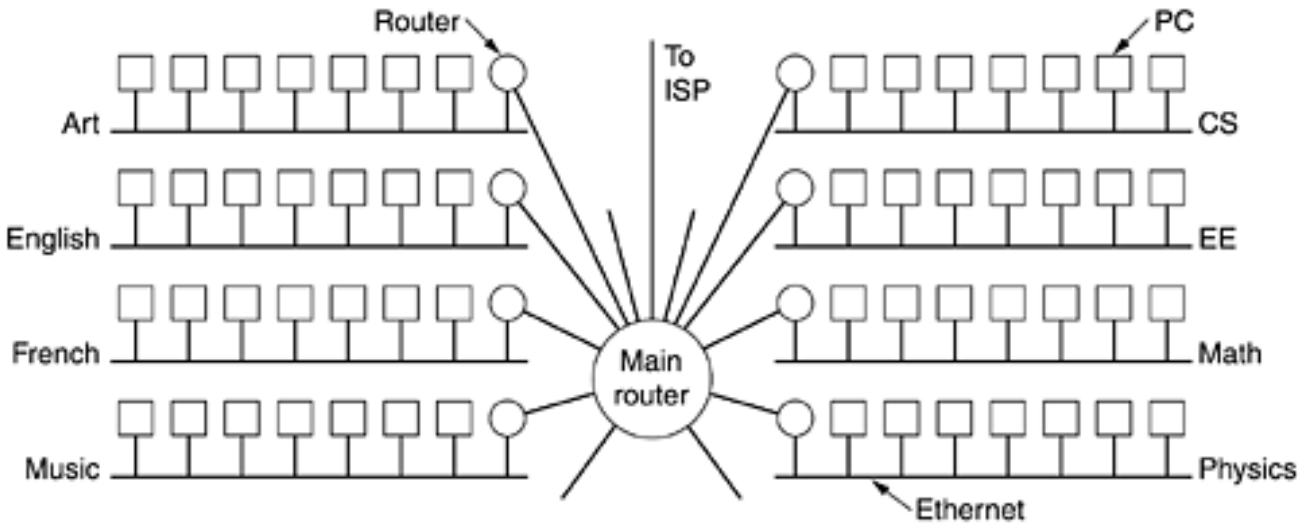


Figura: Reteaua unui campus

Protocolul IP

- **Subretele folosind masti de retea**
  - Divizarea in subretele se va face via masca de retea (*netmask*): bitii NetID sunt 1, bitii HostID sunt 0
  - Identificatorul subretelei (SubnetID) este utilizat in general sa grupeze calculatoarele pe baza topologiei fizice

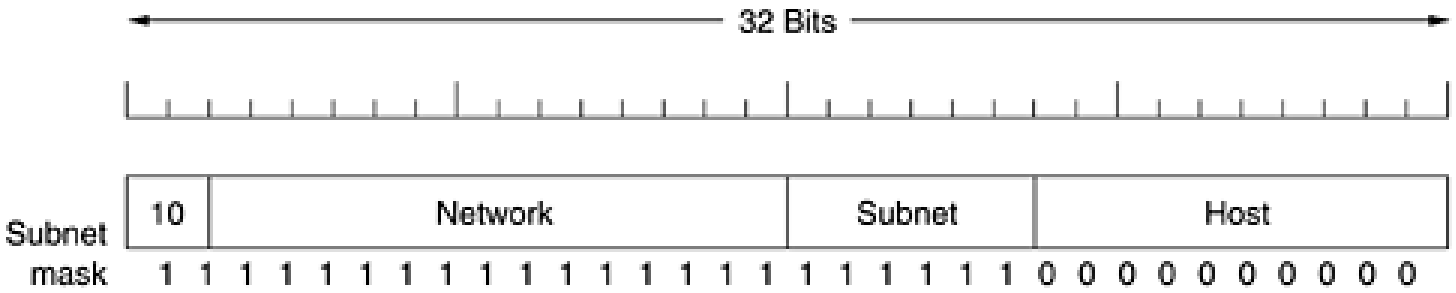


Figura. O cale de a crea o subretea dintr-o retea de clasa B

Protocolul IP

- **Subretele folosind masti de retea**
  - Exemplu:
    - Fie adresa IP: 160.0.6.7

      10100000 00000000 00000110 00000111
    - Masca de retea: 255.255.252.0

      11111111 11111111 11111100 00000000

    - Adresa de retea: 160.0.4.0

      10100000 00000000 00000100 00000000

  Adresa retelei = masca de retea AND adresa IP

- Masti de subretea implicite:
  - 255.0.0.0 Clasa A
  - 255.255.0.0 Clasa B
  - 255.255.255.0 Clasa C

Protocolul IP

- **Conventii de notare**: x.x.x.x/m inseamna ca se aplica o masca de m biti adresei IP precizata de x.x.x.x
- Exemplu:
  - 10.0.0.0/12 – se aplica o masca de 12 biti adresei 10.0.0.0, selectindu-se valorile posibile in ultimii 20 de biti (=32-12) de adresa
  - 85.122.16.0/20 – se aplica o masca de 20 biti adresei 85.122.16.0

Nivelul Retea

- Protocoale
  - ICMP (RFC 792)
  - ARP (RFC 826)
  - RARP (RFC 903)
  - BOOTP (RFC 951,1048,1084)
  - DHCP
- De la IPv4 la IPv6

Protocolul ICMP

- **ICMP – Internet Control Message Protocol**
  - Utilizat pentru schimbul de mesaje de control
  - Foloseste IP
  - Mesajele ICMP sunt procesate de software-ul IP, nu de procesele utilizatorului
  - Tipuri de mesaje :

Tipul mesajului	Descriere
8 Echo Request	Intreaba o masina daca este activa
0 Echo Replay	“Da, sunt activa”
3 Destination Unreachable	Pachetul nu poate fi livrat (e.g. DF setat)
5 Redirect	Schimbarea rutei
11 Time Exceeded	A expirat timpul
… etc (RFC 792)	http://www.iana.org/assignments/icmp-parameters

Protocolul ICMP

- Utilizat de:
  - comanda ping (Packet Internet Gropher)
  - comanda traceroute
    - Se trimite un pachet cu TTL=1 (1 hop)
    - Primul router ignora pachetul si trimite inapoi un mesaj ICMP de tip "*time-to-live exceeded*"
    - Se trimite un pachet cu TTL=2 (2 hop-uri)
    - Al doilea router ignora pachetul si trimite inapoi un mesaj "*time-to-live exceeded*"
    - Se repeta pina cind se primeste raspuns de la destinatie sau s-a ajuns la numarul maxim de hop-uri

Rezolutia adreselor

- **Adrese IP <-> adrese hardware (fizice)**
  - Procesul de a gasi adresa hardware a unei gazde stiind adresa IP se numeste rezolutia adresei (*address resolution*) – protocolul ARP (RFC 826)
    - ARP – protocol de tip broadcast (fiecare masina primeste cererea de trimitere a adresei fizice, raspunde doar cea in cauza)
  - Procesul de a gasi adresa IP pe baza adresei hardware se numeste rezolutia inversa a adresei (*reverse address resolution*) – protocolul RARP (RFC 903)
    - Utilizat la boot-are de statiile de lucru fara disc
    - BOOTP (RFC 951,1048,1084)
    - DHCP (*Dynamic Host Configuration Protocol*) RFC 2131,2132

IPv6

- Context:
  - Probleme de adresabilitate via IPv4 clasic:
    - Cresterea exponentiala a numarului de gazde
    - Aparitia unor tabele de rutare de mari dimensiuni
    - Configuratii tot mai complexe, utilizatori tot mai multi
    - Imposibilitatea asigurarii calitatii serviciilor (QoS)
  - Presiuni din partea operatorilor de telefonie mobila

IPv6

- Obiective pentru un nou protocol:
  - Suport pentru miliarde de gazde
  - Reducerea tabelelor de rutare
  - Simplificarea protocolului
  - Suport pentru gazde mobile
  - Compatibilitatea cu vechiul IP
  - Suport pentru evolutiile viitoare ale Internet-ului
- RFC 2460, 2553

IPv6

- 6 Iunie 2012

IPv6

- Aspecte:
  - Adresele IPv6 au lungime de 16 octeti - 2^{128} adrese
  - Notatie: 16 numere hexa, fiecare de 2 cifre, delimitate de ":"
    - Exemplu: 2001:0db8:0000:0000:0000:0000:1428:57ab
    - Daca unul sau mai multe din grupurile de 4 cifre este 0000, zerourile pot fi omise si inlocuite (o singura data) cu "::"
      - Exemplu: 2001:0db8::1428:57ab
  - Pentru pastrarea compatibilitatii: adresele IP publice sunt considerate un subset al spatiului de adrese IPv6
  - Adresele IPv4 in IPv6 pot fi scrise astfel:

10.0.0.1 -> ::10.0.0.1 sau 0:0:0:0:0:0:A00:1

IPv6

- ICMPv6
  - Ofera functiile ICMP (raportarea transmiterii datelor, erorilor, etc.) plus:
    - Descoperirea vecinilor (*Neighbor Discovery Protocol – NDP*) - Inlocuieste ARP
    - Descoperirea ascultatorilor multicast (*Multicast Listener Discovery*) – inlocuieste IGMP (*Internet Group Management Protocol*)
  - Detalii in RFC 4443

IPv6

- **…continuare -> Curs viitor**

Rezumat

- Nivelul Retea
  - Protocolul IPv4
    - Problematica
    - Caracterizare
    - Subretele
  - Retele Private
  - ICMP
  - Rezolutia adreselor
  - IPv6 – imagine generala

Intrebari?

Nivelul Transport

Lenuta Alboaie

adria@info.uaic.ro

Cuprins

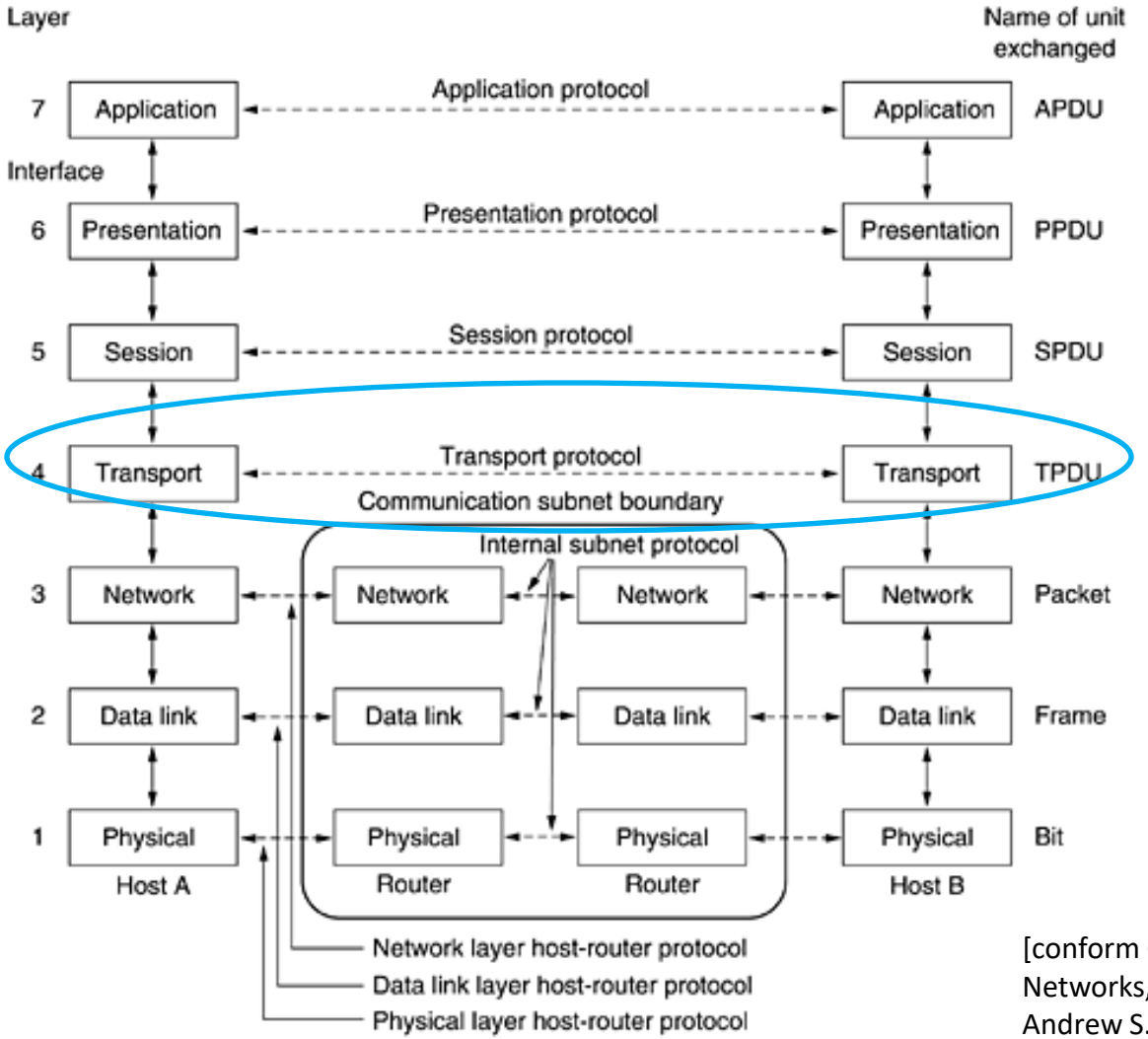
Nivelul Transport

- Preliminarii
- UDP (User Datagram Protocol)
- TCP (Transmission Control Protocol)
- TCP versus UDP

Nivelul transport|Preliminarii

Utilizatoare de servicii de Transport

Furnizoare de servicii de transport

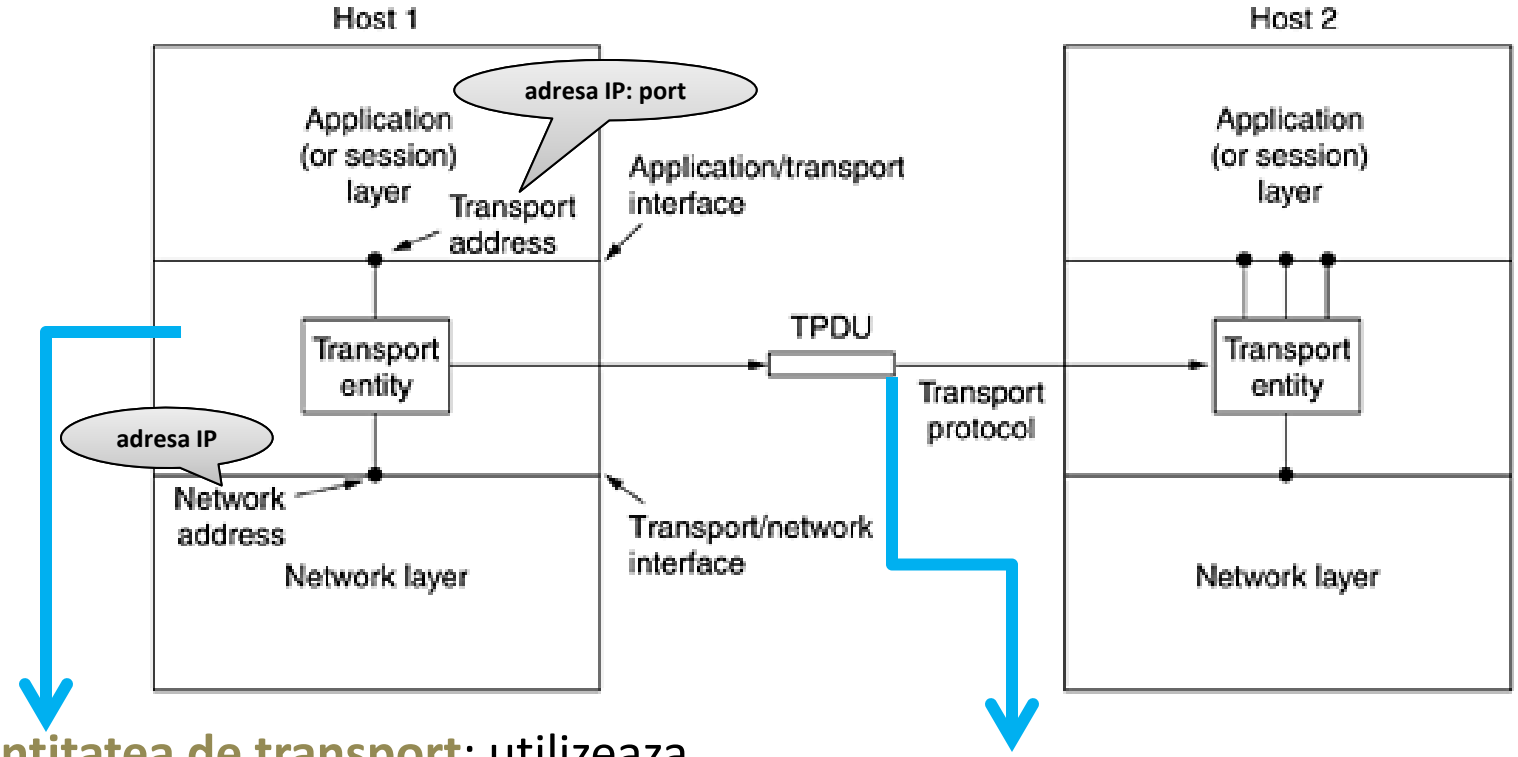


[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Figura. Modelul OSI

Nivelul Transport

- Relatia (logica) intre nivele: retea-**transport**-aplicatie



Entitatea de transport: utilizeaza serviciile nivelului retea si ofera servicii nivelului superior

TPDU (**Transport Protocol Data Unit**) – unitatea de date pentru transport

[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Nivelul Transport

- Ofera servicii mult mai fiabile decat cele de la nivelul retea (e.g. pachetele pierdute/incorecte de la nivelul retea pot fi detectate/corectate la nivelul transport)
- **Calitatea serviciilor** – QoS (*Quality of Service*)
  - Intarzierea la stabilirea conexiunii
  - Productivitatea sau rata de transfer
  - Intirzierea la transfer
  - Rata reziduala a erorilor
  - Protectia
  - Prioritatea
  - Rezilienta

Nivelul Transport

- Asigura comunicarea logica dintre **procese** rulind pe *host*-uri diferite (comunicare *end-to-end*)
  - (Curs anterior!) Nivelul retea asigura comunicarea logica intre ***host*-uri**
- **PORT**
  - Adauga o noua dimensiune adreselor IP de la nivelul retea
  - Se asociaza unei aplicatii (serviciu) si nu unei gazde
  - Un proces poate oferi mai multe servicii => poate utiliza mai multe porturi
  - Un serviciu poate corespunde mai multor procese

Nivelul Transport

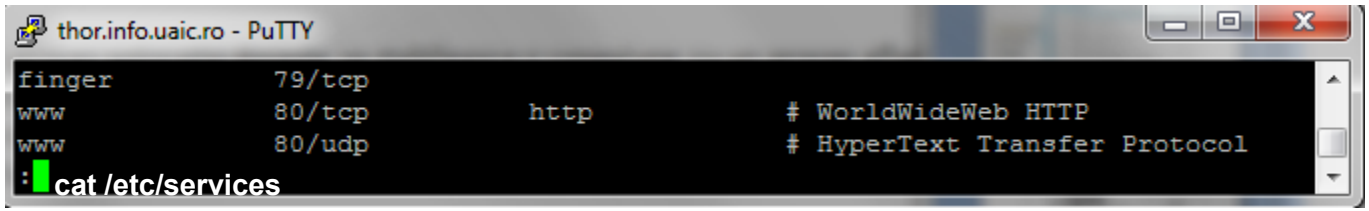
- **PORT**
  - Este un numar pe 16 biti si poate avea valori intre 0 – 65535
  - 1-1024 – valori rezervate (*well-known*), 1-512 servicii de sistem

    (IANA – Internet Assigned Number Authority: RFC 1700)

    Exemple:

    /etc/services : sunt precizate porturile asociate serviciilor sistem

    HTTP – 80; SMTP – 25; telnet – 23; SSH - 22



Nivelul Transport

Primitive generale

– Permit utilizatorilor nivelului transport (e.g. programe de aplicatie) sa acceseze serviciile

Primitiva	Unitatea de date trimisa (TPDU)	Explicatie
LISTEN	(nimic)	Se blocheaza pana cand un proces incearca sa se conecteze
CONNECT	CONNECTION REQUEST	Incearca sa stabileasca conexiunea
SEND	DATA	Transmite informatie
RECEIVE	(nimic)	Se blocheaza pana cand se primeste date trimise
DISCONNECT	DISCONNECTION REQ.	Trimisa de partea care vrea sa se deconecteze

Nivelul Transport

Cele mai importante protocoale la nivelul transport:

- **TCP** (**Transmission Control Protocol**) – protocol de transport orientat conexiune; RFC 793 (1122), 1323
- **UDP** (**User Datagram Protocol**) – protocol de transport neorientat conexiune; RFC 768

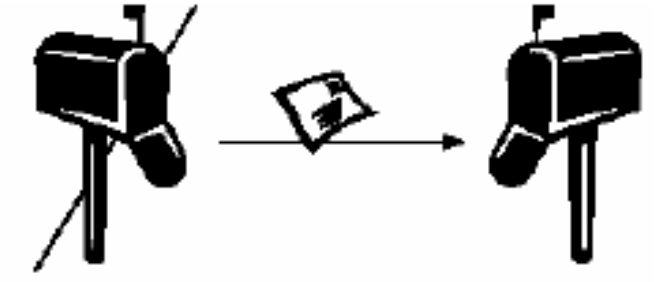
UDP

- Protocol de transport neorientat conexiune, nesigur, minimal
- Nu ofera nici o calitate suplimentara serviciilor
- Nu recurge la negocieri sau la confirmari ale primirii datelor

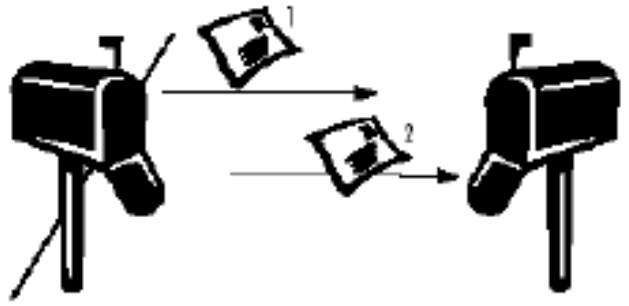
UDP

- **Analogie**: UDP – similar postei terestre

  - Trimiterea unei scrisori



  - Nu se garanteaza ordinea receptionarii



  - Mesajul se poate pierde

[conform Retele de calculatoare – curs 2007-2008, Sabin Buraga]

UDP

- Utilizeaza IP
- Pentru a oferii servicii de comunicare intre procese foloseste **porturi**
- UDP transmite pachete: antet (8 bytes) + continut

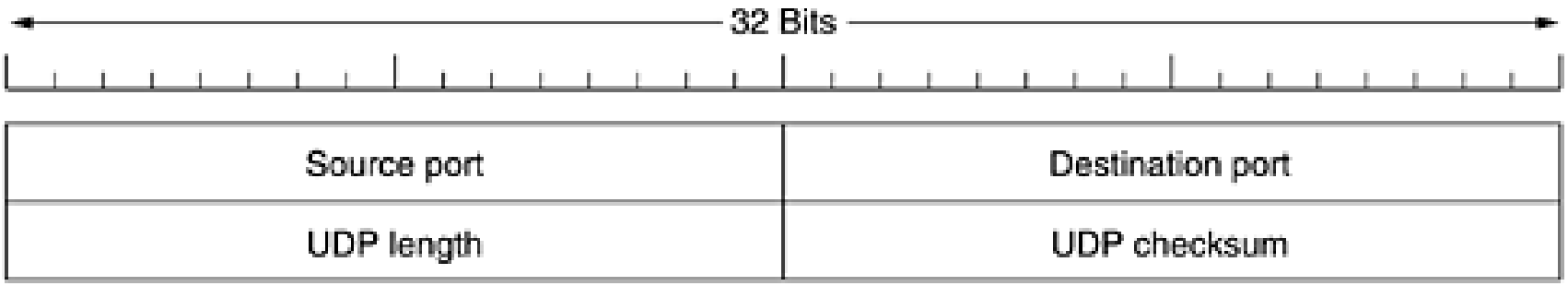


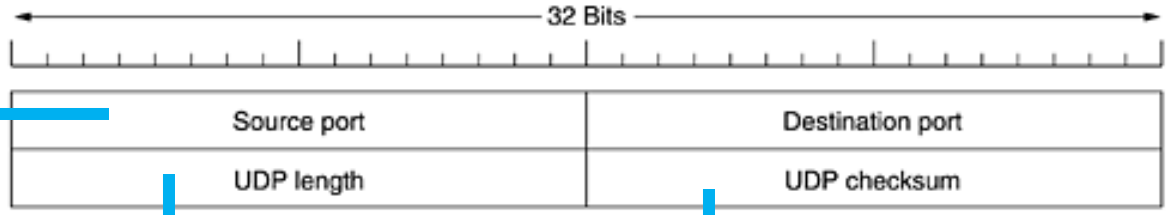
Figura: Antet UDP

[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

UDP

- ***Source port*** si ***Destination port*** identifica “end-point”-urile de pe masinile sursa si destinatie

Figura: Antet UDP



- ***UDP length*** = 8 bytes + dimensiunea continutului

- ***UDP checksum*** nu este obligatoriu

UDP

- Exemple de utilizari:
  - **DNS** (Domain Name System)

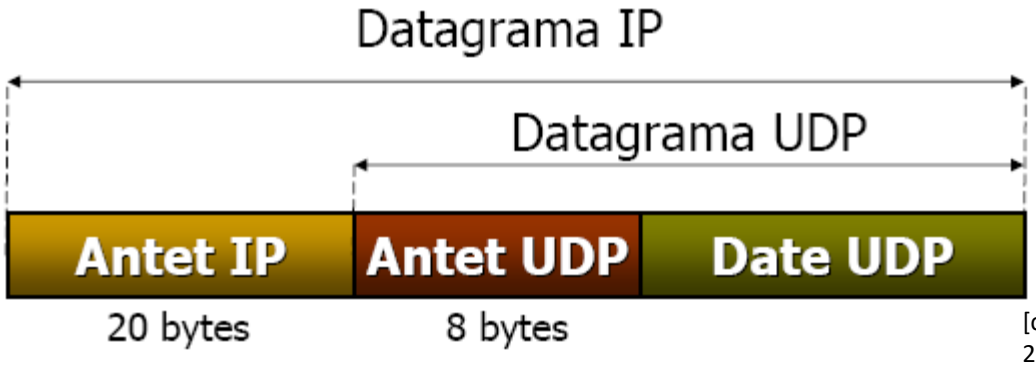
    Use-case: A are nevoie de IP-ul *host*-ului cu numele www.info.uaic.ro

    Pas 1. A trimite un pachet UDP continind numele host-ului : www.info.uaic.ro

    Pas 2. Serverul de DNS trimite un pachet UDP de raspuns continind adresa IP a host-ului : 85.122.23.146
  - **RPC** (Remote Procedure Call) – mecanism de apel al procedurilor la distanta

UDP

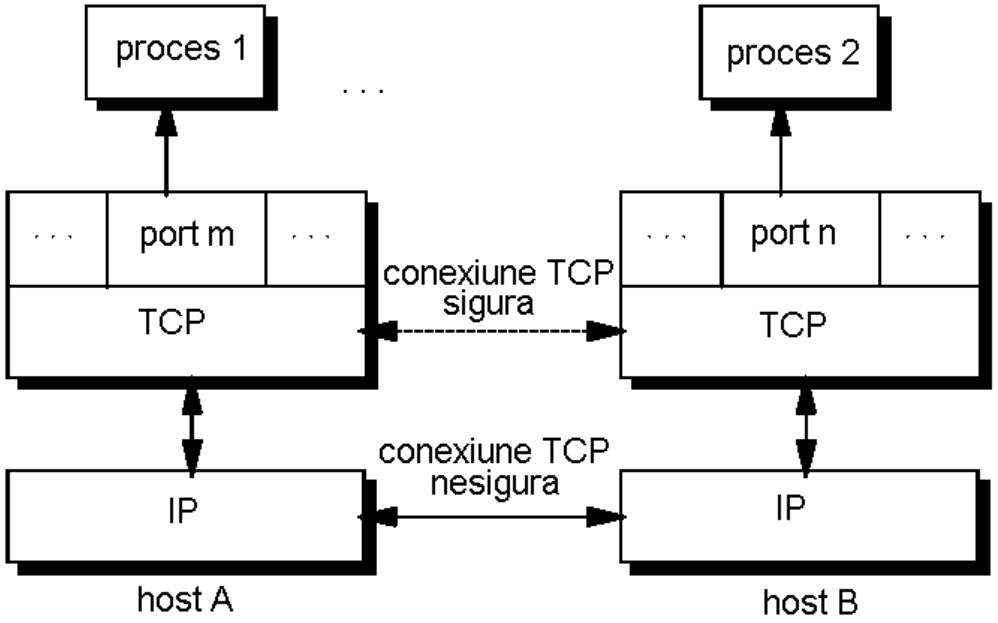
- Ce nu ofera?
  - Controlul fluxului
  - Controlul erorilor
  - Retransmisia la receptionarea unui pachet eronat
- Ce ofera?
  - Folosind port-uri extinde protocolul IP pentru comunicarea intre procese aflate pe host-uri diferite si nu numai intre host-uri



[conform Retele de calculatoare – curs 2007-2008, Sabin Buraga]

TCP - Transmission Control Protocol

- Protocol de transport orientat conexiune, fara pierdere de informatii
- Vizeaza oferirea calitatii maxime a serviciilor
- Integreaza mecanisme de stabilire si de eliberare a conexiunii

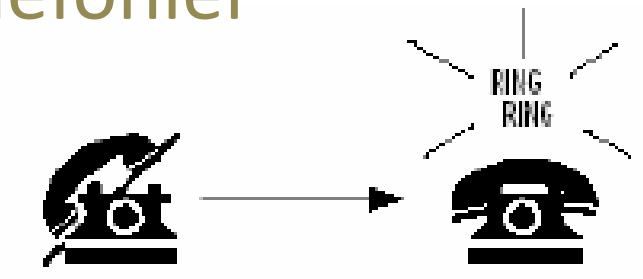


- Controleaza fluxul de date (*stream-oriented*)
- Utilizat de multe protocoale de aplicatii: HTTP, SMTP, …

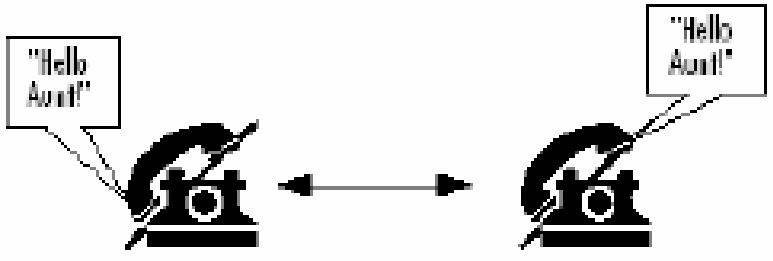
[conform IBM – TCP/IP Tutorial and Technical Overview, 2006]

TCP

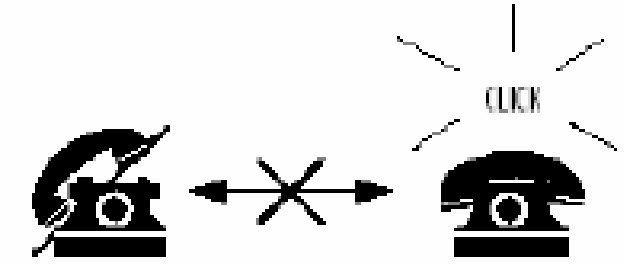
- Analogie: TCP – similar telefoniei



- Initierea unei convorbiri



- Dialogul dintre parti



- Terminarea convorbirii

[conform Retele de calculatoare – curs 2007-2008, Sabin Buraga]

TCP

- Utilizeaza **conexiuni**, nu porturi ca abstractiuni fundamentale
- Ambele parti (expeditorul, destinatarul) trebuie sa participe la realizarea conexiunii
- Conexiunile se identifica prin perechi reprezentate de

  adresa IP:PORT (*soclu - socket*)

  – Exemplu:

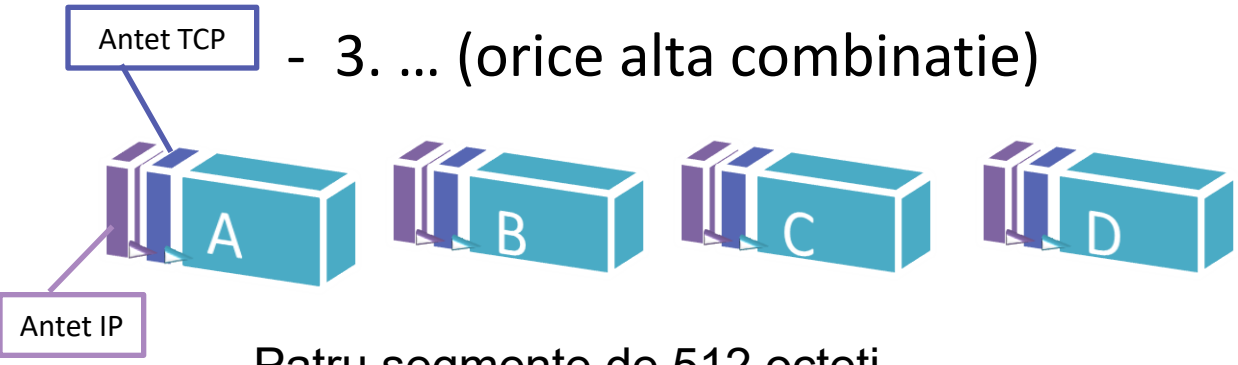
  (85.122.23.146: 12345, 85.122.23.146: 22)
- Una din parti ofera o deschidere pasiva - asteapta aparitia unei cereri de conectare a partenerului de comunicare care realizeaza o deschidere activa
- Un *soclu* poate fi partajat de conexiuni multiple de pe aceeasi masina

TCP

- Conexiunile TCP sunt full-duplex
- O conexiune TCP este un flux de octeti si nu un flux de mesaje

  Exemplu:

  - Emitatorul trimite 4 fragmente de 512 octeti
  - Receptorul poate primi:
    - 1. doua fragmente de 1024 de octeti
    - 2. un fragment de 1 octet si unul de 2047 de octeti
    - 3. … (orice alta combinatie)



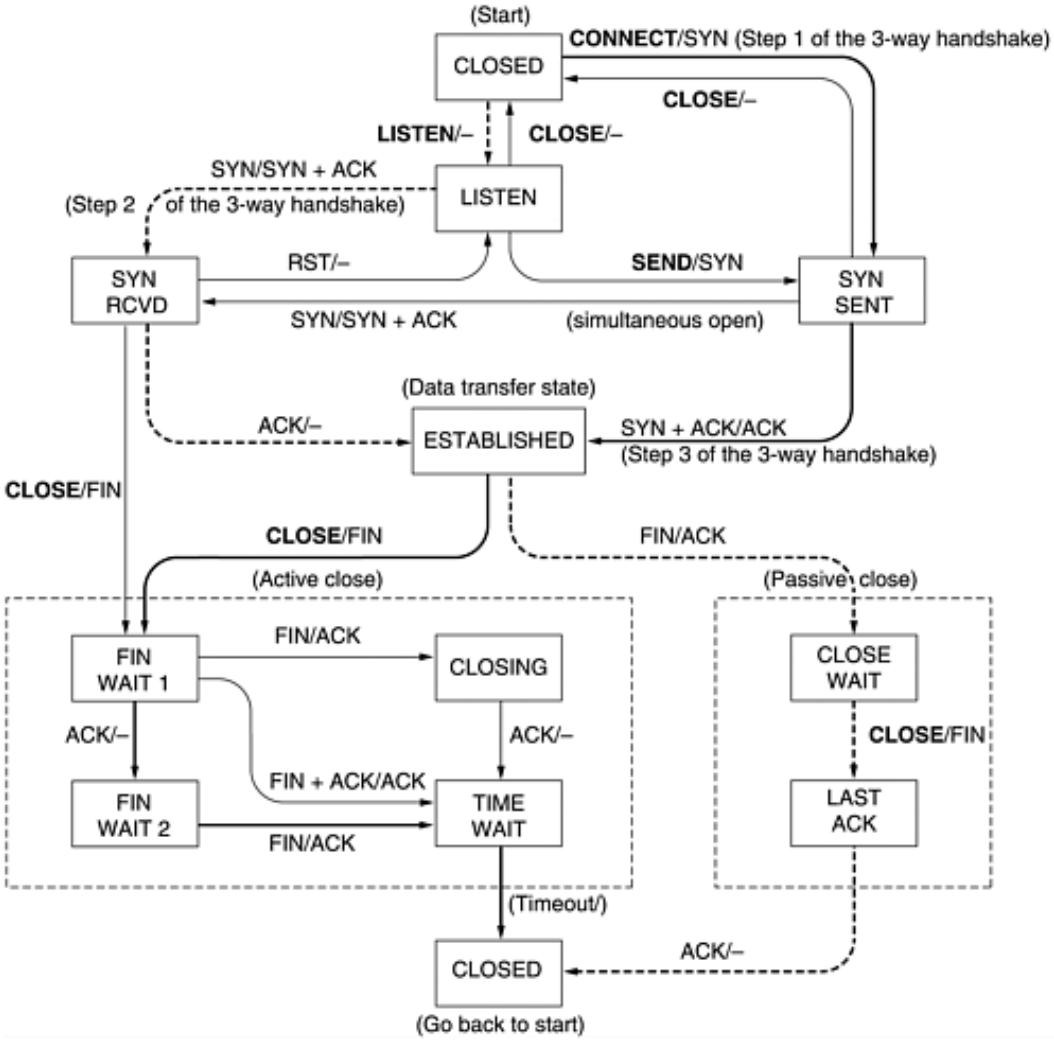
Patru segmente de 512 octeti
trimise ca datagrame IP separate

Livrarea celor 2048
octeti catre aplicatie

TCP

Automatul finit TCP

- Modeleaza comportamentul protocolului
- Starile sunt utilizate la managementul conexiunii



Legenda:

——————— Client

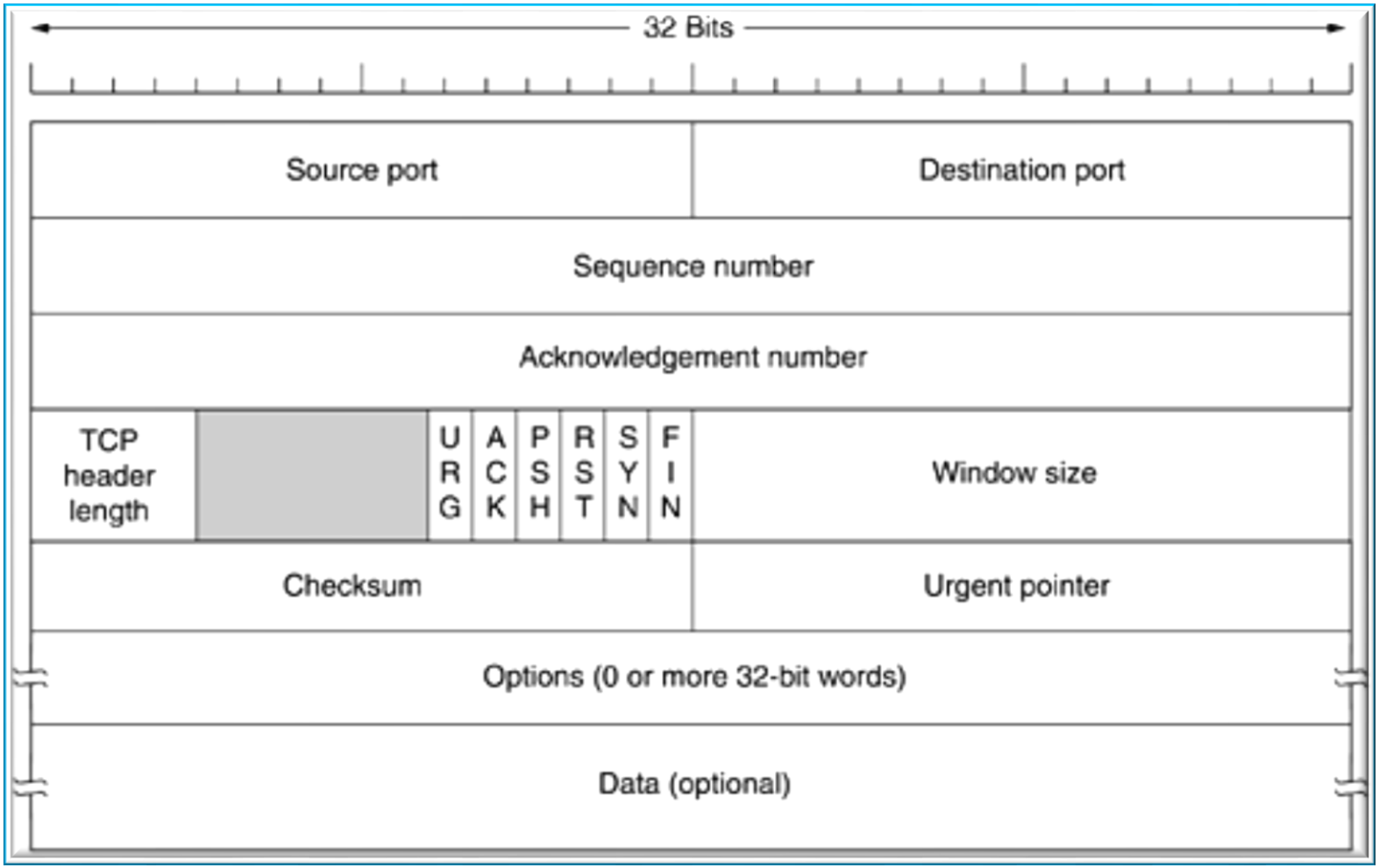
---------- Server

-Fiecare linie este etichetata cu o pereche *eveniment/actiune*
Exemplu: ACK/-

[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

TCP

Antetul TCP

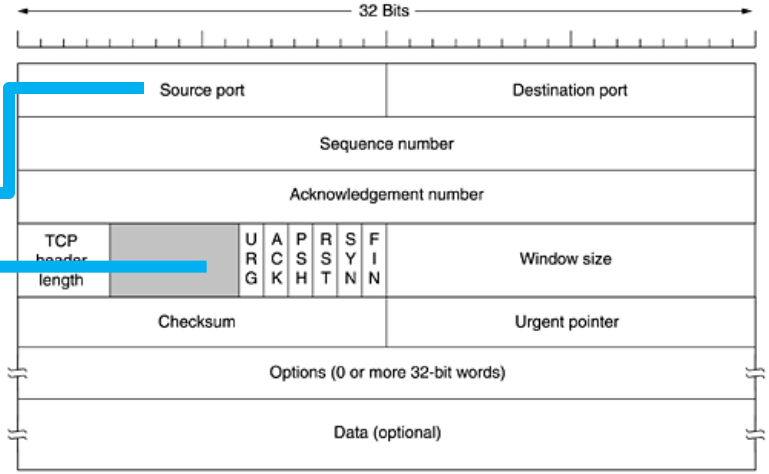


[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

TCP

Antetul TCP

Cimpurile ***Source Port*** si ***Destination Port*** identifica punctele finale ale conexiunii



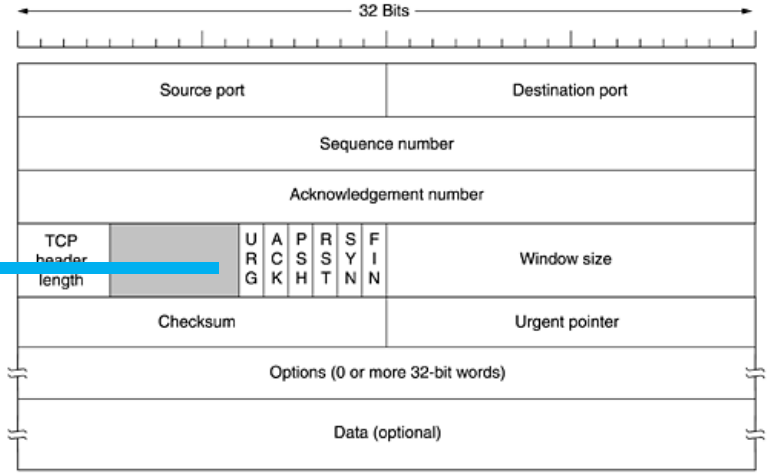
Flaguri de control

- **URG** (URGence) (1 bit) – daca este setat, campul *Urgent Pointer* este folosit
- **ACK** (ACKnowledgment)
  - daca este setat campul *Acknowledge Number* este folosit;
  - este setat in toate pachetele de confirmare

TCP

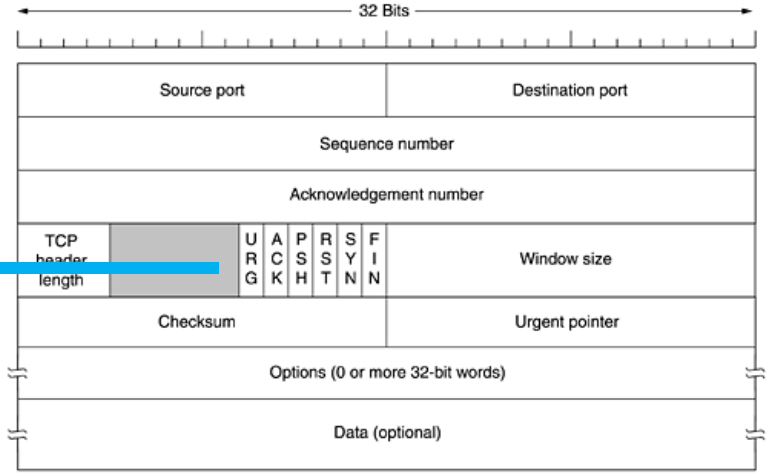
Antetul TCP

- **Flaguri de control**
  - **PSH** (PuSH)– datele vor fi transmise imediat aplicatiei destinatare
  - **RST** (ReSeT) – semnalizeaza erori in conexiune (e.g. deschiderea unei conexiuni este refuzata)



TCP

Antetul TCP



- **Flaguri de control**
  - SYN (SYNchronize)
    - folosit pentru stabilirea conexiunii

(Cererea de conexiune: SYN=1, ACK=0; Raspunsul la cerere: SYN=1, ACK=1;)

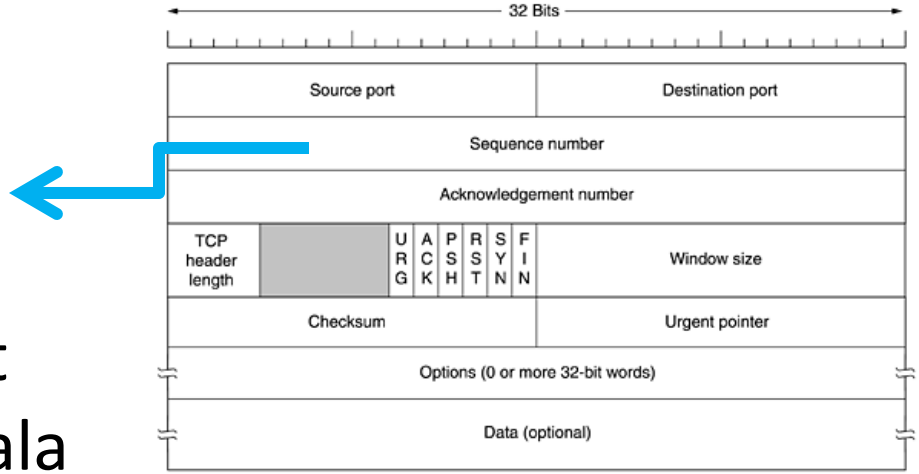
- indica cererea de conexiune si conexiune acceptata

  - **FIN** (FINish)- indica inchiderea conexiunii

TCP

Antetul TCP

Campul ***Sequence Number*** (**SEQ**)



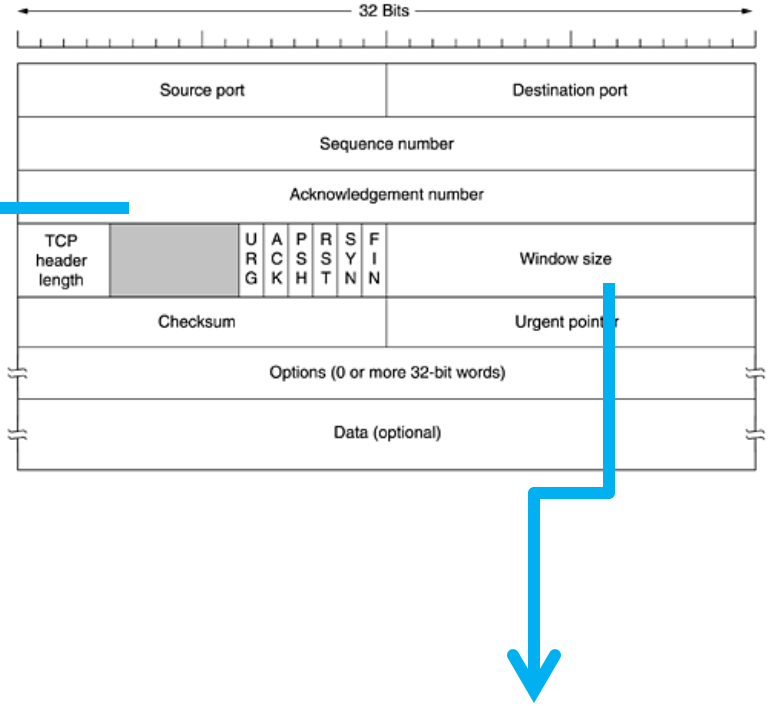
- Daca flagul SYN este setat => SEQ are valoarea initiala a numarului de ordine;
- Daca flagul SYN este nesetat => SEQ are ca valoare numarul de ordine a primului octet trimis in acest pachet

TCP

Antetul TCP

Campul ***Acknowldge Number***

- Daca flagul ACK este setat => are valoarea numarului de ordine al urmatorului octet care se asteapta a fi receptionat;

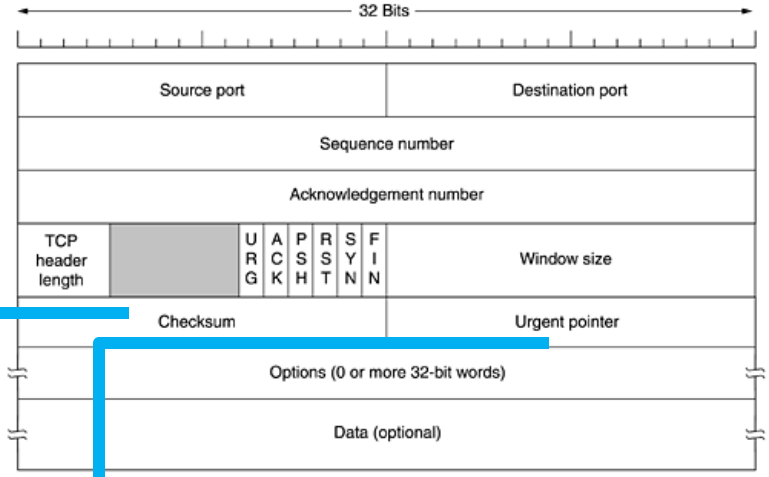


Campul ***Window size*** – numarul de octeti pe care receptorul doreste sa ii receptioneze (controlul fluxului)

TCP

Antetul TCP

Campul **Checksum** – folosit pentru verificarea sumei de control al pachetului TCP (antet si data)

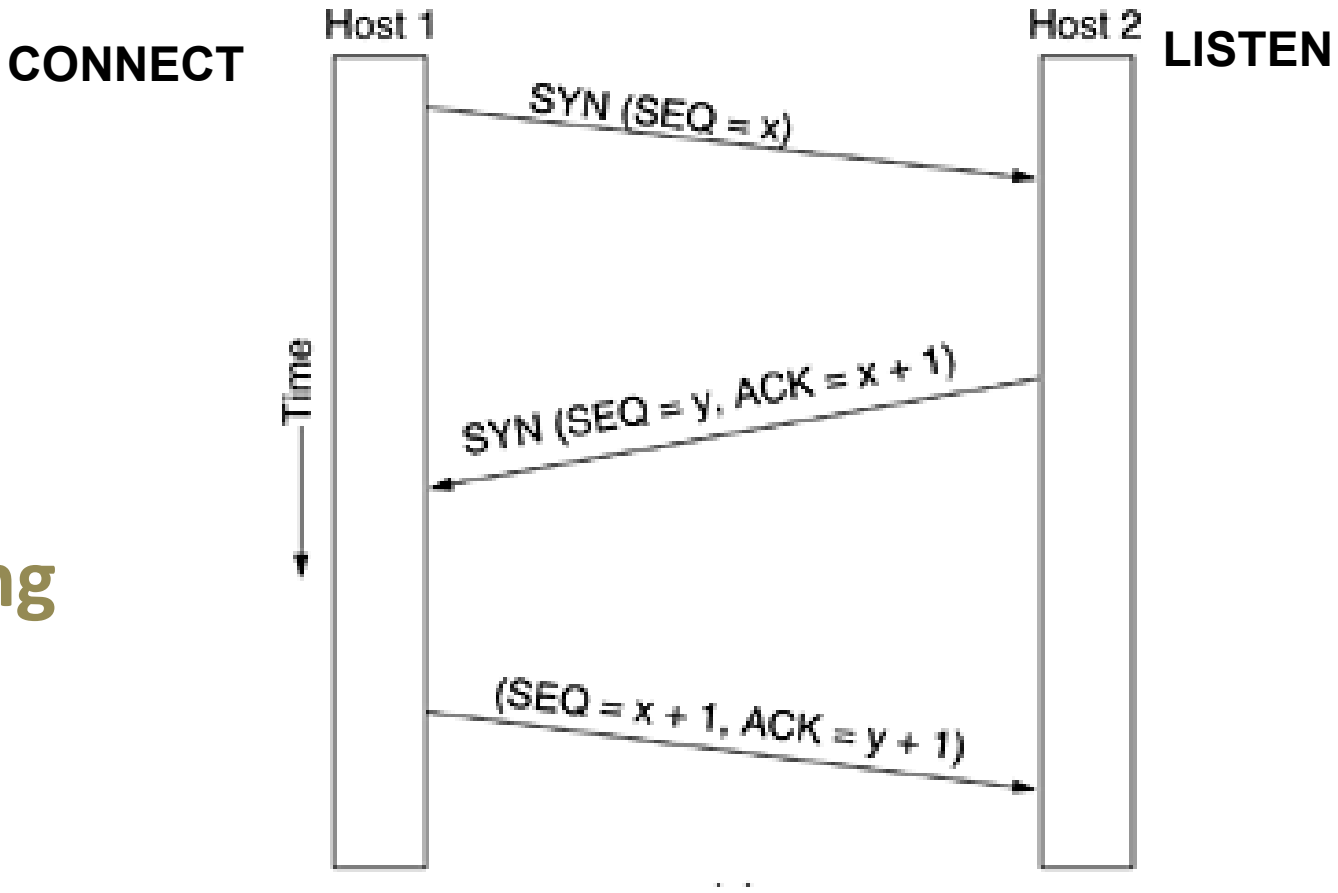


Campul **Urgent Pointer** – daca flagul URG este setat, indica deplasamentul fata de numarul curent de ordine la care se gaseste informatia urgenta

Administrarea conexiunii TCP

Stabilirea conexiunii

Three-way handshaking



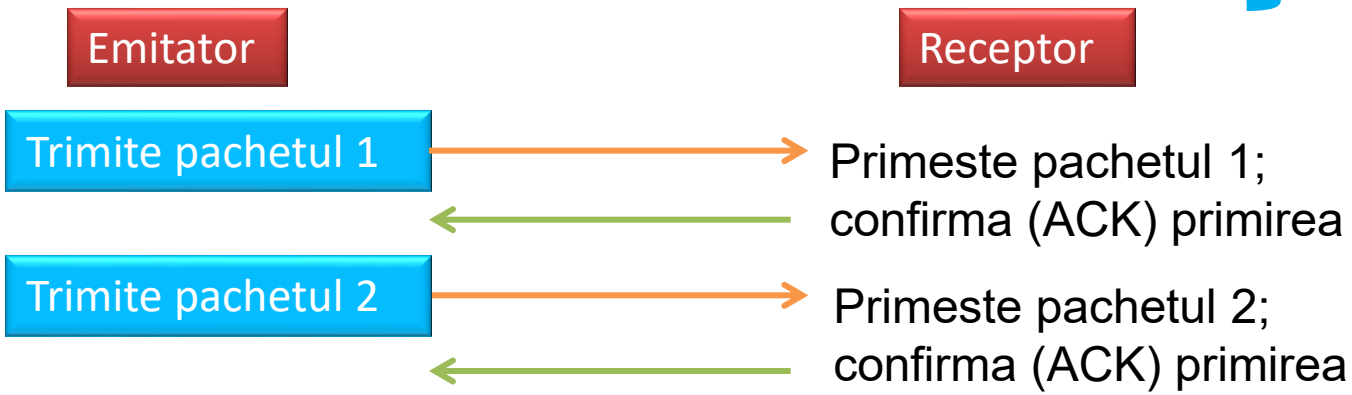
[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Controlul fluxului

Mecanism general:

- Protocol de comunicare simplu: se trimite un pachet si apoi se asteapta confirmarea de primire de la destinatar inainte de a trimite pachetul urmator.
- Daca confirmarea (ACK) nu este primita intr-un interval de timp prestabilit, pachetul este trimis din nou

Fiabilitatea Comunicarii (*reliability*)



Problema: mecanismul asigura siguranta comunicarii dar determina folosirea doar a unei parti din banda disponibila a retelei

Administrarea conexiunii TCP

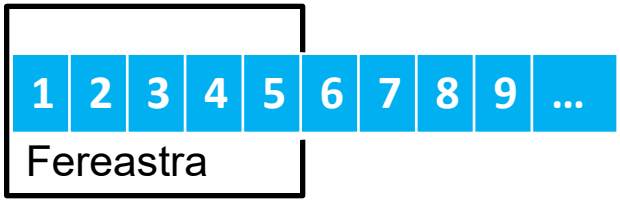
Fereastra glisanta – context:

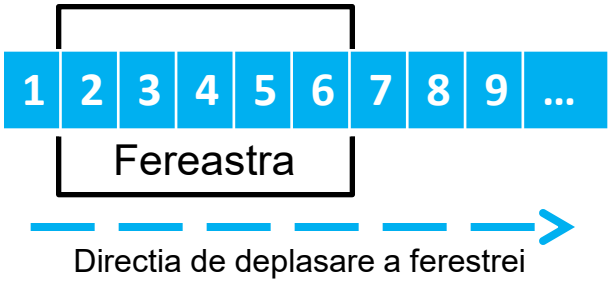
- TCP asigura un flux de octeti => numerele de secventa sunt atribuite fiecarui octet din stream
- TCP imparte fluxul de octeti in segmente TCP; segmentele trimise si confirmarile vor transporta numere de secventa (vezi slide 19)
- Dimensiunea ferestrei este exprimata in octeti si nu in numar de pachete
- Dimensiunea ferestrei este determinata de catre receptor cind conexiunea este stabilita si este variabila in timpul transferului de date; Fiecare mesaj ACK va include dimensiunea ferestrei pe care receptorul este apt sa opereze in momentul respectiv al comunicarii

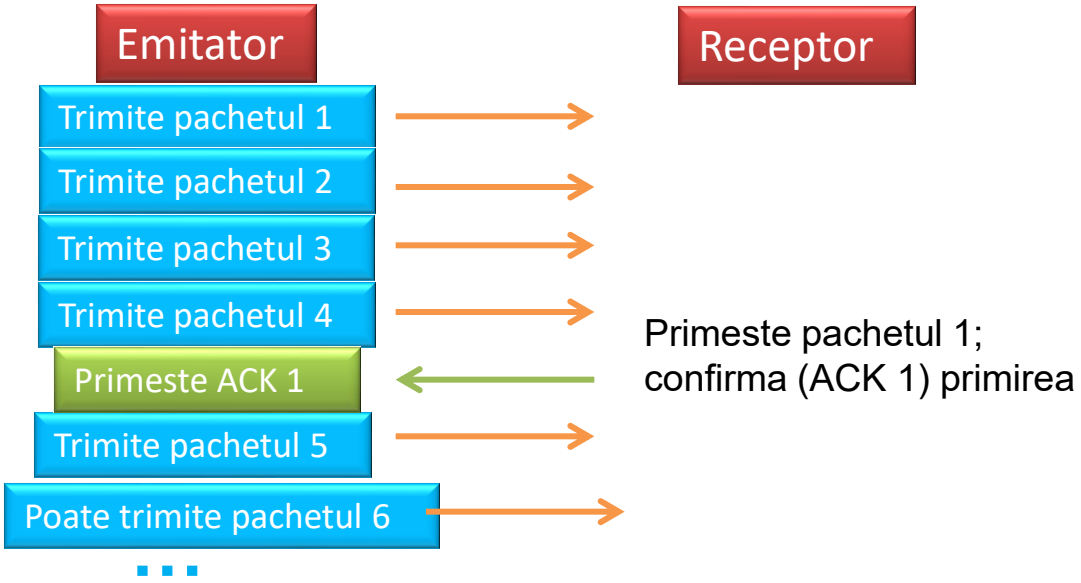
Administrarea conexiunii TCP

Controlul fluxului

- Protocol de comunicare cu **fereastra glisanta** :
  - emitatorul grupeaza pachetele intr-un *buffer*
  - emitatorul poate transmite pachetele din fereastra fara sa receptioneze o confirmare (ACK), dar porneste cate un cronometru (care va semnaliza depasirea unui interval de timp prestabilit) pentru fiecare dintre pachete
  - receptorul trebuie sa confirme fiecare pachet primit, indicand numarul de secventa al ultimului octet **receptionat corect**
  - in urma confirmarilor primite emitatorul deplaseaza fereastra (o gliseaza)







Administrarea conexiunii TCP

Controlul fluxului folosind fereastra glisanta – situatii

- **Pachetul 2 s-a pierdut**
  - emitatorul nu receptioneaza ACK 2, deci fereastra ramine pe pozitia 1
  - receptorul nu primeste pachetul 2, si nu va confirma inca pachetele 3,4,5
  - cronometrul emitatorului va semnaliza timpul de asteptare a confirmarii si pachetul este retransmis
  - receptorul primeste pachetul 2 si va emite confirmarea ACK 5 (secventa de pachete 2-5 au fost receptionate cu succes) => fereastra glisanta se deplaseaza cu patru pozitii dupa receptionarea ACK 5
- **Pachetul 2 a ajuns la destinatie dar confirmarea s-a pierdut**
  - Emitatorul nu primeste ACK 2, dar primeste ACK 3 (toate pachetele pina la 3 au ajuns cu succes) => emitentul deplaseaza fereastra glisanta la pachetul 4

Administrarea conexiunii TCP

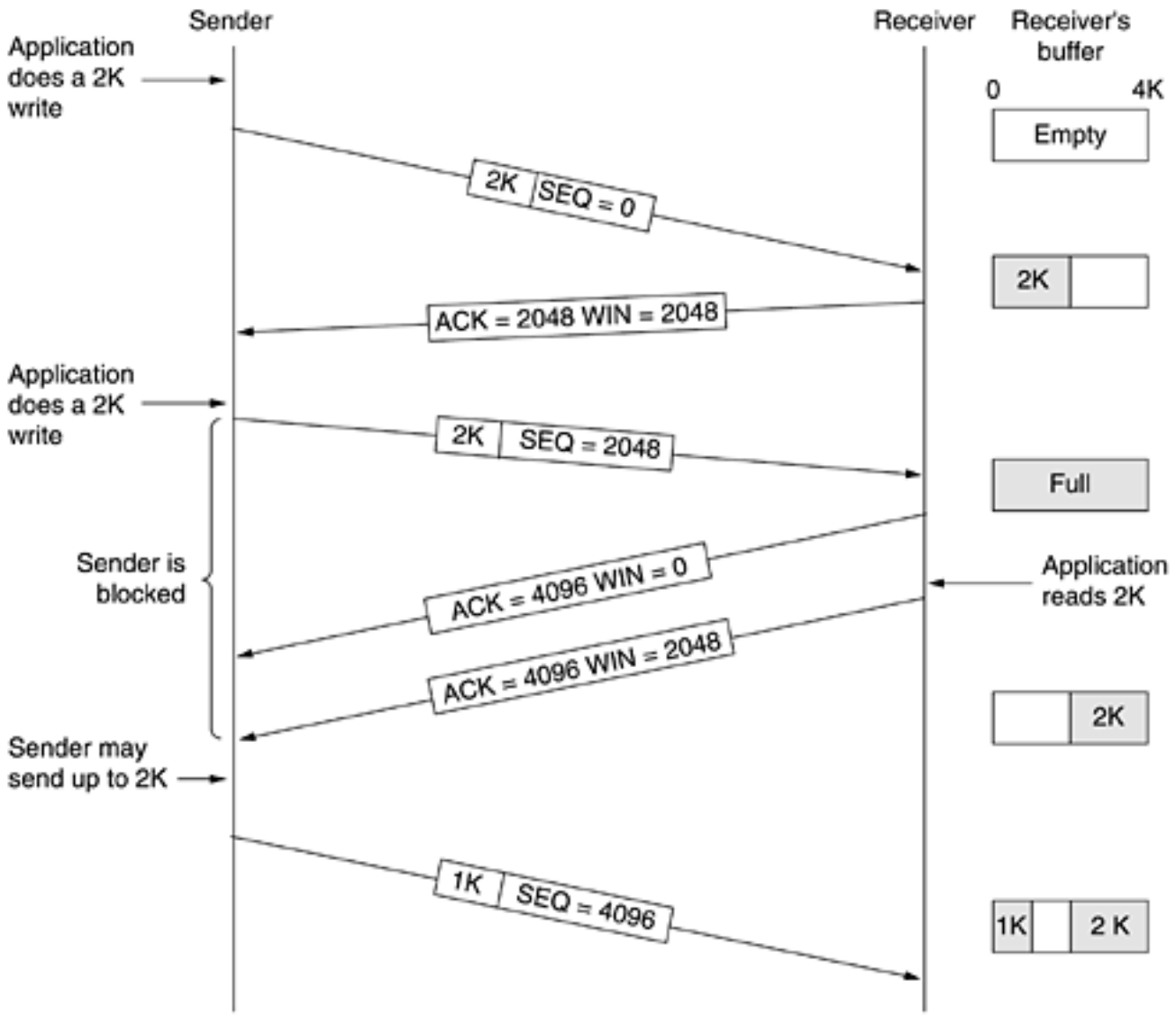
Ferastra glisanta asigura:

- Transmisie sigura
- Folosirea mai buna a latimii de banda => o mai mare viteza de transmisie
- Controlul transmisiei (receptorul poate intarzia confirmarea datelor, cunoscind memoria libera de care dispune si dimensiunea ferestrei de comunicare)

Administrarea conexiunii TCP

Exemplu:

Politica TCP de transmisie a datelor si managmentul ferestrei TCP



[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

Administrarea conexiunii TCP

- Detectia erorilor & retransmiterea datelor
  - Fiecare segment trimis contine un numar de secventa (*Sequence Number*) indicind pozitia octetilor transmisi in cadrul fluxului de date
  - Gazda destinatar verifica numarul de secventa pentru fiecare segment (se testeaza daca anumite segmente se pierd, sunt duplicate sau nu sunt in ordine) si trimite inapoi pentru fiecare segment un numar de confirmare (*Acknowledgment Number*), specificind numarul de secventa al urmatorului octet care se astepta a fi receptionat
  - Segmentele pierdute sunt detectate folosindu-se un timer de retransmisie a datelor
  - Pentru detectarea erorilor se utilizeaza si *checksum*-uri

Administrarea conexiunii TCP

- Resetarea conexiunii
  - Uneori conditii anormale forteaza aplicatiile sau software-ul de retea sa distruga conexiunea
  - Pentru resetarea conexiunii, o parte a comunicarii initiaza terminarea, trimitind un segment cu bitul RST setat
  - Cealalta parte abandoneaza conexiunea, fara a se mai transmite eventuale date ramase netransmise
  - Transferul in ambele directii este oprit, *buffer*-ele sunt golite

Administrarea conexiunii TCP

- Fortarea transmiterii datelor
  - TCP poate divide fluxul de date in segmente de dimensiuni diferite de pachetele vehiculate de aplicatii => eficienta transmisiei
  - Uneori intervine situatia de a transmite datele fara a se mai astepta umplerea buffer-elor (e.g., aplicatii interactive)
  - Fortarea transmiterii se realizeaza prin ***push***: se seteaza bitul **PSH** si se forteaza transmiterea segmentelor, indiferent de starea de umplere a *buffer*-elor

Administrarea conexiunii TCP

- Inchiderea conexiunii
  - Conexiunile TCP fiind full-duplex, cand o aplicatie semnaleaza ca nu mai exista date de trimis, TCP va inchide conexiunea doar intr-o directie
    - Partenerul de comunicatie poate expedia un segment TCP cu bitul FIN setat; confirmarea semnifica ca sensul respectiv de comunicare este efectiv oprit
  - Conexiunea este desfiintata cand ambele directii au fost oprite

TCP

- **Automatul finit TCP**
  - Modeleaza comportamentul protocolului
  - Starile sunt utilizate la managementul conexiunii

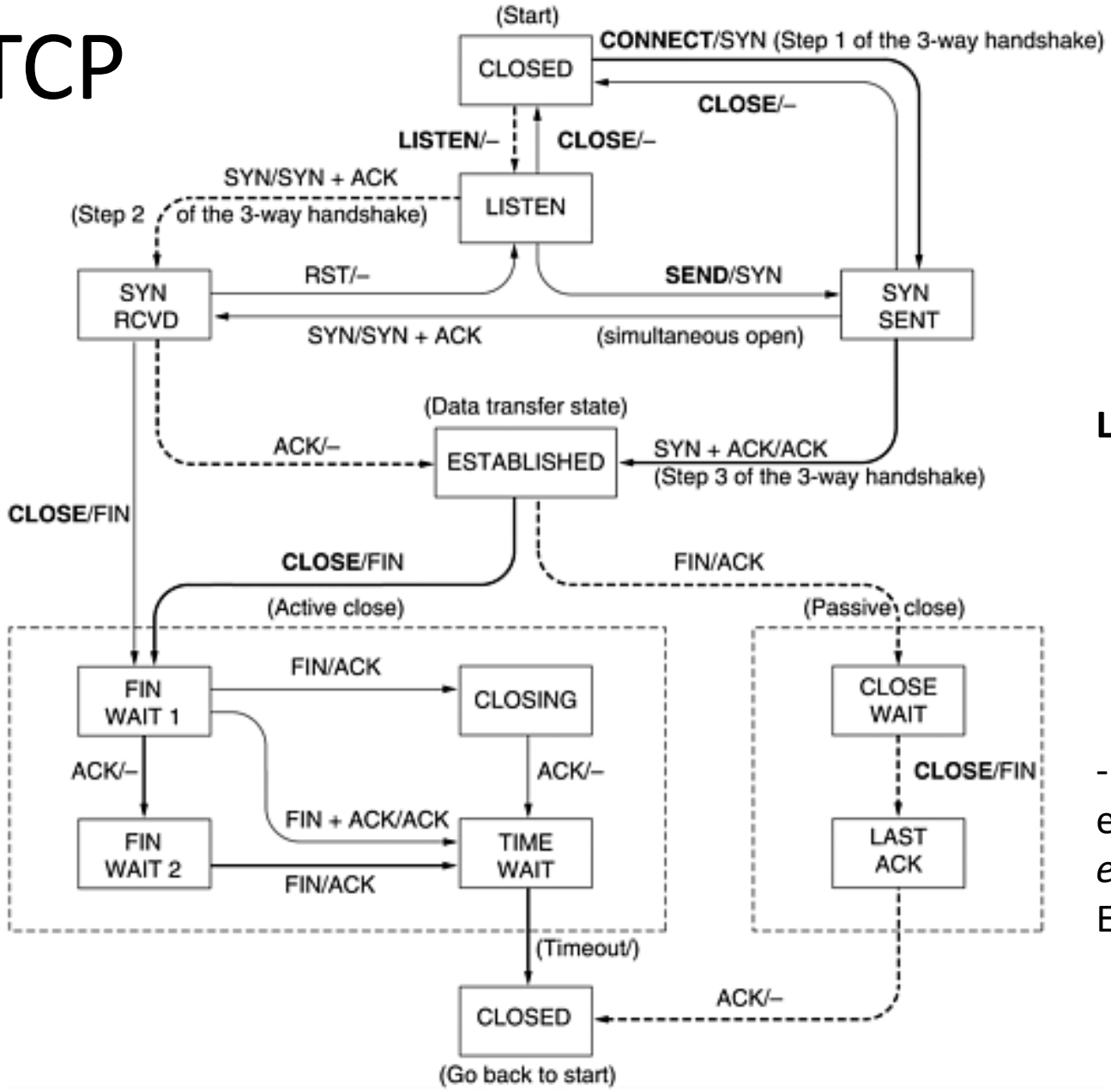
State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

Figura. Starile utilizate in automatul cu stari finite pentru controlul conexiunii TCP

[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

TCP

Automatul finit TCP



Legenda:

——— Client

---------- Server

_____ Evenimente mai putin obisnuite

-Fiecare linie este etichetata cu o pereche *eveniment/actiune*
Exemplu: ACK/-

[conform Computer Networks, 2010 – Andrew S. Tanenbaum, et.al.]

TCP

- Automatul finit TCP
  - Stabilirea conexiunii:
    - CLOSED – din aceasta stare se poate cere o deschidere activa (se trece in SYN_SENT) sau pasiva (SYN_RCVD)
    - LISTEN – se poate trimite o cerere de conexiune activa (se trece in SYN_SENT) ori pasiva (SYN_RCVD)
  - Conexiune stabilita:
    - ESTABLISHED – poate incepe transmisia de date (din aceasta stare se poate trece in CLOSE_WAIT sau FIN_WAIT_1)
  - Deconectarea initiata de procesul partener
    - CLOSE_WAIT, LAST_ACK, CLOSE
  - Stari ce intervin in procesul de deconectare
    - FIN_WAIT_1, FIN_WAIT_2, CLOSING, TIME_WAIT

TCP

Exemplu:

netstat -t

```
adria@thor:~/html/teach/courses/net$ netstat -t
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 thor.info.uaic.ro:smtp  186.122.246.108:33374   SYN_RECV
tcp        0      0 localhost:60000         localhost:45740         ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps mail.traveltech.c:55156 ESTABLISHED
tcp        0      0 localhost:45740         localhost:60000         ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps 81.253.57.112:51462     ESTABLISHED
tcp        0      0 thor.info.uaic.ro:smtp  fenrir.info.uaic.:59806 TIME_WAIT
tcp        0      0 thor.info.uaic.ro:imaps ia.info.uaic.ro:51058   ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps 77-58-250-39.dcli:56424 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:59605 fenrir.info.uaic.ro:ssh ESTABLISHED
tcp        0      0 localhost:57572         localhost:spamd         TIME_WAIT
tcp        0      0 thor.info.uaic.ro:pop3s 79-112-42-154.iasi:2019 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:pop3s info-c-117.info.ua:1302 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:ssh   mail.traveltech.c:52266 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:pop3s info-c-59.info.ua:50149 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps mail.traveltech.c:55152 ESTABLISHED
tcp        0      0 localhost:58053         localhost:10025         TIME_WAIT
tcp        0      0 thor.info.uaic.ro:imaps info-c-70.info.uai:1266 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps info-c-91.info.uai:4285 ESTABLISHED
tcp        0      0 192.168.103.2:39107     pdc:65022               ESTABLISHED
tcp        0      0 thor.info.uaic.ro:pop3s info-c-59.info.ua:55896 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps 77-58-250-39.dcli:56476 ESTABLISHED
tcp        0      0 192.168.103.2:39082     pdc:65022               ESTABLISHED
tcp        0      0 thor.info.uaic.ro:smtp  main.dntis.ro:52854     ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps ws70-228.unine.ch:35907 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps info-c-70.info.uai:1364 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:ssh   ia.info.uaic.ro:40542   ESTABLISHED
tcp        0    396 thor.info.uaic.ro:ssh   79-112-21-031.ias:54540 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps ia.info.uaic.ro:39325   ESTABLISHED
tcp        0      0 thor.info.uaic.ro:pop3s 79-112-42-179.iasi:1146 ESTABLISHED
tcp        0      0 thor.info.uaic.ro:imaps 81.253.57.112:51461     ESTABLISHED
```

TCP

- Exemple de utilizari ale TCP:
  - Majoritatea protocoalelor de aplicatii:
    - HTTP
    - TELNET
    - SMTP
    - SSH
    - …

TCP versus UDP

- Ambele se bazeaza pe IP, utilizeaza porturi
- Unitatea de transmisie
  - TCP -> Segment TCP
  - UDP -> Pachet UDP

TCP versus UDP

- UDP ofera servicii minimale de transport (efort minim de transmisie)

  TCP ofera servicii orientate-conexiune, full-duplex, sigure – pentru transportul fluxurilor de octeti (-> mecanism complex de transmisie)
- Utilizarea TCP sau UDP depinde de aplicatie:

  e-mail, transfer de fisiere, operare in timp-real, transmisii multimedia in timp real, *chat*, …

Rezumat

Nivelul Transport

- Preliminarii
- UDP (User Datagram Protocol)
- TCP (Transmission Control Protocol)
- TCP versus UDP

Bibliografie

- Andrew S. Tanenbaum, David J. Wetherall, Computer Networks (5th Edition), ISBN-10: 0132126958 , Publication Date: October 7, 2010
- A. Tanenbaum, Computer Networks. 4th Edition. Prentice Hall. 2003
- James F. Kurose, Keith W. Ross; Computer Networking: A Top-Down Approach (5th Edition), ISBN-10: 0136079679
- Lydia Parziale, David T. Britt, Chuck Davis, Jason Forrester, Wei Liu, Carolyn Matthews, Nicolas Rosselot , IBM – TCP/IP Tutorial and Technical Overview, 2006
- Tamara Dean, Network +Guide to Networks (Editia 5), ISBN-10: 1-423-90245-9, 2009

Intrebari?

Programarea in retea (I)

Lenuta Alboaie

adria@info.uaic.ro

Cuprins

- Modelul client/server
- API pentru programarea in retea
- *Socket*-uri BSD
  - Caracterizare
  - Creare
  - Primitive
- Modelul client/server TCP

Paradigme ale comunicarii in retea

- Modelul client/server
- Apelul procedurilor la distanta (RPC)
- Mecanismul peer-to-peer (P2P) – comunicare punct-la-punct

Modelul client/server

[Primul server Web]

- Proces **server**
  - Ofera servicii in retea
  - Accepta cereri de la un proces client
  - Realizeaza un anumit serviciu si returneaza rezultatul
- Proces **client**
  - Initializeaza comunicarea cu serverul
  - Solicita un serviciu, apoi asteapta raspunsul serverului

Modelul client/server

- Moduri de interactiune
  - **Orientat-conexiune** – bazat pe TCP
  - **Neorientat-conexiune** – bazat pe UDP

Modelul client/server

- Implementare:
  - **iterativa** – fiecare client e tratat pe rind, secvential

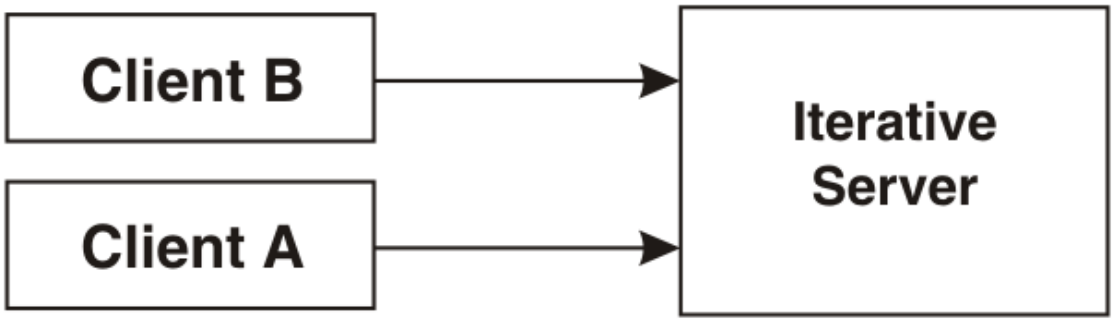


Figura: Exemplu de server iterativ

[http://publib.boulder.ibm.com]

Modelul client/server

- Implementare:
  - **concurenta** – cererile sunt procesate concurent

    Procese copil pentru fiecare cerere de procesat

    Multiplexarea conexiunii

    Tehnici combinate

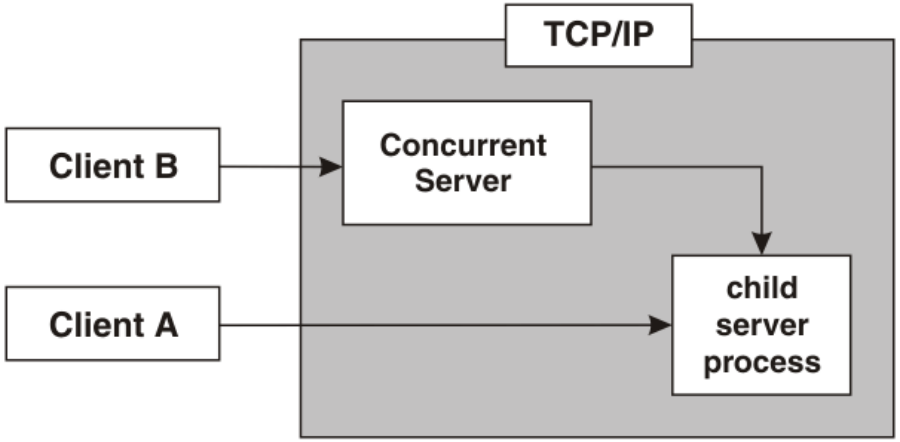


Figura: Exemplu de server concurent

[http://publib.boulder.ibm.com]

API pentru programarea in retea

- Necesitate:
  - Interfata generica pentru programare
  - Independenta de hardware si de sistemul de operare
  - Suport pentru diferite protocoale de comunicatie
  - Suport pentru comunicatii orientate-conexiune si prin mesaje
  - Independenta in reprezentarea adreselor
  - Compatibilitatea cu serviciile I/O comune

API pentru programarea in retea

- Se pot utiliza mai multe API-uri pentru programarea aplicatiilor Internet
  - ***Socket*-uri BSD** (*Berkeley System Distribution*)
  - TLI (*Transport Layer Interface*) – AT&T, XTI (Solaris)
  - Winsock
  - MacTCP
- Functii oferite:

  specificarea de puncte terminale locale si la distanta, initierea si acceptarea de conexiuni, trimitere si receptare de date, terminarea conexiunii, tratare erori
- TCP/IP nu include definirea unui API

...

Interfata de programare a aplicatiilor bazata pe

soket-uri BSD

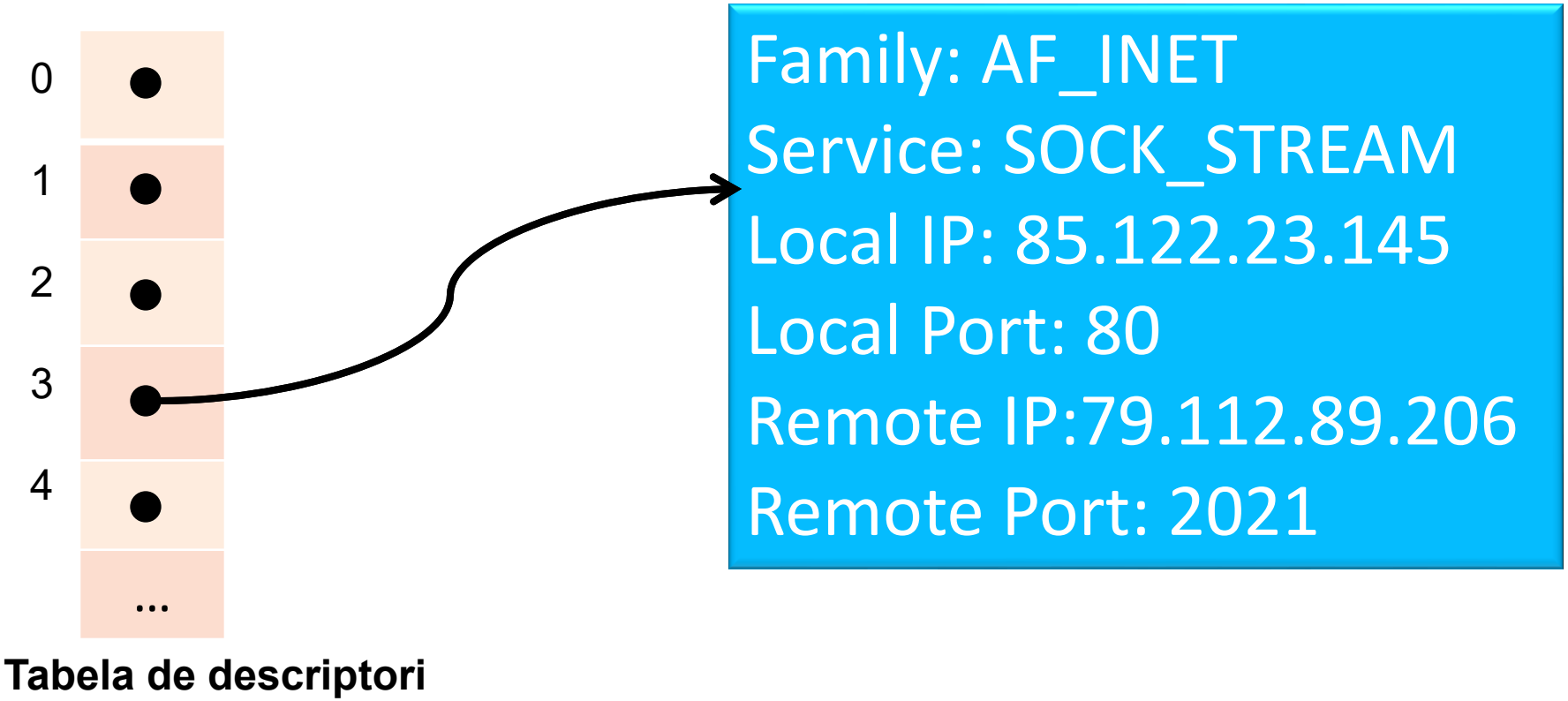
Socket

- Facilitate generala, independenta de arhitectura hardware, de protocol si de tipul de transmisiune a datelor, pentru comunicare intre procese aflate pe masini diferite, in retea
- Ofera suport pentru familii multiple de protocoale
  - Protocolul domeniului UNIX – folosit pentru comunicatii locale
  - Protocolul domeniului Internet folosind TCP/IP
  - Altele: XNS Xerox,…
- Abstractiune a unui punct terminal (*end-point*) la nivelul transport

Socket

- Utilizeaza interfata de programare I/O existenta (similar fisierelor, *pipe*-urilor, *FIFO*-urilor etc.)
- Poate fi asociat cu unul/mai multe procese, existind in cadrul unui domeniu de comunicatie
- Ofera un API pentru programarea in retea, avind implementari multiple
- Din punctul de vedere al programatorului, un *socket* este similar unui descriptor de fisier; diferente apar la creare si la diferite optiuni de control al *socket*-urilor

Socket


0
1
2
3
4
...
Family: AF_INET
Service: SOCK_STREAM
Local IP: 85.122.23.145
Local Port: 80
Remote IP:79.112.89.206
Remote Port: 2021
Tabela de descriptori

Interfata de programare a aplicatiilor bazata pe *socket*-uri BSD

Primitive de baza:

- **socket()** – creaza un nou punct terminal al conexiunii
- **bind()** ataseaza o adresa locala la un *socket*
- **listen()** permite unui *socket* sa accepte conexiuni
- **accept()** blocheaza apelantul pina la sosirea unei cereri de conectare(utilizata de serverul TCP)
- **connect()** tentativa (activa) de stabilire a conexiunii (folosita de clientul TCP)
- **send()** trimitere de date via *socket*
- **recv()** receptarea de date via *socket*
- **close()** elibereaza conexiunea (inchide un *socket*)
- **shutdown()** inchide directional un *socket*

Interfata de programare a aplicatiilor bazata pe *socket*-uri BSD

Alte primitive:

- Citire de date
  - read() / readv() / recvfrom() / recvmsg()
- Trimitere de date
  - write() / writev() / sendto() / sendmsg()
- Multiplexare I/O
  - select()
- Administrarea conexiunii
  - fcntl() / ioctl() / setsockopt() / getsockopt() / getsockname() / getpeername()

Socket-uri|creare

Apelul de sistem **socket()**

```
#include <sys/types.h>
#include <sys/socket.h>
int socket (int domain, int type, int protocol)
```

Domeniul de comunicare: **AF_UNIX, AF_INET, AF_INET6, …**

Protocolul utilizat pentru transmitere (uzual: **0** pentru nivelul transport)

Tipul socketului (modalitatea de realizare a comunicarii): **SOCK_STREAM, SOCK_DGRAM, SOCK_RAW**

Socket-uri|creare

Apelul de sistem **socket()**

Valoarea de retur

- Succes: descriptorul de *socket* creat
- Eroare: -1
  - Raportarea erorii se realizeaza *via* variabilei **errno**
    - **EACCES**
    - **EAFNOSUPPORT**
    - **ENFILE**
    - **ENOBUFS sau ENOMEM**
    - **EPROTONOSUPPORT**
    - **…**

Constante definite in **errno.h**

Socket-uri

Exemplu de combinatii posibile pentru cele trei argumente ale primitivei socket():

int socket (int domain, int type, int protocol)

Domeniu	Tip	Protocol
AF_INET	SOCK_STREAM	TCP
	SOCK_DGRAM	UDP
	SOCK_RAW	IP
AF_INET6	SOCK_STREAM	TCP
	SOCK_DGRAM	UDP
	SOCK_RAW	IPv6
AF_LOCAL	SOCK_STREAM	Mecanism intern de comunicare
	SOCK_DGRAM	

Observatie: AF_LOCAL=AF_UNIX (din motive istorice)

Socket-uri

Observatii

- Primitiva *socket()* aloca resursele necesare unui punct terminal de comunicare, dar nu stabileste modul de adresare
- *Socket*-urile ofera un mod generic de adresare; pentru TCP/IP trebuie specificate (*adresa IP, port*)
- Alte suite de protocoale pot folosi alte scheme de adresare

Tipuri POSIX

int8_t, uint8_t, int16_t, uint16_t, int32_t, int32_t, u_char, u_short, u_int, u_long

Socket-uri

- Tipuri POSIX folosite de socket-uri:
  - sa_family_t – familia de adrese
  - socklen_t – lungimea structurii de memorare
  - in_addr_t – adresaIP (v4)
  - in_port_t – numarul de port
- Specificarea adreselor generice

```
struct sockaddr {
    sa_family_t  sa_family;
    char sa_data[14]
}
```

Familia de adrese: AF_INET, AF_ISO,…

14 bytes - adresa folosita

Socket-uri

- Pentru IPv4 AF_INET vom avea nevoie de o structura speciala: sockaddr_in

```
struct sockaddr_in {
        short int sin_family;
        unisgned short int sin_port;
        struct in_addr sin_addr;
        unsigned char sin_zero[8];
}

struct in_addr{
        in_addr_t s_addr;
}
```

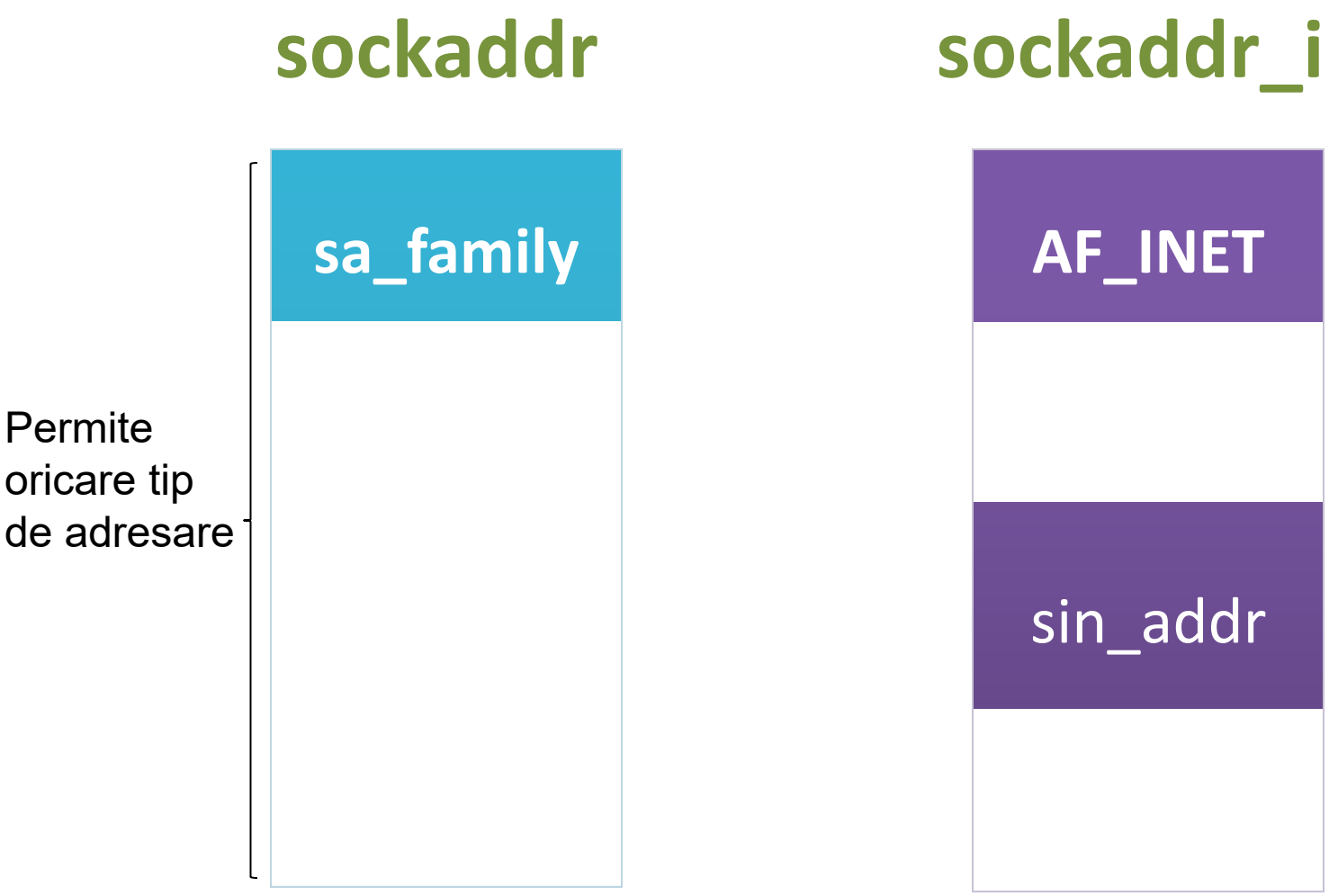
Familia de adrese: **AF_INET**

Portul (2 octeti)

Bytes neutilizati

4 bytes ai adresei IP

Socket-uri


sockaddr
sockaddr_in
sa_family
AF_INET
sin_addr
Permite oricare tip de adresare

Socket-uri

- Toate valorile stocate in sokaddr_in vor respecta ordinea de codificare a retelei (*network byte order*)
- Functii de conversie (netinet/in.h)
  - **uint16_t htons (uint16_t)** – conversie a unui intreg scurt (2 octeti) de la gazda la retea;
  - **uint16_t ntohs (uint16_t)**;
  - **uint32_t ntohl (uint32_t)** – conversie a unui intreg lung(4 octeti) de la retea la gazda;
  - **uint32_t htonl (unit32_t)**;

Discutii|Ordinea octetilor

Ordinea octetilor dintr-un cuvint (*word* – 2 octeti) se poate realiza in doua moduri:

- **Big-Endian** – cel mai semnificativ octet este primul
- **Little-Endian** – cel mai semnificativ octet este al doilea

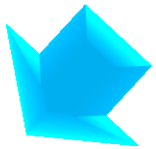
Exemplu:

Masina BigEndian trimite
(e.g. procesor Motorola)

=2

Masina LittleEndian va interpreta:
(e.g. procesor Intel)

=512



Drept conventie, se considera ordinea retelei (*network byte order*) - *BigEndian*

Socket-uri

- Pentru IPv6 AF_INET6 vom avea nevoie de o structura sockaddr_in6:

```
struct sockaddr_in6 {
  u_int16_t sin6_family;   /* AF_INET6*/
  u_int16_t sin6_port;
  u_int32_t sin6_flowinfo;
  struct in6_addr sin6_addr;
  u_int32_t  sin6_scope_id;
}

struct in6_addr{
  unsigned char s6_addr[16];
}
```

Socket-uri

Exemplu:

```
// IPv4:
 struct sockaddr_in ip4addr;  int s;
 ip4addr.sin_family = AF_INET;
 ip4addr.sin_port = htons(2510);
 inet_pton(AF_INET, "10.0.0.1", &ip4addr.sin_addr);
 s = socket(PF_INET, SOCK_STREAM, 0);
 bind(s, (struct sockaddr*)&ip4addr, sizeof (ip4addr));

// IPv6:
 struct sockaddr_in6 ip6addr; int s;
 ip6addr.sin6_family = AF_INET6;
 ip6addr.sin6_port = htons(2610);
 inet_pton(AF_INET6, "2001:db8:8714:3a90::12", &ip6addr.sin6_addr);
 s = socket(PF_INET6, SOCK_STREAM, 0);
 bind(s, (struct sockaddr*)&ip6addr, sizeof (ip6addr));
```

Converteste adrese IPv4 si IPv6 din sir de caractere (x.x.x.x) in ordinea de codificare a retelei (#include <arpa/inet.h>)

? (urmatorul slide)

Socket-uri (slide 19)

Observatii

- Primitiva *socket()* aloca resursele necesare unui punct terminal de comunicare, dar nu stabileste modul de adresare
- *Socket*-urile ofera un mod generic de adresare; pentru TCP/IP trebuie specificate (*adresa IP, port*)
- Alte suite de protocoale pot folosi alte scheme de adresare

Socket-uri| asignarea unei adrese

- Asignarea unei adrese la un socket existent se realizeaza cu **bind()**

```
int bind ( int sockfd,
           const struct sockaddr *addr,
           int addrlen );
```

- Se returneaza: 0 in caz de succes, -1 eroare
  variabila errno va contine codul de eroare corespunzator: EACCES , EADDRINUSE, EBADF, EINVAL, ENOTSOCK,…

Socket-uri| asignarea unei adrese

Exemplu:

```
#define PORT 2021
struct sockaddr_in adresa;
int sd;

sd = socket (AF_INET, SOCK_STREAM, 0))  // TCP
adresa.sin_family = AF_INET; // stabilirea familiei de socket-uri
adresa.sin_addr.s_addr = htonl (adresaIP); //adresa IP
adresa.sin_port = htons (PORT);  //portul

if (bind (sd, (struct sockaddr *) &adresa, sizeof (adresa) == -1)
        {
                perror ("Eroare la bind().\n");
                …
        }
```

Socket-uri| asignarea unei adrese

- Utilizari ale lui bind():
  - Serverul doreste sa ataseze un socket la un port prestabilit (pentru a oferi servicii via acel port)
  - Clientul vrea sa ataseze un socket la un port specificat
  - Clientul cere sistemului de operare sa asigneze orice port disponibil
- In mod normal, clientul nu necesita atasarea la un port specificat
- Alegerea oricarui port liber:

```
adresa.sin_port = htons(0);
```

Socket-uri| asignarea unei adrese

- Alegerea adresei IP la bind()
  - Daca gazda are asignate mai multe adrese IP?
  - Cum se rezolva independenta de platforma?

Pentru a atasa un socket la adresa IP locala, se va utiliza in locul unei adrese IP constanta

INADDR_ANY

Socket-uri| asignarea unei adrese

- Conversia adreselor IP:

  int inet_aton (const char *cp, struct in_addr *inp);

  ASCII “x.x.x.x” -> reprezentare interna pe 32 biti (*network byte order*)

  char *inet_ntoa(struct in_addr in);

  reprezentare pe 32 biti (*network byte order*)-> ASCII “x.x.x.x”

  Obs: **[@fenrir ~]$** man inet_addr

Socket-uri| asignarea unei adrese

- Observatii:
  - PentruIPv6 in locul constantei INADDR_ANY

  se va folosi (vezi antetul netinet/in.h):

  serv.sin6_addr = in6addr_any;
  - Functiile de conversie pentru IPv6 (merg si pentru IPv4) sunt:

  inet_pton()

  inet_ntop()

Socket-uri| listen()

- Stabilirea modului pasiv de interactiune
  - Nucleul sistemului va astepta cereri de conectare directionate la adresa la care este atasat socketul

    3-way handshake
  - Conexiunile multiple receptionate vor fi plasate intr-o coada de asteptare

    int listen(int sockfd, int backlog);

    Numarul de conexiuni din coada de asteptare

    Socket TCP atasat unei adrese
  - Se returneaza: 0 – succes, -1 - eroare

Socket-uri| listen()

- Observatii:
  - Alegerea valorii *backlog* depinde de aplicatie
  - Serverele HTTP trebuie sa specifice o valoare *backlog* cit mai mare (din cauza incarcarii cu cereri multiple)

Socket-uri| accept()

- Acceptarea propriu-zisa a conexiunilor din partea clientilor
  - Cand aplicatia este pregatia pentru a trata o noua conexiune, va trebui sa interogam sistemul asupra unei alte conexiuni cu un client

```
int accept (int sockfd,
            struct sockaddr *cliaddr,
            socklen_t *addrlen);
```

Socket TCP (mod pasiv)

- Trebuie initial sa fie egal cu lungimea structurii **cliaddr**
- Se va returna numarul de bytes folositi in **cliaddr**

Se returneaza descriptorul de socket corespunzator punctului terminal al clientului sau -1 in caz de eroare

Socket-uri| connect()

- Incercarea de a stabili o conexiune cu serverul
  - *3-way handshake*

Socket TCP
- Nu necesita atasarea cu bind(); sistemul de operare va asigna o adresa locala (IP, port)

```
int connect (int sockfd,
             const struct sockaddr *serv_addr,
             socklen_t addrlen);
```

Contine adresa serverului (IP, port)

Se returneaza: succes ->0, eroare -> -1

I/O TCP| read()

int read(int sockfd, void *buf, int max);

- Apelul este blocant in mod normal, read() returneaza doar cind exista date disponibile
- Citirea de la un socket TCP poate returna mai putini octeti decat numarul maxim dorit
  - Trebuie sa fim pregatiti sa citim cate 1 byte la un moment dat (vezi cursul anterior)
- Daca partenerul a inchis conexiunea si nu mai sunt date de primit, se returneaza 0 (EOF)
- Erori: EINTR – un semnal a intrerupt citirea, EIO – eroare I/O, EWOULDBLOCK – *socket*-ul nu are date intr-o citire neblocanta

I/O TCP| write()

int write(int sockfd, const void *buf, int count);

- Apelul este blocant in mod normal
- Erori:
  - EPIPE – scriere la un socket neconectat
  - EWOULDBLOCK – nu se pot accepta date fara blocare, insa operatiunea este setata ca find blocanta

I/O TCP| Exemplu

```
#define MAXBUF 127  /* lungime buffer citire*/
…
char *cerere= “da-mi ceva”;
char buf [MAXBUF]; /* buffer pentru raspuns*/
char *pbuf= buf; /* pointer la buffer */
int n, lung = MAXBUF; /* nr. bytes cititi, nr. bytes liberi in buffer */
…
/* trimite cererea*/
write(sd, cerere, strlen(cerere));

/* asteapta raspunsul*/
while ((n = read (sd, pbuf, lung)) > 0) {
        pbuf+= n;
        lung -= n;
}
```

Exemplu de comunicarea dintre client si sever

Inchiderea conexiunii| close()

int close(int sockfd)

- Efect:
  - terminarea conexiunii;
  - dealocarea memoriei alocate *socket*-ului
    - pentru procese care partajeaza acelasi *socket*, se decrementeaza numarul de referinte la acel socket; cind ajunge la 0 *socket*-ul este dealocat
- Probleme:
  - serverul nu poate termina conexiunea, nu stie daca si cind clientul va mai trimite si alte cereri
  - clientul nu poate sti daca datele au ajuns la server

Inchiderea conexiunii| shutdown()

- Inchidere unidirectionala
  - Cind un client termina de trimis cererile, poate apela shutdown() pentru a specifica faptul ca nu va mai trimite date pe *socket*, fara a dealoca *socket*-ul
  - Serverul va primi EOF si, dupa expedierea catre client a ultimului raspuns, va putea inchide conexiunea

```
#include <sys/socket.h>
int shutdown (int sockfd, int how);
```

0 – viitoare citiri de pe socket nu vor mai fi permise (SHUT_RD);
1 – viitoarele scrieri pe socket nu vor mai fi permise (SHUT_WR);
2 - citirile/scrierile nu vor mai fi permise (SHUT_RDWR)

Metaphor for Good Relationships

Copyright Dr. Laura's Network Programming Corp.

To succeed in relationships…

- you need to establish your own identity. bind()
- you need to be open & accepting. accept()
- you need to establish contacts. connect()
- you need to take things as they come, not as you expect them. *read might return 1 byte*
- you need to handle problems as they arise. *check for errors*

[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Model client/server

- **Modelul unui server TCP iterativ:**
  - Creare *socket* pentru tratarea conexiunilor cu clientii: socket()
  - Pregatirea structurilor de date (sockaddr_in)
  - Atasarea *socket*-ului la adresa locala (port): bind()
  - Pregatirea *socket*-ului pentru ascultarea portului in vederea stabilirii conexiunii cu clientii: listen()
  - Asteptarea realizarii unei conexiuni cu un anumit client (deschidere pasiva): accept()
  - Procesarea cererilor clientului, folosindu-se *socket*-ul returnat de accept(): succesiune de read()/write()
  - Inchiderea (directionata) a conexiunii cu clientul: close(), shutdown()

Model client/server

- **Modelul unui client TCP:**
  - Creare *socket* pentru conectarea la server: socket()
  - Pregatirea structurilor de date (sockaddr_in)
  - Atasarea socket-ului: bind() – optional
  - Conectarea la server (deschidere activa): connect()
  - Solicitarea de servicii si receptionarea rezultatelor trimise de server: succesiune de write()/read()
  - Inchiderea (directionata) a conexiunii cu serverul: close(), shutdown()

Model general - server/client TCP

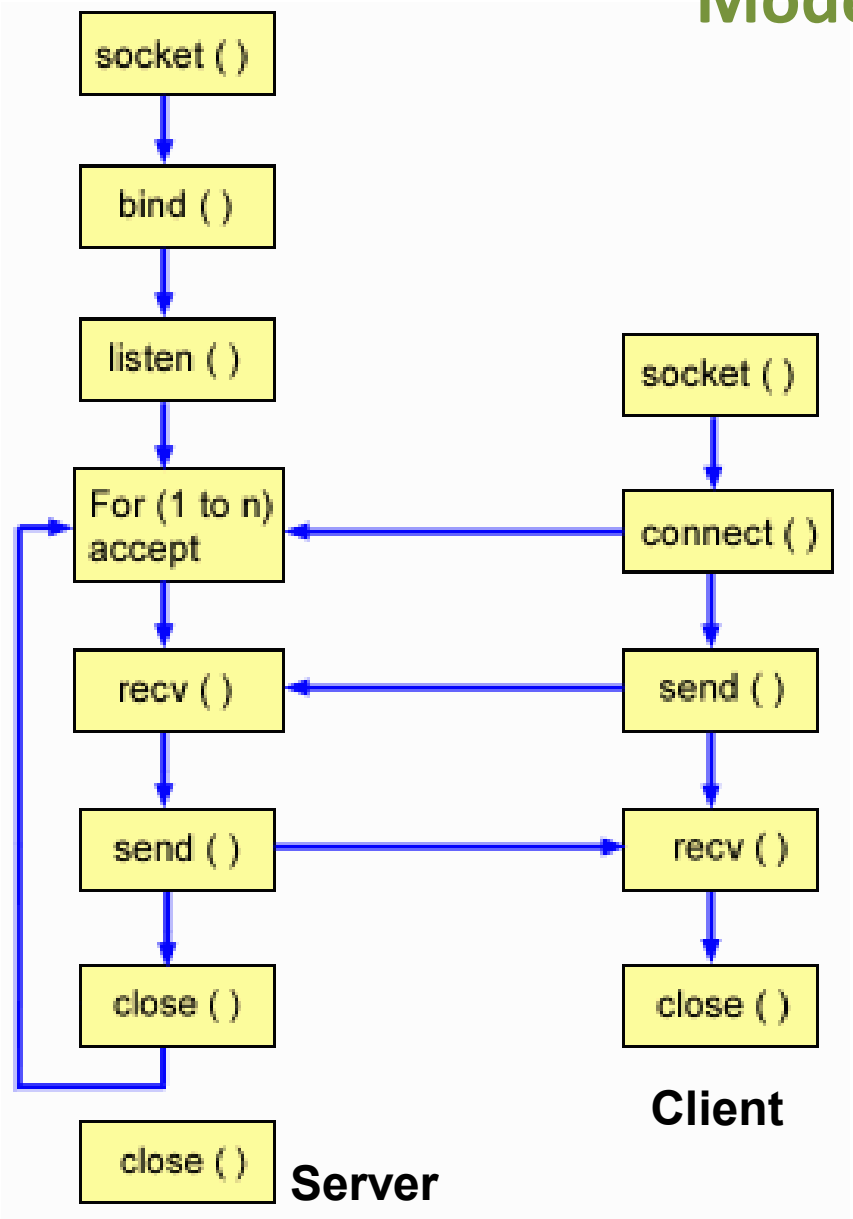
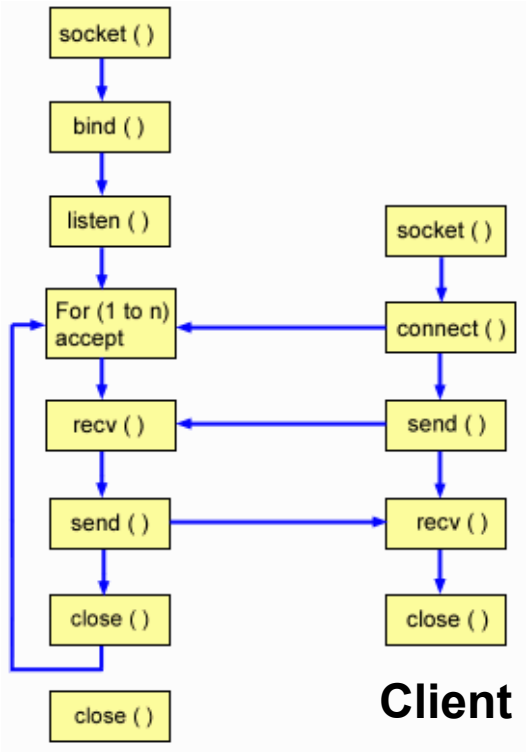


Figura: Server TCP Iterativ-succesiunea de evenimente

[http://publib.boulder.ibm.com]



DEMO ☺

Exemplu de server/client TCP iterativ

Cuprins

- Modelul client/server
- API pentru programarea in retea
- *Socket*-uri BSD
  - Caracterizare
  - Creare
  - Primitive
- Modelul client/server TCP

Intrebari?

Programarea in retea (II)

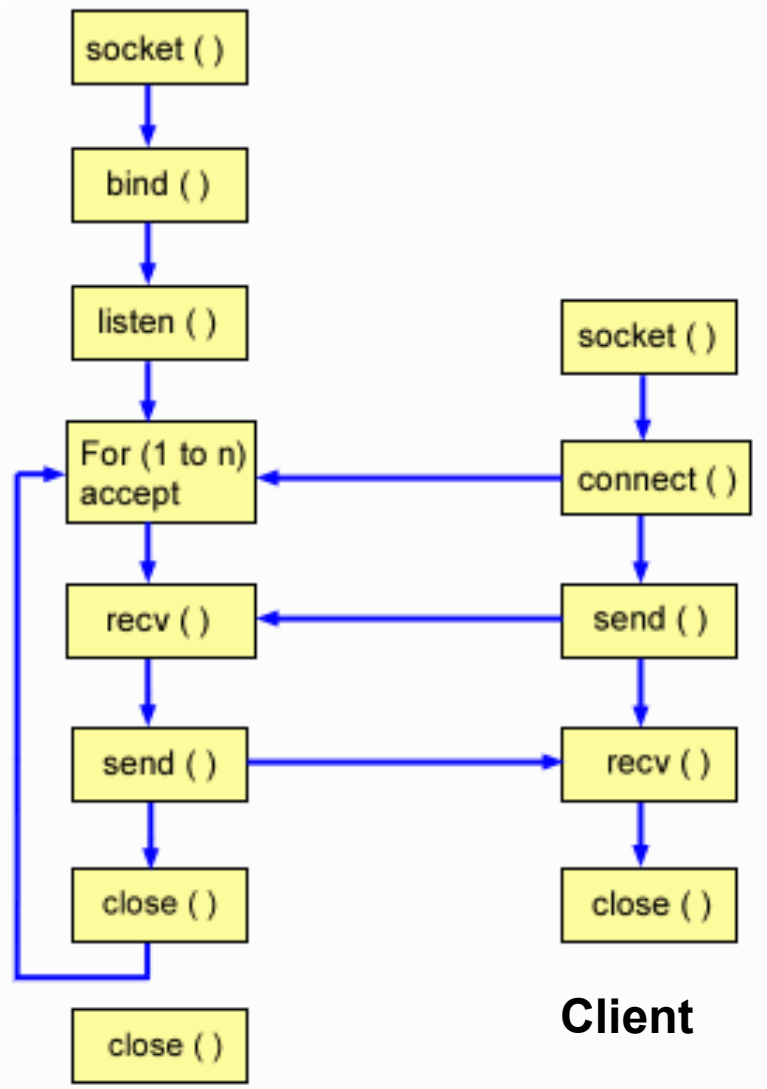
Lenuta Alboaie

adria@info.uaic.ro

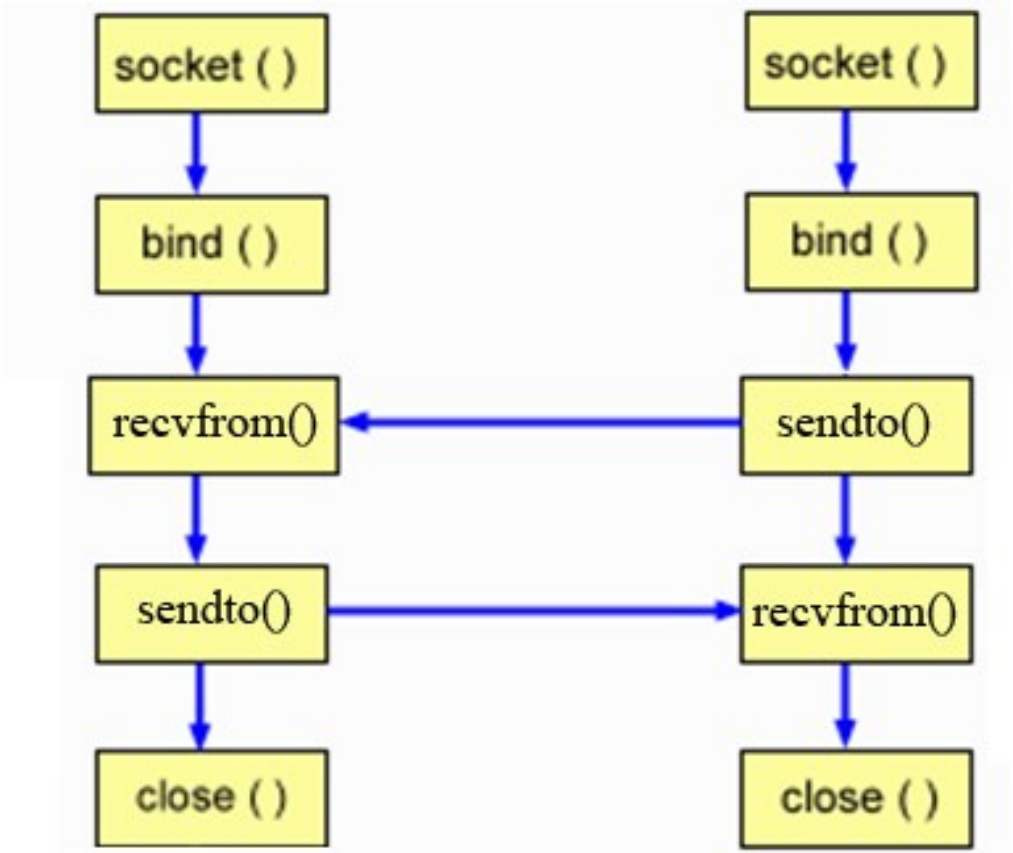
Cuprins

- … sa ne amintim: client/server TCP iterativ - primitive
- Modelul client/server UDP
- Primitive I/O
- Aspecte de programare avansata Internet
- API-ului *socket* – discutii si critici

Model server/client TCP



Client/Server UDP



Client/Server UDP

- Pentru **socket()** se va folosi **SOCK_DGRAM**
- Apelurile listen(), accept(), connect() nu vor mai fi utilizate in mod uzual
- Pentru scriere de datagrame se pot folosi **sendto()** sau **send()** (mai general)
- Pentru citire de datagrame se pot folosi **recvfrom()** sau **recv()**
- Nimeni nu garanteaza ca datele expediate au ajuns la destinatar sau nu sunt duplicate

Client/Server UDP

- Socket-urile UDP pot fi “conectate”: clientul poate folosi connect() pentru a specifica adresa (IP, port) a punctului terminal (serverul) – **pseudo-conexiuni**:
  - Utilitate: trimiterea mai multor datagrame la acelasi server, fara a mai specifica adresa serverului pentru fiecare datagrama in parte
  - Pentru UDP, connect() va retine doar informatiile despre punctul terminal, fara a se initia nici un schimb de date
  - Desi connect() raporteaza succes, nu inseamna ca adresa punctului terminal e valida sau serverul este disponibil

Client/Server UDP

- Pseudo-conexiuni UDP
  - Se poate utiliza shutdown() pentru a opri directionat transmiterea de date, dar nu se va trimite nici un mesaj partenerului de conversatie
  - Primitiva close() poate fi apelata si pentru a elimina o pseudo-conexiune

Alte primitive| I/O

```
#include <sys/types.h>
#include <sys/socket.h>
int send (int sockfd, char *buff, int nbytes, int  flags);
int recv (int sockfd, char *buff, int nbytes, int flags);
```

- Pot fi folosite in cadrul comunicatiilor orientate conexiune sau pentru pseudo-conexiuni

  Apelurile send() si recv() presupun că sunt cunoscute toate elementele unei asocieri, adică a fost efectuat în prealabil un apel connect()
- Primele 3 argumente sunt similare cu cele de la write(), respectiv read()
- Argumentul a patrulea este de regulă 0, dar poate avea si alte valori care precizează condiții de efectuare a apelului
- Cele 2 apeluri returnează la execuție normală lungimea transferului în octeți

Alte primitive| I/O

```
#include <sys/types.h>
#include <sys/socket.h>
int sendto ( int sockfd, char *buff, int nbytes, int flags,
                struct sockaddr *to, int addrlen);
int recvfrom (int sockfd, char *buff, int nbytes, int flags,
                struct sockaddr *from, int *addrlen);
```

- Sunt folosite pentru comunicatii neorientate conexiune
- La sendto() si recvfrom() elementele pentru identificarea nodului la distanță se specifică în apel, prin ultimele 2 argumente
- Cele 2 apeluri returnează la execuție normală lungimea transferului în octeți

Alte primitive| I/O

```
#include <sys/uio.h>
ssize_t readv (int fd, const struct iovec *iov, int iovcnt);
ssize_t writev (int fd, const struct iovec *iov, int iovcnt);
```

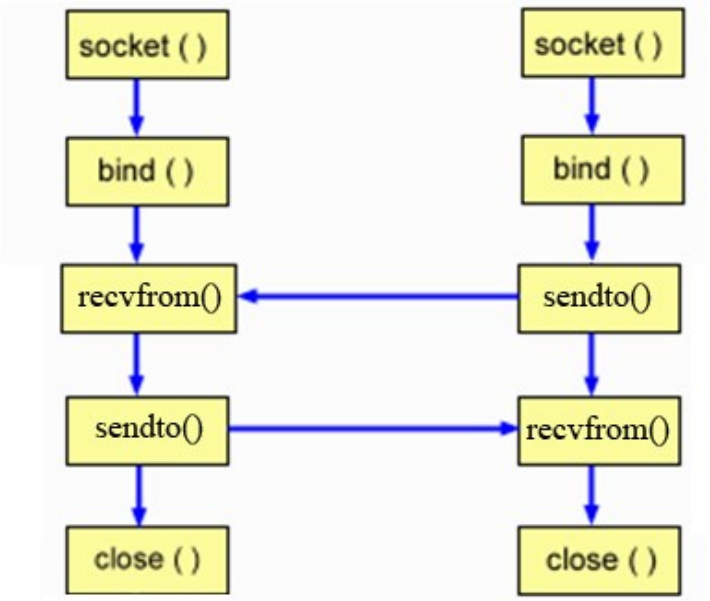
– Mai generale decit read()/write(), ofera posibilitatea de a lucra cu date aflate in zone necontigue de memorie

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t  recvmsg (int s, struct msghdr *msg, int flags);
ssize_t  sendmsg (int s, const struct msghdr *msg, int flags);
```

– Receptioneaza/transmite mesaje extragindu-le din structura *msghdr*

DEMO

Exemplu de server/client UDP



Server UDP Client UDP

Alte primitive|informatii

- **getpeername()** – returneaza informatii despre celalalt capat al conexiunii

```
#include <sys/socket.h>
int getpeername (int sockfd, struct sockaddr *addr,
                 socklen_t *addrlen);
```

- **getsockname()** – returneaza informatii asupra socketului(local) specificat –> (adresa la care este atasat)

```
#include <sys/socket.h>
#include <sys/types.h>
int getsockname( int sockfd, struct sockaddr * addr,
                 socklen_t * addrlen);
```

Programare retea avansata

- Optiuni atasate socket-urilor
  - **getsockopt()** si **setsockopt()**
- Multiplexare I/O

Primitive|optiuni

- Optiuni atasate *socke*t-urilor
  - Atribute utilizate pentru consultarea sau modificarea unui comportament, general ori specific unui protocol, pentru unele (tipuri de) *socke*t-uri
  - Tipuri de valori:
    - Booleene (*flag*-uri)
    - Mai “complexe”:

      int, timeval, in_addr, sock_addr, etc

Primitive|optiuni

- **getsockopt()** – consultarea optiunilor

```
#include <sys/types.h>
#include <sys/socket.h>
int getsockopt (int sockfd, int level, int optname, void *optval,
                socklen_t *optlen);
```

Numele, valoarea si lungimea optiunii

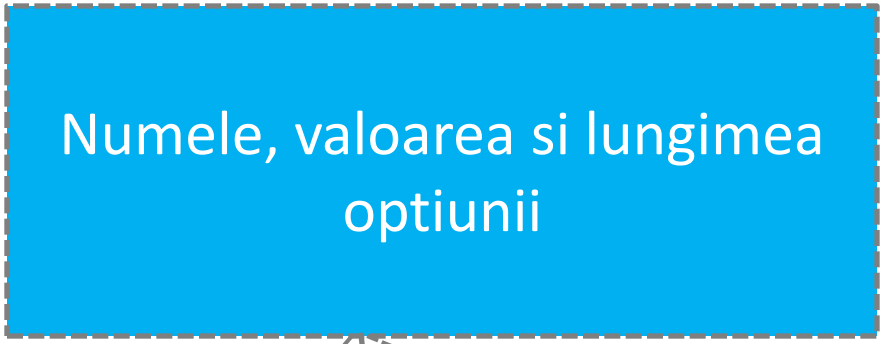
***Level* -** indica daca optiunea este generala sau specifica unui protocol

Exemplu:

```
len = sizeof (optval);
getsockopt (sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, &len);
```

Primitive|optiuni

- **setsockopt()** – setarea optiunilor




```
#include <sys/types.h>
#include <sys/socket.h>
int setsockopt (int sockfd, int level, int optname, void *optval,
                socklen_t *optlen);
```

Returneaza:

- 0 = succes
- -1 = eroare: EBADF, ENOTSOCK, ENOPROTOOPT,...

Primitive|optiuni

Optiuni generale

- Independente de protocol
- Unele suportate doar de anumite tipuri de socketuri (SOCK_DGRAM, SOCK_STREAM)
  - **SO_BROADCAST**
  - **SO_DONTROUTE**
  - **SO_ERROR**
  - **SO_KEEPALIVE**
  - **SO_LINGER**
  - **SO_RCVBUF, SO_SNDBUF**
  - **SO_REUSEADDR**
  - **SO_OOBINLINE**
  - …

[http://www.beej.us/guide/bgnet/output/html/multipage/setsockoptman.html]

Primitive|optiuni

- **SO_BROADCAST** (boolean)
  - Activeaza/dezactiveaza trimiterea de date in regim broadcast
  - Utilizata doar pentru SOCK_DGRAM
  - Previne anumite aplicatii sa nu trimita in mod neadecvat *broadcast*-uri
- **SO_DONTROUTE** (boolean)
  - Utilizat de daemon-ii de rutare
  - Dezactiveaza/activeaza rutarea pachetelor de date

Primitive|optiuni

- **SO_ERROR** (int)
  - Indica eroarea survenita (similara lui errno)
  - Poate fi folosita cu primitiva getsockopt()

- **SO_KEEPALIVE** (boolean)
  - Folosita pentru SOCK_STREAM
  - Se va trimite o informatie de proba celuilalt punct terminal daca nu s-a realizat schimb de date timp indelungat
  - Utilizata de TCP (e.g., telnet): permite proceselor sa determine daca procesul/gazda cealalta a picat

Primitive|optiuni

- **SO_LINGER** (struct linger)
  - Controleaza daca si dupa cit timp un apel de inchidere a conexiunii va astepta confirmari(ACK-uri) de la punctul terminal
  - Folosita doar pentru socket-uri orientate-conexiune pentru a ne asigura ca un apel close() nu va returna imediat
  - Valorile vor fi de tipul:

```
struct linger {
   int l_onoff;      /* interpretat ca boolean */
   int l_linger;     /* timpul in secunde*/
}
```

  - **l_onoff = 0:** *close*() returneaza imediat, dar datele netrimise sunt transmise
  - **l_onoff !=0** si **l_linger=0:** *close*() returneaza imediat si datele netrimise sunt sterse
  - **l_onoff!=0** si **l_linger !=0:** *close*() nu returneaza pina cind datele netrimise sunt transmise (sau conexiunea este inchisa de sistemul remote)

Primitive|optiuni

- **SO_LINGER – Exemplu**

```
int result;
struct linger lin;
lin.l_onoff=1 ;  /*0 -> l_linger este ignorata */
lin.l_linger=1; /* 0 = pierderea datelor; nonzero= asteptare pina
                   se trimit datele */
result= setsockopt( sockfd,
                    SOL_SOCKET,
                    SO_LINGER,
                    &lin, sizeof(lin));
```

Primitive|optiuni

- **SO_RCVBUF/SO_SNDBUF** (int)
  - Modifica dimensiunile *buffer*-elor de receptionare sau de trimitere a datelor
  - Utilizate pentru SOCK_DGRAM si SOCK_STREAM

- **Exemplu**:

  int result; int buffsize = 10000;

  result= **setsockopt** (s, SOL_SOCKET, SO_SNDBUF, &buffsize, sizeof(buffsize));

Primitive|optiuni

- **SO_REUSEADDR** – (boolean)
  - Permite atasarea la o adresa(port) deja in uz
    - nu incalca regula de asociere unica realizata de bind
  - Folosita pentru ca un *socket pasiv* sa poata folosi un port deja utilizat de alte procese

Stare 1

```
Active connections (including   servers)
Proto Recv-Q Send-Q  Local Address Foreign Address (state)
tcp        0      0    *.2000       *.*             LISTEN
```

Stare 2

```
Proto Recv-Q Send-Q  Local Address Foreign Address (state)
tcp 0 0 192.6.250.100.2000 192.6.250.101.4000 ESTABLISHED
tcp 0 0 *.2000 *.* LISTEN
```

- Daca *daemon*-ul care asculta la portul 2000 este *killed*, incercarea de restart a *daemon*-ului va esua daca SO_REUSEADDR nu este setat

Exemplu

```
int optval = 1;
setsockopt (sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));
bind (sockfd, &sin, sizeof(sin) );
```

Primitive|optiuni

Optiuni specifice protocolului IP

- **IP_TOS** permite setarea cimpului "Type Of Service" (e.g., ICMP) din antetul IP
- **IP_TTL** permite setarea cimpului "Time To Live" din antetul IP

Exista si optiuni pentru IPv6.(RFC 2460,2462)

-**IPV6_V6ONLY, …**

Primitive|optiuni

Optiuni specifice protocolului TCP

- **TCP_KEEPALIVE** seteaza timpul de asteptare daca SO_KEEPALIVE este activat
- **TCP_MAXSEG** stabileste lungimea maxima a unui segment (nu toate implementarile permit modificarea acestei valori de catre aplicatie)
- **TCP_NODELAY** seteaza dezactivarea algoritmului Nagle (reducerea numarului de pachete de dimensiuni mici intr-o retea WAN; TCP va trimite intotdeauna pachete de marime maxima, daca este posibil) – utilizata pentru generatori de pachete mici (e.g., clienti interactivi precum *telnet*)

Multiplexare I/O

- Posibilitatea de a monitoriza mai multi descriptori I/O
  - Un client TCP generic (e.g., *telnet*)
  - Un client interactiv (e.g., *ftp*, *scp*, *browser Web*,…)
  - Un server care poate manipula mai multe protocoale (TCP si UDP) simultan
  - Rezolvarea unor situatii neasteptate (i.e. caderea unui server in mijlocul comunicarii)
- Exemplu: datele citite de la intrarea standard trebuie scrise la un socket, iar datele receptionate prin retea trebuie afisate la iesirea standard

Multiplexare I/O| solutii

- Utilizarea mecanismului neblocant folosind primitivele **fnctl()** / **ioctl()**
- Utilizarea mecanismului asincron
- Folosirea **alarm()** pentru a intrerupe apelurile de sistem lente
- Utilizarea unor *procese*/*thread*-uri multiple (*multitasking*)
- Folosirea unor primitive care suporta verificarea existentei datelor de intrare de la descriptori de citire multipli: **select()** si **poll()**

Multiplexare I/O| solutii

- Utilizarea mecanismului neblocant folosind primitiva **fnctl()**
  - Se seteaza apelurile I/O ca neblocante

```
int flags;
flags = fcntl ( sd, F_GETFL, 0 );
fcntl( sd, F_SETFL, flags | O_NONBLOCK);
```

  - Daca nu sunt date disponibile un apel read() va intoarce -1 sau daca nu este suficient spatiu in *buffer* un apel write() va intoarce -1 (cu eroarea EAGAIN)

Multiplexare I/O| solutii

Utilizarea mecanismului neblocant folosind primitiva **ioctl()**

```
#include <sys/ioctl.h>
ioctl (sd, FIOSNBIO, &arg);
```

-arg este un pointer la un int
-Daca int are valoare 0, socketul este setat in mod blocant
-Daca int are valoare 1, socketul este setat in mod neblocant

Daca socketul este in mod neblocant, urmatoarele apeluri sunt afectate astfel:

- accept() – daca nu este prezenta nici o cerere, *accept*() returneaza cu eroarea EWOULDBLOCK
- connect() – daca conexiunea nu se poate stabili imediat, *connect*() returneaza cu eroarea EINPROGRESS
- recv() – daca nu exista date de primit, *recv*() returneaza -1 cu eroarea EWOULDBLOCK
- send() – daca nu exista spatiu in buffer pentru ca datele sa fie transmise, *send*() returneaza -1 cu eroarea EWOULDBLOCK

Multiplexare I/O| solutii

Trimiterea si receptarea datelor in mod asincron

- **Problema**: In conditiile in care *socket*-urile sunt create implicit in mod blocant (I/O), cum s-ar putea instiinta procesul atunci cind se intimpla “ceva” la un socket?
- **Socket**-urile **asincrone** permit trimiterea unui semnal (SIGIO) procesului
- Socket-urile asincrone permit utilizatorului separarea “*procesarilor socket*” de alte procesari
- Generarea semnalului SIGIO este dependenta de protocol

Multiplexare I/O| solutii

Trimiterea si receptarea datelor in mod asincron

- Pentru TCP semnalul SIGIO poate aparea cind:
  - Conexiunea a fost complet stabilita
  - O cerere de deconectare a fost initiata
  - Cererea de deconectare a fost realizata complet
  - *shutdown()* pentru o directie a comunicatiei
  - Au aparut date de la celalalt punct terminal
  - Datele au fost trimise
  - Eroare

Multiplexare I/O| solutii

Trimiterea si receptarea datelor in mod asincron

- Pentru UDP semnalul SIGIO apare cind:
  - Se receptioneaza o datagrama
  - Apare o eroare
- Putem permite proceselor sa realizeze alte activitati si sa monitorizeze transferurile UDP

Multiplexare I/O| solutii

Trimiterea si receptarea datelor in mod asincron

- **Implementarea**
  - *Socket*-ul trebuie setat ca fiind asincron

```
#include <sys/unistd.h>
#include <sys/fcntl.h>
int fcntl (int s, int cmd, long arg)
```

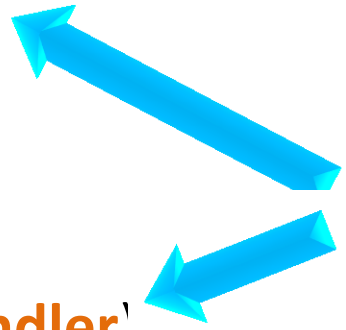
Exemplu:

```
int sd = socket(PF_INET, SOCK_STREAM, 0);
fcntl (sd, F_SETFL, O_ASYNC);  /* setarea asincrona  I/O */
```

Multiplexare I/O| solutii

- Utilizarea alarmelor

```
while(…){
    signal (SIGALRM, alarmHandler);
    alarm (MAX_TIME);
    read (0,…);

    signal (SIGALRM, alarmHandler);
    alarm (MAX_TIME);
    read (tcpsock,…);...
}
```



Functie scrisa de programator

Multitasking

Servere concurente – per-client process

Servere concurente pre-forked

- Se creeaza un numar de procese copil imediat la initializare, fiecare proces liber interactionind cu un anumit client

Servere concurente pre-threaded

- Ca mai sus, dar se folosesc *thread*-uri (fire de executie) in locul proceselor (vezi POSIX *threads* –pthread.h)
- Exemplu: serverul Apache

Probleme:

- Numarul de clienti mai mare decit numarul de *procese/thread*-uri
- Numarul de *procese/thread*-uri prea mare fata de numarul de clienti
- *OS overhead*

..... (curs viitor)

Multiplexare I/O| solutii

Probleme care apar:

- Folosind apeluri **neblocante**, se utilizeaza intens procesorul
- Pentru **alarm(),** care este valoarea optima a constantei **MAX_TIME?**

Multiplexarea I/O|select()

- Permite utilizarea apelurilor blocante pentru un set de descriptori (fisiere, pipe-uri, socket-uri,…)
- Suspenda programul pana cand descriptori din liste sunt pregatiti de operatii de I/O

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
  int select (int nfds,
              fd_set *readfds,
              fd_set *writefds,
              fd_set *exceptfds,
              struct timeval *timeout);
```

Valoarea maxima a descript. plus 1

Multimea descriptorilor de citire, scriere, exceptie

Timpul de asteptare

Multiplexarea I/O|select()

Manipularea elementelor multimii de descriptori (tipul **fd_set**) se realizeaza folosindu-se macrourile:

FD_ZERO (fd_set *set);	Sterge multimea de descriptori din set.
FD_SET (int fd, fd_set *set);	Adauga descriptorul fd in multimea set.
FD_CLR (int fd, fd_set *set);	Sterge descriptorul fd din multimea set.
FD_ISSET(int fd, fd_set *set);	Testeaza daca descriptorul fd apartine sau nu multimii set.

Multiplexarea I/O|select()

Pentru timpul de asteptare se foloseste structura definita in **sys/time.h:**

```
struct timeval {
   long tv_sec;/* secunde*/
   long tv_usec;/* microsecunde*/
   }
```

- Daca timeout este NULL, select() va returna imediat
- Daca timeout este !=0 specifica intervalul de timp in care select() va astepta

Multiplexarea I/O|select()

Un descriptor de socket este gata de citire daca:

- Exista octeti receptionati in *buffe*r-ul de intrare (se poate face read() care va returna >0)
- O conexiune TCP a receptionat FIN (read() returneaza 0)
- *Socket*-ul e *socket* de ascultare si nr. de conexiuni complete este nenul (se poate utiliza accept())
- A aparut o eroare la *socket* (read() returneaza –1, cu errno setat) – erorile pot filtratate via getsockopt() cu optiunea SO_ERROR

Multiplexarea I/O|select()

Un descriptor de *socket* este gata de scriere daca:

- Exista un numar de octeti disponibili in buffer-ul de scriere (write() va returna > 0)
- Conexiunea in sensul scrierii este inchisa

  (incercarea de write() va duce la generarea SIGPIPE)
- A aparut o eroare de scriere (write() returneaza –1, cu errno setat) – erorile pot fitratate via getsockopt() cu optiunea SO_ERROR

Multiplexarea I/O|select()

- Un descriptor de socket este gata de exceptie daca:
  - Exista date *out-of-band* sau *socket*-ul este marcat ca *out-of-band* (curs viitor optional☺)
  - Daca capatul *remote* a socket-ului TCP a fost inchis in timp ce date erau pe canal; urmatoarea operatie de read/write va intoarce ECONNRESET

Multiplexarea I/O|select()

select() poate returna

- Numarul descriptorilor pregatiti pentru o operatiune de citire, scriere sau exceptie
- Valoarea 0 – timpul s-a scurs, nici un descriptor nu este gata
- Valoarea –1 in caz de eroare

Utilizarea lui **select()** – pasii generali:

- Declararea unei variabile de tip **fd_set**
- Initializarea multimii cu **FD_ZERO()**
- Adaugarea cu **FD_SET()** a fiecarui descriptor dorit a fi monitorizat
- Apelarea primitivei **select()**
- La intoarcerea cu succes, verificarea cu **FD_ISSET()** a descriptorilor pregatiti pentru I/O

Demo
Exemplu de utilizare a primitivei **select()**

Socket-uri BSD|utilizare

- Serviciile Internet (serviciile folosesc *socket*-urile pentru comunicarea intre *host*-uri *remote*)
  - Exemplu de aplicatii distribuite
    - World Wide Web
    - Accesul *remote* la o baza de date
    - Distribuirea de *task*-uri mai multor *host*uri
    - Jocuri on-line
    - …

Socket-uri BSD|critici

API-ul bazat pe socket-uri BSD are o serie de limitari:

- Are o complexitate ridicata, deoarece a fost proiectata sa suporte familii de protocoale multiple (dar rar folosite in practica)
- Nu este portabila (unele apeluri/tipuri au alte denumiri/reprezentari pe alte platforme; numele fisierelor - *antet.h* depind de sistem)
- Exemplu: la WinSock descriptorii de *socket* sunt pointeri, in contrast cu implementarile Unix care folosesc intregi

Rezumat

- … sa ne amintim: client/server TCP iterativ - primitive
- Modelul client/server UDP
- Primitive I/O
- Aspecte de programare avansata Internet
- API-ului *socket* – discutii si critici

Intrebari?

Programarea in retea (III)

Lenuta Alboaie

adria@info.uaic.ro

Cuprins

- Primitive I/O - discutii
- Server concurent UDP
- TCP sau UDP – aspecte
- Instrumente
- Alternative de proiectare si implementare al modelului client/server TCP

Primitive I/O

- Citire de date
  - read() / recv() / **readv()** / recvfrom()/ **recvmsg()**
- Trimitere de date
  - write() / send() / **writev()** / sendto()/ **sendmsg()**

Alte primitive| I/O

```
#include <sys/uio.h>
ssize_t readv (int filedes, const struct iovec *iov, int iovcnt);
ssize_t writev (int filedes, const struct iovec *iov, int iovcnt);
        struct iovec
        {
                void *iov_base; /* adresa de start a bufferului */
                size_t iov_len;   /* dimensiunea bufferului */
        };
```

Mai generale decit *read*()/*write*(), ofera posibilitatea de a transmite date aflate in zone necontigue de memorie

Cele 2 apeluri returneaza la execuție normala lungimea transferului in octeti

Alte primitive| I/O

```
#include <sys/socket.h>
ssize_t recvmsg (int sockfd, struct msghdr *msg, int flags);
ssize_t sendmsg (int sockfd, struct msghdr *msg, int flags);
```

Ambele functii au majoritatea optiunilor incorporate in structura *msghrd*

Cele mai generale functii I/O; apelurile read/readv/recv/recvfrom pot fi inlocuite de recvmsg

Cele 2 apeluri returnează la execuție normală lungimea transferului în octeți; -1 in caz de eroare

Alte primitive| I/O

Comparatie intre primitivele I/O:

Function	Orice descriptor	Doar descriptor de socket	Un singur read/write buffer	Scatter/ gather read/write	Flag-uri optionale	Adresa nodului *peer*
read, write	●		●			
readv, writev	●			●		
recv, send		●	●		●	
recvfrom, sendto		●	●		●	●
recvmsg, sendmsg		●		●	●	●

Server UDP | situatii

Majoritatea serverelor UDP sunt iterative

- Server UDP care citeste cererea clientului, proceseaza cererea, trimite raspunsul si termina cu acel client
- Daca este nevoie de schimb de datagrame multiple cu clientul?

Server UDP concurent

- daca elaborarea raspunsului ia mult timp serverul poate crea (*fork*()) un proces copil care va rezolva cererea

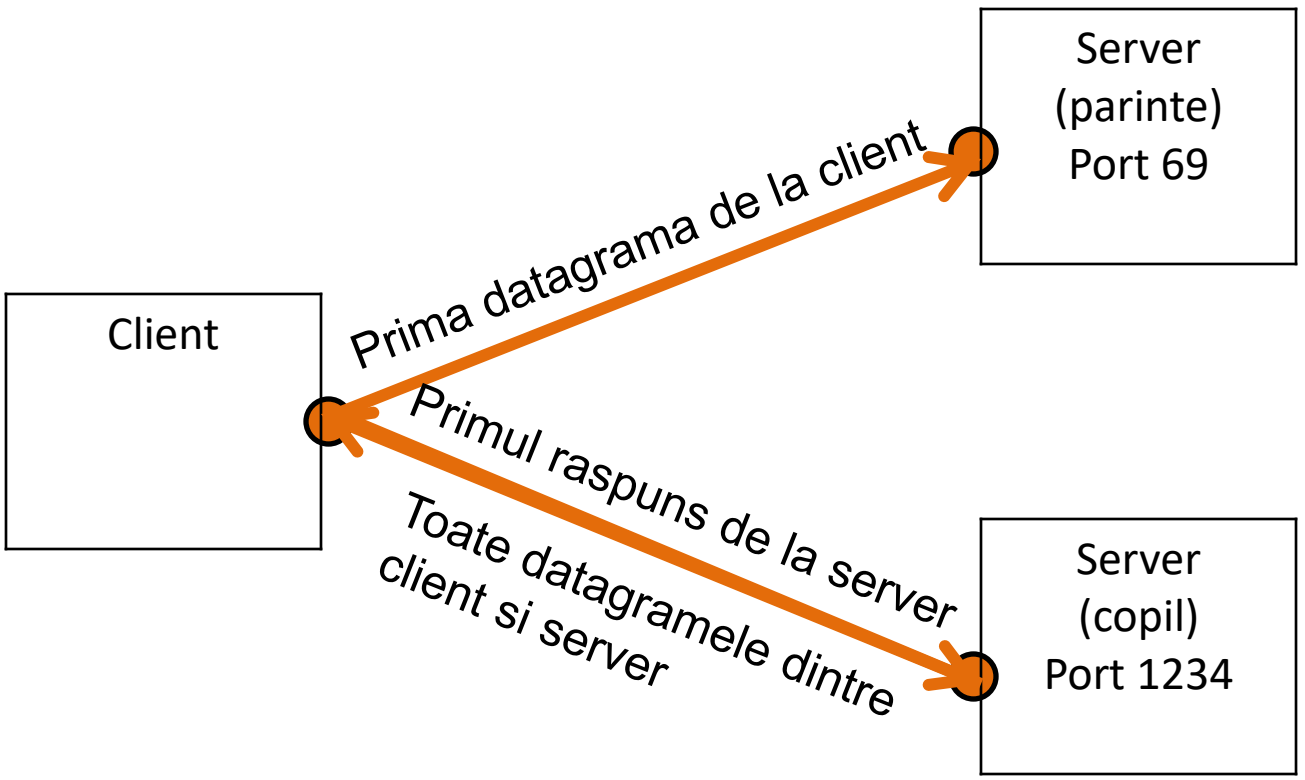
Server UDP | situatii

Server UDP concurent

- Server UDP care schimba datagrame multiple cu un client
  - Problema: Doar un numar de port este cunoscut de client ca fiind un port “*wellknown*”
  - Solutia: serverul creaza un socket nou pentru fiecare client, si il ataseaza la un port “*efemer*”, si utilizeaza acest socket pentru toate raspunsurile.
    - Obligatoriu clientul trebuie sa preia din primul raspuns al serverului noul numar de port si sa faca urmatoarele cereri la acel port
  - Exemplu: TFTP - Trivial File Transfer Protocol

Server concurent UDP

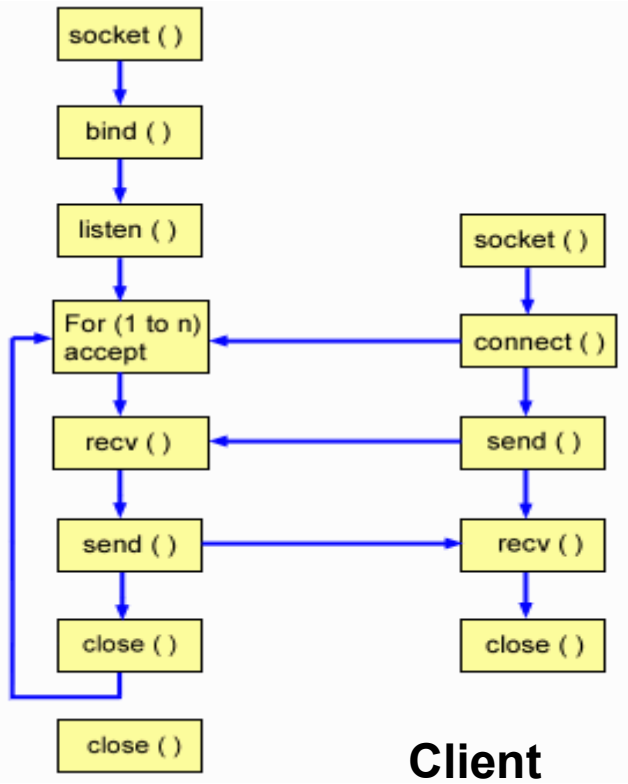
– TFTP utilizeaza UDP si portul 69

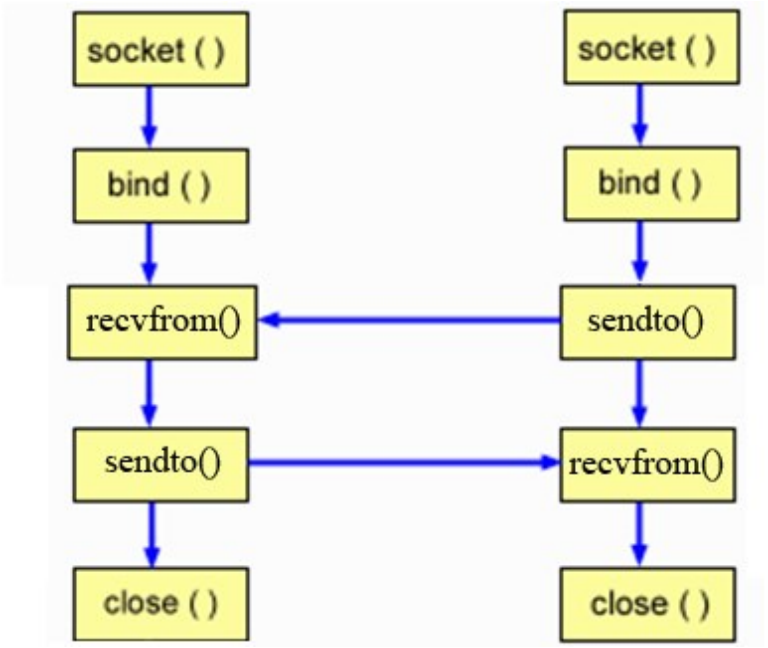


Pasii: socket(), bind() la portul *well-known 69, recvfrom(), fork(), alt recvfrom(),…*

Pasii: socket(), bind() la un port *1234,* procesarea cererii clientului, Schimb de datagrame cu clientul prin noul socket

TCP sau UDP - discutii





Model server/client TCP

Model client/server UDP

TCP sau UDP – discutii

Aspecte privind utilizarea UDP:

- UDP suporta broadcasting si multicasting
- UDP nu are nevoie de un mecanism de stabilire a conexiunii
- Minimul de timp necesar unei tranzactii UDP cerere-raspuns este: RRT(*Round Trip* Time) + SPT (*server processing time*)

Aspecte privind utilizarea TCP:

- TCP suporta point-to-point
- TCP este orientat conexiune
- Ofera siguranta si asigura transmiterea in ordine a datelor;
- Ofera mecanisme de control al fluxului si control al congestiei
- Minimul de timp necesar unei tranzactii TCP cerere-raspuns daca se creaza o noua conexiune este: 2 *RRT + SPT

TCP sau UDP – discutii

[http://www.skullbox.net]

TCP sau UDP – discutii

Folosirea UDP , respectiv TCP – recomandari

- UDP *trebuie* folosit pentru aplicatii multicast sau broadcast

  Controlul erorilor trebuie (eventual) adaugat la nivelul serverului sau clientului

- UDP *poate* fi folosit pentru operatii de cerere-raspuns simple; erorile trebuie tratate la nivelul aplicatiei

Exemple: streaming media, teleconferinte, DNS

TCP sau UDP – discutii

Folosirea UDP , respectiv TCP – recomandari

- TCP *trebuie* folosit pentru *bulk data transfer* (e.g. transfer de fisiere)
  - S-ar putea folosi UDP? → Reinventam TCP la nivelul aplicatiei!

Exemple: HTTP (Web), FTP (File Transfer Protocol), Telnet, SMTP

Instrumente

- Multe sisteme UNIX ofera facilitatea de *"system call tracing"*

adria@ubu: ~/S6

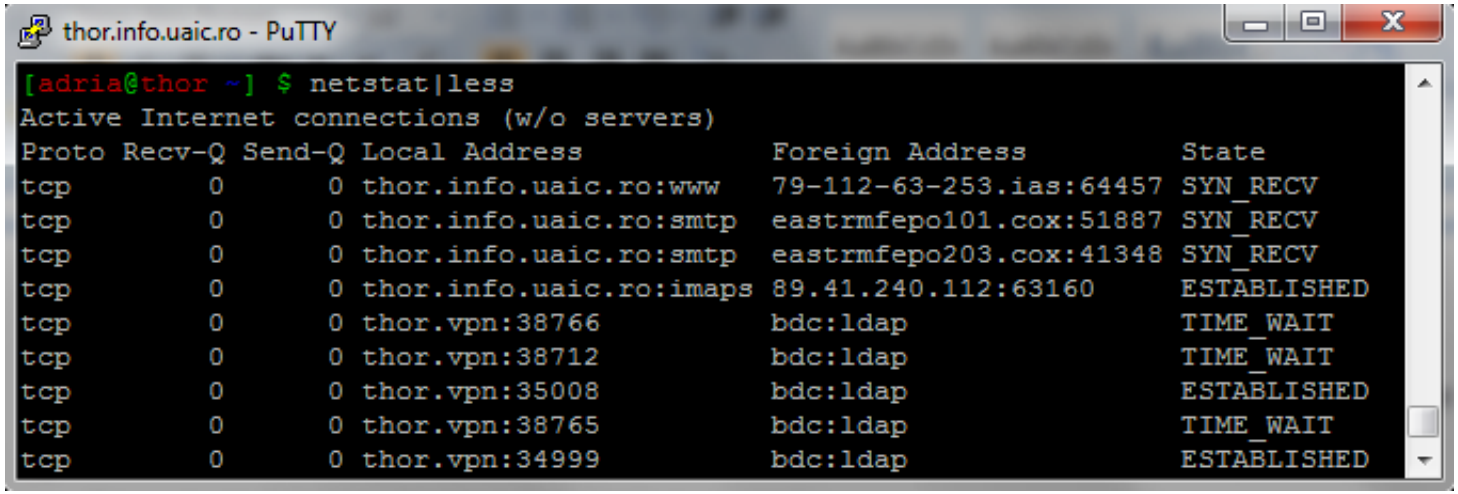
```
  I A  test.c (Modifi  Row 8    Col 28   8:15
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

  char *sir=NULL;
  printf("program de debugs: ");
  //sir = (char *) malloc(100*sizeof(char));
  fgets(sir, 1024, stdin);
  printf(sir);
  return 1;
}
```

strace

```
write(1, "program de debug\n"..., 17program de debug
)    = 17
fstat64(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 3), ...}) = 0
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb7
fdb000
read(0, 0xb7fdb000, 1024)               = ? ERESTARTSYS (To be restarted)
--- SIGWINCH (Window changed) @ 0 (0) ---
read(0, Test in saptamina 6
"Test in saptamina 6\n"..., 1024) = 20
--- SIGSEGV (Segmentation fault) @ 0 (0) ---
+++ killed by SIGSEGV +++
```

Instrumente

- Programe de test de dimensiuni reduse
- Instrumente:
  - **tcpdump** – majoritatea versiunilor de Unix
    - Ofera informatii asupra pachetelor din retea
    - http://www.tcpdump.org/
  - **snoop** – Solaris 2.x
  - **lsof**
    - Identifica ce procese au un socket deschis la o adresa IP sau port specificat
  - **netstat**



Instrumente

- Instrumente:
  - **tcptrack**

```
Client                  Server                  State        Idle A Speed
172.23.195.11:48328     67.39.222.44:22         ESTABLISHED  0s     38 KB/s
172.23.195.11:48646     196.30.80.10:80         ESTABLISHED  1s     30 KB/s
172.23.195.11:48661     64.37.246.17:80         ESTABLISHED  0s     387 B/s
172.23.195.11:48620     216.239.39.99:80        RESET        2s     0 B/s
128.230.225.95:3531     172.23.195.10:1220      ESTABLISHED  5s     0 B/s
172.23.195.11:48621     216.239.39.99:80        ESTABLISHED  7s     0 B/s
172.23.195.11:48606     64.233.167.99:80        ESTABLISHED  10s    0 B/s
172.23.195.11:48014     67.39.222.44:22         ESTABLISHED  16s    0 B/s
172.23.195.11:47988     67.39.222.44:22         ESTABLISHED  18s    0 B/s

TOTAL                                                              69 KB/s
Connections 1-9 of 9                                  Unpaused  Sorted
```

Alternative de proiectare si implementare al modelului client/server TCP

Fire de executie| Necesitate

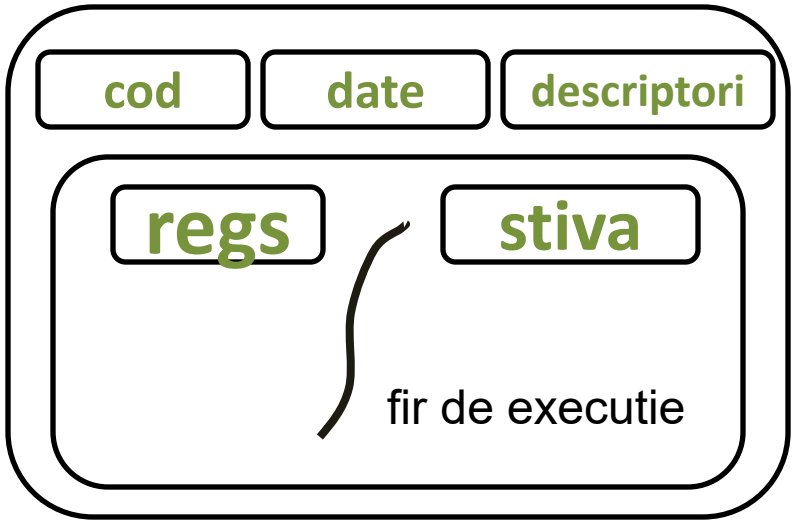
- fork() poate fi un mecanism costisitor
  - implementarile curente folosesc mecanismul *copy-on-write*
- IPC (Inter-Process Comunication) necesita trimiterea informatiei intre parinte si copil *dupa* fork()

Fire de executie| Caracteristici

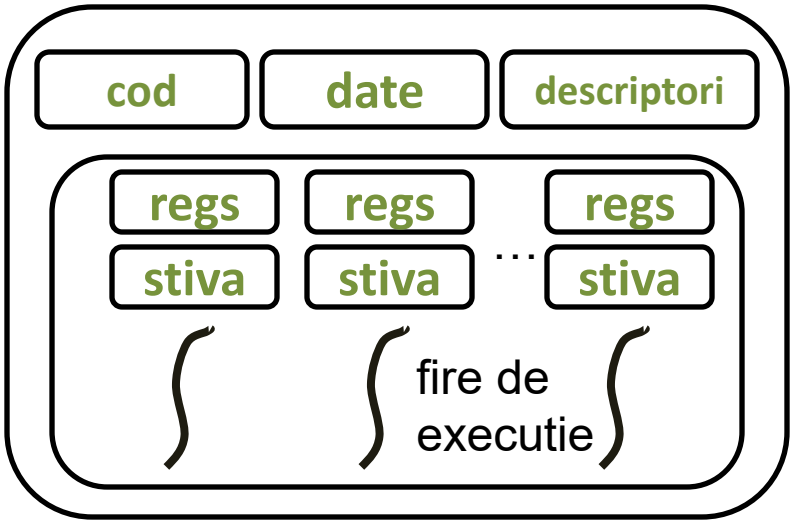
- Firele de executie (*threads*) sunt numite si

lightweight processes (LWP)

- Pot fi vazute ca un program aflat in executie fara spatiu de adresa proprie



Procese cu un fir de executie



Procese cu mai multe fire de executie

Procese, Fire de executie| Comparatii

- Exemplu: Costurile asociate crearii si managementului proceselor (50.000) este mai mare decat in cazul firelor de executie(50.000)

Platform	fork()		pthread_create()	
	user	sys	user	sys
AMD 2.4 GHz Opteron (8cpus/node)	2.2	15.7	0.3	1.3
IBM 1.9 GHz POWER5 p5-575 (8cpus/node)	30.7	27.6	0.6	1.1
IBM 1.5 GHz POWER4 (8cpus/node)	48.6	47.2	1.0	1.5
INTEL 2.4 GHz Xeon (2 cpus/node)	1.5	20.8	0.7	0.9
INTEL 1.4 GHz Itanium2 (4 cpus/node)	1.1	22.2	1.2	0.6

[https://computing.llnl.gov/tutorials/pthreads/]

Fire de executie| Implementare

- **Pthreads** (POSIX Threads) standard ce defineste un API pentru crearea si manipularea firelor de executie
  - FreeBSD
  - NetBSD
  - GNU/Linux
  - Mac OS X
  - Solaris
- Pthread API pentru Windows – pthreads-w32

Fire de executie| Primitive de baza

```
#include <pthread.h>
int pthread_create(
        pthread_t *tid,
        const pthread_attr_t *attr,
        void *(*func) (void *),
        void *arg);
```

pthread_t (-> adeseori un unsigned int) (Identificatorul *thread*-ului)

Structura ce specifica atributele noului fir creat (e.g. dimensiunea stivei, prioritatea, NULL = comportamentul implicit)

Referinta la functia ce va fi executata de *thread*

Argumentul catre *thread* ce este transmis functiei

Returneaza: 0 in caz de succes

o valoare Exxx pozitiva in caz de eroare

Fire de executie| Primitive de baza

```
#include <pthread.h>
int pthread_join(
        pthread_t *tid,
        void **status );
```

Identificatorul *thread*-ului

… va stoca valoarea de *return* a *thread*-ului (un pointer la un obiect)

- Realizeaza asteptarea terminarii unui anumit *thread*

Returneaza: 0 in caz de succes

o valoare Exxx pozitiva in caz de eroare

Fire de executie| Primitive de baza

```
#include <pthread.h>
pthread_t pthread_self();
```

Identificatorul *thread*-ului

Returneaza: ID-ul *thread*-ului care a apelat primitiva

Fire de executie| Primitive de baza

```
#include <pthread.h>
int pthread_detach(pthread_t tid);
```

Identificatorul *thread*-ului

Thread-urile pot fi:

- *joinable*: cind thread-ul se termina, ID-ul si codul de iesire sunt pastrate pina cand se apeleaza pthread_join() <– comportament implicit
- *detached:* cand thread-ul se termina toate resursele sunt eliberate

Returneaza: 0 in caz de succes

o valoare Exxx pozitiva in caz de eroare

Exemplu: pthread_detach(pthread_self());

Fire de executie| Primitive de baza

#include <pthread.h>

void **pthread_exit**(void***status**);

- Terminarea unui *thread*

Thread-urile se pot termina:

- Functia executata de *thread* returneaza (Obs. Valoarea de return este void * si va reprezenta codul de iesire a *thread*-ului)
- Daca functia *main* a procesului returneaza sau oricare din *thread*-uri a apelat *exit*(), procesul se termina

Fire de executie|Exemplu

Exemplu de server TCP concurent care nu foloseste *fork*() pentru a deservi clientii, ci foloseste *thread*-uri

Obs. Compilarea: **gcc –lpthread server.c sau**

gcc server.c –lpthread

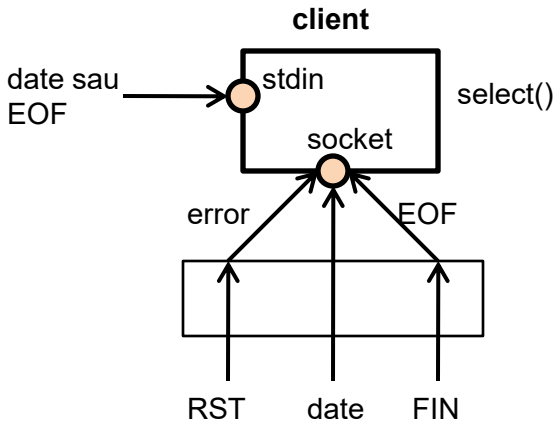
DEMO

Alternative de proiectare al modelului client/server TCP

- **Client TCP - modelul uzual**
  - Aspecte:
    - Atat timp cat este blocat asteptind date de la utilizator, nu sesizeaza evenimentele de retea (e.g. *peer close()*)
    - Functioneaza in modul "*stop and wait*"
    - "*batch processing*"

Alternative de proiectare al modelului client/server TCP

- **Client TCP – utilizind select()**
  - Clientul este notificat de evenimentele din retea in timp ce asteapta date de intrare de la utilizator



Daca *peer*-ul trimite date, *read*() returneaza o valoare >0;

Daca *peer*-ul TCP trimite FIN, *socket*-ul devine "citibil" si *read*() intoarce 0;

Daca *peer*-ul trimite RST (*peer*-ul a cazut sau a *rebootat*), *socket*-ul devine "citibil" si *read*() intoarce -1;

Aspecte:

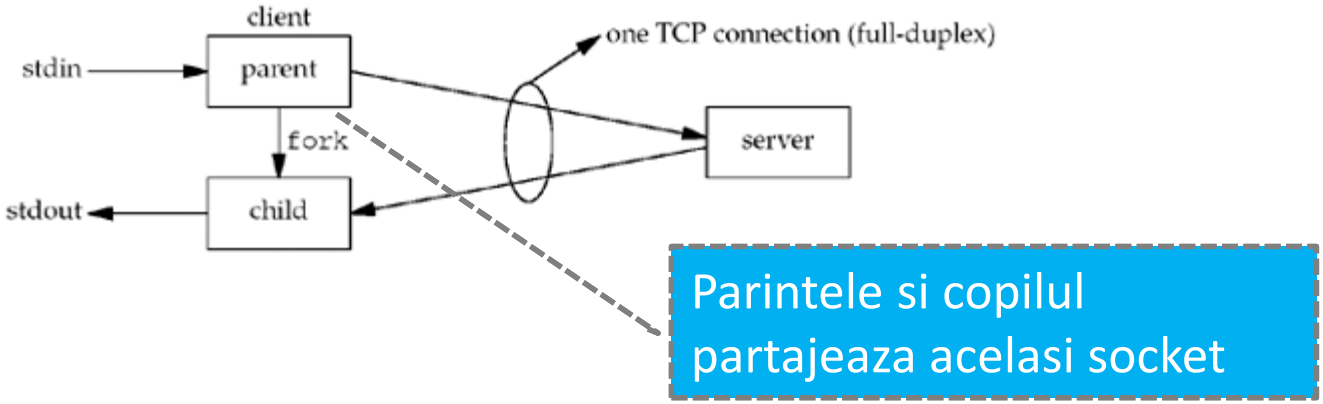
- Apelul *write*() poate fi blocant daca *buffer*-ul de la *socket*-ul emitator este plin

Alternative de proiectare al modelului client/server TCP

- **Client TCP** – **utilizind select() si operatii I/O neblocante**
  - Aspecte:
    - Implementare complexa => cand sunt necesare operatii I/O neblocante se recomanda utilizare de procese (*fork*()) sau de *thread*-uri (vezi *slide*-urile urmatoare)

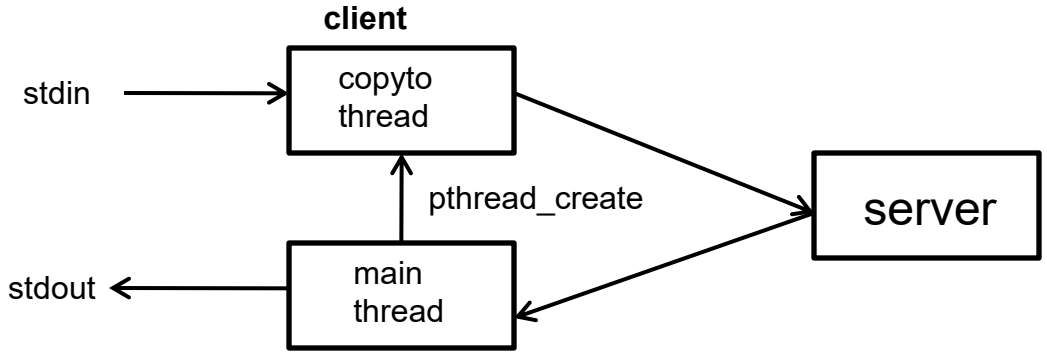
Alternative de proiectare al modelului client/server TCP

- **Client TCP – utilizind fork()**
  - Mecanismul de functionare:
    - exista doua procese
      - Un proces face managementul datelor client-server
      - Un proces face managementul datelor server-client



Alternative de proiectare al modelului client/server TCP

- **Client TCP – utilizind pthread()**
  - Mecanismul de functionare:
    - exista doua fire de executie
      - Un fir de executie face managementul datelor client-server
      - Un fir de executie face managementul datelor server-client



Alternative de proiectare al modelului client/server TCP

- Comparatie a timpilor de executie a clientilor TCP cu arhitecturile client discutate

Tip client TCP	Timp executie (secunde)
Modelul uzual (stop-and-wait)	…
Modelul folosind select si I/O blocante	12.3
Modelul folosind select si I/O neblocante	6.9
Modelul folosind fork()	8.7
Modelul folosind thread-uri	8.5

- Obs. Masuratoarea s-a realizat folosindu-se comanda *time* pentru implementari **client**/server *echo*

[Unix Network Programming, R. Stevens B. Fenner, A. Rudoff - 2003

Alternative de proiectare al modelului client/server TCP

- **Server TCP – iterativ**
  - Se realizeaza procesarea completa a cererii clientului inainte de a deservi urmatorul client

  Aspecte:

  - Sunt destul de rar intilnite in implementarile reale
  - Un astfel de server serveste foarte rapid un client

Alternative de proiectare al modelului client/server TCP

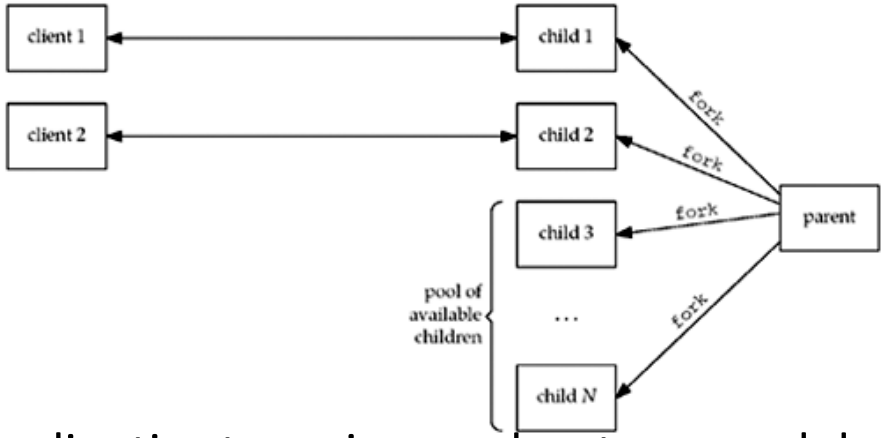
- **Server TCP** – cate un proces copil pentru fiecare client
  - Serverul deserveste clintii in mod simultan
  - Este des intilnit in practica
  - Exemplu de mecanism folosit pentru distribuirea cererilor: *DNS round robin*

Aspecte:

- Crearea fiecarui copil (fork()) pentru fiecare client consuma mult timp de CPU

Alternative de proiectare al modelului client/server TCP

- **Server TCP** – ***preforking;*** **fara protectie pe accept()**
  - Serverul creaza un numar de procese copil cand este pornit, si apoi acestia sunt gata sa serveasca clientii



Aspecte

- Daca numarul de clienti este mai mare decat numarul de procese copil disponibile, clientul va resimti o “degradare” a raspunsului in raport cu factorul timp
- Acest timp de implementare merge pe sisteme ce au *accept*() primitiva de sistem

Alternative de proiectare al modelului client/server TCP

- **Server TCP** – ***preforking;*** **cu blocare pentru protectia *accept*()**

  Implementare:

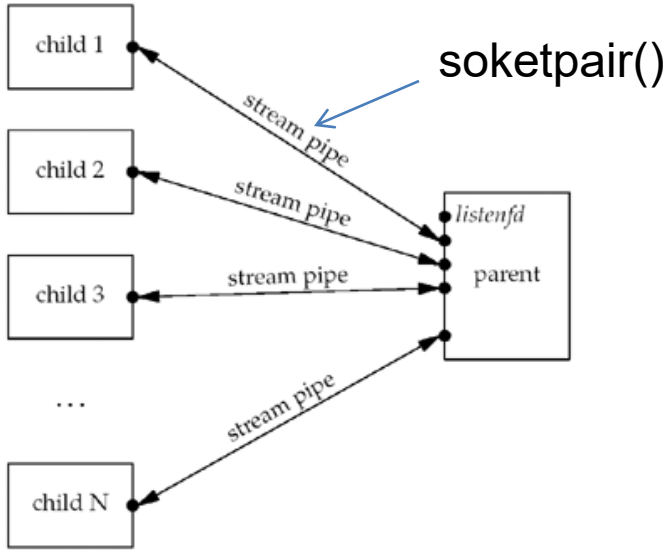
  – Serverul creaza un numar de procese copil cand este pornit, si apoi acestia sunt gata sa serveasca clientii

  – Se foloseste un mecansim de blocare (e.g. *fcntl*()) a apelului primitivei *accept*(), si doar un singur proces la un moment dat va putea apela *accept*(); procesele ramase vor fi blocate pina vor putea obtine accesul

- Exemplu: Apache (http://www.apache.org) foloseste tehnica de *preforking*

Alternative de proiectare al modelului client/server TCP

- **Server TCP** – ***preforking;*** **cu "transmiterea" socket-ului conectat**
  Implementare:
  - Serverul creaza un numar de procese copil cand este pornit, si apoi acestia sunt gata sa serveasca clientii
  - Procesul parinte este cel care apeleaza *accept*() si "transmite" *socket*-ul conectat la un copil



Aspecte:

Procesul parinte trebuie sa aiba evidenta actiunilor proceselor fii => o complexitate mai mare a implementarii

Alternative de proiectare al modelului client/server TCP

- **Server TCP – cate un *thread* pentru fiecare client**

  Implementare:

  Thread-ul principal este blocat la apelul lui accept() si de fiecare data cind este acceptat un client se creaza (*pthread_create*()) un *thread* care il va servi

DEMO (Slide 28)

Aspecte:

Aceasta implementare este in general mai rapida decat cea mai rapida versiune de server TCP *preforked*

Alternative de proiectare al modelului client/server TCP

- **Server TCP** – ***prethreaded;*** **cu blocare pentru protectia *accept*()**

  Implementare:

  – Serverul creaza un numar de *thread*-uri cand este pornit, si apoi acestea sunt gata sa serveasca clientii

  – Se foloseste un mecansim de blocare (e.g. *mutex lock*) a apelului primitivei *accept*(), si doar un singur *thread* la un moment dat va apela *accept*();

  Obs. *Thread*-urile nu vor fi blocate in apelul *accept*()

DEMO

Alternative de proiectare al modelului client/server TCP

- **Server TCP** – ***prethreaded;*** **cu "transmiterea" socket-ului conectat**

  Implementare:

  Serverul creaza un numar de *thread*-uri cand este pornit, si apoi acestia sunt gata sa serveasca clientii

  Procesul parinte este cel care apeleaza *accept*() si "transmite" *socket*-ul conectat la un *thread* disponibil

  Obs. Deoarece *thread*-urile si descriptorii sunt in cadrul aceluiasi proces ,"transmiterea" *socket*-ului conectat inseamna de fapt ca *thread*-ul vizat sa stie numarul descriptorului

Alternative de proiectare al modelului client/server TCP| Discutii

- Daca serverul nu este foarte solicitat, varianta traditionala de server concurent (un *fork*() *per* client) este utilizabila
- Crearea unei multimi de procese copil (eng. *pool of children*) sau multimi de *thread*-uri (eng. *pool of threads*) este mai eficienta din punct de vedere al factorului timp; trebuie avut grija la monitorizarea numarului de procese libere, la cresterea sau descresterea acestui numar a.i. clientii sa fie serviti in mod dinamic
- Mecanismul prin care procesele copil sau *thread*-urile pot apela *accept*() este mai simplu si mai rapid decit cel in care *thread-ul* principal apeleaza *accept*() si apoi “transmite” descriptorul proceselor copil sau *thread*-urilor.
- Aplicatiile ce folosesc *thread*-uri sunt in general mai rapide decat daca utilizeaza procese, dar alegerea depinde de ce ofera SO sau de specificul problemei

Rezumat

- Primitive I/O - discutii
- Server concurent UDP
- TCP sau UDP – aspecte
- Instrumente
- Alternative de proiectare si implementare al modelului client/server TCP

Bibliografie

- UNIX Network Programming: The sockets networking API, W. Richard Stevens, Bill Fenner, Andrew M. Rudoff
- The Illustrated Network: How TCP/IP Works in a Modern Network (The Morgan Kaufmann Series in Networking), Walter Goralski

Intrebari?

Programarea in retea
- OOB-

Lenuta Alboaie

adria@info.uaic.ro

Cuprins

- Trimiterea si receptarea datelor in regim *out-of-band*

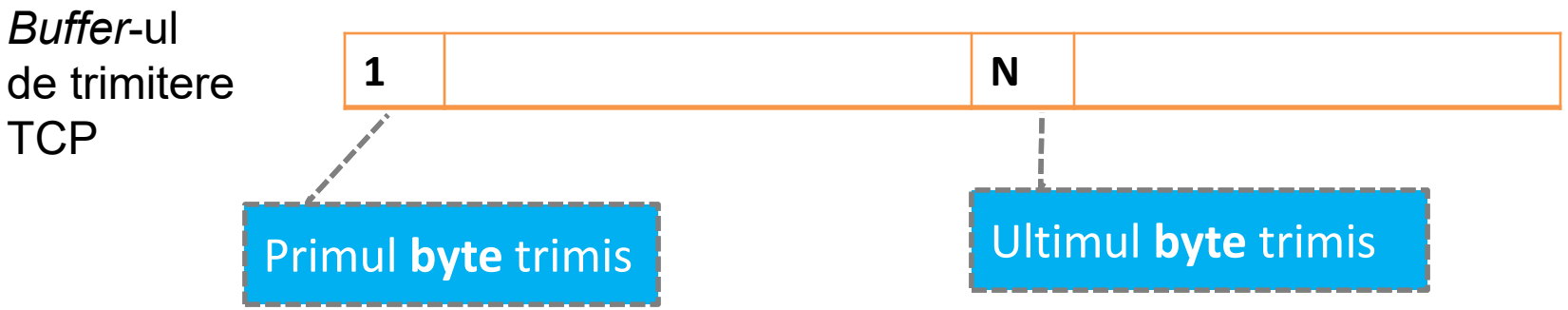
Trimiterea si receptarea datelor in regim out-of-band

- Ideea: in timpul unei conexiuni cind sunt transmise date ("**in-band data**"), si daca la un capat se intimpla "*ceva*" acesta va dori sa transmita rapid celuilalt *peer* in regim de prioritate, o notificare ("**out-of-band data**")
- Mecanismul de realizare
  - Se utilizeaza bitul URG setat in antetul TCP
  - Antetul TCP contine un camp indicind locatia datelor urgente ce trebuie trimise
  - **Trimiterea datelor OOB**:
    - Pentru a expedia un octet urgent intr-un flux de date putem utiliza **send()**:

      send (sd, buff, 1, **MSG_OOB**);

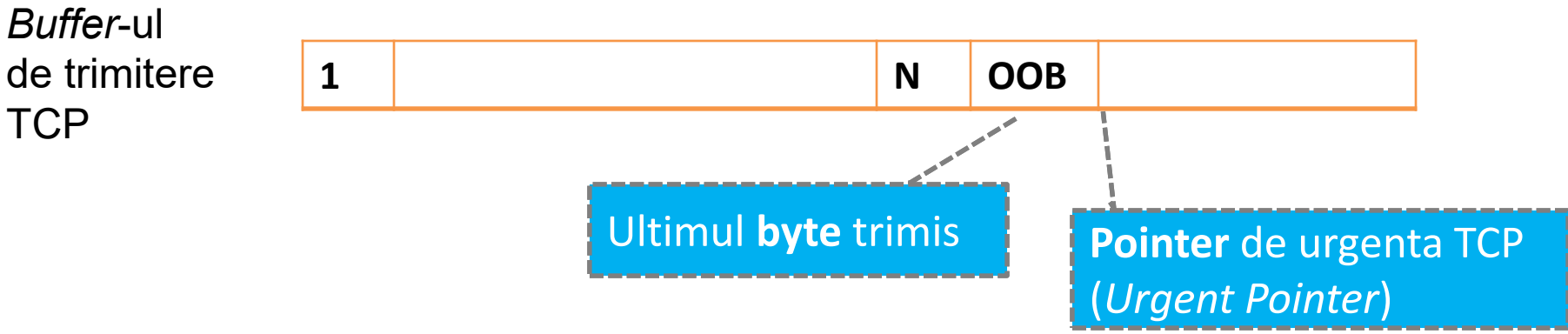
Trimiterea si receptarea datelor in regim out-of-band

Trimiterea unui octet OOB



send (sd, “a”, 1, **MSG_OOB**)

Transmiterea unui *byte* OOB



Trimiterea si receptarea datelor in regim out-of-band

Receptionarea datelor OOB:

- Se genereaza semnalul **SIGURG**
- Apelul **select()** va modifica lista descriptorilor de exceptie

  Citirea datelor OOB:

  - Daca socketul nu are asociata optiunea SO_OOBINLINE, mesajul OOB este plasat intr-un buffer special (*out-of-band buffer)*; citirea bufferului se poate realiza cu **recv()** sau **recvmsg()** setind **MSG_OOB**
  - Daca socket-ul are asociata optiunea SO_OOBINLINE, mesajul OOB este plasat in *buffer*-ul normal de primire;
- Procesul va sti ca a ajuns la *acel octet* in functie de valoarea *out of-band-mark* asociata conexiunii

Trimiterea si receptarea datelor in regim out-of-band

Primitiva sockatmark()

- Cind se primesc date OOB, se face asocierea cu *out-of-band-mark,* reprezentind pozitia datelor OOB in streamul de date trimis de emitator
- **sockatmark**() asigura ca procesul receptor sa determine daca conexiunea are sau nu asociata marca *out-of-band*

```
#include <sys/socket.h>
int sockatmark (int sockfd) ;
```

Returneaza: 1 – este *out-of-band mark*,
0 – nu este *out-of-band mark*
–1 on error

Trimiterea si receptarea datelor in regim out-of-band

Primitiva sockatmark() – observatii

- *out-of-band mark* se aplica indiferent daca procesul receptor primeste datele in mod *out-of-band inline* (*socket*-ul cu optiunea SO_OOBINLINE) sau in mod out-of-band (flagul MSG_OOB)
- Este implementata folosindu-se ioctl() si SIOCATMARK

```
int value;
error = ioctl(tcp_socket, ioctl_type, &value);
```

Trimiterea si receptarea datelor in regim out-of-band

Primitiva sockatmark() – discutii & exemplu

- Daca s-a primit date in mod *OOB inline* sockatmark() intoarce true daca urmatorul octet ce poate fi citit a fost trimis cu flagul MSG_OOB
- Daca *socket*-ul nu are asociata optiunea SO_OOBINLINE, sokatmark() intoarce *true* daca urmatorul octet ce poate fi citit este primul octet care a fost trimis dupa data OOB
- Operatia de citire se opreste in functie de *out of-band-mark*
  - Exemplu:

    Daca sunt 100 de octeti in buffer dar doar 5 octeti pina la *out of-band-mark,* chiar daca procesul citeste 100 de octeti el va primi initial doar primii 5 octeti

Trimiterea si receptarea datelor in regim out-of-band

Erori posibile

- Se asteapta citirea de date OOB, dar ele nu au fost inca trimise – se returneaza EINVAL
- Procesul a fost notificat ca va primi date OOB (via select() sau SIGURG), el incearca sa le citeasca dar ele inca nu au ajuns – se returneaza EWOULDBLOCK
- Se incearca sa se citeasca un acelasi octet OOB de mai multe ori – se returneaza EINVAL
- Daca procesul are setat optiunea SO_OOBINLINE, dar incearca sa citeasca cu flagul MSG_OOB – se returneaza EINVAL

Trimiterea si receptarea datelor in regim out-of-band

- Utilizari:
  - Modalitate de comunicare a celuilalt punct terminal a unei conditii de exceptie chiar si in cazul cind controlul fluxului a oprit emitatorul
  - Pentru a detecta timpuriu erori de comunicare intre client si server (*heart-beat*)

Trimiterea si receptarea datelor in regim out-of-band

Tratare **OOB** prin **SIGURG:**

```
if (listen (sd, 5) == -1)    { … }
…
while (1)
   {…    client = accept (sd, (struct sockaddr *) &from, &len);
         signal(SIGURG, urgHandler);
         fcntl(client, F_SETOWN, getpid()); /*setarea proprietarului socketului conectat */
     for(; ;)    {
          if((n=read(client,msg,sizeof(msg)-1))==0)
          {                    printf("Am primit EOF\n");
                               break;            }
          else
          {
                               msg[n]=0;
                               printf("Am citit %d octeti: %s\n",n, msg);      }
     } … } //while
void urgHandler(int signnr)
{
     int n;                    char buff[100];
     printf("SIGURG e primit\n");
     n=recv(client,buff, sizeof(buff)-1, MSG_OOB);
       buff[n]='\0';
      printf("Am citit %d octet OOB %s\n",n, buff);
}
```

Trimiterea si receptarea datelor in regim out-of-band

Tratare **OOB** folosind primitiva **sokatmark**()

DEMO

- **Emitatorul** trimite 4 octeti de date normale, apoi un octet OOB si inca un octet de date normale
- **Receptorul** nu foloseste SIGURG sau select(); el apeleaza primitiva **sockatmark**()

Rezumat

- Trimiterea si receptarea datelor in regim out-of-band

Bibliografie

- UNIX Network Programming: The sockets networking API, W. Richard Stevens, Bill Fenner, Andrew M. Rudoff
- The Illustrated Network: How TCP/IP Works in a Modern Network (The Morgan Kaufmann Series in Networking), Walter Goralski

Intrebari?

Sistemul numelor de domenii

Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- Domain Name System (DNS)
  - Caracterizare
  - Organizare
  - Configurare
  - Comenzi, Primitive
  - IDN

DNS

- Adresele IP (ex. 85.122.23.145) sunt dificil de memorat
- Se utilizeaza un **sistem al numelor de domenii** pentru a translata adresele IP in nume de domenii si invers
- Numele de domenii se organizeaza in ierarhii

- RFC 1034, 1035, 1123, 2181

DNS| organizare

- Initial: **/etc/hosts** – perechi (nume, IP)
  - Probleme de scalabilitate

- Actual: DNS consta dintr-o schema ierarhica de nume de domenii si a unui sistem de baze de date distribuite pentru implementarea acestei scheme de nume

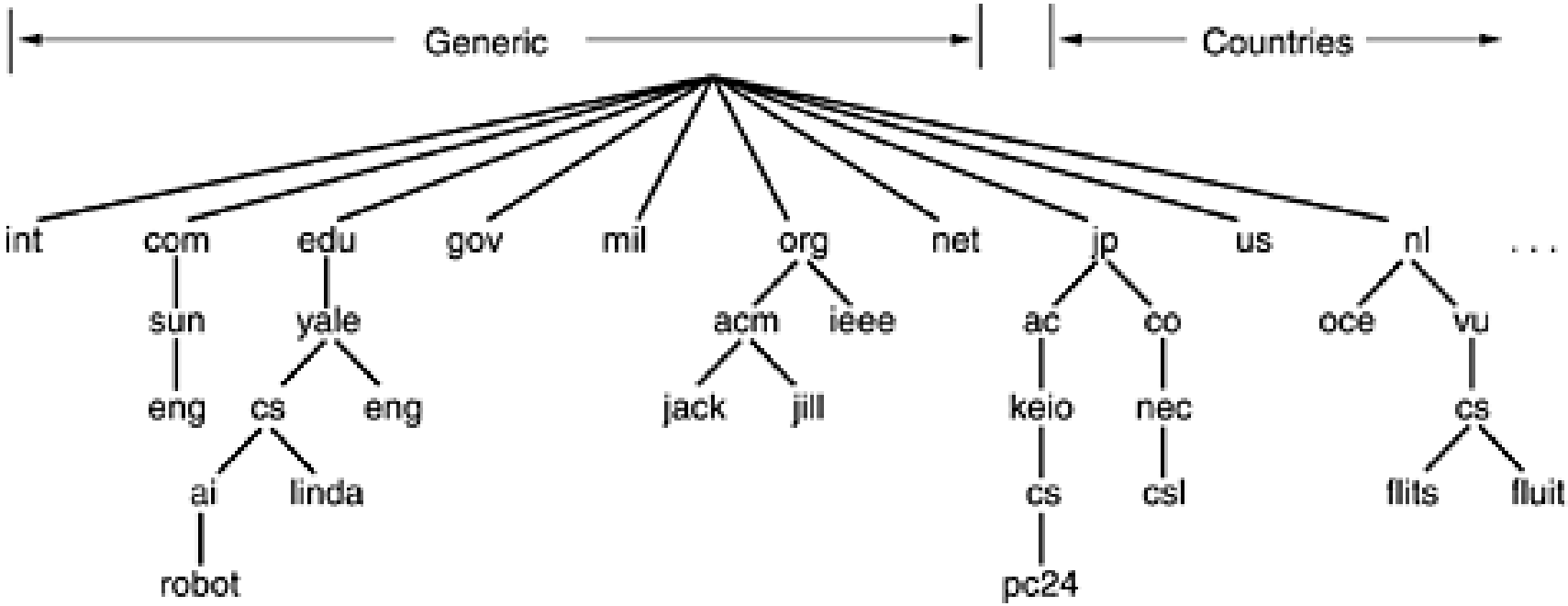


Figura. O portiune a spatiului numelor de domenii in Internet

[Computer Networks, 2003
Andrew S. Tanenbaum]

DNS| Tipuri de domenii

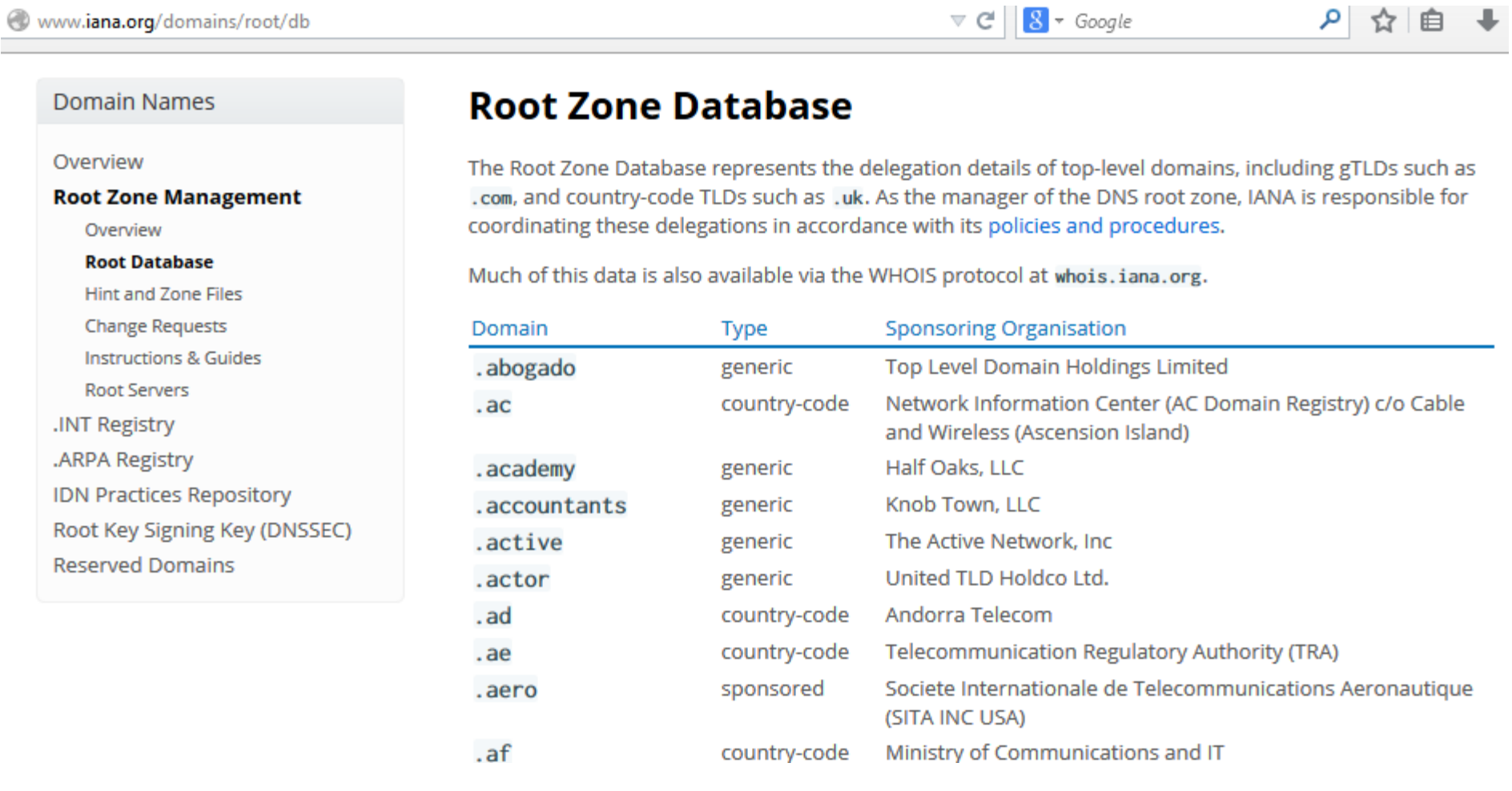
- **Primare** (*Top Level Domains* – TLD)
  - pentru Infrastructura Internet – un singur domeniu **.arpa** ARPA (Address and Routing Parameter Area)
    - *"Changes to the .arpa zone must be coordinated manually with IANA"*
  - State (*ccTLD*) – coduri de state: .ro, .fr, .jp, …
  - IDN ccTLD (*Internationalized Country Code Top-Level Domains*)

    http://例子.测试 http://*example.test*

  - Generice: .biz, .com, .info, .name, .net, .org, .pro
  - Sponsorizate: .aero, .edu, .gov, .int, .jobs, .mil, .tel
  - Rezervate: .example, .invalid, .localhost, .test
  - Pseudo-domenii: .bitnet, .local, .root, .uucp etc.

http://www.iana.org/domains/root/db/

DNS| Tipuri de domenii



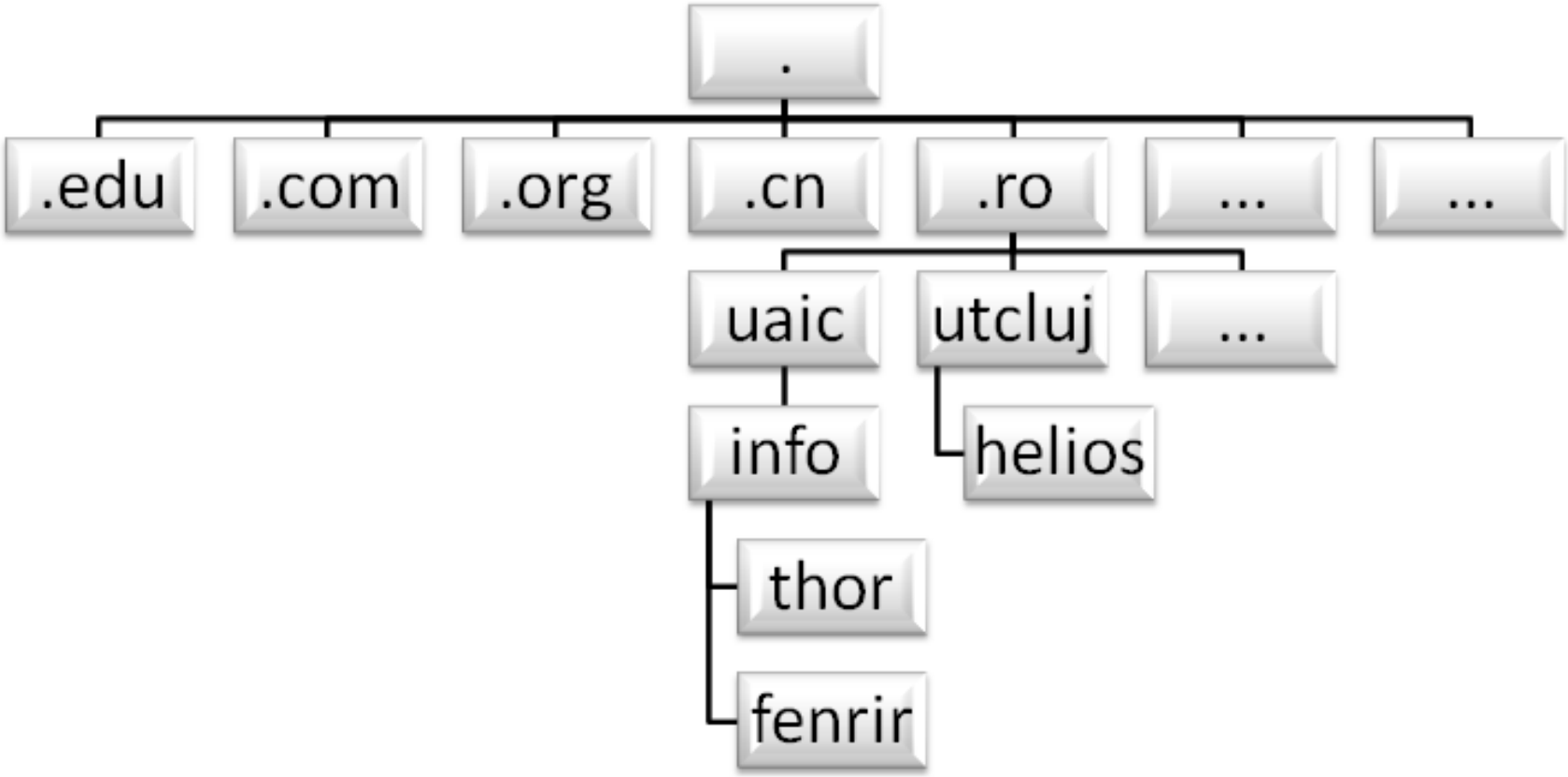
DNS| Tipuri de domenii

- Domeniu de nume
  - Subarbore al arborelui de domenii
  - Nu trebuie sa respecte topologia retelei fizice

- **Sub-domenii**:
  - intreaga cale de nume nu depaseste 255 de caractere
- **Nume de calculatoare** (gazde)

DNS

- Exemplu:



DNS| organizare

- Reguli de alocare a numelor de domenii:
  - Fiecare domeniu controleaza cum sunt alocate subdomeniile sale
  - Pentru a crea un nou subdomeniu, se cere permisiunea domeniului in care va fi inclus (un domeniu de la un anumit nivel va avea o autoritate)
  - Atribuirea de nume de domenii respecta granitele organizationale, nu pe cele ale retelelor
  - Un anumit nivel din ierarhia de niveluri poate fi controlat de mai multe servere

DNS| organizare

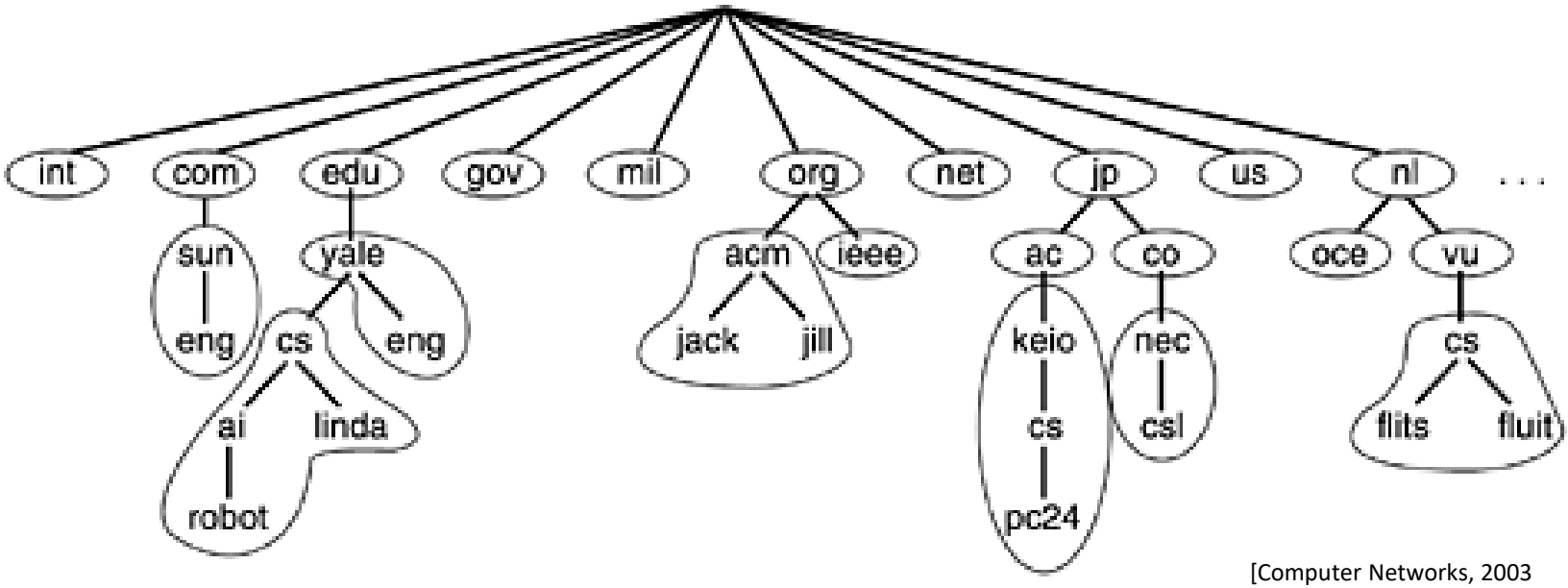
- Servere de nume (*name servers*)
  - Teoretic, un singur server de nume poate contine intreaga baza de date DNS si poate raspunde tuturor cererilor
    - Probleme: incarcarea si "*single point of failure*"

  - Spatiul de nume DNS se divide in zone ne-suprapuse

DNS| organizare

- Servere de nume (name servers)

  Exemplu: O posibila impartire a spatiului de nume DNS in zone



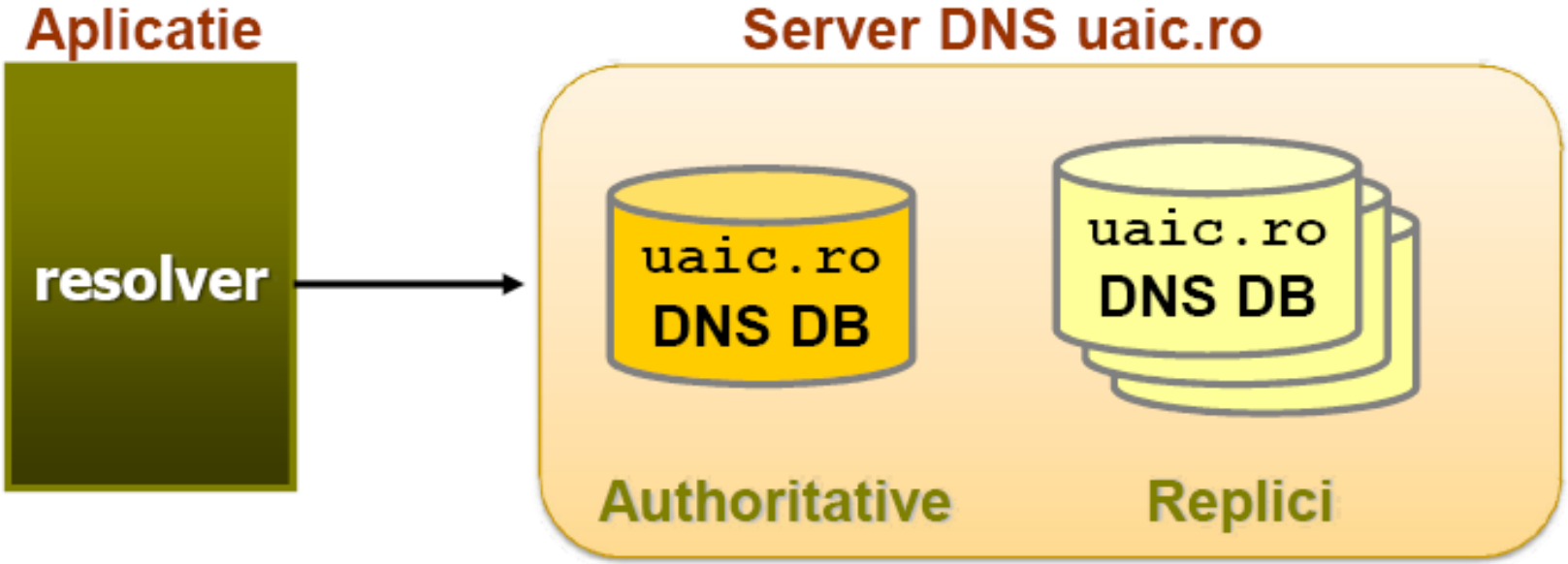
[Computer Networks, 2003 Andrew S. Tanenbaum]

DNS| organizare

- Servere de nume (name servers)
  - Exista un server primar (*primary/authoritative name server*) care deserveste un anumit domeniu si, eventual, mai multe servere secundare continind baze de date replicate
  - TCP se utilizeaza pentru replicarea DNS
  - UDP pentru interogari (*lookups*)

DNS| organizare

- Client DNS
  - Denumit ***resolver***, trimite un pachet UDP serverului DNS care cauta numele si returneaza adresa IP



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

DNS| organizare

- Exemplu de implementari ale serverului de nume: **BIND** (Berkeley Internet Name Domain), MSDNS, PowerDNS etc.

- Ca resolver (client) interactiv, se poate folosi de exemplu una dintre comenzile: **nslookup**, **host** sau **dig**.

DNS| interogari

- Interogari:
  - **Recursiva** – daca un server DNS nu cunoaste adresa pentru numele solicitat, atunci va interoga alt server DNS
  - **Incrementala** – daca serverul DNS nu stie sa raspunda, returneaza eroare si adresa altui server DNS (numit si *referral*) care ar putea cunoaste raspunsul la interogare

[http://technet.microsoft.com/en-us/library/cc775637%28v=ws.10%29.aspx]

DNS| interogari

- Fiecarui domeniu ii este asociata o multime de inregistrari de resurse (***resource record*** – **RR**)
- Mecanismul:
  - Cererea: *resolver*-ul trimite un nume de domeniu
  - Raspunsul: inregistrarile de resurse asociate acelui nume (stocate in bazele de date DNS)

DNS realizeaza corespondenta dintre numele de domenii si inregistrarile de resurse

DNS| interogari

- Forma generala RR este:

  Nume_domeniu Timp_de_viata Tip Clasa Valoare

  Nume_domeniu (*domain name*) – precizeaza domeniul caruia i se aplica aceasta inregistrare

  Timp_de_viata (time-to-live) – da o indicatie despre cat de stabila este inregistrarea

DNS| interogari

Tip - precizeaza tipul inregistrarii

- **SOA** (*Start Of Authority*) : domeniul curent, adresa e-mail a administratorului, etc.
- **A** – adresa IP a gazdei
- **MX** (mail exchangers) – precizeaza numele domeniului pregatit sa accepte posta electronica pentru domeniul specificat
- **CNAME** (*Canonical Name*) – permite crearea pseudonimelor
- **PTR** (Pointer) – Pseudonim pentru adresa IP
- **HINFO**- permit aflarea tipului de masina si de sistem de operare carora le corespunde domeniul
- **TXT**: text neinterpretat (comentarii)

DNS| interogari

Clasa: pentru Internet valoarea acestuia este IN

Valoare: acest camp poate fi un numar, un nume de domeniu sau un sir ASCII; semantica depinde de tipul de inregistrare

Exemple de inregistrari de resurse DNS

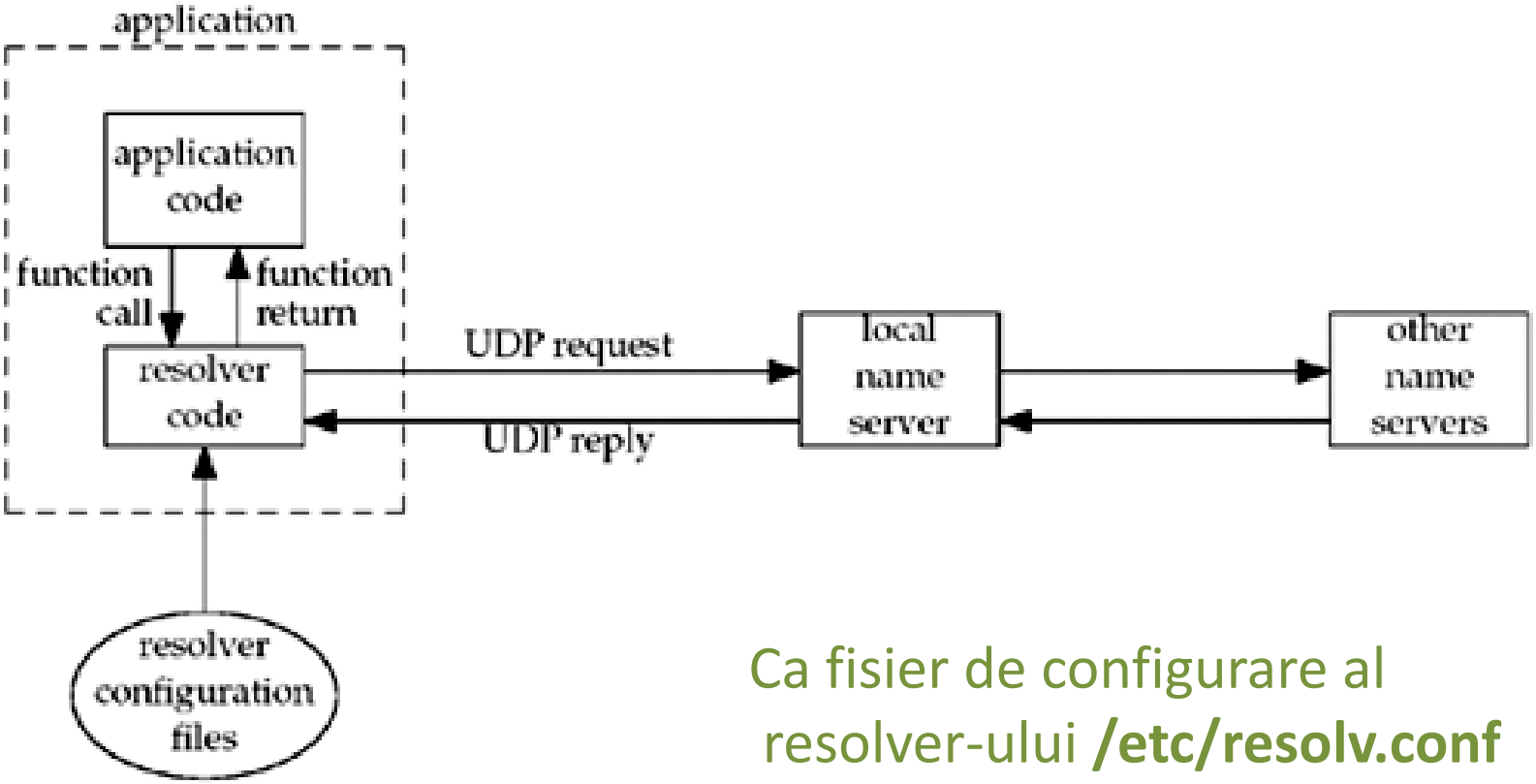
Type	Meaning	Value
SOA	Start of Authority	Parameters for this zone
A	IP address of a host	32-Bit integer
MX	Mail exchange	Priority, domain willing to accept e-mail
NS	Name Server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
HINFO	Host description	CPU and OS in ASCII
TXT	Text	Uninterpreted ASCII text

DNS| configurare

- Exemplu de fisier pentru specificarea unei zone DNS

```
; Zone file for axiologic.ro
;
; The full zone file
;
$TTL 3D
@       IN      SOA     ns1.axiologic.ro. abss.axiologic.ro. (
                        2007050103        ; serial, todays date + todays serial #
                        14400             ; refresh, seconds
                        7200              ; retry, seconds
                        1209600           ; expire, seconds
                        1D )              ; minimum, seconds
;
@                NS      ns1.axiologic.ro.              ; Inet Address of name server
@                NS      ns2.axiologic.ro.              ; Inet Address of name server
@                MX      5 mailx.axiologic.ro.     ; Primary Mail Exchanger
;
localhost       A       127.0.0.1
axiologic.ro.   A       72.249.105.153
www             A       72.249.105.153
mailx           CNAME   axiologic.net.
mail            A       207.210.101.144
ftp             A       72.249.105.153
axiologic.ro. IN TXT "v=spf1 mx mx:mailx.axiologic.ro. ~all"
ns1             A       207.210.101.144
ns2             A       207.210.101.216
~
(END)
```

DNS| clienti, resolveri, servere



Ca fisier de configurare al resolver-ului **/etc/resolv.conf**

[Unix Network Programming, R. Stevens
B. Fenner, A. Rudoff - 2003

DNS| configurare

- Exemplu de fisier /etc/resolv.conf

```
[adria@thor ~] $ cat /etc/resolv.conf
domain info.uaic.ro
search info.uaic.ro
nameserver 85.122.16.1
nameserver 85.122.16.4
[adria@thor ~] $
```

DNS| interogari inverse

- Problema:
  - Data o adresa, care va fi numele ei simbolic?

  (*reverse DNS resolution* sau *reverse DNS lookup*)

Exemple:

1)

```
[adria@ns1 ~]$ host 85.122.23.1
1.23.122.85.in-addr.arpa domain name pointer thor.info.uaic.ro.
[adria@ns1 ~]$
```

2) 2001:db8::567:89ab

b.a.9.8.7.6.5.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa

DNS| optimizari

Proximitatea spatiala: serverele locale vor fi interogate mai des decat cele la distanta

Proximitatea temporala: daca un set de domenii sunt referentiate repetat atunci se apeleaza la *caching DNS*

Fiecare intrare DNS va avea stabilita o valoare TTL (*time to live*)

Se va utiliza si replicarea (servere multiple, servere root multiple) – se va interoga cel mai apropiat (geografic) server

DNS| comenzi

Ca resolver interactiv se pot folosi comenzile:

- **nslookup**
- **dig**
- **host**
- **whois**
- …

DNS| nslookup

Exemple de utilizari:

- **nslookup www.info.uaic.ro**

- Returneaza RR de tip A folosind serverul DNS local

```
[adria@thor ~] $ nslookup www.info.uaic.ro
Server:         85.122.16.1
Address:        85.122.16.1#53

www.info.uaic.ro        canonical name = vidar.info.uaic.ro.
Name:   vidar.info.uaic.ro
Address: 85.122.23.146
```

Host Lookup

- **nslookup 85.122.23.1**

- Returneaza RR de tip PTR pentru 85.122.23.1 in ierarhia de domenii in-addr.arpa

```
[adria@thor ~] $ nslookup 85.122.23.1
Server:         85.122.16.1
Address:        85.122.16.1#53

1.23.122.85.in-addr.arpa        name = thor.info.uaic.ro.
```

Reverse IP Lookup

[http://www.zytrax.com/books/dns/ch3/]

DNS| nslookup

Exemple de utilizari:

- **nslookup www.axiologic.ro**

- Returneaza RR de tip A folosind serverul DNS specificat

```
adria@thor:~$ nslookup www.axiologic.ro 207.210.101.144
Server:         207.210.101.144
Address:        207.210.101.144#53

Name:   www.axiologic.ro
Address: 72.249.105.153
```

Host Lookup

- **man nslookup**

DNS| dig

dig – un instrument mai puternic decat nslookup

Exemplu de utilizare:

- **dig www.info.uaic.ro A**

```
[adria@thor ~] $ dig www.info.uaic.ro A

; <<>> DiG 9.6-ESV-R4 <<>> www.info.uaic.ro A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19336
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 3, ADDITIONAL: 4

;; QUESTION SECTION:
;www.info.uaic.ro.              IN      A

;; ANSWER SECTION:
www.info.uaic.ro.       86400   IN      CNAME   vidar.info.uaic.ro.
vidar.info.uaic.ro.     86400   IN      A       85.122.23.146

;; AUTHORITY SECTION:
info.uaic.ro.           86400   IN      NS      orion.uaic.ro.
info.uaic.ro.           86400   IN      NS      onix.uaic.ro.
info.uaic.ro.           86400   IN      NS      ns.iasi.roedu.net.

;; ADDITIONAL SECTION:
ns.iasi.roedu.net.      86400   IN      A       192.129.4.100
ns.iasi.roedu.net.      86400   IN      AAAA    2001:b30:1:100::100
onix.uaic.ro.           86400   IN      A       85.122.16.4
orion.uaic.ro.          86400   IN      A       85.122.16.1

;; Query time: 1 msec
;; SERVER: 85.122.16.1#53(85.122.16.1)
;; WHEN: Mon Nov 14 11:57:27 2011
;; MSG SIZE  rcvd: 216
```

DNS| comenzi

host

Exemplu de utilizare:

```
adria@thor:~$ host 128.30.52.45
45.52.30.128.in-addr.arpa domain name pointer dolph.w3.org.
```

DNS| whois

whois ibm.com

```
Registrant:
International Business Machines Corporation
   New Orchard Road
   Armonk, NY 10504
   US

   Domain Name: IBM.COM

   ------------------------------------------------------------------------
   Promote your business to millions of viewers for only $1 a month
   Learn how you can get an Enhanced Business Listing here for your domain nam
   Learn more at http://www.NetworkSolutions.com/
   ------------------------------------------------------------------------

   Administrative Contact:
      IBM DNS Admin              dnsadm@us.ibm.com
      IBM Corporation
      New Orchard Road
      Armonk, NY 10504
      US
      +1.9147654227 fax: +1.9147654370

   Technical Contact:
      IBM Corporation            ipreg@us.ibm.com
      New Orchard Road
      Armonk, NY 10504
      US
      +1.9192544441 fax: +1.9147654370


   Record expires on 20-Mar-2018.
   Record created on 19-Mar-1986.
   Database last updated on 8-Nov-2010 04:12:22 EST.

   Domain servers in listed order:

   INTERNET-SERVER.ZURICH.IBM.COM 195.176.20.204
   NS.WATSON.IBM.COM              129.34.20.80
   NS.ALMADEN.IBM.COM             198.4.83.35
   NS.AUSTIN.IBM.COM              192.35.232.34


adria@thor:~$
```

DNS| primitive

- Nu trebuie scris un resolver pentru a afla adresa IP a unei gazde
- Functii principale:
  - **gethostbyname(); getaddrinfo();**
  - **gethostbyaddr() ; getnameinfo();**
- La unele sisteme de operare (e.g., Solaris) va trebui la compilare sa folosim biblioteca nsl (*Name Server Library*): gcc … -lnsl

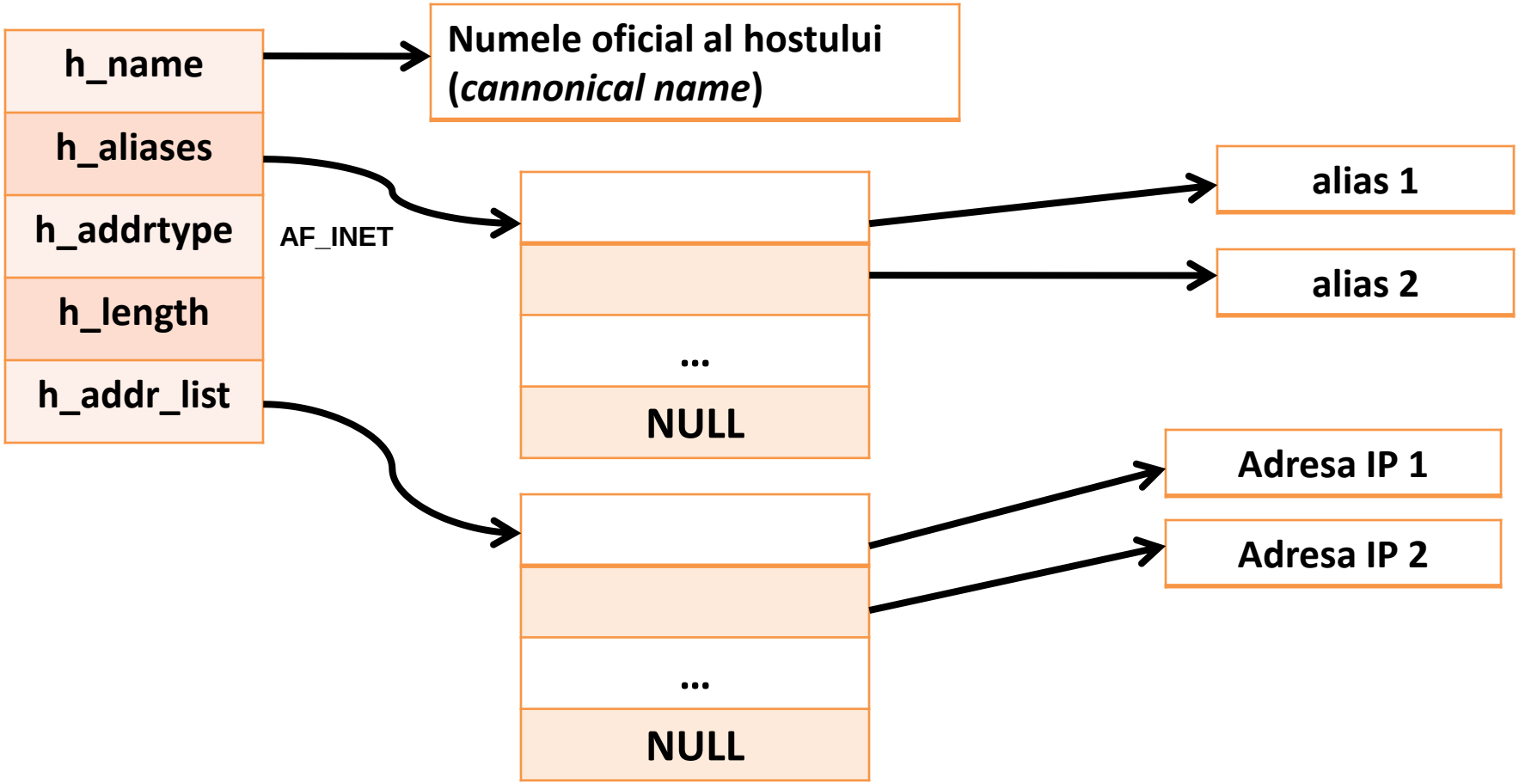
DNS| primitive

Una din structurile folosite: **hostent**

```
struct hostent {
    char *h_name;       /* nume oficial (canonical) */
    char **h_aliases;   /* alias-uri */
    int h_addrtype;     /* AF_INET */
    int h_length;       /* lungimea adresei: 4 sau 6 */
    char **h_addr_list; /*pointeri la adresele IP */
};
```

DNS| primitive

Structura **hostent:**



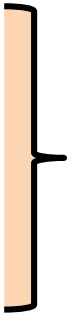
DNS| gethostbyname()

```
#include <netdb.h>
struct hostent *gethostbyname
                    (const char *hostname);
```

- In termenii DNS, gethostbyname() realizeaza o cerere pentru o inregistrare **A**
- Obs. gethostbyname() se foloseste in special pentru IPv4

DNS| gethostbyname()

- Returneaza:
  - In caz de succes returneaza un pointer la **hostent**, ce contine adresa IP a *host*-ului
  - In caz de eroare NULL, iar variabila h_errno indica eroarea aparuta:
    - HOST_NOT_FOUND
    - …
    - NO_RECOVERY
    - …



Constante definite in netdb.h

DNS| gethostbyname()

- Exemplu de utilizare: completarea structurii *sockaddr_in* avind in loc de adresa IP un nume simbolic:

```
struct sockaddr_in server;
struct hostent *hos;
if(!( hos = gethostbyname("fenrir.info.uaic.ro") )
  {/*Eroare la rezolvarea adresei*/}
server.sin_family=AF_INET
  /* adresa IP o luam din structura hos */
memcpy(&server.sin_addr.s_addr, hos->h_addr_list[0],
                        sizeof(hos->h_addr_list));
server.sin_port=htons(4321);
```

DNS| gethostbyaddr()

```
#include <netdb.h>
        struct hostent *gethostbyaddr (
                        const char *addr,
                        socklen_t len,
                        int family);
```

- In termenii DNS, **gethostbyaddr()** realizeaza o cerere la serverul de nume pentru o inregistrare **PTR** in domeniul in-addr.arpa
- Returneaza: In caz de succes returneaza un pointer la **hostent**, ce contine numele oficial al *host*-ului ; In caz de eroare NULL, iar variabila h_errno indica eroarea aparuta

Obs. gethostbyaddr() se foloseste in special pentru IPv4

DNS| getservbyname()

```
#include <netdb.h>
struct servent *getservbyname (const char *servname, const char
    *protoname);
```

- Returneaza: un pointer la *struct servent* in caz de sucess, NULL in caz de eroare

```
        struct servent {
            char *s_name;      /* numele oficial al serviciului*/
            char **s_aliases;  /* alias-uri  */
            int s-port;        /* portul (network-byte order) */
            char *s_proto;     /* protocolul */ };
```

Exemplu: struct servent *pserv;

```
        pserv=getservbyname("ftp","tcp"); /*FTP folosind TCP */
```

DNS| getservbyport()

#include <netdb.h>

struct **servent** ***getservbyport** (int **port,** const char ***protoname**);

- Cauta un serviciu dupa un numar de port si dupa protocol (optional)
- Returneaza: un pointer la *struct servent* in caz de sucess, NULL in caz de eroare

Obs. port este in *network byte order*

Exemplu:

```
struct servent *pserv;
pserv=getservbyport( htons(53), ”udp”); /*DNS folosind UDP */
pserv=getservbyport( htons(21),”tcp”); /*FTP folosind TCP */
```

DNS| getaddrinfo()

```
#include <netdb.h>
int getaddrinfo (
    const char *hostname,
    const char *service,
    const struct addrinfo *hints,
    struct addrinfo **result ) ;
```

Numele host-lui sau o adresa IPv4 sau IPv6 ca string

Portul serviciului sau numele serviciului (“http”,”pop”,..) (vezi /etc/services)

Contine informatii despre tipul de informatii pe care trebuie sa le intoarca primitiva

- Obs. *hostname*, *service*, *hints* – parametri de intrare
- Returneaza: 0 in caz de sucess, !=0 in caz de eroare
- Se recomanda a fi folosita si pentru IPv4 si pentru IPv6
- Combina functionalitati ale: gethostbyname(), getservbyname(), getservbyport()

DNS| getaddrinfo()

```
struct addrinfo {
    int ai_flags;         /* AI_PASSIVE, AI_CANONNAME */
    int ai_family;        /* AF_INET, AF_INET6, AF_UNSPEC */
    int ai_socktype;      /* SOCK_STREAM sau SOCK_DGRAM */
    int ai_protocol;      /* 0 (auto) sau IPPROTO_TCP, IPPROTO_UDP */
    socklen_t ai_addrlen; /* lungimea lui ai_addr */
    char *ai_canonname; /* numele canonic al host-ului */
    struct sockaddr *ai_addr; /*  adresa binara a socket-ului */
    struct addrinfo *ai_next; /* pointer la  urmatoarea structura din
    lista */
};
```

DNS| getaddrinfo()

Discutii:

- Daca functia returneaza cu succes **result** va pointa la lista de **struct addrinfo.**

  Cazuri cind se pot obtine structuri multiple:

  - Exista mai multe adrese asociate cu numele hostului, si cate o structura este returnata pentru fiecare adresa
  - Daca serviciul este furnizat pentru tipuri diferite de *socket*-uri, atunci cate o structura este returnata pentru fiecare tip de *socket*
- Informatia returnata de **getaddrinfo**() in structura **struct addrinfo** ****result** poate fi utilizata astfel:
  - Pentru **socket**() : **ai_family**, **ai_socktype**, **ai_protocol**
  - Pentru **connect**() sau **bind**(): **ai_addr** si **ai_addrlen**
- freeaddrinfo()

DNS| getnameinfo()

```
#include <netdb.h>
int getnameinfo (
    const struct sockaddr *sockaddr,
    socklen_t addrlen,
    char *host,
    socklen_t hostlen,
    char *serv,
    socklen_t servlen,
     int flags) ;
```

Adresa socket-ului trimisa ca argument

numele host-ului intors

Numele serviciului

NI_NOFQDN -> **host** va contine doar numele host-ului si nu intreg numele al domeniului

- Inlocuieste gethostbyaddr() si getservbyport()
- Returneaza: 0 in caz de sucess, !=0 in caz de eroare

DNS| IDN

- **International Domain Names (IDN)**
  - Extensie care permite folosirea caracterelor Unicode in numele de domenii, nu doar a celor ASCII

    http://www.icann.org/en/topics/idn/

    16 Noiembrie 2009 - Inregistrarea de domenii ccIDN sau IDN ccTLD

    > 2010-01: ICANN announces that Egypt, the Russian Federation, Saudi Arabia, and the United Arab Emirates were the first countries to have passed the Fast Track String Evaluation within the IDN ccTLD domain application process.

  - Pot fi exploatate pentru atacuri de tip *phishing*

    (… detalii intr-un curs viitor)

DNS| administrare

- Radacina DNS este oficial administrata de Internet Corporation for Assigned Names and Numbers (ICANN)

- Exista si alte organizatii care ofera radacini alternative (alt DNS roots), precum OpenNIC (Network Information Center) sau New.Net

Rezumat

- Domain Name System (DNS)
  - Caracterizare
  - Organizare
  - Configurare
  - Comenzi, Primitive
  - IDN

Intrebari?

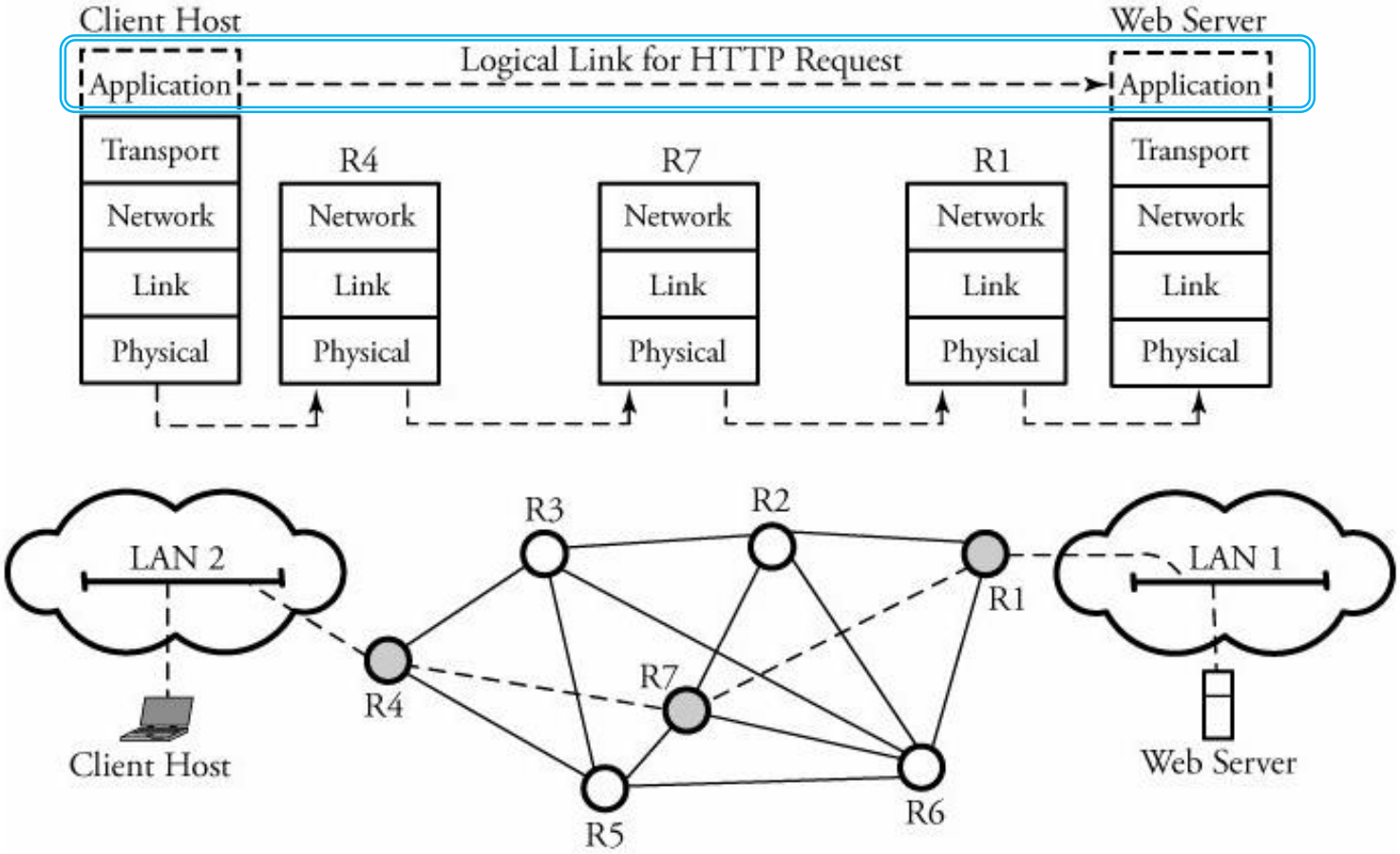
Nivelul Aplicatie

Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- **Protocoale la nivelul aplicatie**
  - Preliminarii
  - Caracteristici de proiectare
  - Accesul la terminal de la distanta
  - Posta Electronica
    - SMTP (Simple Mail Transfer Protocol)
    - POP (Post Office Protocol)
  - Transferul de fisiere
    - TFTP (Trivial File Transfer Protocol)
    - FTP (File Transfer Protocol)
  - World-Wide Web (HTTP)
  - Privire de ansamblu

Preliminarii



Comunicare intre doua *end-systems*

[Computer and Communication Networks , Nader F. Mir, 2006]

Preliminarii

Application	Application layer protocol	Underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (eg Youtube), RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary (e.g., Skype)	typically UDP

Preliminarii

- **La nivelul aplicatie sunt puse la dispozitie o serie de servicii :**
  - Terminal la distanta (TELNET, SSH, …)
  - Posta electronica (SMTP, IMAP, POP, …)
  - Transferul de fisiere (TFTP, FTP si altele)
  - World-Wide Web (HTTP)
  - Conversatii instantanee (ICQ, XMPP (din Mai 2014 -> nu mai are suport in Google Voice),Hangouts IM, WhatsApp…)
- **Se ofera si protocoale pentru rezolvarea unor sarcini de sistem - /etc/services, /etc/protocols**
  - Sistemul de fisiere in retea (NFS)
  - Conectivitatea cu alte sisteme de fisiere (SMB)
  - Servicii de baze de date (MySQL, PostgreSQL, …, Hive,…)

Caracteristici de proiectare

- **Tipuri de protocoale in functie de natura datelor transferate**
  - **Fluxuri de caractere generate de utilizator**
    - Folosite pentru aplicatii interactive la distanta (*telnet*, *rlogin*, *IRC*, …)
    - Traficul este in mare masura compus din date neinterpretate
    - Se pot include secvente de control (i.e. controlul terminalului, coduri de culoare) – coduri ANSI

    (Exemplu: **CSI *n* E -->** numit: CNL – Cursor Next Line

    Moves cursor to beginning of the *n*-th (default 1) following line)

Caracteristici de proiectare

- **Tipuri de protocoale in functie de natura datelor transferate**
  - **Mesaje intrebare/raspuns ASCII**
    - Serverul si clientul vehiculeaza siruri de caractere care pot fi citite si de utilizatori umani (SMTP, FTP, HTTP, XMPP, SIP, …)
    - Uzual, sunt compuse din linii de text
  - **Formate binare**
    - Utilizate pentru protocoale de nivel inferior (SNMP – Simple Network Management Protocol) sau de nivel inalt (NFS peste RPC)
    - Apar probleme la reprezentarea datelor (i.e. ordinea octetilor)
  - **Protocoale ad-hoc folosite de aplicatiile (nestandard) scrise de utilizatori**
    - Pot adopta unele dintre tipurile anterioare

Caracteristici de proiectare

- Cerinte referitoare la proiectarea unui protocol
  - Parametri critici: lungimea numelui comenzilor, marimea *buffer*-elor, modul de adresare
  - Definirea operatiilor permise (e.g., creare, citire, scriere, stergere, actualizare)
  - Raportarea erorilor: coduri de eroare, mesaje
  - Formatul mesajelor: sursa, destinatie, parametri, codificarea datelor, lungime fixa/variabila, …

Caracteristici de proiectare

- Scenariul uzual
  - Serverul – citeste coduri de operatii (*opcode*-uri) si raporteaza starea folosind coduri de eroare
  - Clientul – construieste mesaje folosind *opcode*-urile permise
- Moduri de adresare
  - Proces executat pe o singura masina

    adresa (fizica/logica) a masinii: thor.info.uaic.ro
  - Procese executate pe masini diferite:
    - Adrese formate din 2 parti (proces, masina)

      thor.info.uaic.ro:80
    - Adrese ca nr. generate aleatoriu (*universal ID*)
      - fiecare ID trebuie difuzat tuturor

Caracteristici de proiectare

- Problema fiabilitatii (engl. *Reliability*) comunicarii
  - Reteaua poate pierde mesaje
  - Abordari:
    - Posta clasica (*post-office*)
      - Nu asteapta nici un fel de confirmari
    - *Handshaking* – toate mesajele sunt confirmate
    - Raspuns confirmat (*acknowledged reply*)
      - Se asteapta un raspuns, iar expeditorul raspunsului asteapta confirmarea primirii lui
    - Cerere/raspuns (*request/reply*) – expeditorul asteapta (un timp) venirea raspunsului (e.g. RPC, SOAP)

Accesul la terminal

- Serviciu “antic” standard Internet
- Folosit prin comenzi precum rlogin, telnet, ssh (varianta securizata a *telnet*)
- Utilizeaza modelul client/server:
  - Clientul interactioneaza cu utilizatorul
  - Serverul furnizeaza acces la un *shell* (e.g., *bash*)

Aplicatie de tip
remote login

Accesul la terminal

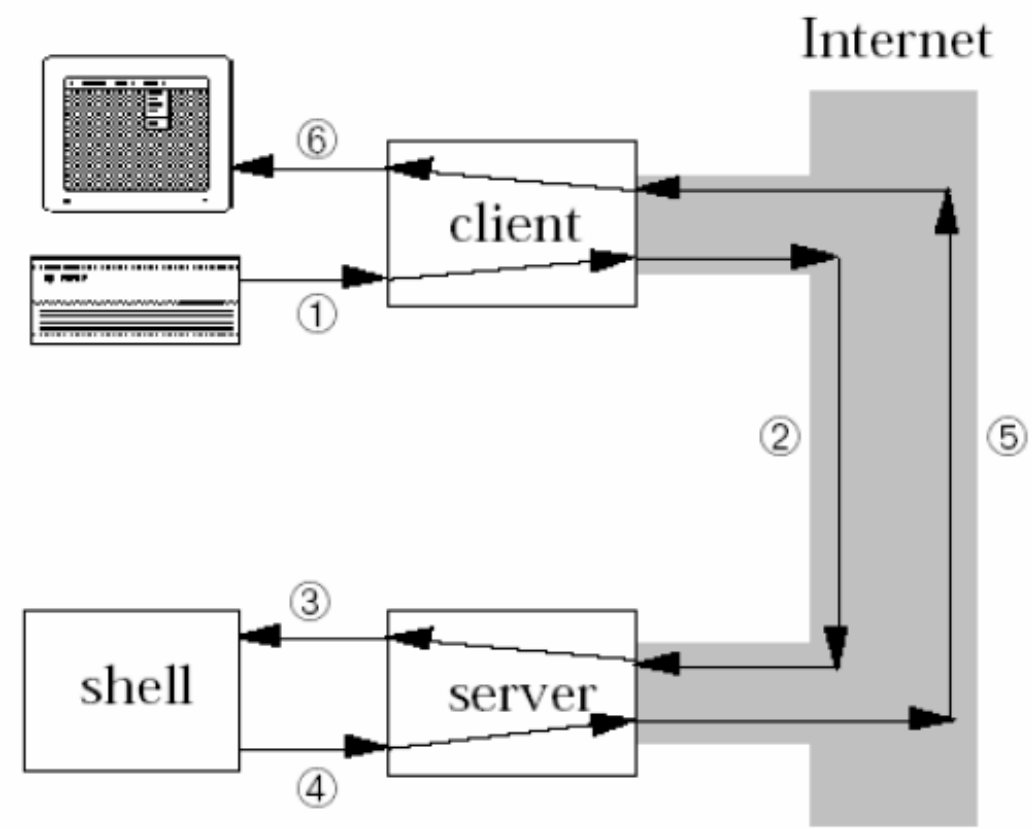


Figura: Mecanismul de functionare a unei aplicatii de tip *remote login*

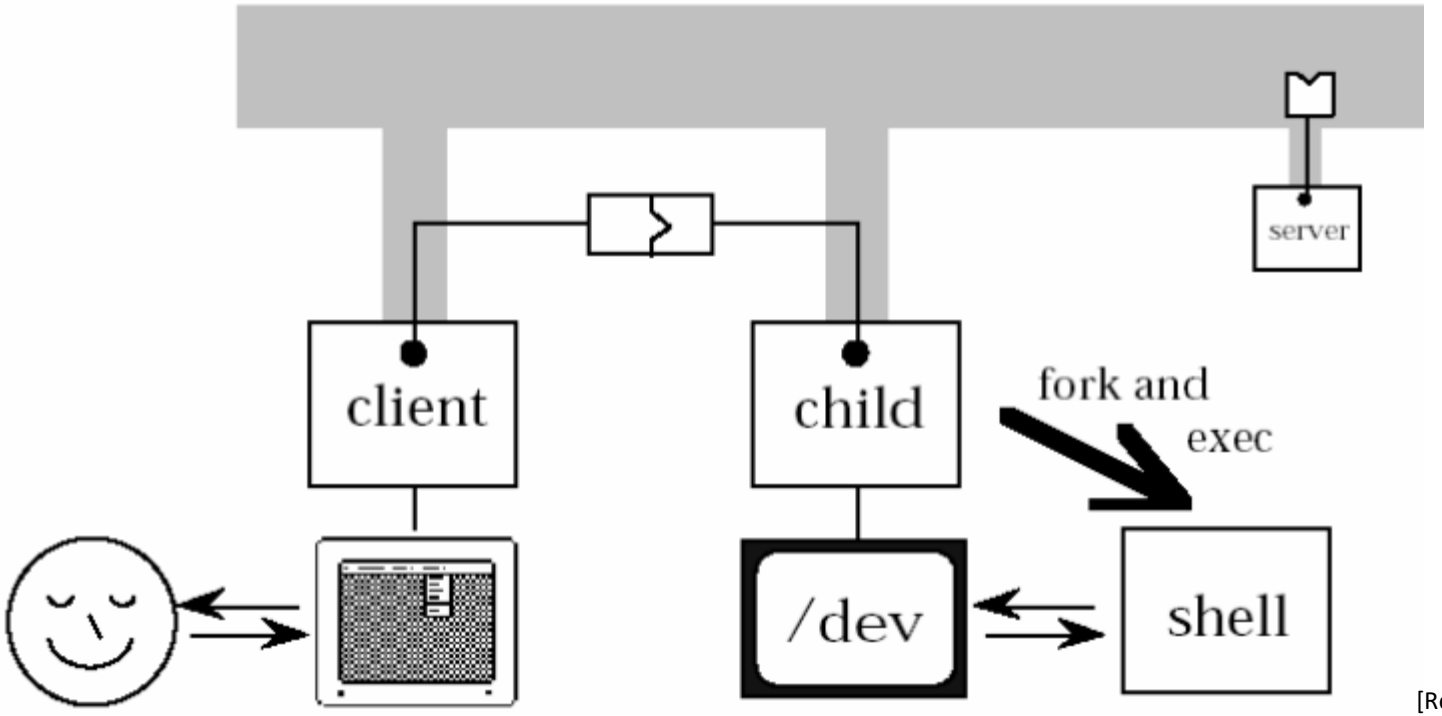
[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Accesul la terminal

- Implementare – mecanism general

Pentru conectarea la distanta, fiecare client va fi deservit de un proces copil al serverului

Procesul copil va crea un alt proces care va conecta clientul la un pseudo-terminal si va executa *shell*-ul



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Accesul la terminal

- **Probleme**
  - Initializarea si autentificarea
    - Cum identificam clientul?
    - Cum stim ca serverul este unul oficial?
  - Procesarea caracterelor speciale (inclusiv sfarsitul de linie – EOL)
  - Cine proceseaza actiuni precum editarea liniei, afisarea caracterelor testate (*echoing*), suspendarea terminalului (CTRL + S) etc.?
  - Modul de comunicare intre client si server
    - Intreruperi din partea utilizatorului
    - Controlul dimensiunii ferestrei de afisare

Accesul la terminal

- **rlogin**
  - protocol simplu de acces la distanta
  - utilizat exclusiv pentru conectarea de masini UNIX
  - RFC 1258: "*The rlogin facility provides a remote-echoed, locally flow-controlled virtual terminal with proper flushing of output*"

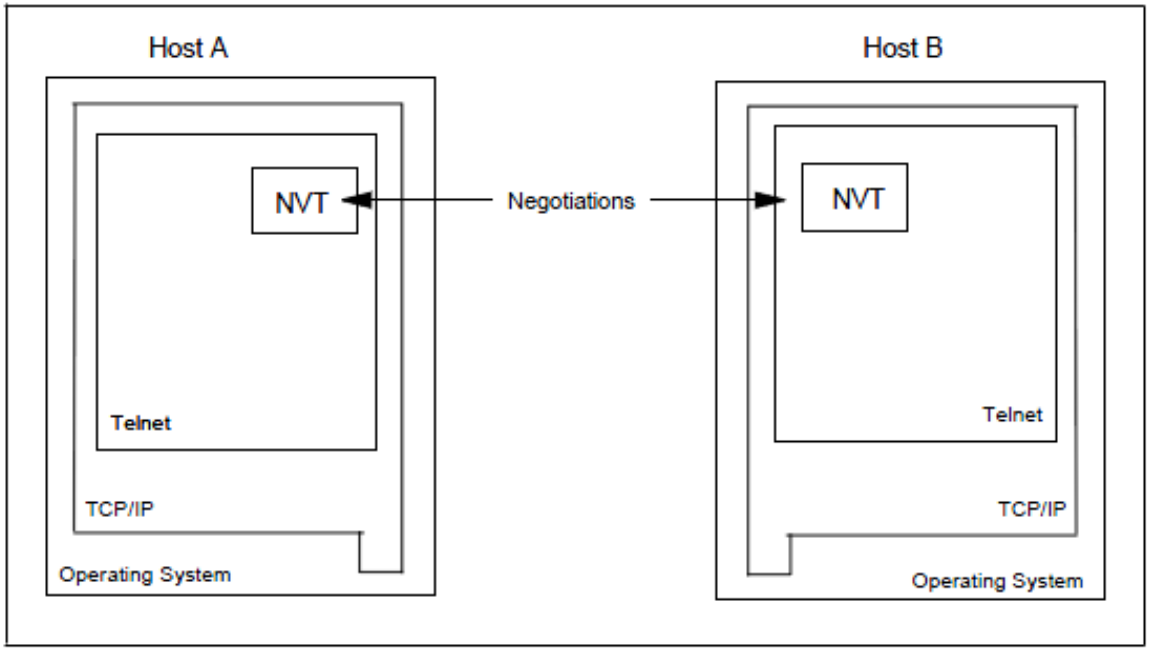
  Functionare:
  - rlogin comunica cu un *daemon* rlogind de pe gazda *remote*
  - autentificarea se face prin apelarea la gazde "de incredere" ("*trusted*" *hosts*)
    - rlogind permite logarea fara parola daca gazda *remote* apare in fisierul /etc/hosts.equiv sau daca utilizatorul are un fisier .rlogin in directorul *home*

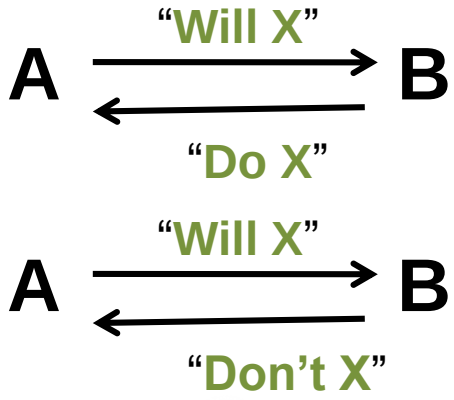
Accesul la terminal

- **telnet** (*terminal network*)
  - Protocol standard TCP/IP de acces la distanta (RFC 854,855)
  - Utilizat indiferent de platforma
  - Poate fi utilizat drept client generic fara a sti detalii despre server
  - Autentificarea clientilor nu se face de catre protocol, ci de catre aplicatie
  - Protocolul se bazeaza pe:
    - Conceptul NVT (Network Virtual Terminal): un dispozitiv virtual cu o structura generala comuna cu o gama larga de terminale; fiecare host face maparea caracteristicilor propriului terminal cu cele ale NVT
    - Odata ce a fost stabilita o conexiune prin TELNET, ambele capete ale comunicarii sunt tratate simetric

Accesul la terminal

- **telnet** (*terminal network*)
  - Protocolul se bazeaza pe:
    - Ambele parti ale comunicarii pot sa negocieze utilizarea de optiuni aditionale care sa reflecte partea hardware utilizata
      - Optiuni pentru : editarea liniei, dimensiunea ferestrei de afisare etc.





telnet ofera compatibilitate cu terminale
vechi (vt52, vt100,…)

[TCP/IP Tutorial and Technical Overview, IBM, 2006]

Accesul la terminal

- **telnet** (*terminal network*)
  - Comunicare dintre client si server se realizeaza prin comenzi de tipul:
    - **IP** (Interrupt Process; 244) -> terminarea programului care ruleaza
    - **AO** (Abort output; 245) -> elibereaza orice buffer de iesire
    - **AYT** (Are you there; 246) -> permite clientului trimiterea unei interogari OOB pentru verificarea faptului ca partea *remote* este activa
    - **EC** (Erase character; 247) -> stergerea caracterului anterior
    - **EL** (Erase Line; 248) -> stergerea intregii linii curente
    - ... (RFC 854)
  - **Trimiterea unei comenzi:** comanda (1 octet) precedata de un octet cu valoarea 255 - IAC (*Interpret As Command*)

SSH

- **SSH** (*secure shell*)
  - Fata de *telnet*, furnizeaza o comunicare sigura (bazata pe TCP) prin mesaje criptate si mesaje de autentificare
  - SSH foloseste modelul client/server
    - Un program client SSH este utilizat pentru stabilirea unei conexiuni cu un *daemon* SSH
  - Utilizari:
    - logarea pe o masina la distanta si executarea de comenzi
    - suport pentru *tunneling* (Curs viitor)
    - permite si transfer de fisiere in asociere cu protocoalele SFTP sau SCP
  - Are suport in majoritatea sistemelor de operare moderne

E-mail

- **Protocoale bazate pe TCP:**
  - **SMTP (*Simple Mail Transfer Protocol*)**
    - RFC 821 (specifica modul de schimb a mail-ului intre doua host-uri)
  - **POP (*Post Office Protocol*)**
    - RFC 1939
  - **POP3S –** varianta securizata a **POP3**
  - A se vedea si: RFC 822 (specificatii privind antetul unui mail), 2049 (specificatii privind documente diferite *de plain text ASCII* ce pot fi continute intr-un email), RFC 974 (standard privind rutarea mailurilor folosind DNS)
  - RFC 822 si 974 -> consolidate in RFC 2821, 2822

E-mail (*Electronic Mail*)

- **Terminologie**
  - **Agent utilizator** (**MUA** – *Mail User Agent*): client (local) pentru posta electronica

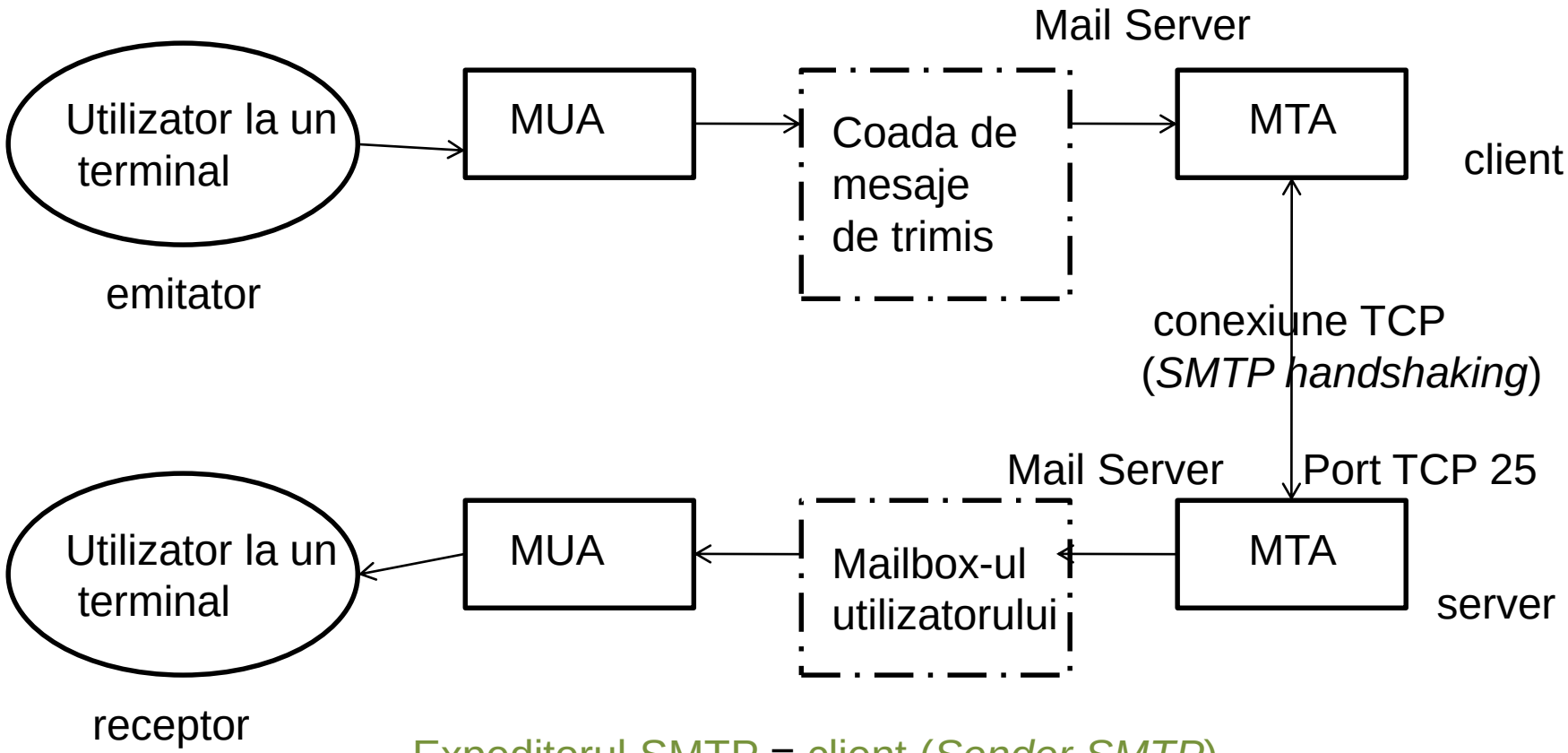
    Ex: alpine, mutt, Mozilla Thunderbird, Kmail, Outlook etc.
  - **Agent de transfer** (**MTA** – *Mail Transport Agent*)

    responsabil cu comunicarea cu gazdele la distanta si cu trimiterea/receptionarea de posta

    (client & server) - sendmail, qmail
  - **Agent de distributie** (**MDA** - *Mail Distribution Agent* sau LDA – *Local Delivery Agent*) - directioneaza mesajele primite catre casuta postala a utilizatorului; Ex: maildrop, Sieve, procmail
  - **Mail exchanger** (**MX**) – gazda responsabila cu e-mail-urile unui domeniu (masina intermediara)

E-mail| SMTP

- Utilizat in schimbul de mesaje de posta intre serverele de mail (MTA-uri)



Expeditorul SMTP = client (*Sender SMTP*)
Destinatarul SMTP = server (*Receiver SMTP*)

E-mail

- **Caracteristici**
  - Distinctia dintre **plic** si **continut**
    - **Plicul** incapsuleaza mesajul, contine date necesare pentru transportul mesajului: destinatar, adresa, prioritate, securitate, …
    - Plicul este folosit pentru dirijarea mesajului la destinatar
    - **Mesajul** din plic contine un **antet** (date de control pentru MUA) si un **corp** (date pentru utilizator)
  - Fiecare utilizator este identificat printr-o adresa de e-mail: **cutie_postala@locatie** (**cont@adresaInternet**)

E-mail|SMTP

- Componente:
  - Plic *(envelope*)– folosit de MTA pentru livrare

  Exemplu:

  MAIL From: <adria@info.uaic.ro>

  RCPT to: <adria@info.uaic.ro>
  - Anteturi (*headers*) – folositi de MUA

  Exemplu: Received, Message-ID, From, Date, Reply-To, Subject,…
  - Continut –ul mesajului (*body*) -
- Mecanism: MUA preia continutul, adauga anteturi si il transmite la MTA; MTA adauga anteturi, adauga plicul si il trimite la un alt MTA

E-mail|SMTP

- Campuri de antet utilizate in transportul de e-mail-uri:

Header	Meaning
To:	Email address(es) of primary recipient(s)
Cc:	Email address(es) of secondary recipient(s)
Bcc:	Email address(es) for blind carbon copies
From:	Person or people who created the message
Sender:	Email address of the actual sender
Received:	Line added by each transfer agent along the route
Return-Path:	Can be used to identify a path back to the sender

E-mail|SMTP

- Campuri de antet utilizate in transportul de e-mail-uri:

Header	Meaning
Date:	The date and time the message was sent
Reply-To:	Email address to which replies should be sent
Message-Id:	Unique number for referencing this message later
In-Reply-To:	Message-Id of the message to which this is a reply
References:	Other relevant Message-Ids
Keywords:	User chosen keywords
Subject:	Short summary of the message for the one-line display

E-mail|SMTP

Exemplu

thor.info.uaic.ro - PuTTY

```
Un mesaj din univers:)

X-UID: 22949
X-Keywords:
X-Status:
Status: RO
To: undisclosed-recipients:;
From: anonimus@spacesome.ro
Date: Mon,  6 Dec 2010 10:48:53 +0200 (EET)
Message-Id: <201012060849l0.0164CD00C@fenrir.info.uaic.ro>
        for <adria@infoiasi.ro>; Mon,  6 Dec 2010 10:48:53 +0200 (EET)
        by fenrir.info.uaic.ro (Postfix) with SMTP id 0164CD00C
Received: from fenrir.info.uaic.ro (fenrir.info.uaic.ro [127.0.0.1])
Delivered-To: adria@infoiasi.ro
        id B2F69CFB6; Mon,  6 Dec 2010 10:51:35 +0200 (EET)
Received: by fenrir.info.uaic.ro (Postfix)
        for <adria@thor.info.uaic.ro>; Mon,  6 Dec 2010 10:51:37 +0200 (EET)
        by thor.info.uaic.ro (Postfix) with ESMTP id C71932FC61
Received: from fenrir.info.uaic.ro (fenrir.info.uaic.ro [85.122.23.145])
Delivered-To: adria@thor.info.uaic.ro
X-Original-To: adria@thor.info.uaic.ro
        NO_DNS_FOR_FROM shortcircuit=no autolearn=no version=3.2.5
X-Spam-Status: No, score=3.1 required=4.5 tests=BAYES_40,MISSING_SUBJECT,
X-Spam-Level: ***
X-Spam-Checker-Version: SpamAssassin 3.2.5 (2008-06-10) on thor.info.uaic.
X-Spam-ASN: AS12675 85.122.16.0/20
Return-Path: <anonimus@spacesome.ro>
From anonimus@spacesome.ro  Mon Dec  6 10:51:37 2010
```

Anteturi nestandard

E-mail|SMTP

- Comunicarea:
  - Se realizeaza o conexiune TCP intre *Sender SMTP* si *Receiver SMTP* (intre MTA-uri). Obs. Receiver SMTP poate fi destinatia finala sau un intermediar (*mail gateway*)
  - Clientul trimite comenzi SMTP, iar serverul raspunde cu coduri de stare
  - Mesajele de stare include coduri numerice NNN si texte explicative
  - Ordinea comenzilor este importanta
  - Se utilizeaza portul 25

E-mail|SMTP

- Comenzi uzuale:
  - **HELO**: identifica gazda expeditoare
  - **MAIL FROM**: porneste o tranzactie si identifica orginea e-mail-ului
  - **RCPT TO**: identifica recipientii individuali ai mesajului (adrese de e-mail); pot exista comenzi RCPT TO: multiple
  - **DATA** desemneaza o serie de linii text terminate cu \r\n, ultima linie continind doar “.”
  - **QUIT**

E-mail|SMTP

- Exemplu:

fenrir.info.uaic.ro - PuTTY

```
[adria@fenrir ~] $ telnet fenrir.info.uaic.ro 25
Trying 127.0.0.1...
Connected to fenrir.info.uaic.ro.
Escape character is '^]'.
220 fenrir.info.uaic.ro ESMTP
HELO fenrir.infoiasi.ro
250 fenrir.info.uaic.ro
MAIL from: anonimus@spacesome.ro
250 2.1.0 Ok
RCPT TO:adria@infoiasi.ro
250 2.1.5 Ok
DATA
354 End data with <CR><LF>.<CR><LF>
Un mesaj din univers:)
.
250 2.0.0 Ok: queued as 049E5CF9C
```

```
1327 12:02       anonimus@spacesom   (2K)
```

E-mail|SMTP

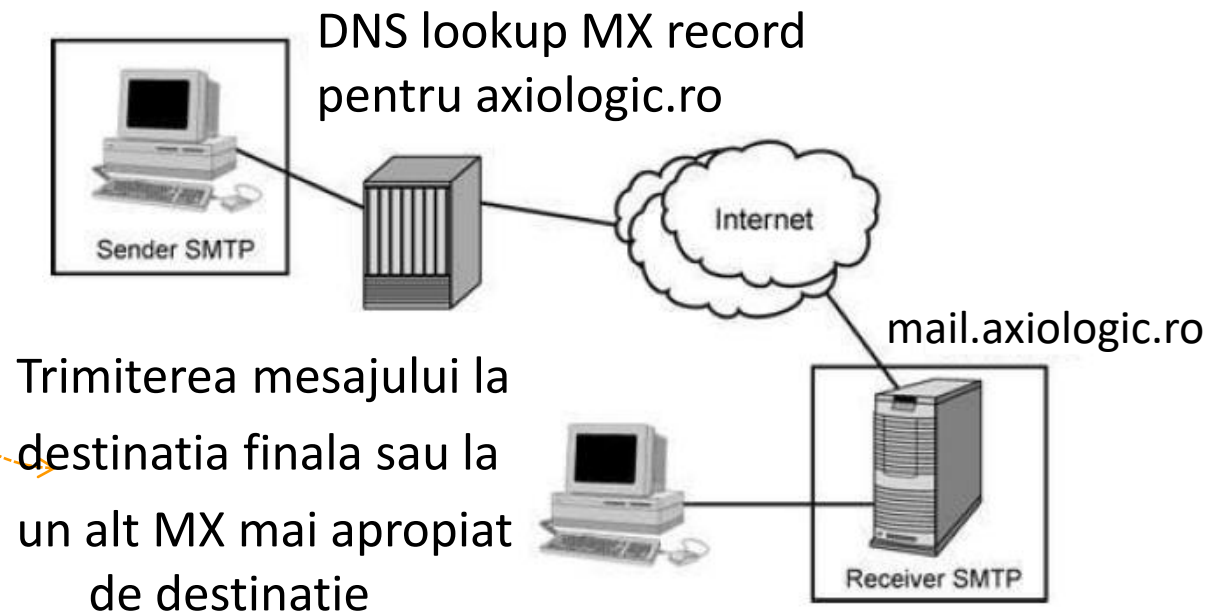
- Alte comenzi:
  - **VRFY:** permite verificarea validitatii unui *recipient*
  - **NOOP**: forteaza serverul sa raspunda cu un cod de OK (200)
  - **EXPN:** expandeaza un grup de adrese (*alias*)
  - **TURN**: interschimba destinatarul cu expeditorul fara a fi necesara crearea unei noi conexiuni TCP

    (s*endmail* nu suporta aceasta comanda)
  - **RSET** abandoneaza tranzactia curenta

E-mail|SMTP

- DNS si e-mail-ul

- Inregistrarea de tip MX din DNS identifica gazda (MX) cu rol de procesare si *forward*-are a mailurilor pentru respectivul domeniu



- Mecanism general:
  - Serverul SMTP verifica inregistrarea MX a domeniului specificat in adresa de email (e.g. axiologic.ro pentru adresa b@axiologic.ro) si sa zicem ca aceasta inregistrare este mail.axiologic.ro.
  - Se va trimite acest mail pe serverul SMTP de pe mail.axiologic.ro .

E-mail|SMTP

- RFC 822: SMTP este limitat la text ASCII pe 7 biti
- RFC 1521: defineste un standard care sa rezolve limitarile anterioare -> MIME (*Multipurpose Internet Mail Extensions*)
  - Standard de codificare a continutului mesajelor non-ASCII
    - Limbi cu accente, cu alfabete non-latine, fara alfabet, mesaje non-textuale
  - Permite atasarea la e-mail a fisierelor de orice tip
  - Se foloseste campul:

    Content-Type: tip/subtip

    Exemplu: Mime-Version: 1.0

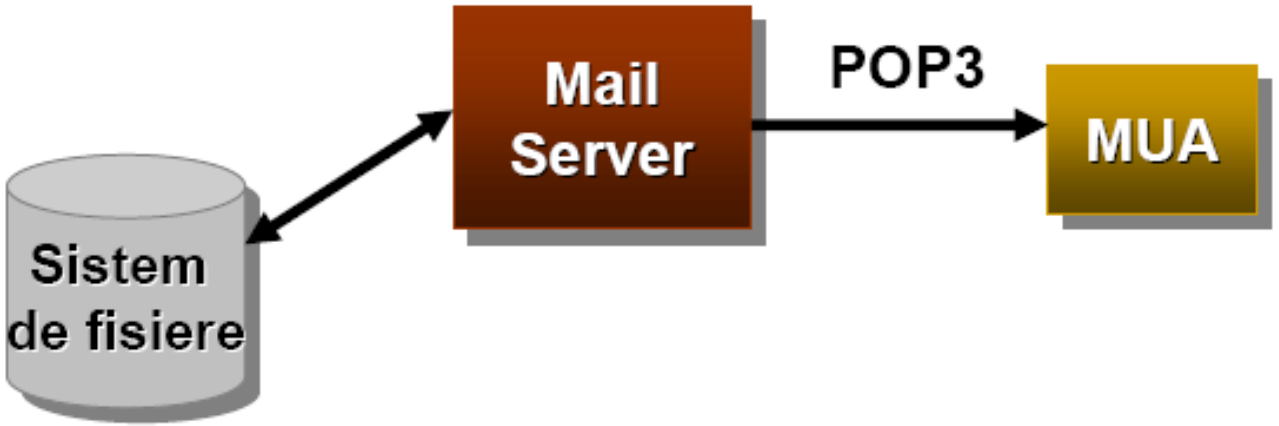
    Content-Type: TEXT/PLAIN

E-mail|SMTP

- Tipuri MIME principale:
  - **application** defineste aplicatiile client (application/executable)
  - **text** defineste formatele text (text/plain, text/html)
  - **image** specifica formatele grafice (image/gif, image/jpeg)
  - **audio** specifica formatele audio (audio/basic)
  - **video** specifica formatele video (video/mpeg)
  - **multipart** utilizat pentru transportul datelor compuse (multipart/mixed, multipart/alternative)

E-mail|POP

- POP (Post Office Protocol) – RFC 1939
- Utilizat la transferul de mesaje de pe un server de posta la un MUA – portul 110
- Comenzile si raspunsurile sunt mesaje ASCII
- Raspunsurile incep cu +OK sau -ERR



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

E-mail|POP

- Comenzi uzuale:
  - **USER** specifica numele de cont
  - **PASS** specifica parola
  - **STAT** furnizeaza numarul de mesaje din cutia postala (*mailbox*)
  - **LIST** afiseaza lista de mesaje si lungimea, cate 1 pe linie
  - **RETR** preia un mesaj
  - **DELE** reseteaza tranzactia, iar orice marcaj de stergere este eliminat
  - **QUIT** sterge mesajele marcate si inchide conexiunea

E-mail|POP

Exemplu

thor.info.uaic.ro - PuTTY

```
adria@thor:~$ telnet thor 110
Trying 85.122.23.1...
Connected to thor.info.uaic.ro.
Escape character is '^]'.
+OK POP3 thor.info.uaic.ro 2007b.104 server ready
user adria
+OK User name accepted, password please
pass [redacted]
+OK Mailbox open, 1108 messages
stat
+OK 1108 194519602
retr 1108
+OK 1115 octets
Return-Path: <anonimus@spacesome.ro>
X-Spam-ASN: AS12675 85.122.16.0/20
X-Spam-Checker-Version: SpamAssassin 3.2.5 (2008-06-10) on thor.info.uaic.ro
X-Spam-Level: ***
X-Spam-Status: No, score=3.1 required=4.5 tests=BAYES_40,MISSING_SUBJECT,
        NO_DNS_FOR_FROM shortcircuit=no autolearn=no version=3.2.5
X-Original-To: adria@thor.info.uaic.ro
Delivered-To: adria@thor.info.uaic.ro
Received: from fenrir.info.uaic.ro (fenrir.info.uaic.ro [85.122.23.145])
        by thor.info.uaic.ro (Postfix) with ESMTP id C71932FC61
        for <adria@thor.info.uaic.ro>; Mon,  6 Dec 2010 10:51:37 +0200 (EET)
Received: by fenrir.info.uaic.ro (Postfix)
        id B2F69CFB6; Mon,  6 Dec 2010 10:51:35 +0200 (EET)
Delivered-To: adria@infoiasi.ro
Received: from fenrir.info.uaic.ro (fenrir.info.uaic.ro [127.0.0.1])
        by fenrir.info.uaic.ro (Postfix) with SMTP id 0164CD00C
        for <adria@infoiasi.ro>; Mon,  6 Dec 2010 10:48:53 +0200 (EET)
Message-Id: <20101206084910.0164CD00C@fenrir.info.uaic.ro>
Date: Mon,  6 Dec 2010 10:48:53 +0200 (EET)
From: anonimus@spacesome.ro
To: undisclosed-recipients:;
Status:  O

Un mesaj din univers:)
.
```

Parola necriptata

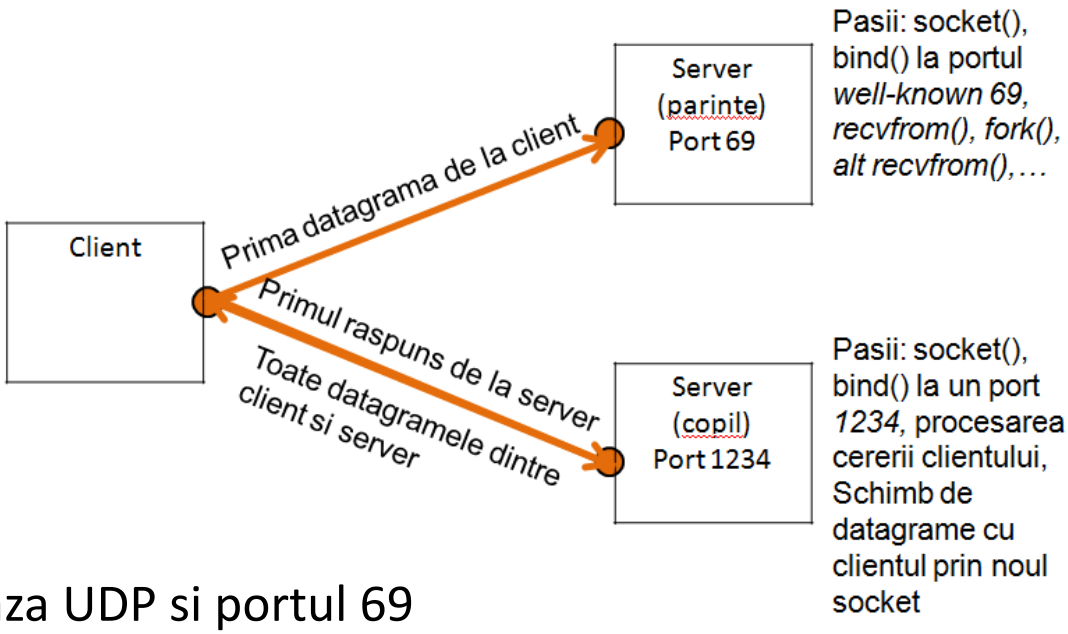
E-mail|POP

- POP 3 - caracteristici:
  - In general, daca utilizatorul schimba clientul el nu-si mai poate reciti mailurile; Obs: Clienti cu optiunea: '*keep a copy of the email on the server*'
  - Foloseste mecanismul "*download-and-keep*": copierea mesajelor pe clienti diferiti
  - POP3 este fara stare intre sesiuni
- Alte solutii:

  IMAP (Interactive Mail Access Protocol) – RFC 1730
  - Pastreaza toate mesajele intr-un singur loc: pe server
  - Permite utilizatorului sa organizeze mesajele in directoare
  - Pastreaza starea "utilizatorului" intre sesiuni
    - Numele directoarelor si maparea dintre ID-urile mesajelor si numele folderului

Transferul de fisiere|TFTP

- TFTP (Trivial File Transfer Protocol) -> …Cursul 6 & RFC 1350



  - utilizeaza UDP si portul 69
  - utilizat deseori la initializarea statiilor de lucru fara disc sau a altor dispozitive
  - nu are mecanisme de autentificare si criptare => este utilizat in retele locale
  - RFC 1785, 2347, 2348, 2349

Transferul de fisiere|TFTP

- TFTP (Trivial File Transfer Protocol)

Implementarile TFTP utilizeaza comenzi de tipul:

`Connect <host>`	Specifies the destination host ID.
`Mode <ascii|binary>`	Specifies the type of transfer mode.
`Get <remote filename> [<local filename>]`	Retrieves a file.
`Put <remote filename> [<local filename>]`	Stores a file.
`Verbose`	Toggles verbose mode, which displays additional information during file transfer, on or off.
`Quit`	Exits TFTP.

– RFC 1785, 2347, 2348, 2349

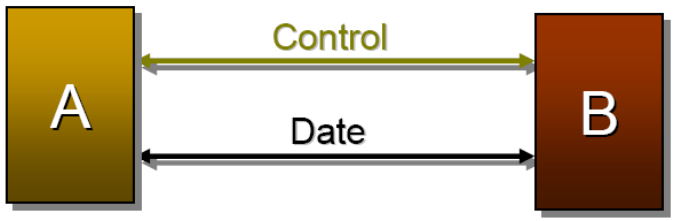
[TCP/IP Tutorial and Technical Overview, IBM, 2006]

Transferul de fisiere|FTP

FTP – caracterizare

- Folosit atat interactiv, cat si de programe
- Asigura transferul sigur si eficient al fisierelor
- Se bazeaza pe modelul client/server
- FTP utilizeaza doua conexiuni TCP pentru transferul fisierelor:
  - **Conexiune de control**
    - folosita pentru trimiterea comenzilor si receptionarea codurilor de stare
    - Conexiunea de control utilizeaza portul 21
  - **Conexiunea de date**
    - folosita pentru transferul efectiv
    - conexiunea de date foloseste portul 20

    sau unul aleator (P > 1023)
    - nu este obligatorie intr-o sesiune FTP



Transferul de fisiere|FTP

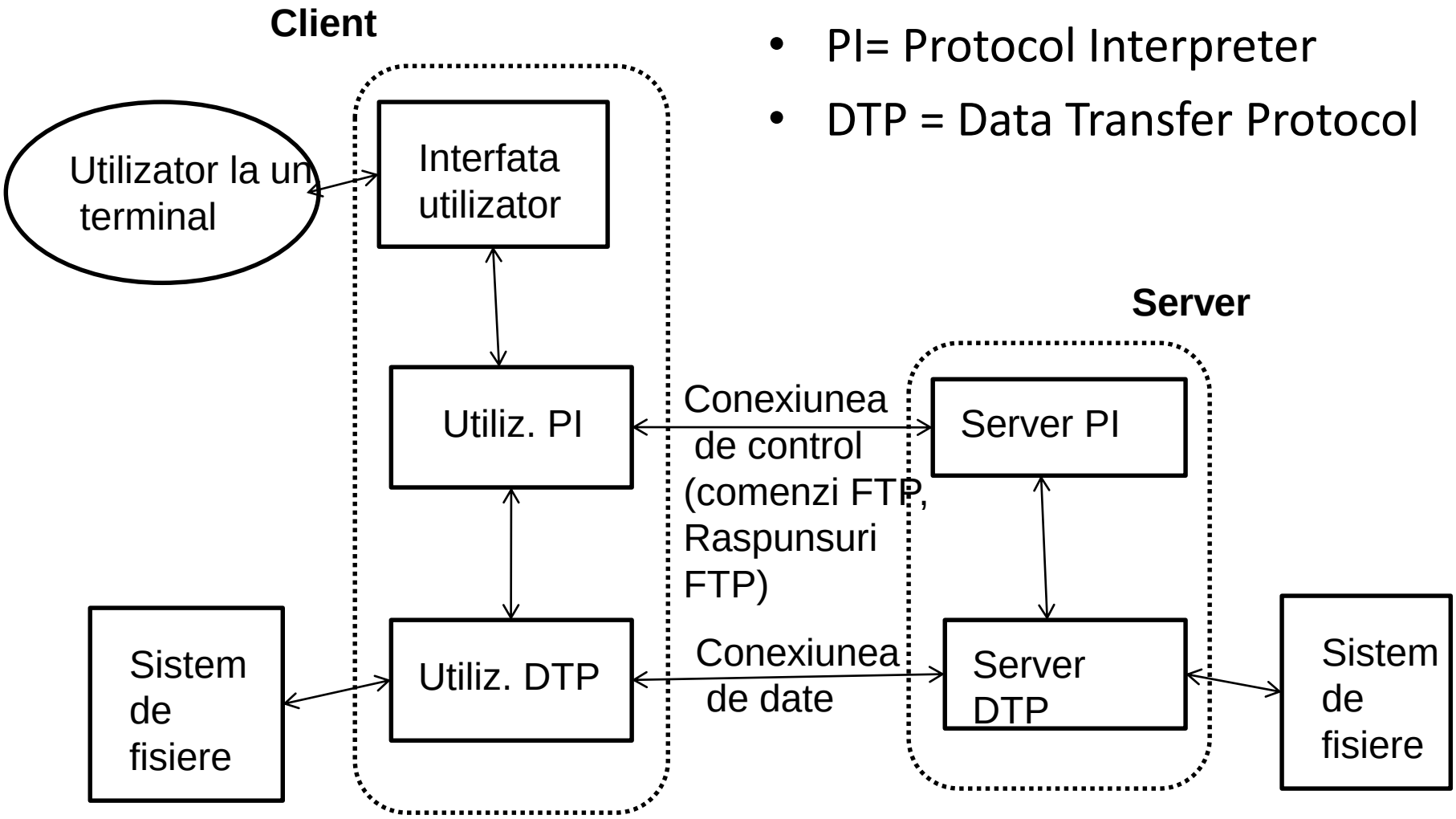
FTP – caracterizare

- Comenzile si raspunsurile sunt linii de text
- Obs. (FTP->)file transfer != file access (->NFS)
- Vezi si RFC 956, 1068, 2228, (FTP Security Extensions), 2428 (FTP Extensions for IPv6 and NATs)
- Pentru interactivitate se foloseste protocolul TELNET

  Tipuri de acces:
  - Anonim (FTP anonymous) – RFC 1635
    - Autentificare cu numele *anonymous* si drept parola o adresa de e-mail
    - Acces public la o serie de resurse (aplicatii, date, multimedia etc.)
  - Autentificat
    - Necesita un nume de utilizator existent, insotit de o parola valida
    - Pentru transferul de date in/din contul personal

Transferul de fisiere|FTP

FTP- model



- PI= Protocol Interpreter
- DTP = Data Transfer Protocol

Transferul de fisiere|FTP

FTP – comenzi (client)

```
adria@thor:~$ ftp
ftp> help
Commands may be abbreviated.  Commands are:

!               debug           mdir            qc              send
$               dir             mget            sendport        site
account         disconnect      mkdir           put             size
append          exit            mls             pwd             status
ascii           form            mode            quit            struct
bell            get             modtime         quote           system
binary          glob            mput            recv            sunique
bye             hash            newer           reget           tenex
case            help            nmap            rstatus         tick
cd              idle            nlist           rhelp           trace
cdup            image           ntrans          rename          type
chmod           lcd             open            reset           user
close           ls              prompt          restart         umask
cr              macdef          passive         rmdir           verbose
delete          mdelete         proxy           runique         ?
ftp>
```

Transferul de fisiere | FTP

FTP – comenzi uzuale (client)

Command	Meaning
Open	Create an FTP connection between the two hosts.
Close	Close an FTP connection between two hosts.
Bye	End the FTP session.
Get	Retrieve a remote file from the remote host.
Put	Store a file on the remote host.
Mget	Get multiple files using wildcards (for example, mget a* fetches all files that being with the letter "a" in the current directory).
Mput	Put multiple files on the remote host using wildcards.
Glob	Enable wildcard interpretation. This is usually on by default.
Ascii	The file transferred is in ASCII representation (a common default).
Binary	The file is in image (binary) format (sometimes the default), and is useful for programs and formatted word processing files.
Cd	Change the directory on the remote host.
Dir	Get a directory listing from the remote host.
Ldir	Get a directory listing from the local host.
Hash	Display hash marks (dots) to show file transfer progress.

→ **RETR** (*retrive*)
→ **STOR** (*store*)

Transferul de fisiere|FTP

FTP – comenzi (protocol)

- Comenzi de control al accesului
  - USER *username*, PASS *password*, QUIT, ChangeWorkingDir,…
- Comenzi de transfer a parametrilor
  - PORT, TYPE, MODE
- Comenzi de realizarea a serviciilor FTP
  - RETR *filename*, ABOR, STOR *filename*, LIST, PrintWorkingDir

Raspunsul de stare

Linie de text continind:

XYZ un cod de stare (utilizat de software) + un mesaj explicativ (destinat oamenilor)

Transferul de fisiere|FTP

FTP – codul de stare (xyz)

Prima cifra semnifica:

1 replica pozitiva preliminara (“am indeplinit, dar asteapta”)

2 replica pozitiva finala (“succes”)

3 replica pozitiva intermediara (“am nevoie si de alte informatii”)

4 replica negativa tranzitorie (“eroare, incerc iar”)

5 replica negativa finala (“eroare fatala”)

Transferul de fisiere|FTP

FTP – codul de stare (xyz)

A doua cifra specifica grupuri de functii:

- **0** erori de sintaxa
- **1** informare (ajutor, informatii de stare)
- **2** referitor la conexiuni
- **3** privitor la autentificarea utilizatorului
- **4** nespecificat
- **5** referitor la sistemul de fisiere

Transferul de fisiere|FTP

FTP – codul de stare (**xy**z)

A treia cifra da informatii suplimentare asupra mesajelor de eroare

Exemple:

125 Conexiune deschisa; transfer pornit

200 Comanda OK

226 Transfer complet

331 Nume utilizator OK, se cere parola

452 Eroare la scrierea fisierului

500 Eroare de sintaxa (comanda necunoscuta)

501 Eroare sintaxa (argumente invalide)

221 Goodbye /*rezultat al comenzii QUIT */

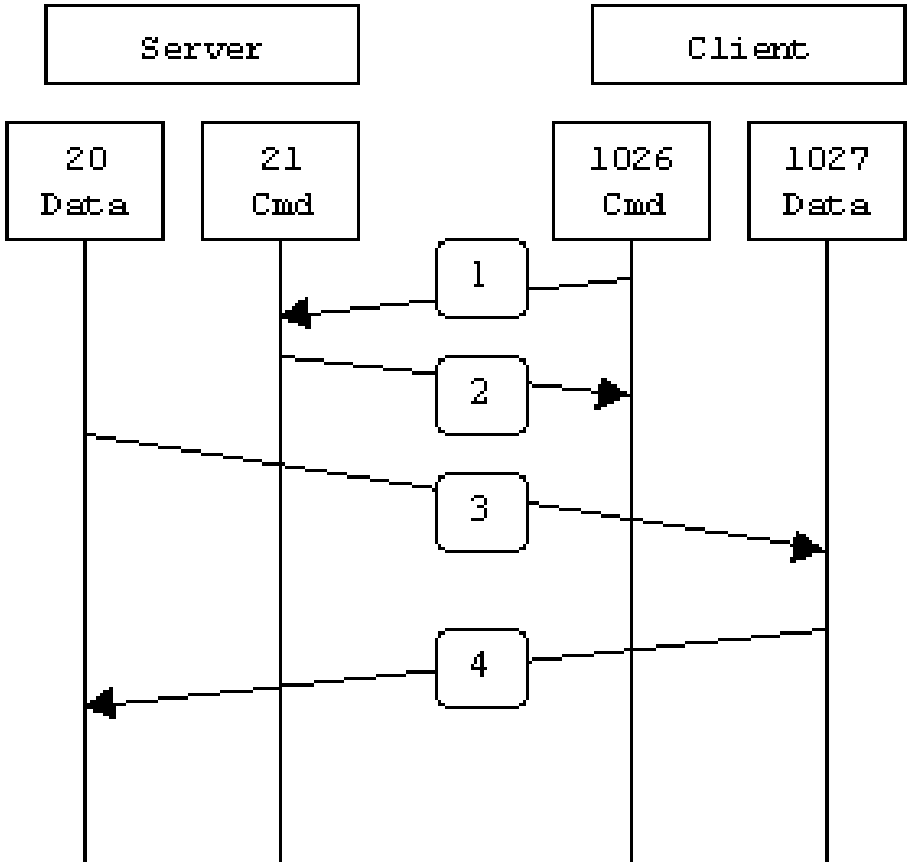
Transferul de fisiere|FTP

FTP – Moduri de transfer

- **STREAM**
  - Fisierul este trimis ca un flux de octeti; sfirsitul transmisiei este indicat de inchiderea normala a conexiunii;
- **BLOCK**
  - Fisierul este transmis ca o serie de blocuri de date precedate de antete continand contoare si descriptori de bloc (e.g. End of data block)
- **COMPRESSED**
  - Fisierele sunt compresate, conform unui algoritm de compresare (e.g., gzip) si sunt trimise ca date binare

Transferul de fisiere|FTP

Active FTP – exemplu

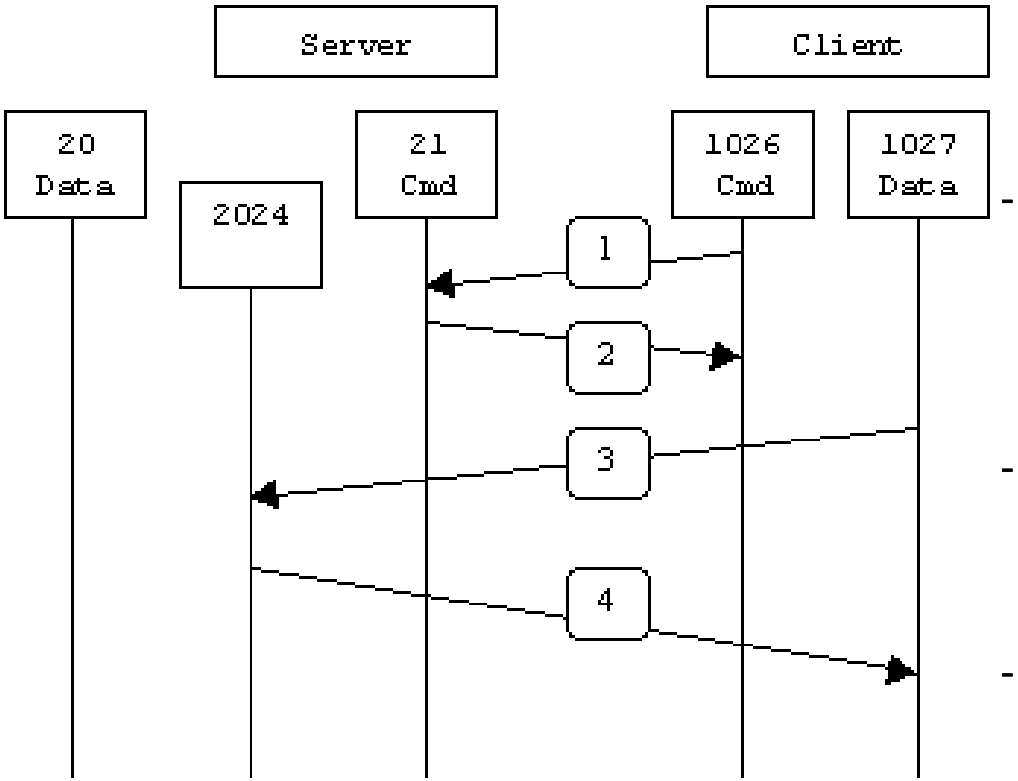


- Clientul se conecteaza la un server (85.122.23.145:**21**) de la un port P > 1023
- Clientul trimite comanda PORT 85.122.23.1.**4.2** (4*256+2=**1026**) ce indica Server-ului sa initieze o conexiune cu clientul la portul P+1
- Clientul asculta la P+1 si primeste datele trimise de server prin portul **20**

Obs. Conexiunea initiata de server poata fi interpretata ca un potential atac de firewall-ul clientului

Transferul de fisiere|FTP

Pasive FTP – exemplu



- La initierea unei conexiuni FTP clientul foloseste doua porturi (P>1023 si P+1)
- Clientul se conecteaza la un server (85.122.23.145:21) de la portul P si trimite comanda PASV
- Serverul deschide un port **Ps** > 1023 si trimite comanda PORT Ps clientului
- Clientul va initia o conexiune (de la portul P+1) cu serverul folosind portul primit (Ps)

HTTP

- **Hyper Text Transfer Protocol**
  - Protocol utilizat in Internet, bazat pe stiva TCP/IP
  - Sta la baza comunicarii dintre serverele si clientii Web
    - Client: in mod uzual poate fi un browser
    - Server: server Web care trimite raspunsuri la cererile primite
  - HTTP 1.0 - RFC 1945
  - HTTP 1.1 - RFC 2616
  - HTTP 1.1 revised - RFC 723X (https://www.w3.org/Protocols/)

Protocolul HTTPS – asigura comunicatii “sigure” HTTP via TLS (Transport Layer Security):

- autentificare pe baza certificatelor digitale + criptare bidirectionala
- RFC 2818 – https://tools.ietf.org/html/rfc2818

HTTP

- Protocolul SPDY – un experiment Google, disponibil ca Internet Draft la care Google a renuntat in 2016
  - Reducerea latentei incarcarii si cresterea securitatii
  - https://www.chromium.org/spdy
  - Implementari SPDY existau in **:** Chrome, Mozilla Firefox, Opera, Amazon Silk, Internet Explorer

- **Protocolul HTTP/2.0**
  - RFC 7540
  - Extinde ideile SPDY, focalizat asupra performantei
  - www.slideshare.net/mnot/what-http20-will-do-for-you

HTTP

- **Hyper Text Transfer Protocol**

  Mecanism general:

  Clientul initiaza o conexiune TCP cu serverul folosind portul 80

  Serverul accepta conexiunea TCP

  Are loc schimbul de mesaje HTTP intre clientul HTTP (browser) si server-ul Web

  Se inchide conexiunea TCP

HTTP

- HTTP – nu se ocupa de partea de rutare sau verificarea cererilor
  - ? Cine: TCP&IP
  - HTTP lucreaza cu cereri la nivel inalt: *Fetch IndexPage* al https://www.google.com
  - *Live HTTP Headers* (Firefox) ->

http://127.0.0.1:8080/Curs_ProgramareJS/test.html

```
GET /Curs_ProgramareJS/test.html HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: Mozilla/5.0 (Windows NT 6.2; WOW64; rv:23.0) Gecko/20100101 Firef
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive

HTTP/1.1 200 OK
Date: Thu, 05 Sep 2013 15:57:00 GMT
Server: Apache/2.4.3 (Win32) OpenSSL/1.0.1c PHP/5.4.7
Last-Modified: Thu, 05 Sep 2013 15:51:41 GMT
Etag: "95-4e5a4e5efb9d1"
Accept-Ranges: bytes
Content-Length: 149
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html
```

http://127.0.0.1:8080/Curs_ProgramareJS/imgsrc2.jpg

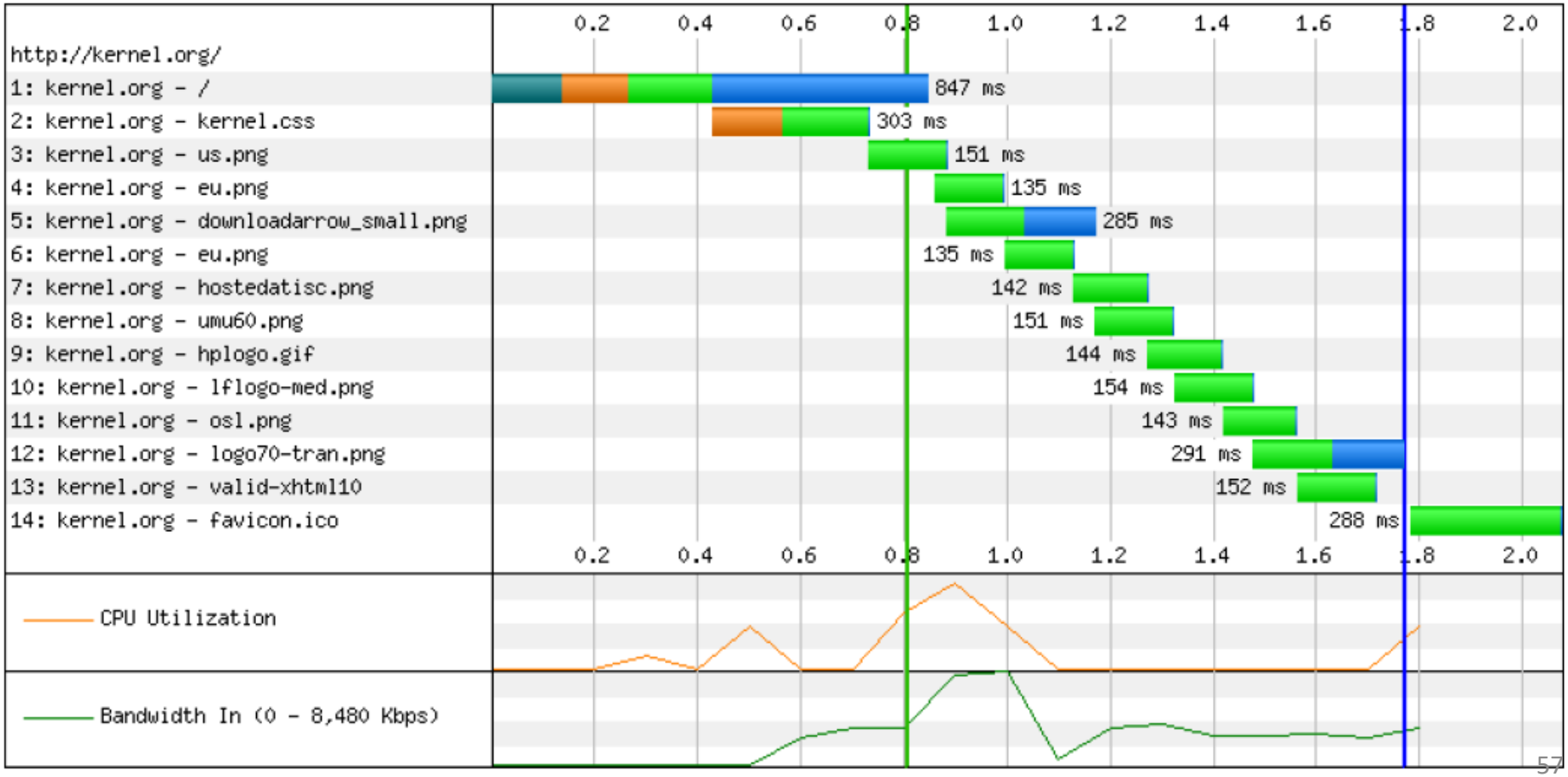
```
GET /Curs_ProgramareJS/imgsrc2.jpg HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: Mozilla/5.0 (Windows NT 6.2; WOW64; rv:23.0) Gecko/20100101 Firefox/23.0
Accept: image/png,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://127.0.0.1:8080/Curs_ProgramareJS/test.html
Connection: keep-alive

HTTP/1.1 200 OK
Date: Thu, 05 Sep 2013 15:57:00 GMT
Server: Apache/2.4.3 (Win32) OpenSSL/1.0.1c PHP/5.4.7
Last-Modified: Wed, 28 Aug 2013 06:23:31 GMT
Etag: "3c9e-4e4fc0742da40"
Accept-Ranges: bytes
Content-Length: 15518
Keep-Alive: timeout=5, max=99
Connection: Keep-Alive
Content-Type: image/jpeg
```

Detalii asupra portului: https://www.grc.com/port_8080.htm

Web Browser| Download

- Modul cum browserul reda un website este esential in procesul de optimizare

Exemplu: download resursa http://kernel.org



Web Browser| Download

- Pasii realizati de browser:
  - Rezolvarea *kernel.org* folosind DNS pentru aflarea IP (primul segment)
  - Al doilea segment indica incercarea de a crea o conexiune HTTP catre kernel.org
  - La inceputul celui de-al treilea segment, conexiunea TCP a fost creata si browserul isi poate primi raspunsul; in cazul nostru poate datorita latentei serverului, abia la la inceputul celui de-al patrulea segment serverul web trimite continutul
  - Total: 847 milisecunde (ms) – si documentul HTML a fost obtinut
- Obs. In general paginile web constau din legaturi catre foi de stiluri, imagini, JavaScript etc.
  - Imediat ce documentul HTML incepe sa vina, browserul incepe operatia de fetch pe alta resursa (kernel.css in cazul nostru)
    - Obs. De data aceasta nu mai sunt intarzieri datorate DNS lookup, deoarece raspunsul anterior a fost plasat in cache-ul browserului
      - Sunt intarzieri datorate initierii conexunii TCP catre server

Web Browser| Download

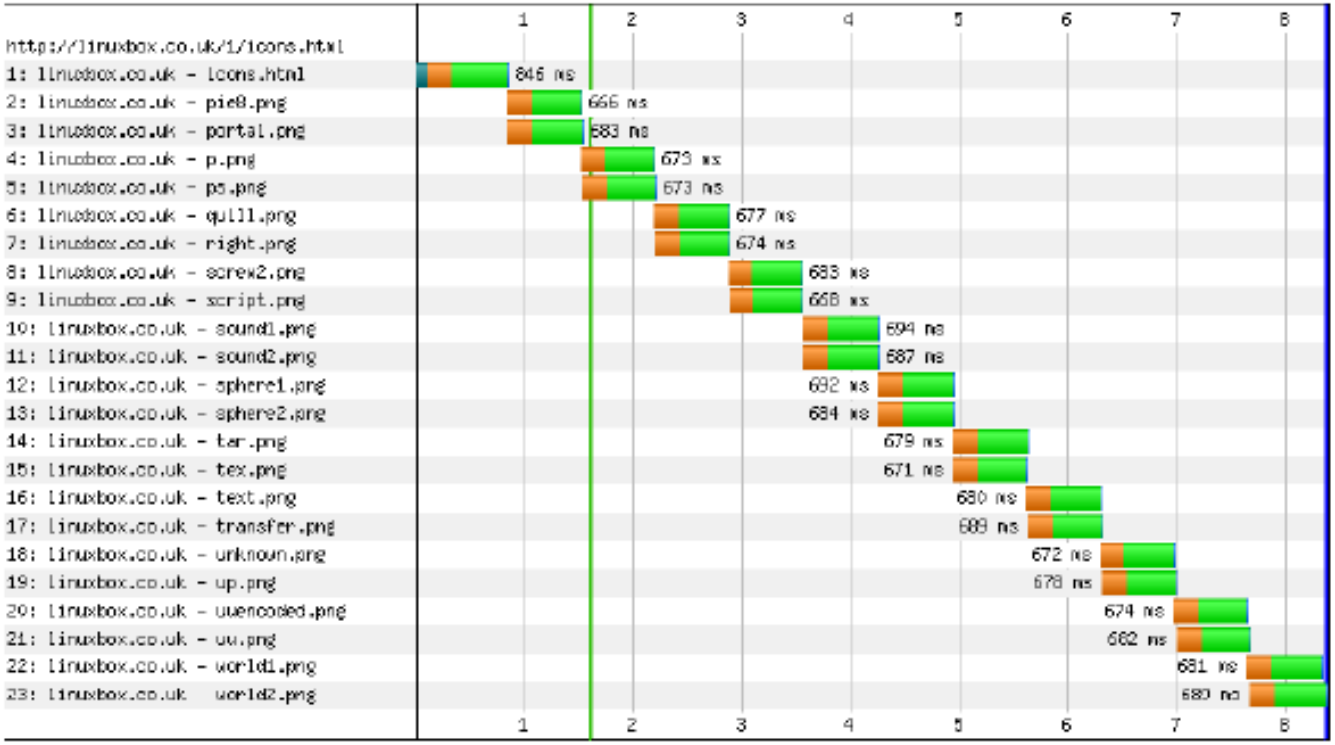
- ? De ce exista o asa mare intarziere pentru cererea ***us.png*****?**
  - “Motiv istoric”: in exemplul nostru browserul downlodeaza doar doua resurse in paralel de la acelasi host (cum s-a terminat kernel.css, incepe us.png)
  - Pentru resursele us.png si cele care urmeaza nu mai exista nici un segment de conexiune TCP ⇔ browserul reutilizeaza conexiunea TCP existenta cu serverul => o optimizare (salveaza 0.1 secunde per cerere)
  - Obs. Pentru resursele 1,5,12 timpul pentru descarcarea resursei e jumatate din intregul timp de *fetching* a resursei; la restul timpul de descarcare este insignifiant

Web Browser| Download

Conexiuni persistente si *Keep-Alive*

- In HTTP 1.0 – comportamentul implicit era ca dupa fiecare obtinere de resursa sa se inchida conexiunea

Efect => latenta in primirea raspunsului, utilizarea resurselor (CPU, RAM) la nivel de client si server



Web Browser| Download

Conexiuni persistente si *Keep-Alive*

- Problema a fost partial rezolvata prin introducerea lui *Keep-Alive*
  - Clientul include in antetul cererii campul: **Content: *Keep-Alive***
  - Daca serverul suporta acest aspect, trimite inapoi un *header* cu aceeasi valoare
    - => conexiunea ramine deschisa pana cand una din parti decide inchiderea ei
  - ? Dar daca un client nu inchide conexiunea?
    - Serverul este *idle* si consuma memorie
      - Majoritatea serverelor web implementeaza un ***Keep-Alive timeout***
      - De asemenea serverele pot limita numarul de resurse care se cer per o conexiune

        Keep-Alive: timeout = 5, max 100
    - Obs. Keep-Alive nu a fost oficial recunoscuta si nu era suportata de toti clientii

Web Browser| Download

Conexiuni persistente si *Keep-Alive*

- HTTP/1.1 a formalizat Keep-Alive => conexiuni persistente in mod implicit
- Daca un client|server nu doreste atunci poate utiliza un camp in antet:
  Connection: close
- ? Cand comportamentul implicit al lui Keep-Alive nu este de dorit?

Download Paralel

- RFC 2616: *"Clients that use persistent connections should limit the number of simultaneous connections that they maintain to a given server. A single-user client should not maintain more than 2 connections with any server or proxy…. These guidelines are intended to improve HTTP response times and avoid congestion."*
- Scopul furnizorilor de clienti browser: cresterea gradului de interactiune cu utilizatorul
  - E problema serverelor web

Web Browser| Download

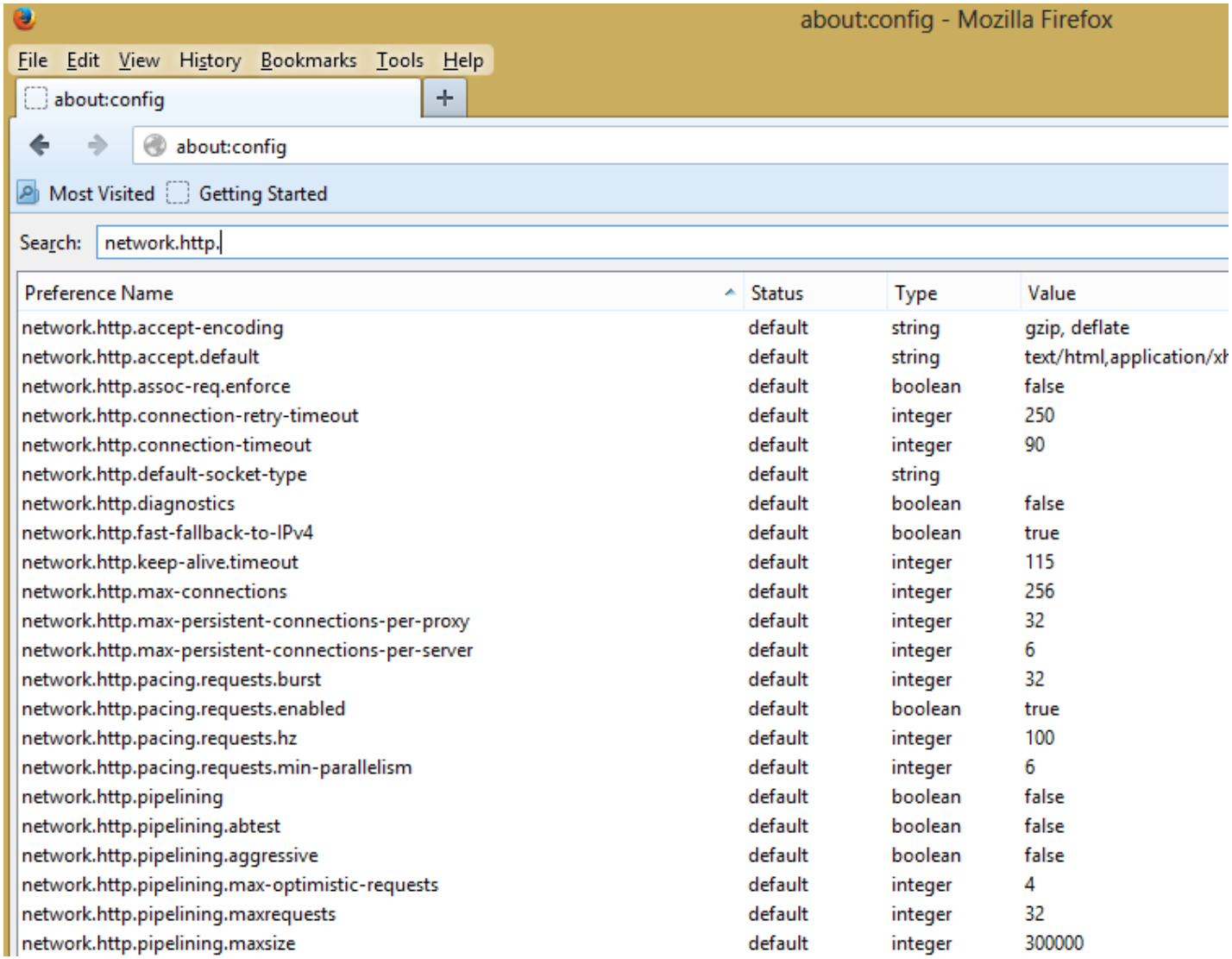
Maximul de conexiuni paralele per host:

BROWSER	MAX PARALLEL CONNECTIONS PER HOST
IE 6 and 7	2
IE 8	6
IE 9	6
IE 10	8
Firefox 2	2
Firefox 3	6
Firefox 4 to 17	6
Opera 9.63	4
Opera 10	8
Opera 11 and 12	6
Chrome 1 and 2	6
Chrome 3	4
Chrome 4 to 23	6
Safari 3 and 4	4

Web Browser| Download

Firefox:
ajustarea parametrilor vizand conexiunile HTTP

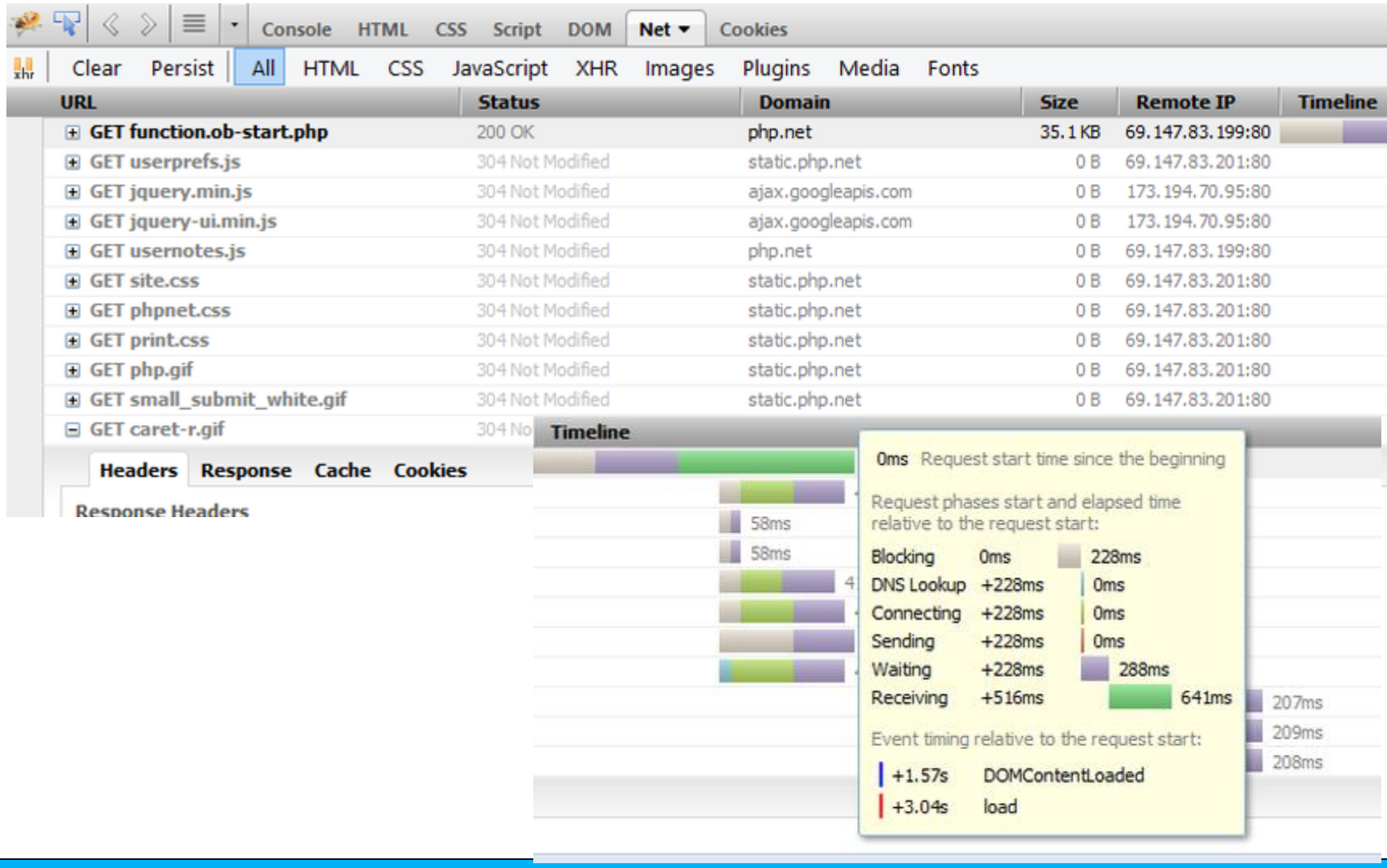
via schema URI
about:config



Preference Name	Status	Type	Value
network.http.accept-encoding	default	string	gzip, deflate
network.http.accept.default	default	string	text/html,application/xh
network.http.assoc-req.enforce	default	boolean	false
network.http.connection-retry-timeout	default	integer	250
network.http.connection-timeout	default	integer	90
network.http.default-socket-type	default	string	
network.http.diagnostics	default	boolean	false
network.http.fast-fallback-to-IPv4	default	boolean	true
network.http.keep-alive.timeout	default	integer	115
network.http.max-connections	default	integer	256
network.http.max-persistent-connections-per-proxy	default	integer	32
network.http.max-persistent-connections-per-server	default	integer	6
network.http.pacing.requests.burst	default	integer	32
network.http.pacing.requests.enabled	default	boolean	true
network.http.pacing.requests.hz	default	integer	100
network.http.pacing.requests.min-parallelism	default	integer	6
network.http.pipelining	default	boolean	false
network.http.pipelining.abtest	default	boolean	false
network.http.pipelining.aggressive	default	boolean	false
network.http.pipelining.max-optimistic-requests	default	integer	4
network.http.pipelining.maxrequests	default	integer	32
network.http.pipelining.maxsize	default	integer	300000

Web Browser| Download

Firefox –> Firebug



HTTP

- **Hyper Text Transfer Protocol**

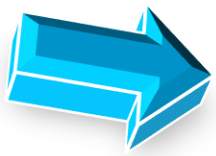
  Conexiunile HTTP sunt persistente

  ⇒Inchiderea si deschiderea a mai putine conexiuni TCP=> timp de CPU este salvat in rutere si host-uri (clienti, servere, proxy, ...) si se economiseste memoria folosita

  ⇒Clientii pot face cereri multiple in cadrul aceleiasi conexiuni, fara a astepta raspunsul pentru fiecare

  ⇒Congestia in retea este redusa datorita numarului de pachete mai mic

  ⇒Cererile se desfasoara mai rapid deoarce nu mai este nevoie de un *handshake* la fiecare cerere

Mai multe detalii?  **Cursul de Tehnologii Web!**

Rezumat

- **Protocoale la nivelul aplicatie**
  - Preliminarii
  - Caracteristici de proiectare
  - Accesul la terminal de la distanta
  - Posta Electronica
    - SMTP (Simple Mail Transfer Protocol)
    - POP (Post Office Protocol)
  - Transferul de fisiere
    - TFTP (Trivial File Transfer Protocol)
    - FTP (File Transfer Protocol)
  - World-Wide Web (HTTP)
  - Privire de ansamblu

Bibliografie

Content Networking Fundamentals, Silvano Da Ros, Publisher: Cisco Press Pub Date: March 30, 2006 Print ISBN-10: 1-58705-240-7 Print ISBN-13: 978-1-58705-240-8 Pages: 576

Computer and Communication Networks, Nader F. Mir, Publisher: Prentice Hall Pub Date: November 02, 2006 Print ISBN-10: 0-13-174799-1 Print ISBN-13: 978-0-13-174799-9 Pages: 656

TCP/IP Tutorial and Technical Overview, IBM, 2006

Network + Guide to Networks, Tamara Dean, 2009

Intrebari?

Paradigma P2P

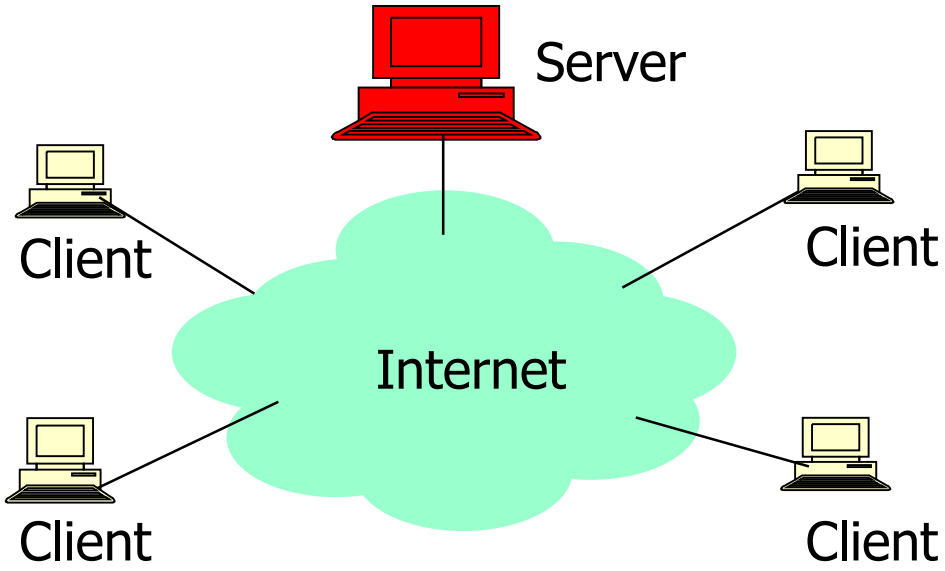
Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- **Paradigma peer-to-peer (P2P)**
  - Preliminarii
  - Definitii
  - Caracterizare
  - Tipuri de aplicatii
  - Infrastructuri
  - Instrumente

Preliminarii

...sa ne reamintim modelul client/server



Preliminarii

...sa ne reamintim modelul client/server

- Uzual, privim clientul ca fiind o componenta avand capacitati computationale reduse
- Serverul este mentinut si administrat in mod centralizat

Probleme ale arhitecturii client/server:

- Lipsa robustetii
- Lipsa sigurantei (eng. Reliability)
- Lipsa scalabilitatii
- Vulnerabilitate la atac

Definitii

Peer *= one that is of equal standing with another* (conform Webster)

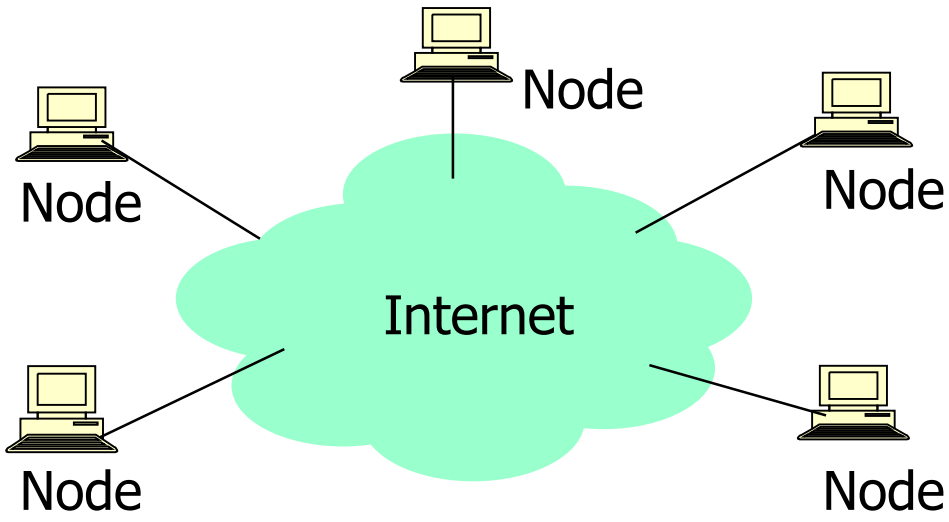
Peer-to-peer (P2P) = arhitectura de retea in care nodurile sunt relativ egale

- In sensul ca fiecare nod este, in principiu, capabil sa realizeze functii specifice retelei

Definitii

Sistemele P2P, in sens strict, sunt sisteme complet distribuite

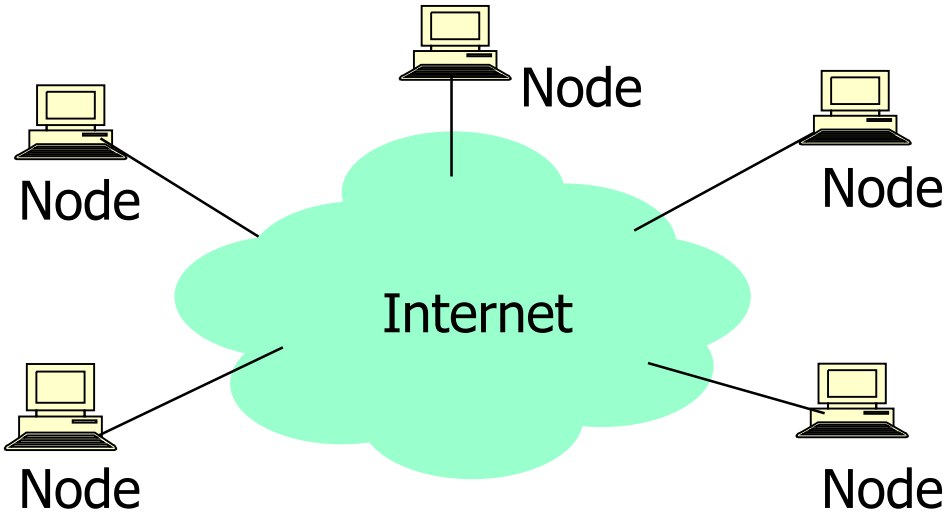
- Toate nodurile sunt total echivalente, in termeni de functionalitate si a activitatilor pe care le pot desfasura



Obs.: Sistemele P2P pure sunt rare (de ex. Gnutella); majoritatea sunt hibride, avand supernoduri sau servere cu diferite roluri (de ex. cautare de date, control, etc.)

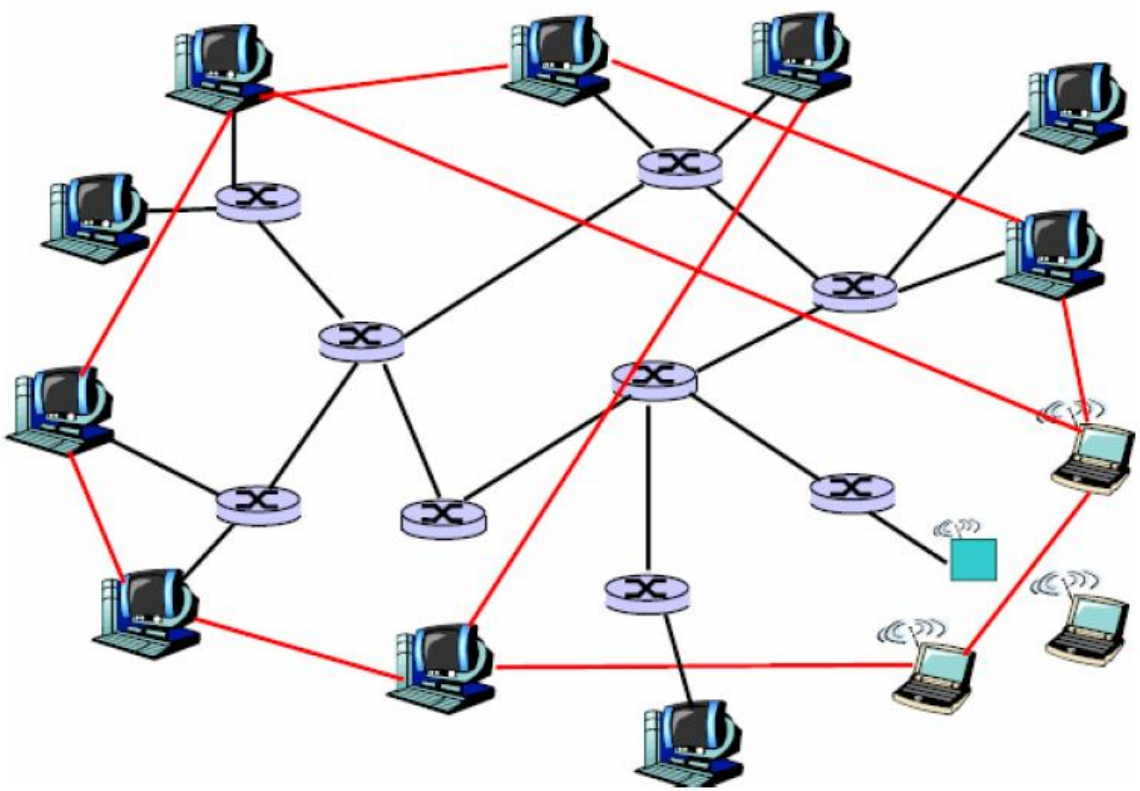
Definitii

- Nodurile
  - Pot consuma si oferi date
  - Orice nod poate initia o conexiune
- Nu exista sursa de date centralizata =>
  - Forma de democratie pe Internet
  - Protectie *copyright* amenintata



Definitii

- "**P2P** este clasa de aplicatii care se bazeaza pe resursele (de stocare, de procesare, continut, prezente umane) disponibile la marginile (edges) Internet-ului



Edges of the Internet
(*overlay networks*)

Definitii

"**P2P** is a class of applications that take advantage of resources – storage, cycles, content, human presence – available at the edges of the Internet. Because accessing these decentralized resources means operating in an environment of unstable and unpredictable IP addresses, P2P nodes must operate outside the DNS system and have significant, or total autonomy from central servers"

"A distributed network architecture may be called a **P2P** network if the participants share a part of their own resources. These shared resources are necessary to provide the service offered by the network. The participants of such a network are both resource providers and resource consumers"

Caracterizare

Caracteristici definitorii:

- Partajarea resurselor computationale prin interschimb direct si mai putin prin intermedieri oferite de o autoritate centralizata (server)
  - Serverele centralizate pot fi folosite insa pentru a realiza activitati specifice (initializarea retelei P2P, adaugarea de noi noduri in retea,…)
  - Ideal, nodurile participa activ si unilateral la realizarea de operatii ca localizarea & caching-ul nodurilor/continutului, dirijarea informatiilor, managementul resurselor transferate etc.

Caracterizare

Caracteristici definitorii:

- Abilitatea de a trata instabilitatea si variatiile conctivitatii retelei, adaptandu-se automat la erorile survenite sau la dinamicitatea nodurilor
  - Topologia retelei P2P e adaptiva si toleranta la defecte, nodurile auto-organizandu-se in vederea mentinerii conectivitatii si performantei retelei

Caracterizare

Reteaua P2P este una suprapusa (*overlay*) peste cea fizica

- Se situeaza la nivelul aplicatie => flexibilitate
- Muchiile virtuale sunt conexiuni TCP sau pointeri la adrese IP
- Mentinerea retelei P2P se face prin verificarea periodica a conectivitatii (*ping*) ori a existentei (mesaje "*still alive*?")
- Cand un nod pica, sistemul P2P ar putea stabili noi muchii
- Proximitatea (fizica) a nodurilor nu e importanta
- Reteaua P2P poate fi structurata sau nu

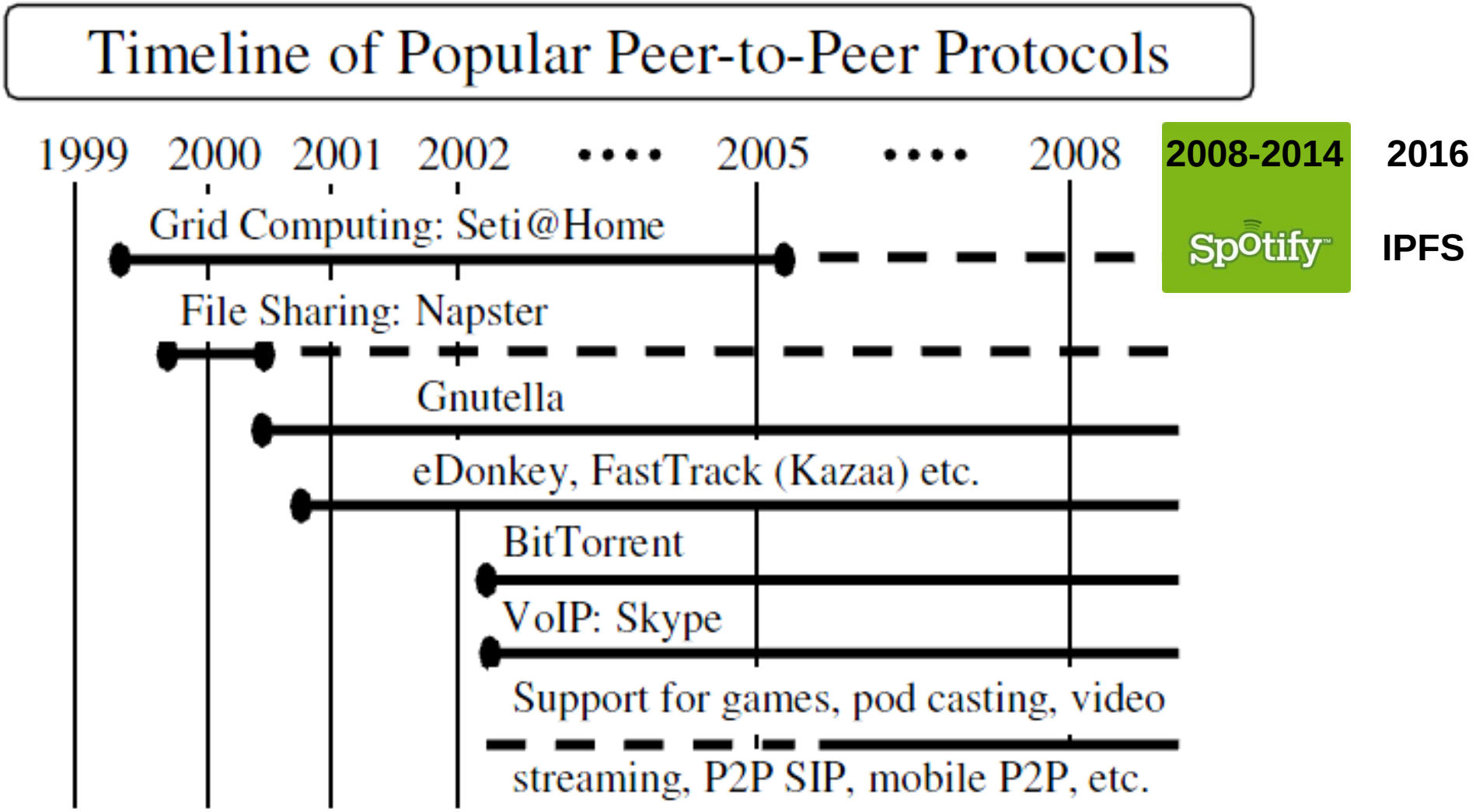
Scopuri si beneficii

- **Utilizarea eficienta a resurselor**
  - Latimea de banda neutilizata, resurse de stocare, putere de procesare disponibile la marginile (*edges*) retelei
- **Scalabilitate**
  - Fara informatii centralizate, fara *bottleneck*-uri (de comunicare si de calcul)
  - Agregarea resurselor se face in mod natural odata cu utilizarea sistemului
- **Siguranta** (eng. Reliability)
  - Existenta de copii a datelor
  - Distribuire geografica
  - Nu mai exista "*single point of failure*"
- **Administrare usoara**
  - Nodurile se auto-organizeaza
  - Cresterea tolerantei la erori si a echilibrarii incarcarii
  - Cresterea autonomiei
- **Anonimitatea**
  - Greu de realizat intr-un sistem centralizat
- **Dinamism**
  - Mediu dinamic
  - Colaborare si comunicare ad-hoc

Dezavantaje/Probleme

- Arhitecturile P2P sunt probabilistice
  - Localizare impredictibila a resurselor
  - Resursele sunt volatile
- Inexistenta unui control centralizat
  - Probleme privind impunerea unei autoritati asupra aplicatiilor, continutului si utilizatorilor
  - Dificultati in detectarea si identificarea utilizatorilor (aspecte anti-sociale)
- Incurajarea folosirii sistemelor P2P in scop abuziv si ilegal (e.g. drepturile de autor asupra continutului digital)
- Lipsa increderii la nivel comercial, de afaceri
- Probleme de securitate (curs viitor)

Evolutie…


Timeline of Popular Peer-to-Peer Protocols
1999
2000
2001
2002
2005
2008
Grid Computing: Seti@Home
File Sharing: Napster
Gnutella
eDonkey, FastTrack (Kazaa) etc.
BitTorrent
VoIP: Skype
Support for games, pod casting, video
streaming, P2P SIP, mobile P2P, etc.
2008-2014
Spotify
2016
IPFS

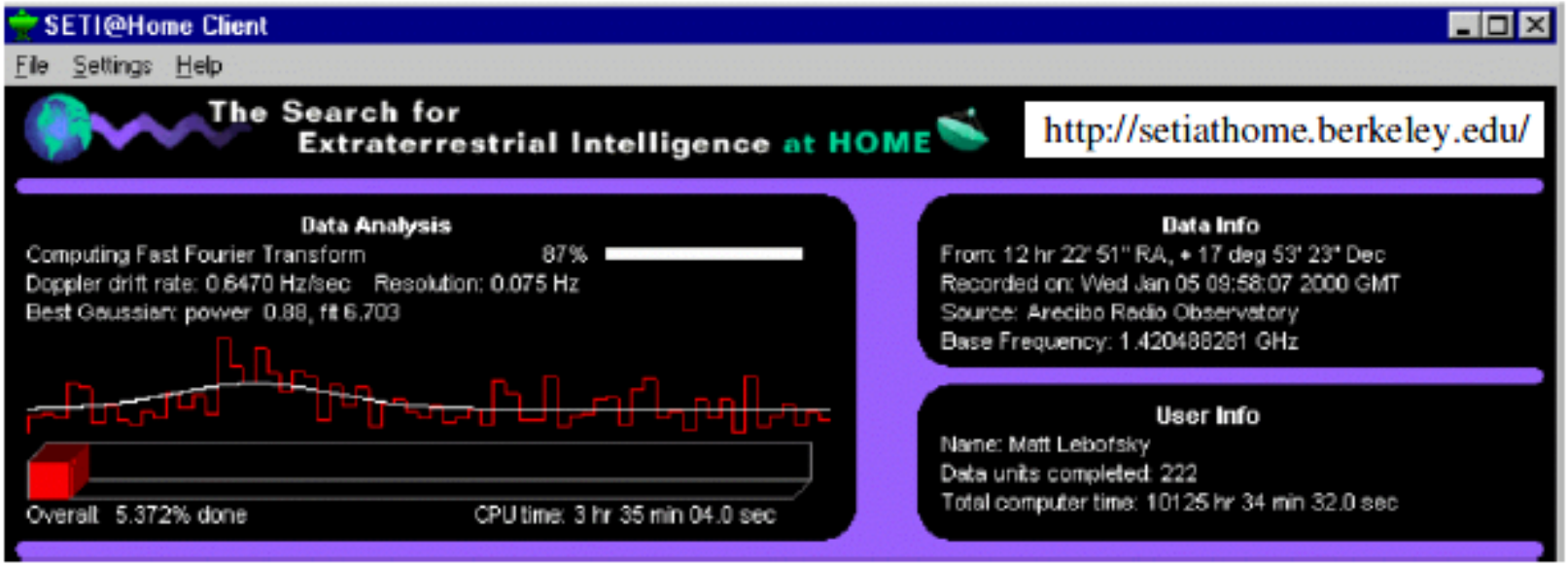
Tipuri de aplicatii

- **Comunicare & colaborare**
  - Sisteme ce ofera o infrastructura pentru facilitarea comunicarii & colaborarii directe, deseori in timp real, intre noduri
    - Sisteme conversationale (chat, mesagerie instantanee):

    IRC (Internet Relay Chat), ICQ (1996), YM!, MSN Messenger, Skype, Sisteme multicast P2P (e.g. Cirrus – Adobe Flash http://labs.adobe.com/technologies/cirrus/; WebRTC), …
- **Calcul distribuit**
  - Sisteme ce folosesc puterea computationala a nodurilor disponibile (cicli de procesor)
    - Rezolvarea unor probleme prin *divide-et-impera*: SETI@home (*Search for Extra-Terrestrial Intelligence*-Berkeley), genome@home
    - Reteaua P2P reprezinta un gen de Grid computational (…curs master)

Tipuri de aplicatii| Calcul distribuit - Exemplu

SETI@Home: A Public-Resource Computing Experiment



- Radio telescope signal analysis has insatiable appetite for computing power
- Usage of computers in homes and offices around the world has provided unprecedented computing power
- Grid computing application via peer-to-peer approach under central control

Tipuri de aplicatii

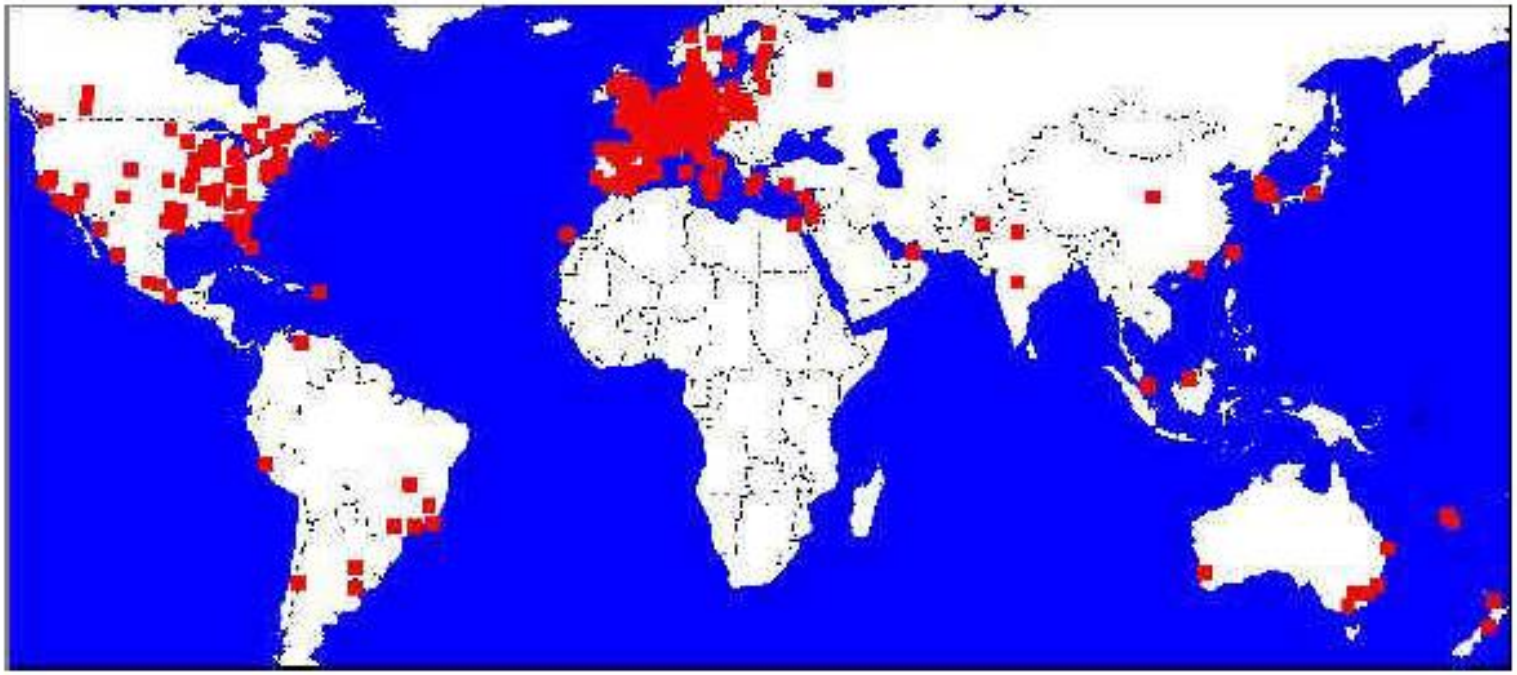
- **Sisteme de stocare (baze de date)**
  - Proiectare de sisteme de baze de date distribuite bazate pe infrastructuri P2P
    - PIER – motor scalabil de interogare distribuita (http://pier.cs.berkeley.edu/)
    - Edutella – proiect open-source pentru interogari si stocare de *meta-date* (P2P pentru Semantic Web)
- **Distribuire de continut digital**
  - Sisteme & infrastructuri pentru partajarea resurselor digitale (multimedia si alte date) intre utilizatori
    - Aplicatii pentru partajarea fisierelor (e.g. Napster, Gnutella, KaZaA, Freenet, BitTorrent, eDonkey etc.)
    - Medii de stocare distribuita pentru publicarea, organizarea, indexarea, cautarea si regasirea datelor in maniera securizata & eficienta

      (PAST, Chord, Groove, Mnemosyne, Avalanche,…)

Tipuri de aplicatii

- **Distribuire de continut digital | Exemplu**

P2P File-Sharing: Fast distribution of large files



Example: Harry Potter III early propagation after 2 hours on May 28th 2004 (Source: *www.itic.ca/DIC/News/archive.html*)

Tipuri de aplicatii

- **Distribuire de continut prin P2P**
  - Sisteme P2P de “interschimb de fisiere”
    - Nodurile transfera un fisier la un moment dat
    - Se ofera facilitati pentru realizarea unei retele P2P si pentru cautarea&transferul de fisiere intre noduri
    - Nu se ofera suport pentru securitate, disponibilitate si persistenta
    - Exemple: Napster, KaZaA, Gnutella

Tipuri de aplicatii

- **Distribuire de continut prin P2P**
  - Sisteme P2P pentru publicarea & stocarea continutului
    - Utilizatorii pot publica, stoca si distribui continut digital, pe baza unor drepturi de acces (privilegii)
    - Se focalizeaza asupra securitatii si persistentei
    - Unele ofera si facilitati privind colaborarea intre utilizatori
    - Exemple: Scan, Groove, Freenet, MojoNation, Tangler

Tipuri de aplicatii

- **Distribuire de continut prin P2P**
  - Infrastructuri pentru:
    - Dirijare & Localizare:

      Chord, Can, Pastry, Tapestry, Kademila
    - Anonimitate:

      Onion Routing, ZeroKnowledge, Freedom, Tarzan
    - Managementul reputatiei:

      Eigentrust, PeerTrust

Infrastruc.(localizare & dirijare)

- Mecanismele de localizare si dirijare ce pot fi adoptate depind de:
  - Topologia
  - Structura
  - Gradul de centralizare

  ale retelei suprapuse, acoperitoare (*overlay network*)

Infrastruc.(localizare & dirijare)

- **Aspecte privind centralizarea**
  - **Arhitecturi pur descentralizate**: toate nodurile realizeaza exact aceleasi activitati, jucand simultan roluri de servere si clienti, fara a beneficia de o coordonare centrala
    - Nodurile se numesc servents (SERVers + clieENTS)

Infrastruc.(localizare & dirijare)

- **Aspecte privind centralizarea**
  - **Arhitecturi partial centralizate**: unele noduri au un rol mai important (de ex. stocand indecsi locali pentru fisierele partajate)
    - Nodurile devin supernoduri conform politicilor fiecarui sistem P2P
    - Rolul de supernod este stabilit dinamic
  - **Arhitecturi descentralizate hibride**: exista un server central facilitand interactiunea intre noduri, mentinand cataloage de meta-date ale fisierelor
    - Serverele pot identifica si verifica nodurile de stocare
    - Sistemele se mai numesc *broker mediated*

Infrastruc.(localizare & dirijare)

- **Aspecte privind structura retelei:**
  - **Nestructurata:** plasarea continutului este complet independenta de topologia retelei suprapuse
    - Continutul trebuie localizat
    - Strategii de cautare prin "*forta bruta*": inundarea retelei – cereri propagate via BFS/DFS
    - Strategii mai sofisticate: drumuri aleatorii, probabilistice, etc.
  - **Slab structurata** (*loosely structured*): desi localizarea continutului nu e complet specificata, aceasta este afectata de dirijare
    - Categorie aflata intre retele structurate si cele nestructurate

Infrastruc.(localizare & dirijare)

- **Aspecte privind structura retelei:**
  - **Structurata:** topologia este controlata, iar fisierele (sau pointerii la ele) sunt plasate in locatii precise
    - Se realizeaza o asociere (*mapping)* intre continut (identificatorul de fisier) si locatie (adresa nodului)
      - In genul unei tabele de rutare distribuita
    - Cautarile exacte (*exact-match queries*) pot fi realizate in mod scalabil
    - Structura folosita la dirijarea eficienta a mesajelor este dificil de mentinut in cazul unor noduri tranziente, cu rata mare de atasare si deconectare de la retea

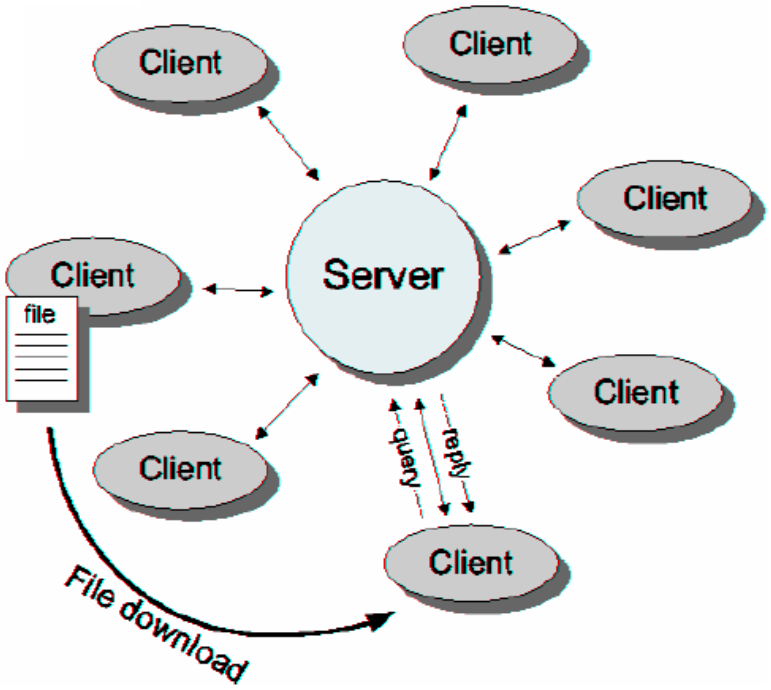
Infrastruc.(localizare & dirijare)

	Centralizare		
	Hibridă	Parțială	Absentă
Nestructurată	Napster Publius	KaZaA Morpheus Edutella	Gnutella FreeHaven
Infrastructură structurată			Chord, CAN, Tapestry, Pastry
Sisteme structurate			OceanStore Scan, PAST, Kademlia, Tarzan

Arhitecturi nestructurate

Descentralizate hibride

- Fiecare calculator client stocheaza continut (fisiere) partajat(e)
- Serverul central mentine o tabela cu conexiunile utilizatorilor inregistrati (IP, latime de banda,…) + o tabela cu lista fisierelor fiecarui utilizator &meta-date
- Exemple: **Napster, Publius**



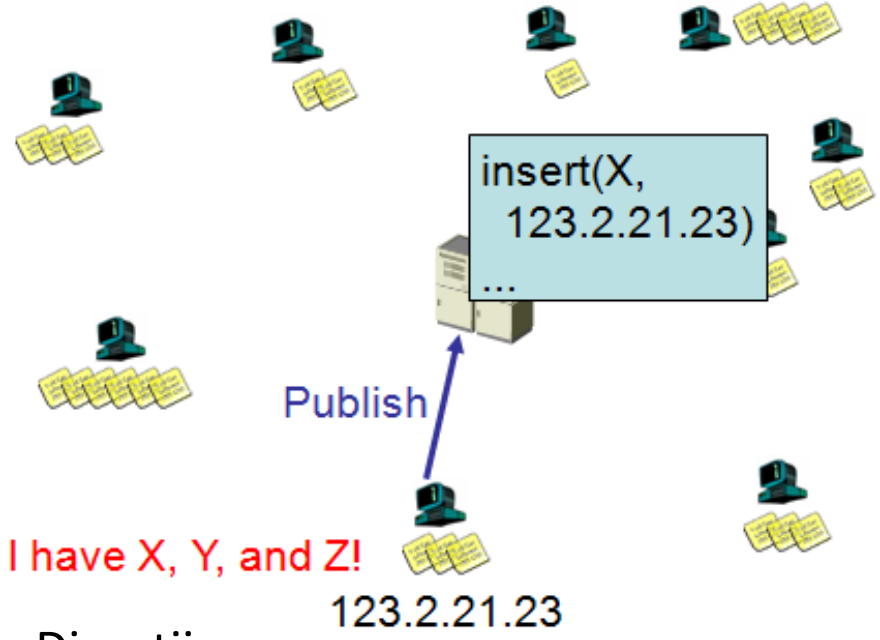
Arhitecturi nestructurate

Napster

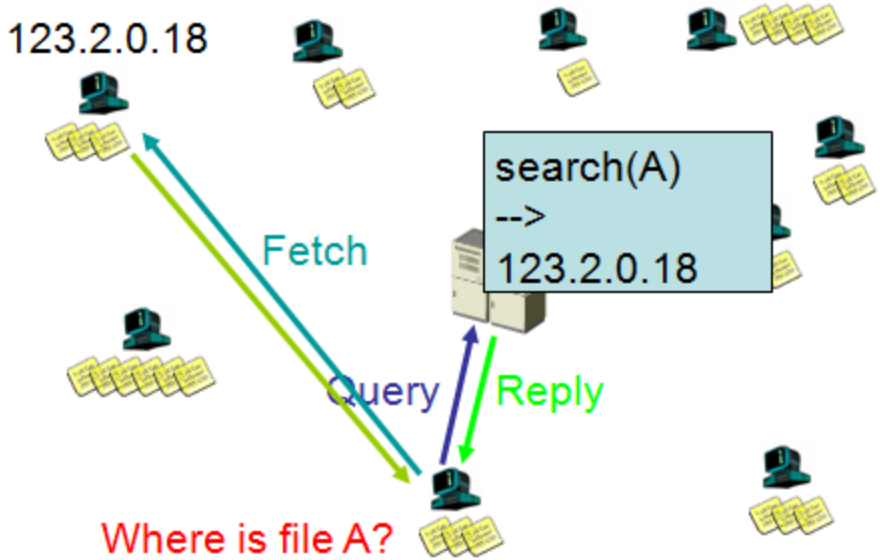
- 1999: Sean Fanning lanseaza Napster
- A atins cota de 1.5 milioane de utilizatori simultan
- Baza de date centralizata - operatii:
  - **Join**: clientul contacteaza serverul central (via TCP)
  - **Publish**: raportarea unei liste de fisiere serverului central
  - **Search**: interogarea serverului => se intoarce cineva care stocheaza fisierul cerut
  - **Fetch**: ia fisierul direct de la *peer* (cel cu cea mai buna rata de transfer)
- Iulie 2001: Napster a fost inchis

Arhitecturi nestructurate

Napster: Publish



Napster: Search



Discutii:

- Serverul face toate procesarile
- Avem "*single point of failure*"
- Probleme de scalabilitate, unele sisteme nu permit adaugarea altor servere (lista serverelor disponibile este statica)

Arhitecturi nestructurate

Descentralizate pure

- Se construieste o retea acoperitoare (*overlay*) cu propriile mecanisme de rutare prin IP
- Nu exista o coordonare centrala
- Utilizatorii se conecteaza via o aplicatie care are rol dublu – *servent*
- Comunicarea intre *serventi* se bazeaza pe un protocol la nivel de aplicatie, cu 4 tipuri de mesaje:
  - Ping – cere ca un nod sa se anunte
  - Pong – replica la mesajul *ping* (IP, port, numarul & marimea fisierelor)
  - Query – cerere de cautare (sir de cautare + viteza minima de transfer)
  - Query hints – raspuns (IP, port, viteza, dim. fis., index fis.)

Arhitecturi nestructurate

Descentralizate pure

- Cautarea se realizeaza prin inundare (flooding)
  - Daca nu ai fisierul dorit, intreaba pe *n* vecini
  - Daca nici ei nu au fisierul, vor intreba pe vecinii lor in maxim *m* hop-uri
  - Pe calea de intoarcere se vor intoarce raspunsurile (nu continutul fisierelor)
- Fiecare mesaj are un TTL atasat
- Exemplu: **Gnutella**

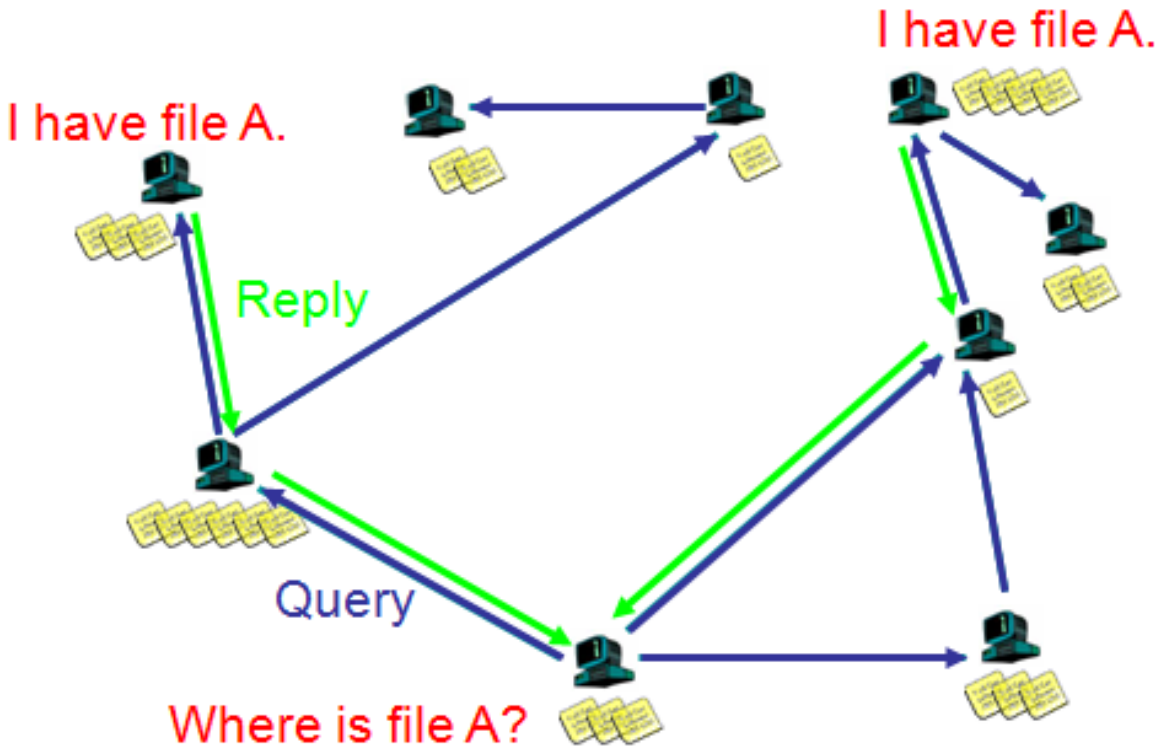
Arhitecturi nestructurate

Gnutella

- 2000: L. Frankel si T. Pepper(Nullsoft) lanseaza Gnutella
- Apar clienti: Bearshare, Morpheus, LimeWire
- *Query Flooding*:
  - **Join**: la intrare in sistem, clientul contacteaza cateva noduri care devin “vecinii” sai
  - **Publish**: nu este necesar
  - **Search**: se intreaba vecinii, care isi intreaba vecinii lor, …
    - Exista un TTL ce limiteaza propagarea
  - **Fetch**: preia fisierul direct de la *peer*

Arhitecturi nestructurate

Gnutella



Aspecte:

- Timpul de cautare este… O(?)
- Nodurile pleaca adesea => reteaua instabila

Arhitecturi nestructurate

Partial centralizate

- Folosesc conceptul de supernod: are activitati de servire a unei sub-retele P2P (indexare, *caching*)
- Nodurile sunt alese automat ca fiind supernoduri daca au suficienta latime de banda si putere computationala
- Toate cererile sunt trimise initial la supernoduri
- Avantaje: timpul descoperirii resurselor e mai redus + eterogenitatea este exploatata
- Exemplu: **KaZaA**

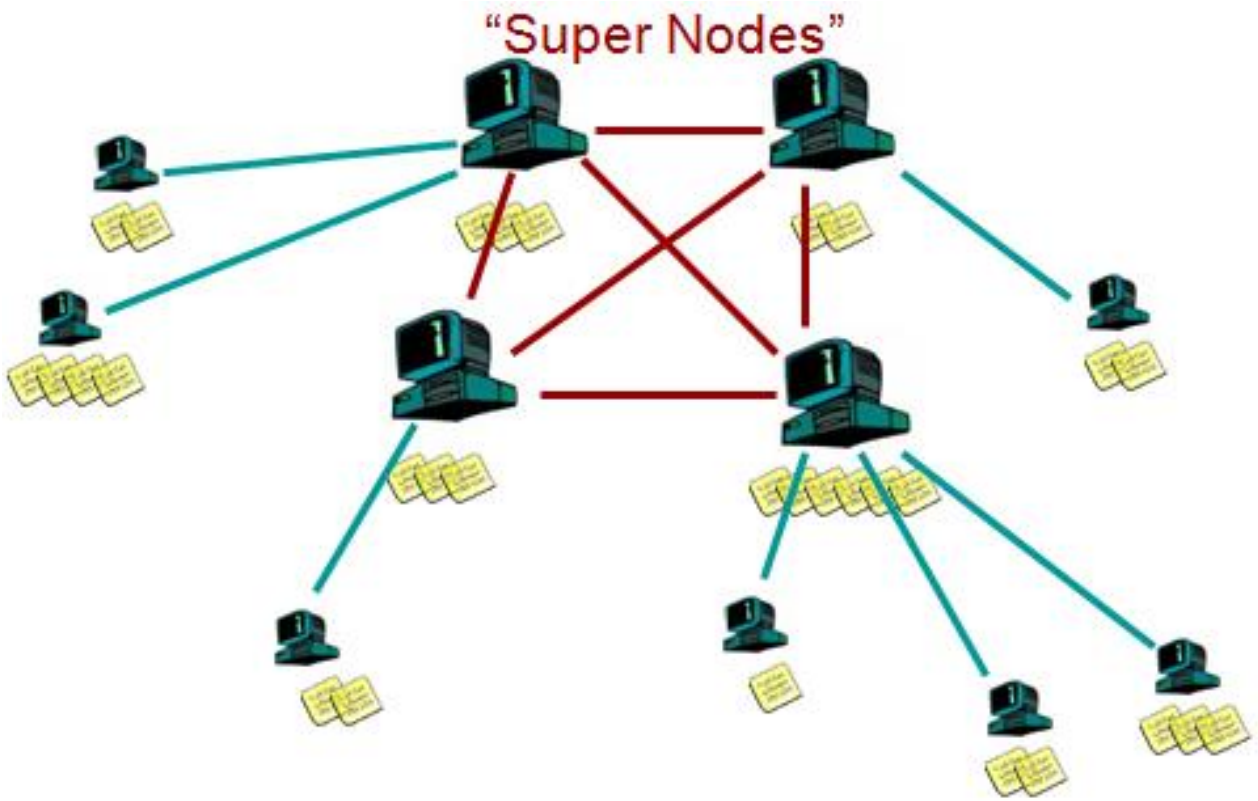
Arhitecturi nestructurate

KaZaA

- 2001: Se lanseaza KaZaA
- Apar clienti: Morpheus, giFT
- Se utilizeaza un mecanism de tip "*smart*" *query flooding*:
  - **Join**: la intrare in sistem, clientul contacteaza un "*supernode*" (poate deveni si el *supernod* la un moment dat)
  - **Publish**: trimite lista de fisiere *supernodului*
  - **Search**: trimite interogarea *supernodului*, si *supernodurile* se interogheaza intre ele
  - **Fetch**: ia fisierul direct de la *peer(s);* poate prelua fisierul simultan de la mai multe *peer*-uri

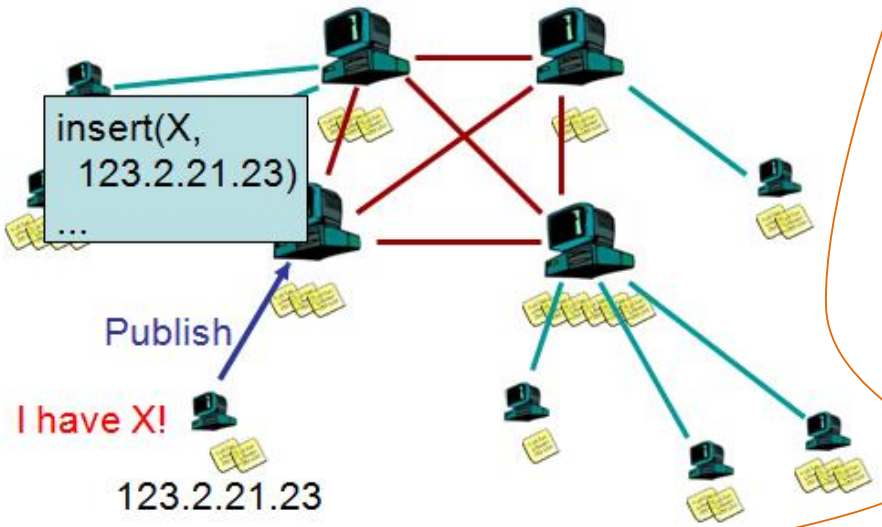
Arhitecturi nestructurate

KaZaA: Designul retelei

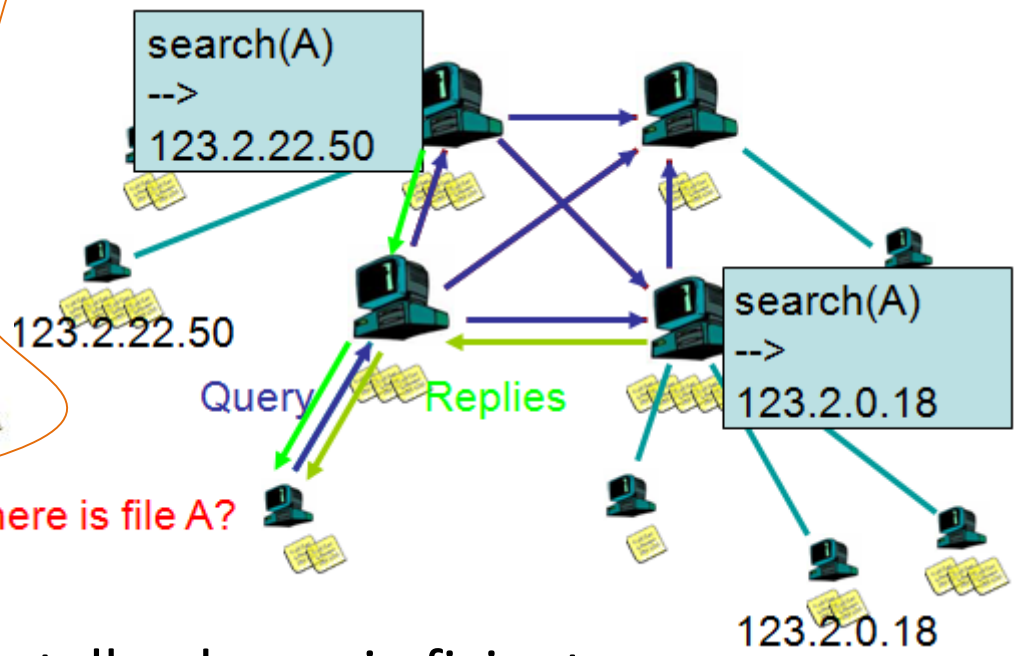


Arhitecturi nestructurate

KaZaA: Inserarea de Fisiere



KaZaA: Cautarea de Fisiere



Discutii:

- Comportament similar cu Gnutella, dar mai eficient
- Nu este nici o garantie asupra timpului de cautare sau a domeniului de cautare

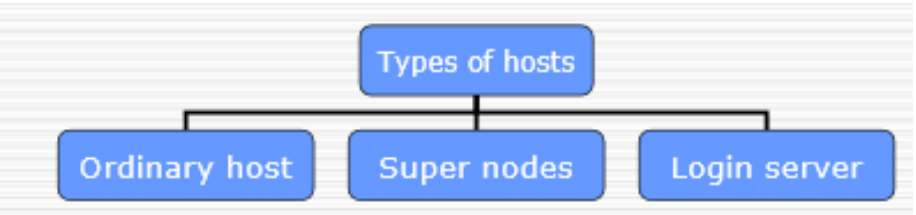
Arhitecturi nestructurate

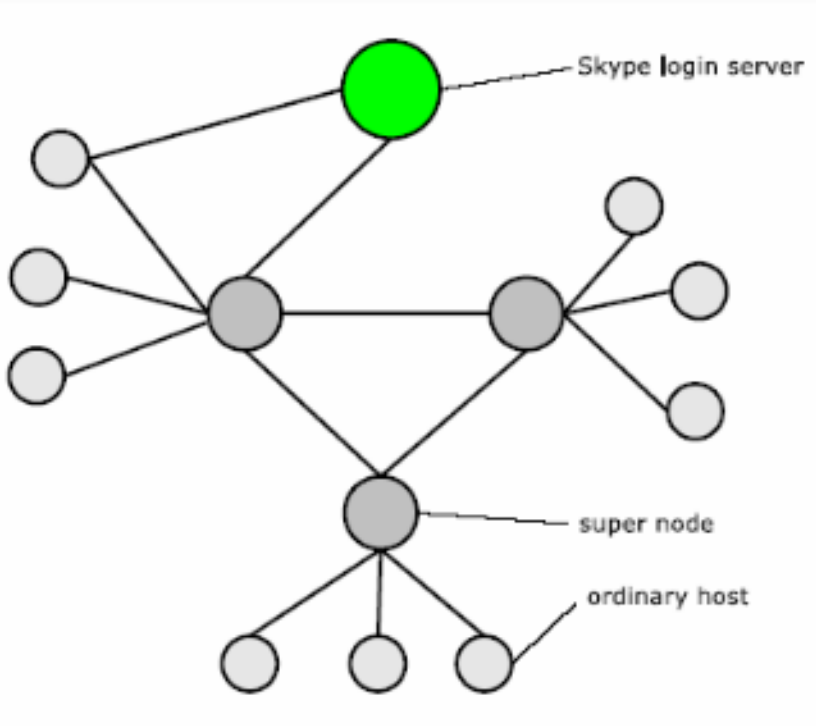
Partial centralizate

- Software-ul KaZaA este proprietar
- Datele de control P2P sunt criptate
- Mesajele folosesc HTTP
- Un nod e fie un supernod, fie asignat unui supernod
- Un supernod are 100-150 noduri-copil
- O retea poate avea ~30000 supernoduri
- Fiecare supernod are conexiuni TCP cu 30-50 supernoduri
- Pentru fiecare fisier se mentin meta-date (nume, dimensiune, *content hash*, descriptor de fisier)
- *Content hash*-ul este folosit pentru cautarea altei copii a unui fisier partial transferat
- Varianta fara *spyware* si *pop-up*-uri: **KaZaA-lite**

Arhitecturi nestructurate

Skype



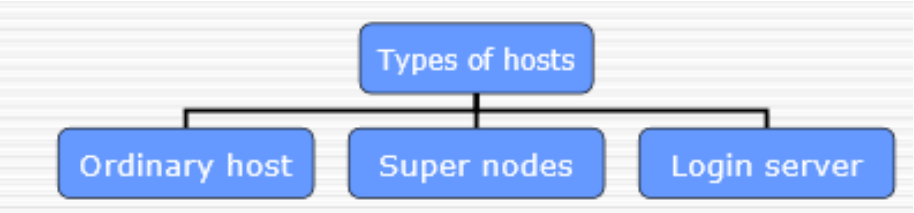


- Prima retea de telefonie p2p bazata pe IP
- din Iunie 2014, Microsoft a anuntat incompatibilitatea cu protocolul anterior Skype
- foloseste Microsoft Notification Protocol 24 (prima utilizare -> MSN Messenger in 1999)
- arhitectura era similara cu KaZaA

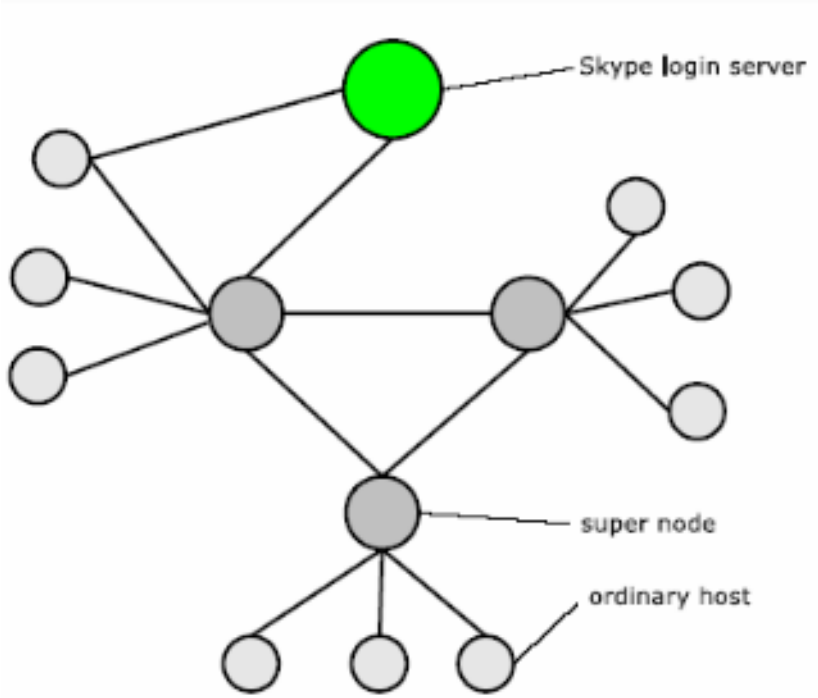
http://www1.cs.columbia.edu/~salman/publications/skype1_4.pdf

Arhitecturi nestructurate

Skype



- Fiecare client mentinea un *host cache* cu adresele IP si numerele de port ale supernodurilor accesibile
- Orice client cu latime de banda (si fara restrictii de firewall sau NAT) putea deveni supernode
- din 2012, Microsoft a inceput gazduirea supernodurilor in servere din centrele sale de date



http://en.wikipedia.org/wiki/PRISM_%28surveillance_program%29

Arhitecturi nestructurate

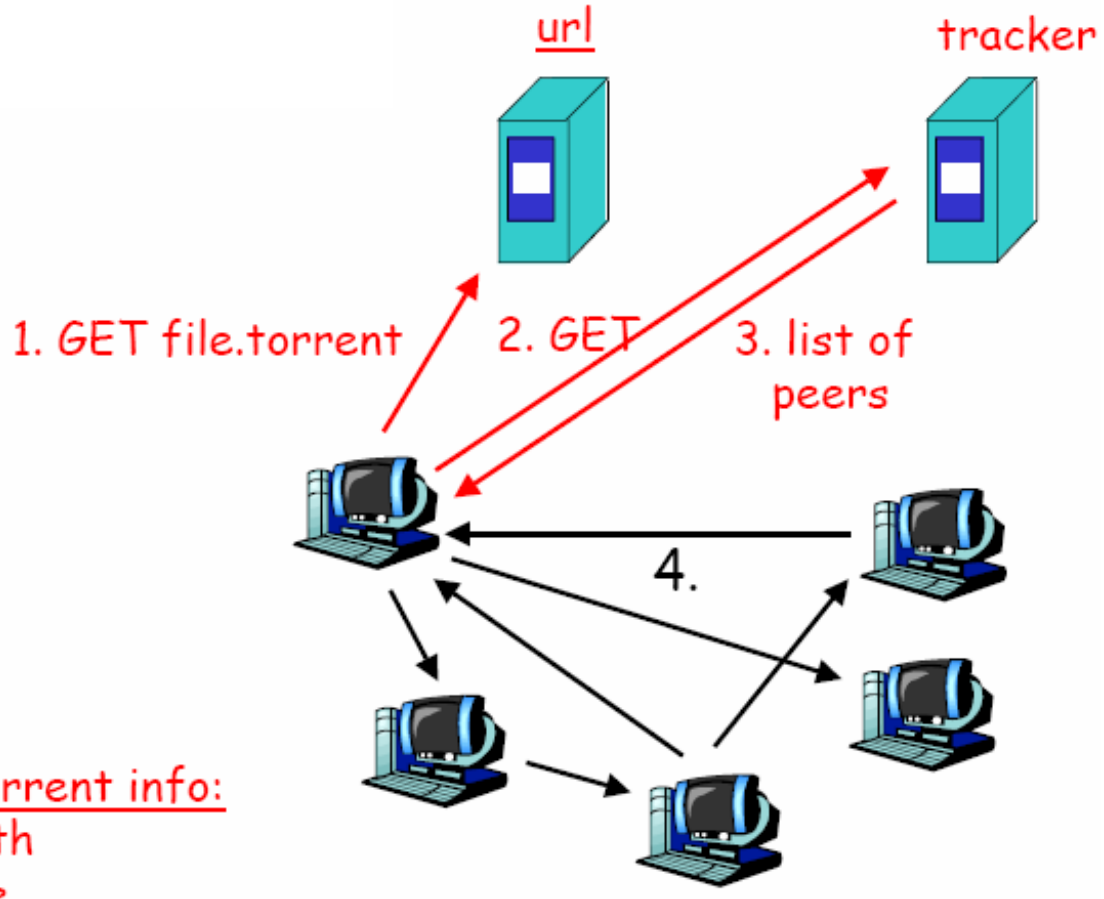
Partial centralizate

- Daca un fisier este gasit pe mai multe noduri, transferul poate fi realizat in paralel
  - Copiile identice se identifica via *content hash*
- Diferite portiuni din fisier sunt transferate de pe noduri diferite
- Pentru transferurile intrerupte, se face o recuperare automata *(automatic recovery)*
- Exemplu: **BitTorrent**
  - In 2002, B. Cohen a lansat BitTorrent
  - Si-a propus concentrarea pe problema legata de obtinerea eficienta a resurselor (*efficient fetching)* si nu pe cautare (*searching*)
  - Sustinatori inca de la aparitie
    - Blizzard Entertainment folosea BitTorrent pentru distributia versiunilor beta a noilor jocuri

Arhitecturi nestructurate

Partial centralizate

BitTorrent - **arhitectura**



Arhitecturi nestructurate

Partial centralizate

BitTorrent

- Se bazeaza pe mecanismul de *swarming*:
  - Join: contacteaza un server centralizat (*tracker*) si obtine o lista de *peer*-uri
  - Publish: ruleaza un server *tracker*
  - Search: de ex. foloseste Google pentru a gasi un *tracker* pentru fisierul dorit
  - Fetch: Preia bucati de fisiere de la *peer*-uri; incarca bucatile de fisier pe care le ai
- Obs. Diferenta fata de Napster
  - *Downlod-ul* de bucati (*chunk*) de fisiere
  - Utilizarea strategiei “tit-for-tat”: daca A face *download* de la alte noduri, atunci A trebuie sa permita si *download-ul* de la el (*free-rider problem*)

Arhitecturi nestructurate

Probleme

- **Noduri ale caror adrese IP sunt disponibile via NAT (cu restrictii)**
  - Nu pot fi servere TCP pentru reteaua P2P
  - Solutie partiala: *reverse call*
    - A vrea sa transfere de la B, iar B foloseste NAT
    - A si B stabilesc conexiuni TCP cu serverul C (IP rutabil)
    - A poate cere lui B, via C, sa realizeze o conexiune TCP de la B la A
    - A poate trimite o cerere lui B, iar B ii ofera raspunsul
  - Daca A si B utilizeaza NAT?
- **Flash crowd: o crestere neasteptata de cereri pentru o anumita resursa**
  - Pentru continutul dorit nu exista suficiente copii incarcate
  - Cat timp ia unui utilizator sa localizeze fisierul?
  - Cate mesaje va primi un nod datorita cautarilor realizate de alte noduri?
  - Se poate folosi un protocol de cautare generic, bazat pe TTL

Arhitecturi structurate

- Reprezinta solutia **academica** pentru P2P
- Scop:
  - Cautare cu succes
  - Timp de cautare in limite cunoscute
  - Scalabilitate demonstrata
- Abordare: **DHT (Distributed Hash Table)**
  - Se stocheaza perechi (key, value)
    - Key – nume de fisiere
    - Value – continut de fisier sau pointer la o locatie
  - Fiecare *peer* stocheaza o multime de (key, value)
  - *Operatii: gaseste nodul responsabil cu un Key*
    - Mapare *key – node*
    - Rutarea eficienta a cererilor de *insert/lookup/delete* asociate cu acest nod
  - Se permite o mare fluctuatie a nodurilor

Arhitecturi structurate

- Aspect de interes: **localizarea continutului**
- Idee: Responsabilitatea este distribuita mai multor noduri ale retelei de acoperire, intr-un mod adaptiv
- Fiecarei resurse i se asociaza o cheie unica via o functie *hash*: h(“Curs Retele”)->7929; Intervalul de valori ale functiei *hash* se distribuie in reteaua P2P
- Fiecare nod trebuie sa “cunoasca” locatia macar a unei singure copii a fiecarei resurse pentru care functia sa *hash* ia valori in intervalul lui
- Nodurile pot mentine in cache-ul propriu o copie a fiecarei resurse pe care trebuie sa o “cunoasca”

Arhitecturi structurate

- Aspect de interes: **dirijarea**
- Pentru fiecare resursa, un nod ce “cunoaste” resursa trebuie sa fie accesat pe calea cea mai “scurta”
- Abordarile de sisteme P2P structurate difera prin strategia de dirijare
- Nodurile din sistem formeaza o structura de date distribuita care poate fi: inel, arbore, hypercub, skip list, etc.
- Se ofera un API pentru tabelele distribuite de hash-uri (DHT – Distributed Hash Table)
  - Dand o cheie *k*, API-ul va returna adresa IP a nodului responsabil pentru valoarea cheii k

Arhitecturi structurate

- **Implementari**
  - Chord [MIT]
  - Pastry [Microsoft Research UK, Rice University]
  - Tapestry [UC Berkeley]
  - Content Addressable Network (CAN) [UC Berkeley]
  - SkipNet [Microsoft Research US, Univ. of Washington]
  - Kademlia [New York University]
  - Viceroy [Israel, UC Berkeley]
  - P-Grid [EPFL Switzerland]

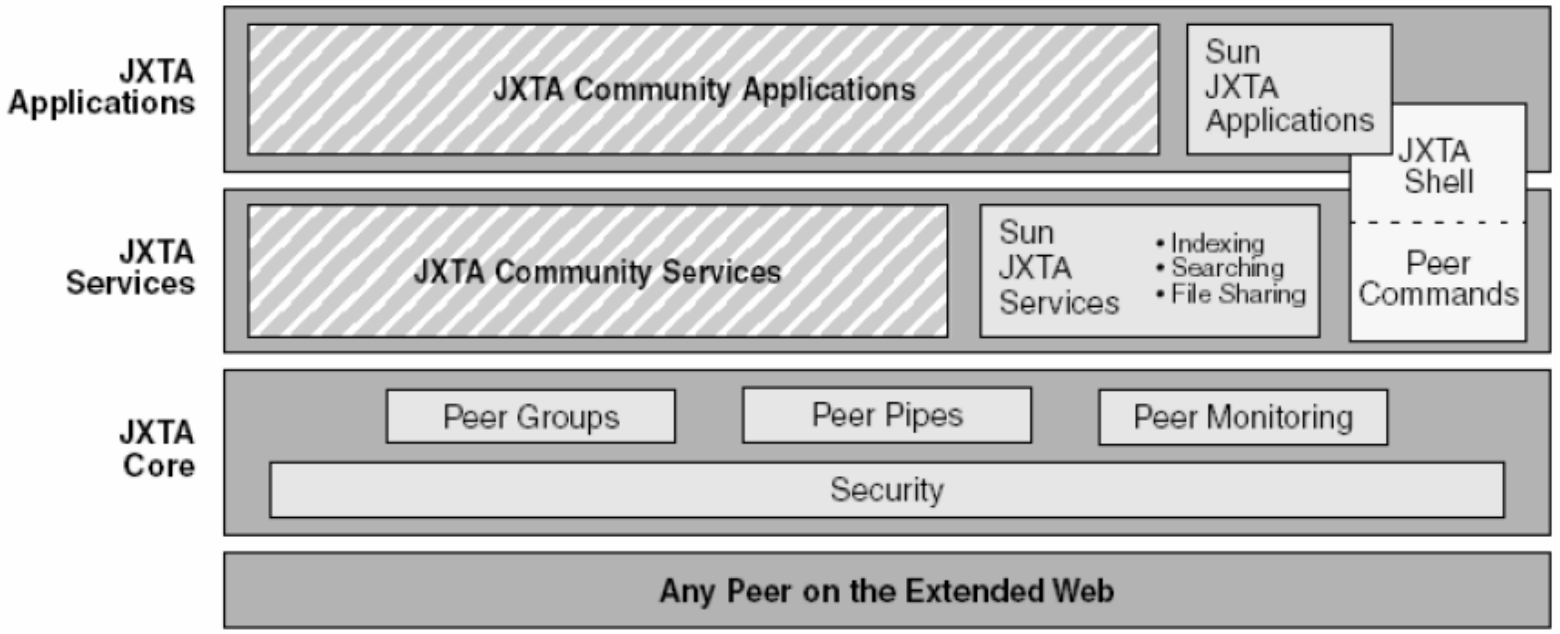
Arhitecturi structurate

- **Slab structurate**
  - Nodurile pot estima ce noduri stocheaza resursele cautate
    - Se evita broadcasturile oarbe
    - Se foloseste o propagare in lant (*chain mode propagation*): fiecare nod ia decizii locale privitoare la care va fi nodul urmator interogat
  - Cautarea unui fisier presupune utilizarea unei chei si a unui mecanism de *timeout*
  - Exemplu: **Freenet**

[https://en.wikipedia.org/wiki/Freenet]

Instrumente

- JXTA – www.jxta.org
  - Mediu de dezvoltare a sistemelor & aplicatiilor P2P
  - Bazat pe java, disponibil in regim open source



Instrumente

- P2P – framework pentru Android

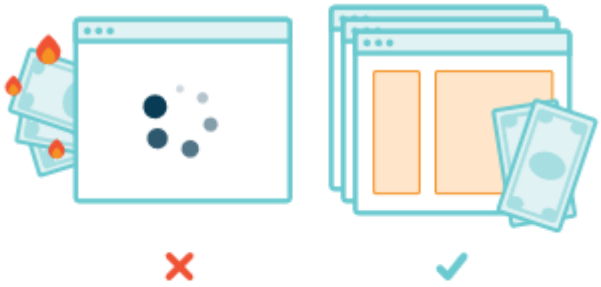
  https://code.google.com/p/p2p-communication-framework-for-android/
- p2psim – simulator pentru protocoalele p2p

  http://pdos.csail.mit.edu/p2psim/
- Instrumente si protocoale pentru P2P:

  http://en.wikibooks.org/wiki/The_World_of_Peer-to-Peer_%28P2P%29/Networks_and_Protocols/Other_Software_Implementations
- “ IPFS is the Distributed Web” - https://ipfs.io/

Instrumente

- " IPFS is the Distributed Web" - https://ipfs.io/
  - **A peer-to-peer hypermedia protocol to make the web faster, safer, and more open**



HTTP is inefficient and expensive

HTTP downloads a file from a single computer at a time, instead of getting pieces from multiple computers simultaneously. With video delivery, a P2P approach could save 60% in bandwidth costs.

IPFS makes it possible to distribute high volumes of data with high efficiency. And zero duplication means savings in storage.

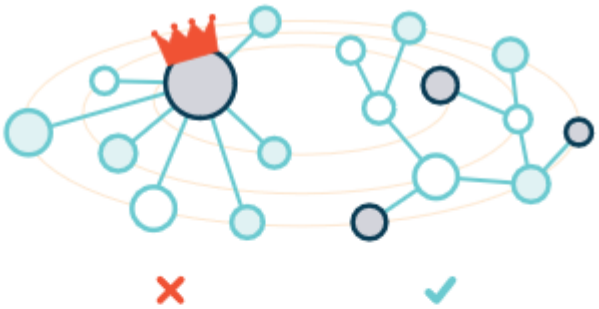
Humanity's history is deleted daily

The average lifespan of a web page is 100 days. Remember GeoCities? The web doesn't anymore. It's not good enough for the primary medium of our era to be so fragile.

IPFS provides historic versioning (like git) and makes it simple to set up resilient networks for mirroring of data.

Instrumente

- " IPFS is the Distributed Web" - https://ipfs.io/
  - **A peer-to-peer hypermedia protocol to make the web faster, safer, and more open**



The web's centralization limits opportunity

The Internet has been one of the great equalizers in human history and a real accelerator of innovation. But the increasing consolidation of control is a threat to that.

IPFS remains true to the original vision of the open and flat web, but delivers the technology which makes that vision a reality.

Our apps are addicted to the backbone

Developing world. Offline. Natural disasters. Intermittent connections. All trivial compared to interplanetary networking. The networks we're using are so 20th Century. We can do better.

IPFS powers the creation of diversely resilient networks which enable persistent availability with or without Internet backbone connectivity.

Instrumente

- “ IPFS is the Distributed Web” - https://ipfs.io/

Let's take a look at what happens when you add files to IPFS:

Each file and all of the **blocks within it** are given a **unique fingerprint** called a **cryptographic hash**.

Each **network node** stores only content it is interested in, and some indexing information that helps figure out who is storing what.

IPFS **removes duplications** across the network and tracks **version history** for every file.

When **looking up files**, you're asking the network to find nodes storing the content behind a unique hash.

Every file can be found by **human-readable names** using a decentralized naming system called **IPNS**.

- https://github.com/ipfs/papers/raw/master/ipfs-cap2pfs/ipfs-p2p-file-system.pdf

Statistici

Global Consumer Internet Traffic 2005–2011

Consumer Internet Traffic 2005–2011							
	2005	2006	2007	2008	2009	2010	2011
By Sub-Segment (terabytes per month)							
Web, e-mail, file transfer	362,084	505,996	692,812	948,425	1,233,172	1,603,615	2,756,415
P2P	1,060,226	1,329,770	1,772,403	2,379,025	3,111,891	4,040,403	5,269,360
Gaming	66,844	91,943	133,367	188,680	250,574	318,212	386,832
Video Communications	11,629	15,575	24,932	36,638	47,173	66,101	92,453
VoIP	10,965	23,035	39,339	57,653	75,575	92,815	110,456
Internet Video to PC	53,074	174,427	484,027	838,154	1,232,461	1,726,114	2,331,908
Internet Video to TV	0	12,727	110,692	353,095	620,197	936,580	1,342,482
By Geography (TB per month)							
North America	534,236	618,765	917,365	1,287,026	1,698,700	2,242,841	2,861,772
Western Europe	334,600	505,329	814,015	1,281,041	1,856,310	2,515,070	3,458,721
Asia Pacific	565,782	819,072	1,201,277	1,742,834	2,315,755	3,049,294	4,663,774
Japan	60,080	98,747	147,733	223,120	319,788	436,057	556,631
Latin America	19,917	33,755	57,083	90,765	130,466	189,992	268,559
Central Eastern Europe	40,773	59,097	86,196	122,272	165,387	222,895	294,901
Middle East and Africa	9,435	18,708	33,904	54,613	84,637	127,689	185,549

Statistici

Legenda:

Web, E-mail, and File Transfer – includes Web, e-mail, instant messaging, newsgroups, and file transfer (excluding P2P and commercial file transfer such as iTunes)

P2P – includes peer-to-peer traffic from all recognized P2P systems such as BitTorrent, eDonkey, etc.

Gaming – includes casual online gaming, networked console gaming, and multiplayer virtual world gaming

Video Communications – includes PC-based video calling, Webcam viewing, and Web-based video monitoring

VoIP – includes traffic from retail VoIP services and PC-based VoIP, but excludes wholesale VoIP transport

Internet Video to PC – free or pay TV or VoD viewed on a PC, excludes P2P video file downloads

Internet Video to TV – free or pay TV or VoD delivered via Internet but viewed on a TV screen using a STB or media gateway

Global Consumer Peer-to-Peer Traffic 2005–2011

Consumer Peer-to-Peer Traffic 2005–2011							
	2005	2006	2007	2008	2009	2010	2011
By Geography (TB per month)							
North America	381,746	378,538	462,356	560,817	673,083	852,483	1,080,979
Western Europe	223,519	304,988	411,057	540,032	757,818	991,817	1,330,885
Asia Pacific	391,235	550,664	762,276	1,074,759	1,401,028	1,811,094	2,327,648
Japan	28,621	42,883	58,463	87,446	117,967	154,868	206,803
Latin America	8,732	14,358	23,247	37,284	53,587	80,043	117,731
Central Eastern Europe	22,075	31,009	43,117	59,928	79,589	106,543	141,282
Middle East and Africa	4,297	7,329	11,886	18,759	28,819	43,553	64,033
Total (TB per month)							
Peer-to-Peer Traffic	1,060,226	1,329,770	1,772,403	2,379,025	3,111,891	4,040,403	5,269,360

Table 10. Global Consumer File-Sharing Traffic, 2015–2020

Consumer File Sharing, 2015–2020							
	2015	2016	2017	2018	2019	2020	CAGR 2015–2020
By Network (PB per Month)							
Fixed	5,942	5,909	5,829	5,713	5,616	5,939	0%
Mobile	22	28	29	29	29	35	9%
By Subsegment (PB per Month)							
P2P file transfer	4,798	4,550	4,224	3,840	3,438	3,633	-5%
Other file transfer	1,166	1,388	1,634	1,902	2,207	2,340	15%
By Geography (PB per Month)							
Asia Pacific	2,335	2,269	2,186	2,098	2,004	2,098	-2%
North America	1,015	1,137	1,260	1,371	1,478	1,576	9%
Western Europe	1,124	1,105	1,096	1,075	1,053	1,131	0%
Central and Eastern Europe	829	763	691	646	621	666	-4%
Latin America	554	573	558	514	454	463	-4%
Middle East and Africa	107	91	68	39	34	39	-18%
Total (PB per Month)							

File Sharing
This category includes traffic from P2P applications such as BitTorrent and eDonkey, as well as web-based file sharing. Note that a large portion of P2P traffic is due to the exchange of video files, so a total view of the impact of video on the network should count P2P video traffic in addition to the traffic counted in the Internet video-to-PC and Internet video-to-TV categories. Table 10 shows the forecast for consumer P2P traffic from 2015 to 2020. Note that the P2P category is limited to traditional file exchange and does not include commercial video-streaming applications that are delivered through P2P, such as PPStream or PPLive.

- http://www.cisco.com/c/en/us/solutions/collateral/service-provider/visual-networking-index-vni/complete-white-paper-c11-481360.html

Rezumat

- **Paradigma peer-to-peer(P2P)**
  - Preliminarii
  - Definitii
  - Caracterizare
  - Tipuri de aplicatii
  - Infrastructuri
  - Instrumente

Bibliografie

- P2P Networking and Applications, John F. Buford, Heather Yu, Eng Keong Lua ,2009, Elsevier
- http://mtc.sri.com/Conficker/P2P/
- http://www.cisco.com/en/US/docs/cable/serv_exch/serv_control/broadband_app/protocol_ref_guide/01_p2p.pdf
- http://pdos.csail.mit.edu/p2psim/
- Statistici: http://www.sandvine.com/news/pr_detail.asp?ID=312
- Statistici: http://www.hbtf.org/files/cisco_IPforecast.pdf
- http://en.wikibooks.org/wiki/The_World_of_Peer-to-Peer_%28P2P%29/Networks_and_Protocols/Other_Software_Implementations
- https://www.kirsle.net/blog/entry/skype-switched-to-the-msn-messenger-protocol

Intrebari?

Paradigma RPC

Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- **Remote Procedure Call (RPC)**
  - Preliminarii
  - Caracterizare
  - XDR (External Data Representation)
  - Functionare
  - Implementari
  - Utilizari

Preliminarii

- **Proiectarea aplicatiilor distribuite**
  - Orientata pe protocol – ***socket***-uri
    - Se dezvolta protocolul, apoi aplicatiile care il implementeaza efectiv
  - Orientata pe functionalitate – **RPC**
    - Se creeaza aplicatiile, dupa care se divid in componente si se adauga protocolul de comunicatie intre componente

RPC|Caracterizare

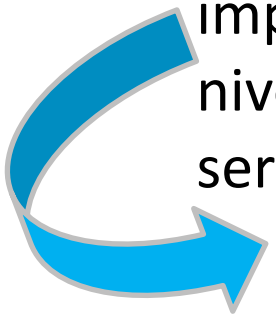
- **Idee:** In loc de accesarea serviciilor la distanta prin trimiterea si primire de mesaje, clientul apeleaza o procedura care va fi executata pe alta masina
- **Efect**: RPC “ascunde” existenta retelei de program
  - Mecanismul de *message-passing* folosit in comunicarea in retea este ascuns fata de programator
  - Programatorul nu trebuie sa mai deschida o conexiune, sa citeasca sau sa scrie date, sa inchida conexiunea etc.
- Este un instrument de programare mai simplu decat interfata *socket* BSD

RPC|Caracterizare

- O aplicatie RPC va consta dintr-un client si un server, serverul fiind localizat pe masina pe care se executa procedura
- La realizarea unui apel la distanta, parametrii procedurii sunt transferati prin retea catre aplicatia care executa procedura; dupa terminarea executiei procedurii rezultatele sunt transferate prin retea aplicatiei client
- Clientul si serverul –> procese pe masini diferite

RPC|Caracterizare

- **RPC realizeaza comunicarea dintre client si server prin *socket*-uri TCP/IP (uzual, UDP), via doua interfete *stub* (*ciot*)**
  - Obs.: Pachetul RPC (*client stub* si *server stub*|*skeleton*) ascunde toate detaliile legate de programarea in retea
- **RPC** implica urmatorii **pasi**:
  1. Clientul invoca procedura *remote*
     - Se apeleaza o procedura locala, numita *client stub* care impacheteaza argumentele intr-un mesaj si il trimite nivelului transport, de unde este transferat la masina server *remote*

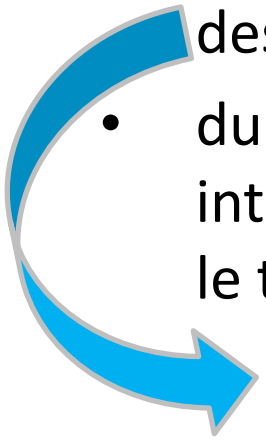


Marshalling (serializare) = mecanism ce include codificarea argumentelor intr-un format standard si impachetarea lor intr-un mesaj

RPC|Caracterizare

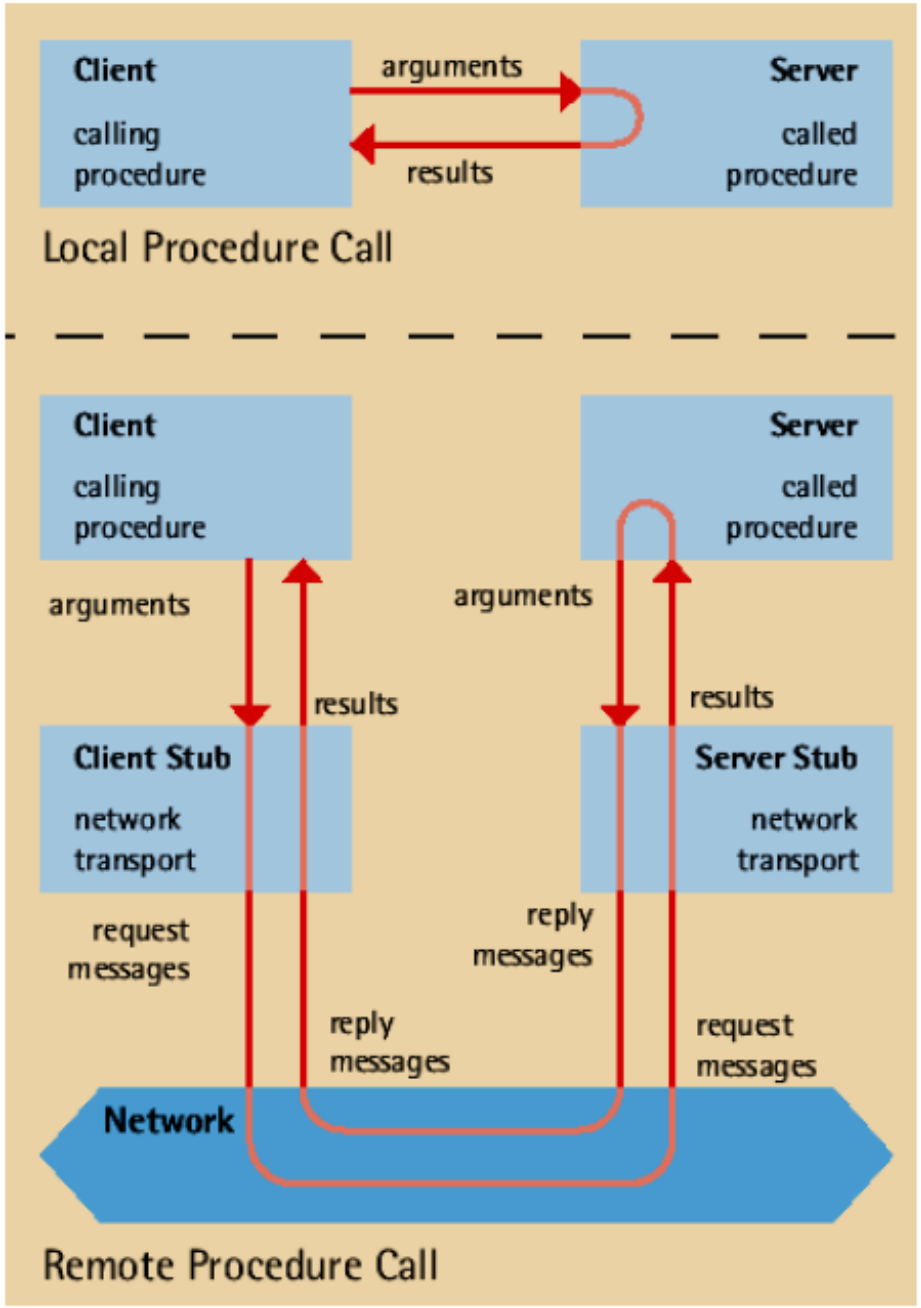
- **RPC** implica urmatorii **pasi**:

  2. Server-ul:
     - nivelul transport trimite mesajul catre *server stub,* care despacheteaza parametrii si apeleaza functia dorita;
     - dupa ce functia returneaza, *server stub* preia valorile intoarse, le impacheteaza (*marshalling*) intr-un mesaj si le trimite la *client stub*



un-marshalling (deserializare) = decodificare

  3. *Client stub* preia valorile primite si le returneaza aplicatiei client



RPC|Caracterizare

- Interfetele ciot implementeaza protocolul RPC
- Diferente fata de apeluri locale:
  - Performanta poate fi afectata de timpul de transmisie
  - Tratarea erorilor este mai complexa
  - Locatia server-ului trebuie sa fie cunoscuta (Identificarea si accesarea procedurii la distanta)
  - Poate fi necesara autentificarea utilizatorilor

[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

RPC|Caracterizare

- Procedurile ciot se pot genera automat, dupa care se “leaga” de programele client si server
- Ciotul serverului asculta la un port si realizeaza invocarea rutinelor printr-o interfata de apel de proceduri locale
- Clientul si serverul vor comunica prin mesaje, printr-o reprezentare independenta de retea si de sistemul de operare:

  External Data Representation (**XDR**)

RPC|Caracterizare

- **External Data Representation (XDR)**

  XDR defineste numeroase tipuri de date si modul lor de transmisie in mesajele RPC (RFC 1014)

  – Tipuri uzuale:
    - Preluate din C: int, unsigned int, float, double, void,…
    - Suplimentare: string, fixed-length array, variable-length array, …

  – Functii de conversie (rpc/xdr.h)
    - xdrmem_create() – asociaza unei zone de memorie un flux de date RPC
    - xdr_*numetip*() – realizeaza conversia datelor

RPC|Caracterizare

- **External Data Representation (XDR)**

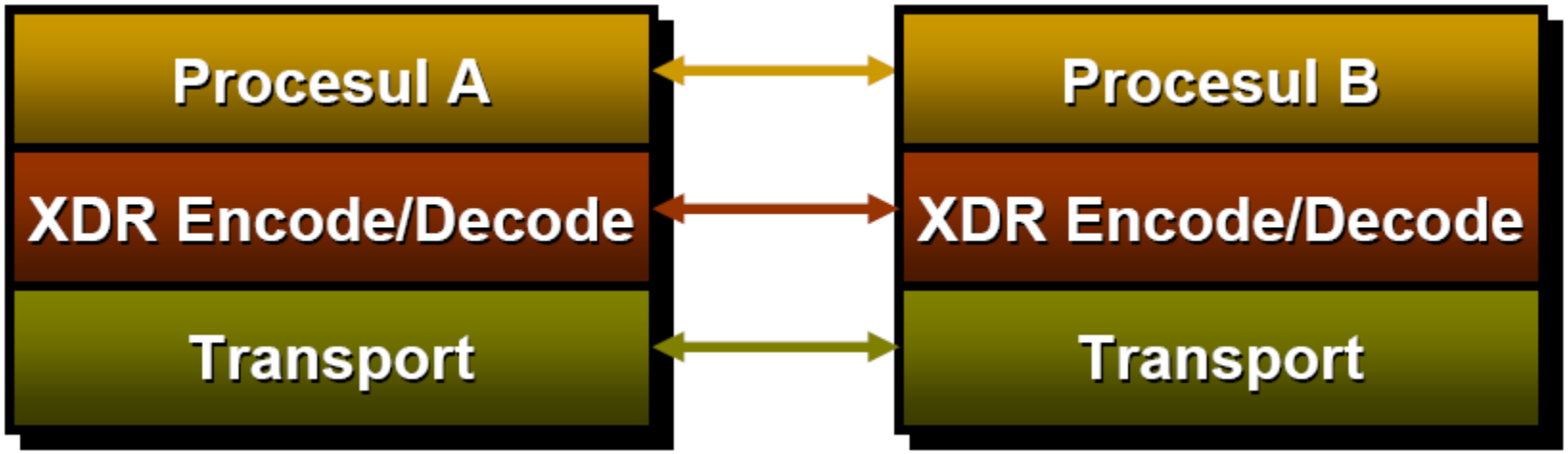
Exemplu

```
#include <rpc/xdr.h>
#define BUFSIZE 400 /* lungimea zonei de memorie */
/* conversia unui intreg in format XDR */
...
XDR *xdrm; /* zona de memorie XDR */
char buf[BUFSIZE];
int intreg;
...
xdrmem_create (xdrm, buf, BUFSIZE, XDR_ENCODE);
...
intreg = 33;
xdr_int (xdrm, &intreg);
...
```

Inlocuit la celalalt capat al comunicatiei cu XDR_DECODE

RPC|Caracterizare

- **External Data Representation (XDR)**
  - Poate fi vazut ca nivel suplimentar intre nivelul transport si nivelul aplicatie
  - Asigura conversia simetrica a datelor client si server

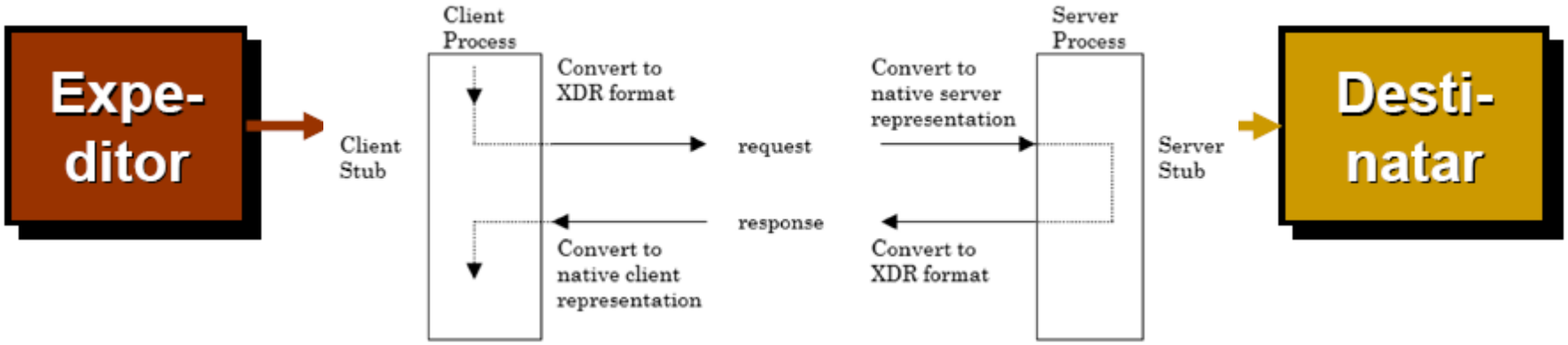


[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

RPC|Caracterizare

External Data Representation (XDR)

- Activitatea de codificare/decodificare



- In prezent, poate fi inlocuit de reprezentari XML-RPC sau SOAP sau JSON-RPC (in contextul serviciilor Web)

vezi cursul de Tehnologii Web!

RPC|Functionare

Context:

- Un serviciu de retea este identificat de portul la care exista un *daemon* asteptand cereri
- Programele server RPC folosesc porturi efemere

De unde stie clientul unde sa trimita cererea?

Portmapper = serviciu de retea responsabil cu asocierea de servicii la diferite porturi

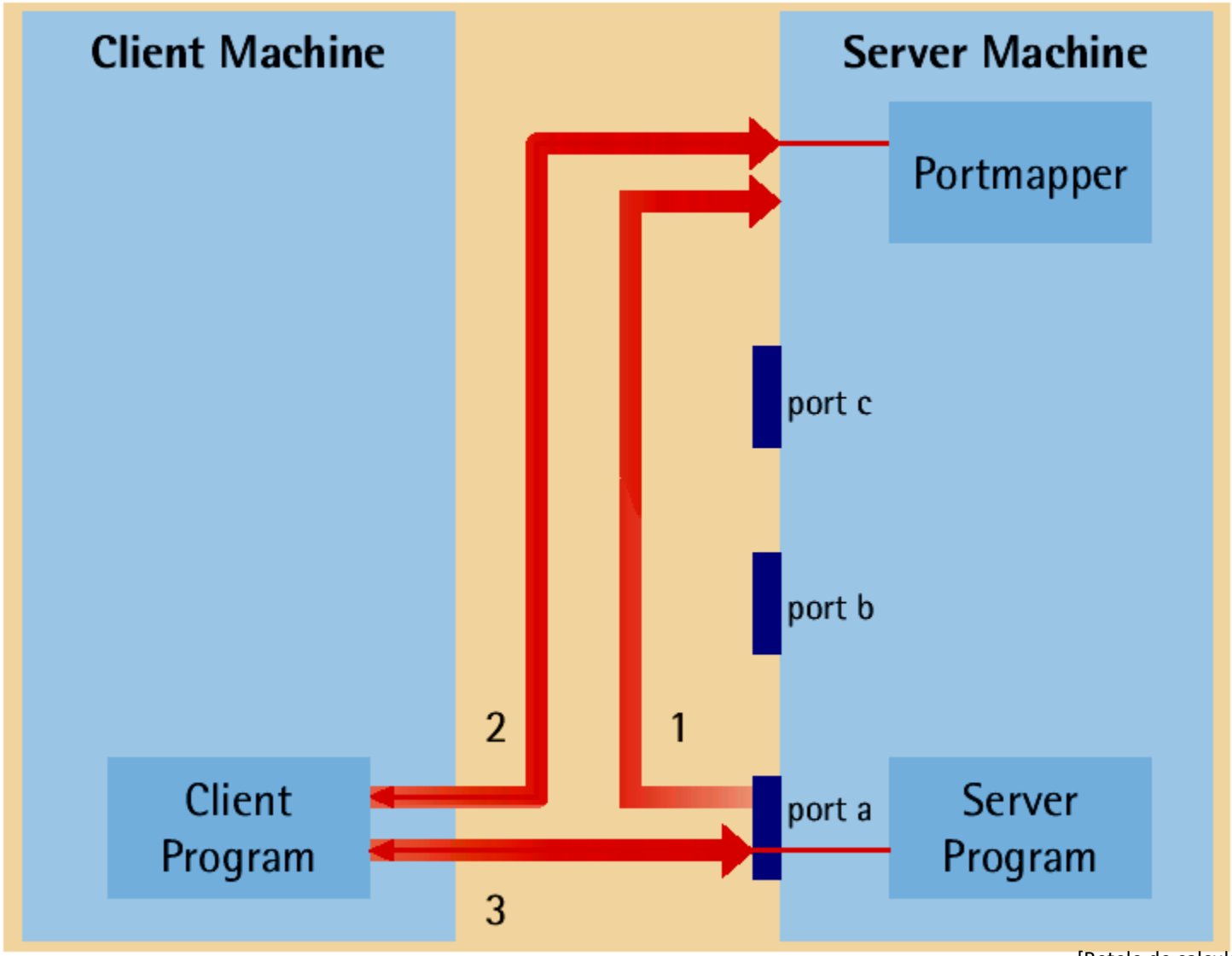
=> Numerele de port pentru un anumit serviciu nu sunt fixe

- Este disponibil la portul 111 (*well-known port*)

```
[adria@thor ~] $ rpcinfo -p
   program vers proto   port
    100000    2   tcp    111  portmapper
    100000    2   udp    111  portmapper
    100024    1   udp  56660  status
    100024    1   tcp  48918  status
```

RPC|Functionare

Mecanism general



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

RPC|Functionare

Mecanism general:

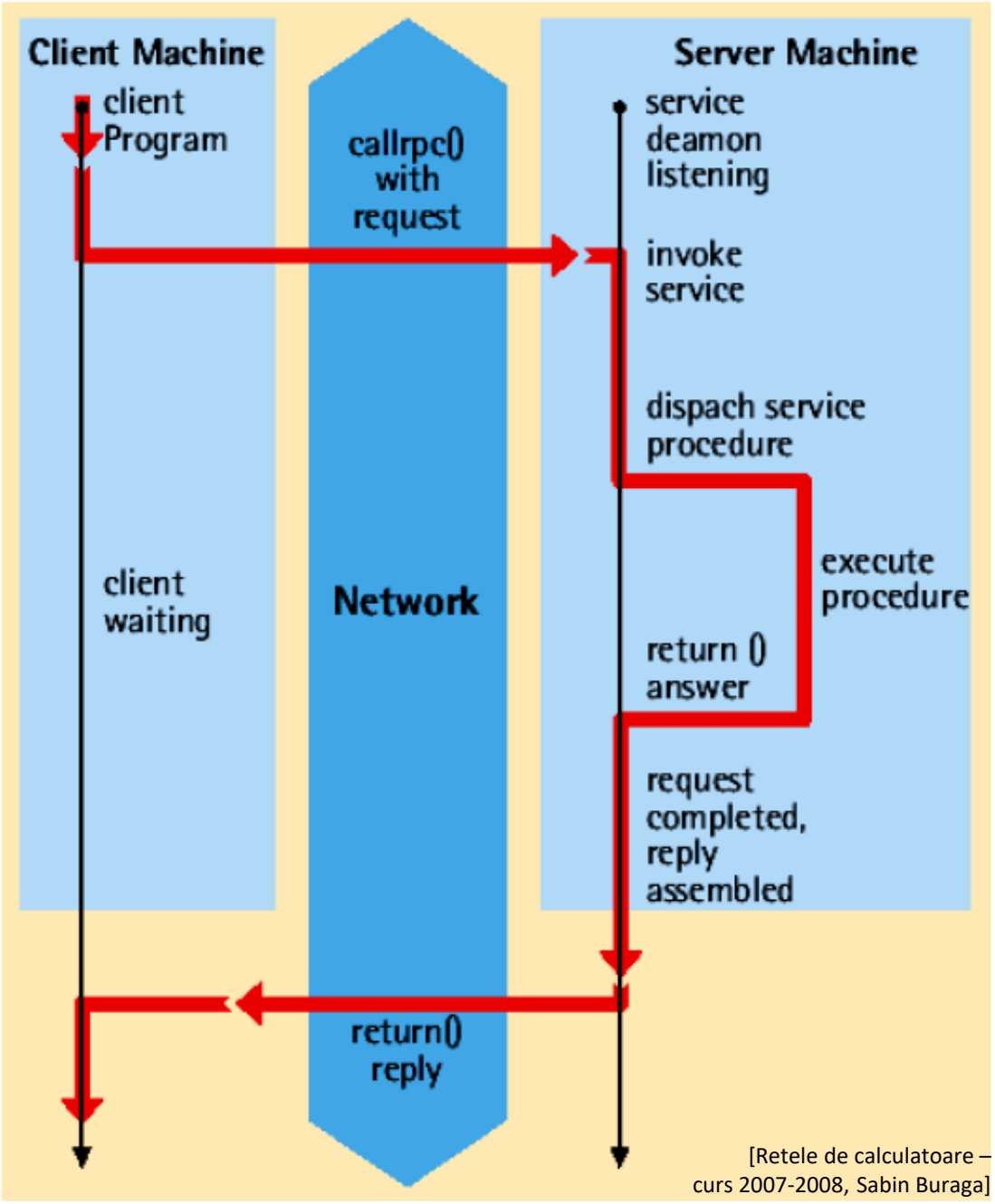
Pas 1: Se determina adresa la care serverul va oferi serviciul

- La initializare, serverul stabileste si inregistreaza via *portmapper* portul la care va oferi serviciul (portul **a**)

Pas 2: Clientul consulta *portmapper*-ul de pe masina serverului pentru a identifica portul la care trebuie sa trimita cererea RPC

Pas 3: Clientul si serverul pot comunica pentru a realiza executia procedurii la distanta

- Cererile si raspunsurile sunt (de)codificate prin XDR



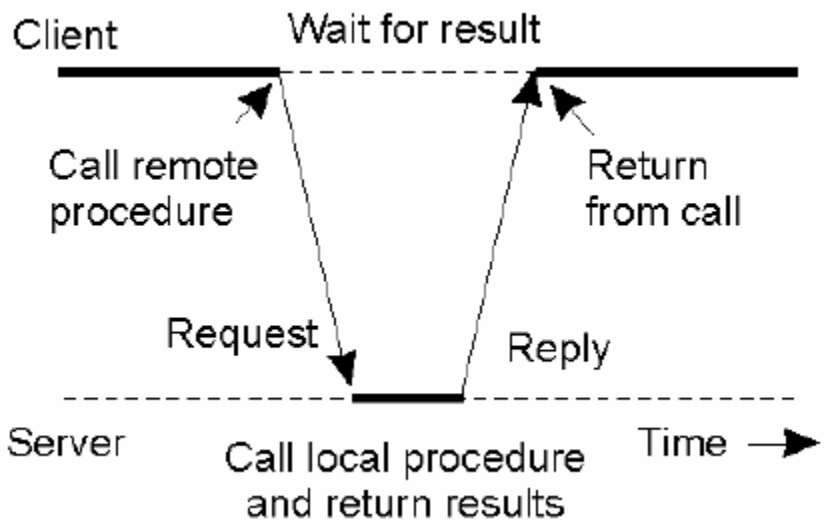
[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

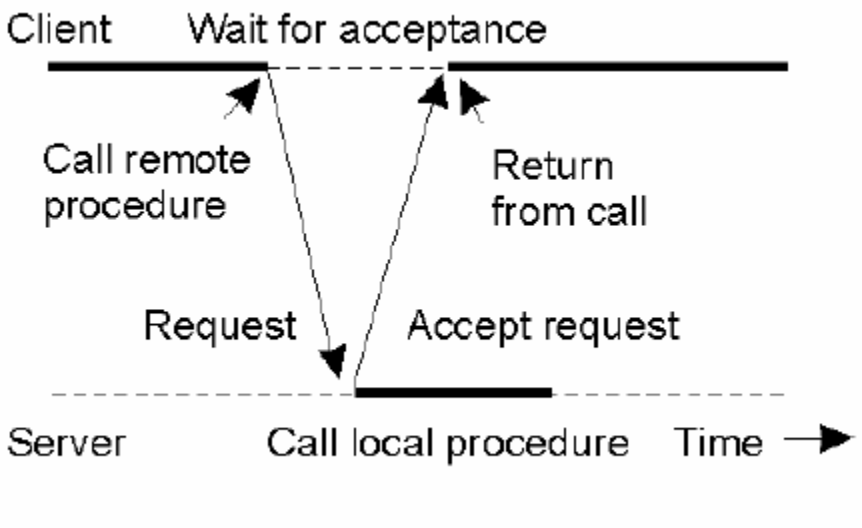
RPC|Functionare

- Atunci cind un server furnizeaza mai multe servicii, este de obicei folosita o rutina *dispatcher*
- *Dispatcher-ul* identifica cererile specifice si apeleaza procedura corespunzatoare, dupa care rezultatul este trimis inapoi clientului pentru a-si continua executia

RPC|Functionare

- Transferurile de date RPC pot fi:
  - Sincrone
  - Asincrone





[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

RPC|Implementare

- Open Network Computing RPC (ONC RPC) - cea mai raspandita implementare in mediile Unix (Sun Microsystems)
  - RFC 1057
  - Interfata RPC este structurata pe 3 niveluri:
    - **Superior**: independent de sistem, hardware sau retea
      - Exemplu: man **rcmd** -> *routines for returning a stream to a remote command ….*
    - **Intermediar**: face apel la functiile definite de biblioteca RPC:
      - **registerrpc**() – inregistreaza o procedura spre a putea fi executata la distanta
      - **callrpc(**) – apeleaza o procedura la distanta
      - **svc_run(**) – ruleaza un serviciu RPC
    - **Inferior:** da posibilitatea de a controla in detaliu mecanismele RPC (de ex. alegerea modului de transport al datelor, etc.)

RPC|Implementare

- Open Network Computing RPC (ONC RPC)
  - Procedurile la distanta se vor include intr-un **program la distanta** - unitate software care se va executa pe o masina la distanta
  - Fiecare program la distanta corespunde unui server: putand contine proceduri la distanta + date globale; procedurile pot partaja date comune;
  - Fiecare program la distanta se identifica printr-un identificator unic stocat pe 32 biti; Conform implementarii Sun RPC avem urmatoarele valori ale identificatorilor:
    - 0x00 00 00 00 – 0x1F FF FF FF – aplicatiile RPC ale sistemului
    - 0x20 00 00 00 – 0x3F FF FF FF – programele utilizator
    - 0x40 00 00 00 – 0x5F FF FF FF – identificatori temporari
    - 0x60 00 00 00 – 0xFF FF FF FF – valori rezervate
  - Fiecare procedura (din cadrul unui program) este identificata printr-un index (1..n)

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

  Exemple:

  - 10000 meta-serverul *portmapper*
  - 10001 pentru rstatd care ofera informatii despre sistemul *remote*; se pot utiliza procedurile rstat() sau perfmeter()
  - 10002 pentru rusersd ce furnizeaza informatii despre utilizatorii conectati pe masina la distanta
  - 10003 serverul nfs – ce ofera acces la sistemul de fisiere in retea NFS (*Network File System*)

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

  Fiecare program la distanta are asociat un numar de versiune

  - Initial versiunea 1
  - Urmatoarele versiuni se identifica in mod unic prin alte numere de versiune

    Se ofera posibilitatea de a schimba detaliile de implementare sau extinderea capabilitatilor aplicatiei fara a asigna un alt identificator unui program

  Un program la distanta este un 3-uplu de forma:

  <id_Program, versiune, index_procedura>

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

Programare de nivel inalt:

fenrir.info.uaic.ro - PuTTY

```
    I A  prog.c (Modified) (c)  }        Row 18    Col
#include <rpc/rpc.h>
#include <string.h>
#include <rpcsvc/rusers.h>
#include <stdio.h>

int main(int argc, char * argv[])
{
  int nrUsers;
  if (argc!=2){/*eroare*/}
  if ((nrUsers=rusers(argv[1]))<0){/*eroare RPC */}
  printf("Sunt %d utilizatori pe masina %s\n",
                 nrUsers, argv[1]);
  exit(0);
}
```

Compilare: gcc prog.c – lrpcsvc –o prog

Executie: ./prog fenrir.infoiasi.ro

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

Programare la nivel intermediar:

```
callrpc (char *host,    /* numele serverului */
        u_long prognum,  /* numarul programului server */
        u_long versnum,  /* numarul de versiune a serv.*/
         u_long procnum, /* numarul procedurii */
         xdrproc_t inproc,  /* fol. pentru codificare XDR*/,
        char *in,  /* adresa argumentelor procedurii*/,
        xdrproc_t outproc,  /* fol.  pentru  decodificare  */,
        char *out,  /* adresa de plasare a rezultatelor*/
            );
```

Apelata de clientul RPC

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

Programare la nivel intermediar:

registerrpc(

u_long **prognum** /* numarul programului server */,
u_long **versnum** /* numarul de versiune a serv*/,
u_long **procnum** /* numarul procedurii */,
void *(***procname**)*() /* numele functiei *remote* */,
xdrproc_t **inproc** /* fol. pt. decodificarea param. */,
xdrproc_t **outproc** /* fol. pt. codificarea result. */
);

Apelata de serverul RPC

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

  Programare la nivel intermediar:

  svc_run ()

  Apelata de serverul RPC,
  reprezinta *dispatcher*-ul

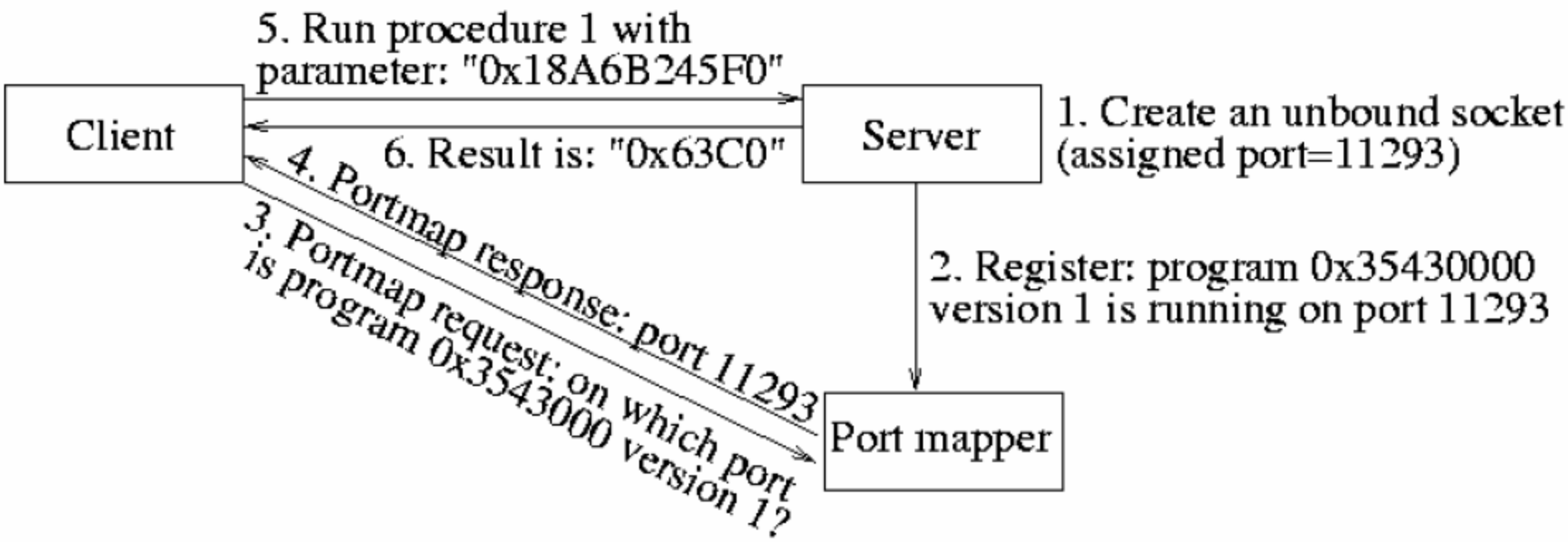
  - Se asteapta venirea de cereri RPC, apoi se apeleaza folosindu-se *svc_getreq*() procedura corespunzatoare

  Obs.: Functiile de nivel intermediar utilizeaza doar UDP

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

Programare la nivel inferior:



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

  Realizarea de aplicatii RPC cu **rpcgen**

  - Se creeaza un fisier cu specificatii RPC (Q.x)
    - Declaratii de constante utilizate de client si server
    - Declaratii de tipuri de date globale
    - Declaratii de programe la distanta, proceduri, tipuri de parametri, tipul rezultatului, identificatorul unic de program
  - Programul server.c care contine procedurile
  - Programul client.c care invoca procedurile

  Pentru server: gcc server.c Q_svc.c Q_xdr.c –o server

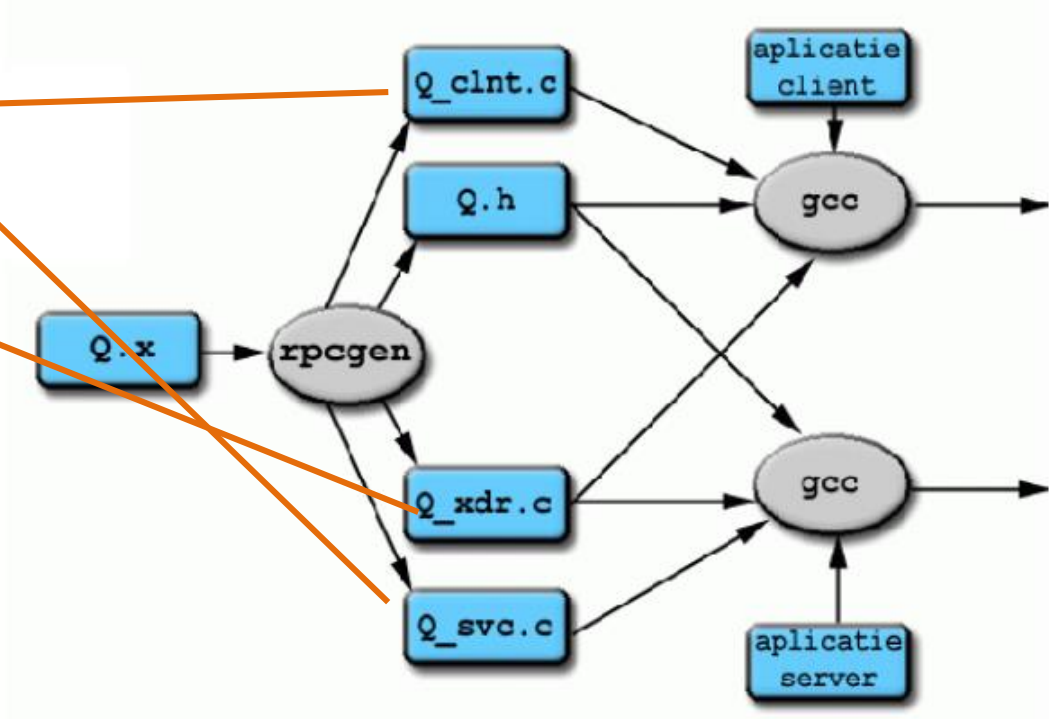
  Pentru client: gcc client.c Q_clnt.c Q_xdr.c –o client

RPC|Implementare

- Open Network Computing RPC (ONC RPC)

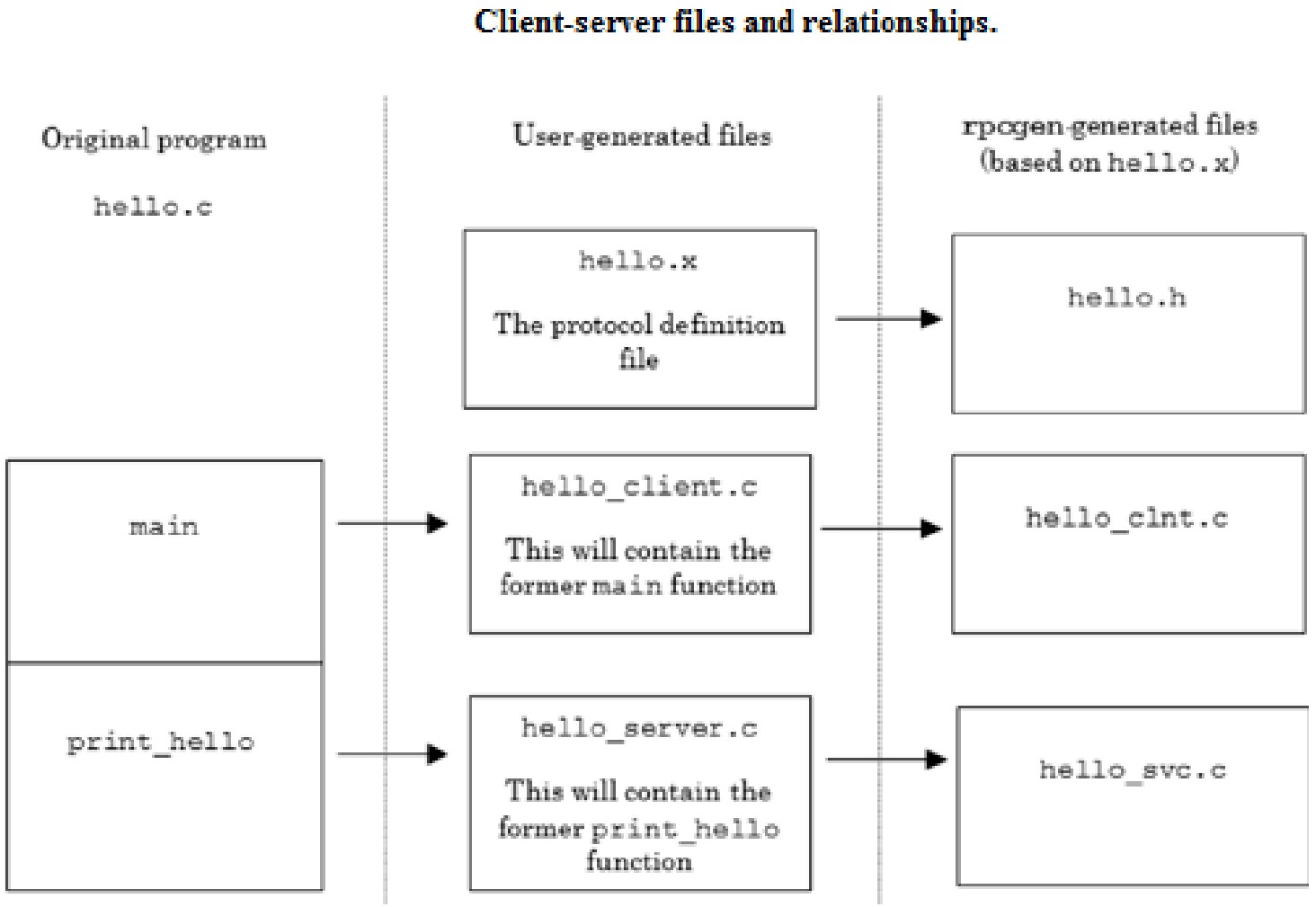
In implementarea unei aplicatii RPC se utilizeaza utilitarul **rpcgen**

- Genereaza *client stub* si *server stub*
- Generaza functiile de codificare si decodificare XDR
- Genereaza rutina *dispatcher*



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

RPC|Implementare



[Interprocess Communications in Linux, J.S. Gray]

RPC|Implementare

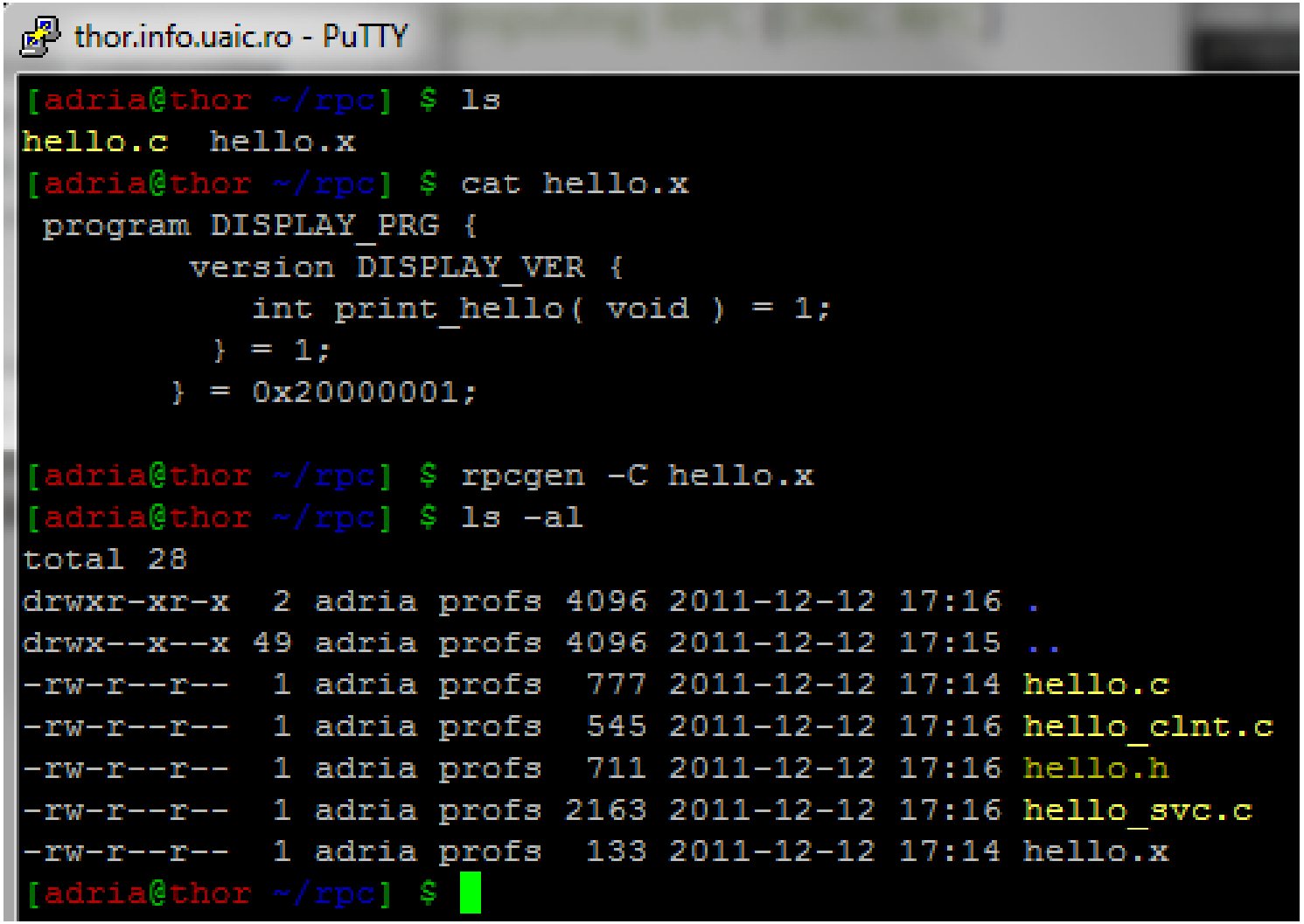
thor.info.uaic.ro - PuTTY

I A hello.c (c) Row 20

```c
/*
        A C program with a local function
*/
#include <stdio.h>
int print_hello( );

int main( )
{
   printf("main : Calling function.\n");
   if (print_hello())
     printf("main : Mission accomplished.\n");
   else
     printf("main : Unable to display message.");
    return 0;
}

int print_hello( )
{
      return printf("funct: Hello, world.\n");
}
```

[Interprocess Communications in Linux, J.S. Gray]

RPC|Implementare



[Interprocess Communications in Linux, J.S. Gray]

RPC|Implementare

```
   I A  hello_client.c (c)          Row 1    Col 1    10:01  Ctrl-K H for help
/*
        The CLIENT program:  hello_client.c
        This will be the client code executed by the local client process.
 */
#include <stdio.h>
 #include "hello.h"              /* Generated by rpcgen from hello.x  */
 int
 main(int argc, char *argv[]) {
   CLIENT         *client;
  int            *return_value, filler;
   char           *server;
 /*
     We must specify a host on which to run.  We will get the host name
     from the command line as argument 1.
  */
   if (argc != 2) {
     fprintf(stderr, "Usage: %s host_name\n", *argv);
     exit(1);
   }
   server = argv[1];
 /*
     Generate the client handle to call the server
  */
    if ((client=clnt_create(server, DISPLAY_PRG, DISPLAY_VER, "tcp")) ==
     clnt_pcreateerror(server);
     exit(2);
   }
   printf("client : Calling function.\n");
 return_value = print_hello_1((void *) &filler, client);
   if (*return_value)
     printf("client : Mission accomplished.\n");
   else
     printf("client : Unable to display message.\n");
  return 0;
 }
```

[Interprocess Communications in Linux, J.S. Gray]

RPC|Implementare

```
I A  hello_server.c (c)          Row 1    Col 1   10:02  Ctrl-K H for help

/*
    The SERVER program: hello_server.c
    This will be the server code executed by the "remote" process
*/
#include <stdio.h>
#include "hello.h"          /* is generated by rpcgen from hello.x  */
int *
print_hello_1_svc(void * filler, struct svc_req * req) {
  static int  ok;
   ok = printf("server : Hello, world.\n");
  return (&ok);
}
```

[Interprocess Communications in Linux, J.S. Gray]

RPC|Implementare

```
[adria@thor ~/html/teach/courses/net/files/NetEx/S11/RPC/tempRPC] $ rpcinfo -p
   program vers proto   port
    100000    2   tcp    111  portmapper
    100000    2   udp    111  portmapper
    100024    1   udp  56604  status
    100024    1   tcp  34914  status
[adria@thor ~/html/teach/courses/net/files/NetEx/S11/RPC/tempRPC] $
```

```
/tempRPC] $ gcc hello_client.c hello_clnt.c -o client
/tempRPC] $ gcc hello_server.c hello_svc.c -o server
/tempRPC] $ ./server
```

```
[adria@thor ~/html/teach/courses/net/files/NetEx/S11/RPC/tempRPC] $ rpcinfo -p
   program vers proto   port
    100000    2   tcp    111  portmapper
    100000    2   udp    111  portmapper
    100024    1   udp  56604  status
    100024    1   tcp  34914  status
 536870913    1   udp  37547
 536870913    1   tcp  43833
[adria@thor ~/html/teach/courses/net/files/NetEx/S11/RPC/tempRPC] $
```

```
[adria@thor ~/html/teach/courses/net/files/NetEx/S11/RPC/tempRPC] $ ./client 127.0.0.1
client : Calling function.
client : Mission accomplished.
```

```
[adria@thor ~/html/teach/courses/net/files/NetEx/S11/RPC/tempRPC] $ ./server
server : Hello, world.
```

[Interprocess Communications in Linux, J.S. Gray]

RPC|Implementare

- Alte implementari:
  - DCE/RPC (Distributed Computing Environment/RPC)
    - Alternativa la Sun ONC RPC
    - Utilizat si de serverele Windows
  - ORPC (Object RPC)
    - Mesajele de cerere/raspuns la distanta se incapsuleaza in obiecte
    - Descendenti directi:
    - (D)COM (Distributed Component Object Model) & CORBA (Common Object Request Broker Architecture)
    - In Java: RMI (Remote Method Invocation)
    - .Net Remoting , WCF
  - SOAP (Simple Object Access Protocol)
    - XML ca XDR, HTTP ca protocol de transfer
    - Baza de implementare a unei categorii de servicii Web

RPC|Utilizari

- **Accesul la fisiere la distanta NFS (Network File System)**
  - Protocol proiectat a fi independent de masina, sistem de operare si de protocol – implementat peste RPC (… conventia XDR)
  - Protocol ce permite partajare de fisiere in retea => NFS ofera acces transparent clientilor la fisiere
    - Obs.: Diferit fata de FTP (vezi curs anterior)
  - Ierarhia de directoare NFS foloseste terminologia UNIX (arbore, director, cale, fisier, etc.)
  - NFS este un protocol => client - nfs , server –nfsd comunicand prin RPC
  - Modelul NFS
    - Operatii asupra unui fisier la distanta: operatii I/O, creare/redenumire/stergere, stat, listarea intrarilor
    - Comanda **mount** - specifica gazda *remote*, sistemul de fisiere ce trebuie accesat si unde trebuie sa fie localizat in ierarhia locala de fisiere
  - RFC 1094

RPC|Utilizari

- Accesul la fisiere la distanta **NFS** (**Network File System**)
  - Este transparent pentru utilizator
  - Clientul NFS trimite o cerere RPC serverului RPC, folosind TCP/IP
    - Obs.: NFS a fost folosit predominant cu UDP
  - Serverul NFS primeste cererile la portul 2049 si le trimite la modulul de accesare a fisierelor locale

Obs.: Pentru deservirea rapida a clientilor, serverele NFS sunt in general *multi-threading* sau pentru sisteme UNIX care nu sunt *multi-threading*, se creeaza si raman in kernel instante multiple a procesului (*nfsd*-uri)

RPC|Utilizari

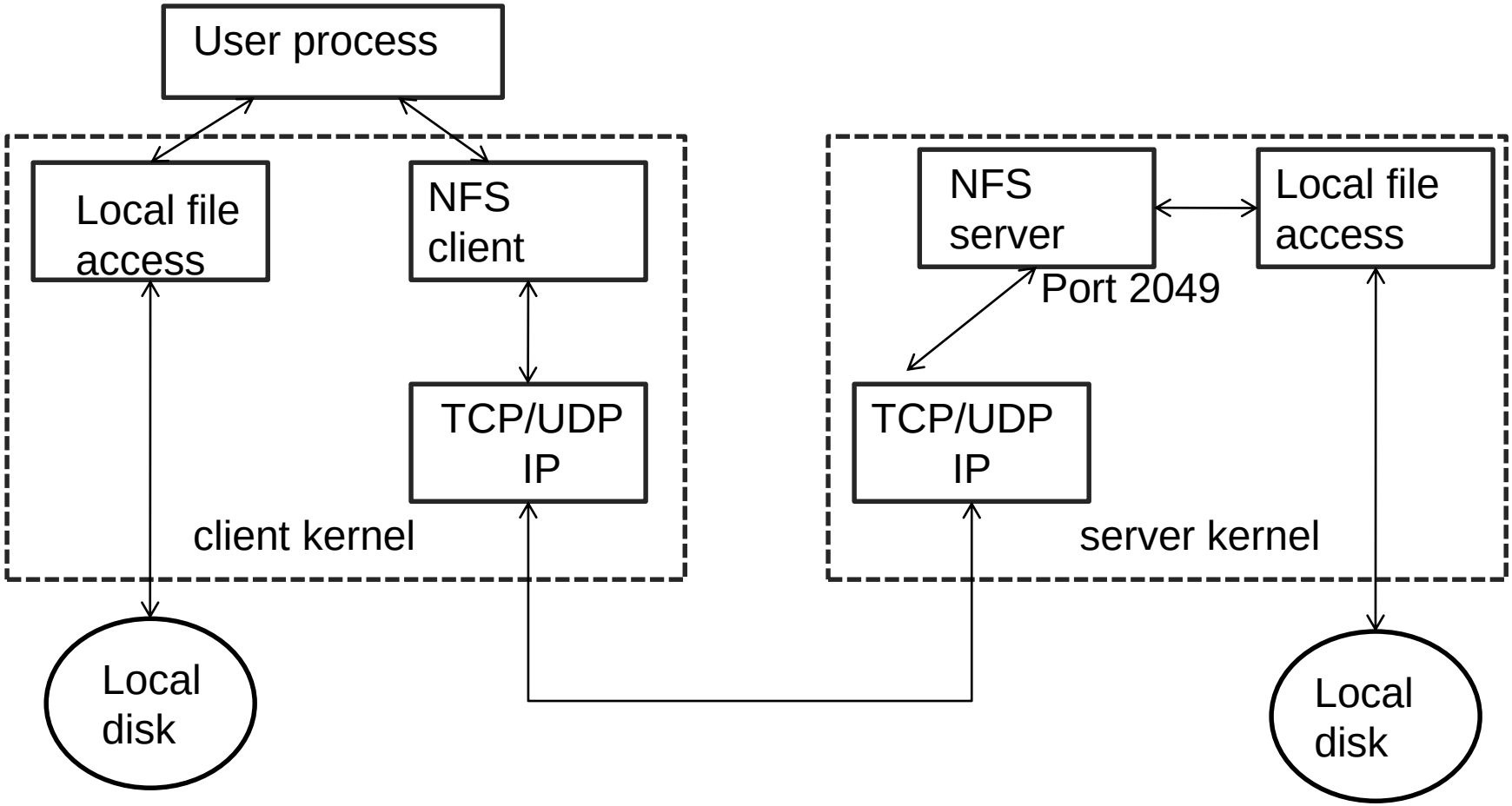


Figura: Arhitectura NFS

RPC|Utilizari

- Accesul la fisiere la distanta **NFS** (**Network File System**)
  - (0) este pornit *portmapper*–ul la *boot*-area sistemului
  - (1) daemonul *mountd* este pornit pe server; creeaza *end-point*-uri TCP si UDP, le asigneaza porturi efemere si apeleaza la *portmapper* pentru inregistrarea lor
  - (2) se executa comanda *mount* si se face o cerere la *portmapper* pentru a obtine portul serverului *demon* de *mount*
  - (3) *portmapper*–ul intoarce raspunsul
  - (4) se creeaza o cerere RPC pentru montarea unui sistem de fisiere
  - (5) serverul returneaza un *file handle* pentru sistemul de fisiere cerut
  - (6) Se asociaza acestui *file handle* un punct de montare local pe client (*file handle* este stocat in codul clientului NFS si orice cerere pentru sistemul de fisiere respectiv va utiliza acest *file handle*)

RPC|Utilizari

- Accesul la fisiere la distanta **NFS** (**Network File System**)
  - Procesul de montare (protocolul **mount**)
    - Pentru ca un client sa poata accesa fisiere dintr-un sistem de fisiere, clientul trebuie sa foloseasca protocolul **mount**

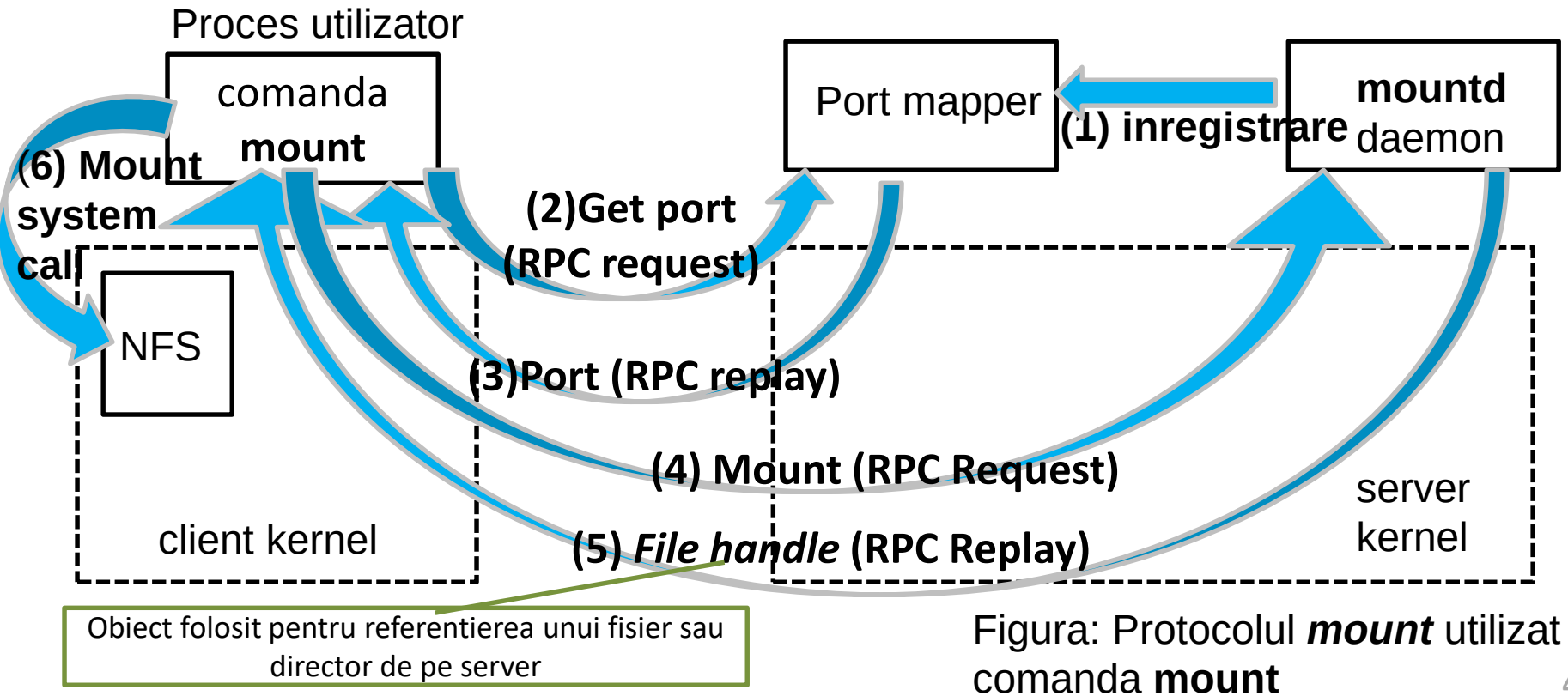


Figura: Protocolul ***mount*** utilizat de comanda **mount**

Rezumat

- **Remote Procedure Call (RPC)**
  - Preliminarii
  - Caracterizare
  - XDR (External Data Representation)
  - Functionare
  - Implementari
  - Utilizari

Intrebari?

Nivelul Retea (II)

Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- **Nivelul retea**
  - **Activitatea de rutare (dirijare)**
    - **Preliminarii**
    - **Caracterizare**
    - **Rutare**
    - **Protocoale de rutare**
      - **RIP & OSPF**
      - **BGP & EGP**
- **Congestie – discutii generale**

Rutare|Preliminarii

- Partea software-ului nivelului retea care alege calea pe care un pachet receptionat trebuie trimis pentru a ajunge la destinatie
- Daca se folosesc datagrame, decizia de rutare trebuie luata pentru fiecare pachet
- Daca se utilizeaza circuite virtuale, decizia de rutare se ia la stabilirea unui nou circuit
- Cerintele pentru un algoritm de rutare: corect, simplu, robust, optim, rapid convergent
- Activitati
  - Determinarea caii optime de rutare (*routing*)
  - Transportarea pachetelor: comutare (*packet switching*)

Terminologie

- ***end systems*** – dispozitive de retea fara capacitati de redirectat pachete catre subretele
- ***intermediate systems*** – dispozitive de retea avand capacitati de redirectat pachete
  - Intradomain IS – comunicare in cadrul unui domeniu de rutare
  - Interdomain IS – comunicare si intre domenii de rutare
- **sistem autonom** – AS (eng. *Autonomous system*) – colectie de retele care partajeaza aceeasi strategie de dirijare

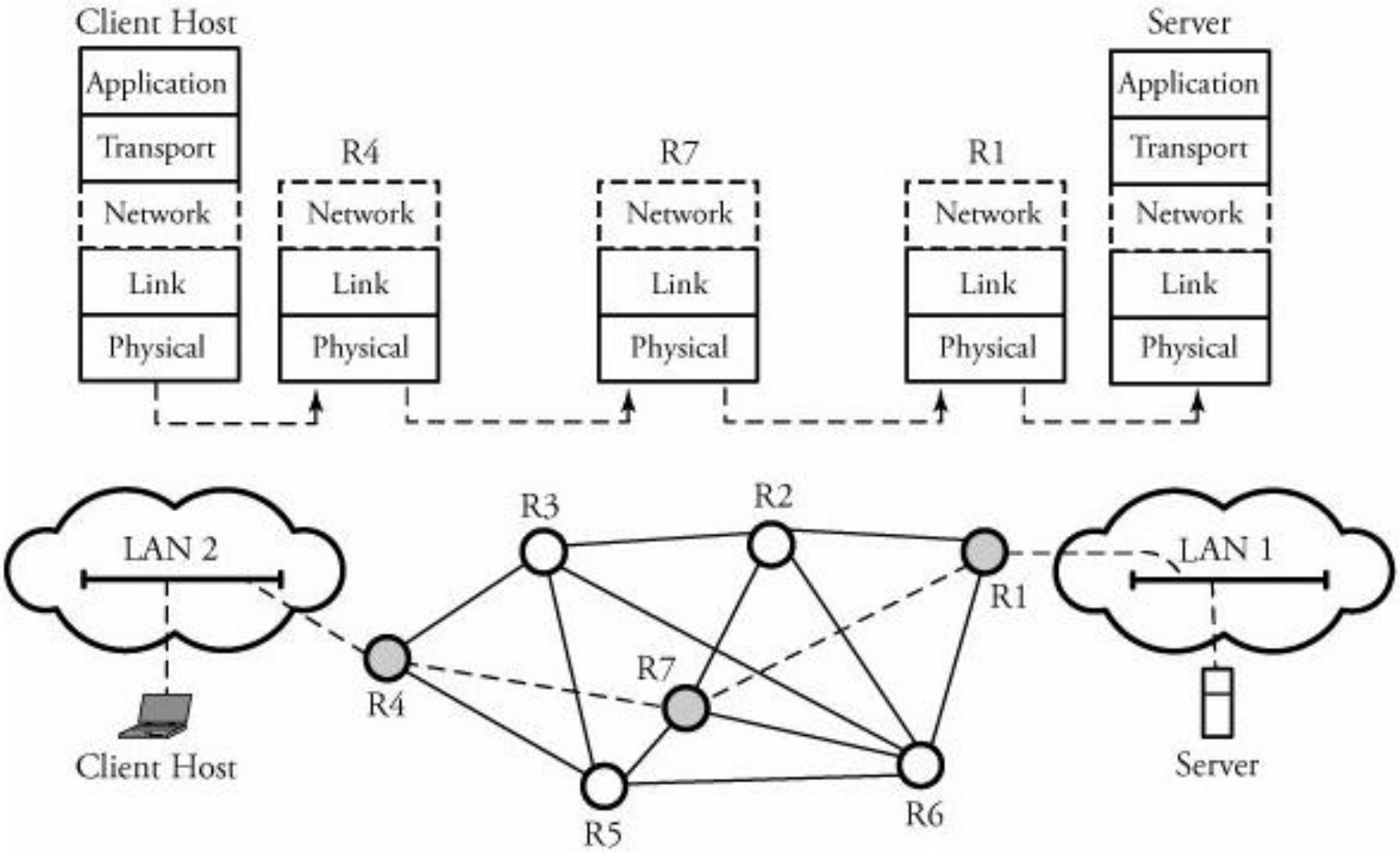
Nivelul retea | ...sa ne reamintim

- La nivelul retea, Internetul poate fi vazut ca o colectie de subretele sau sisteme autonome conectate intre ele

  IP-ul este liantul care face posibila aceasta interconectare. (a se vedea Cursul 2)
- Nivelul retea se ocupa cu trimiterea pachetelor de la sursa la destinatie (mecanism care implica trecerea printr-o serie de noduri intermediare) => nivelul retea este nivelul cel mai de jos care se ocupa cu transmisia *end-to-end*

  Obs.: Nivelul legaturii de date are rolul de transport a *frame*-urilor de la un punct la altul

Nivelul retea| …sa ne reamintim



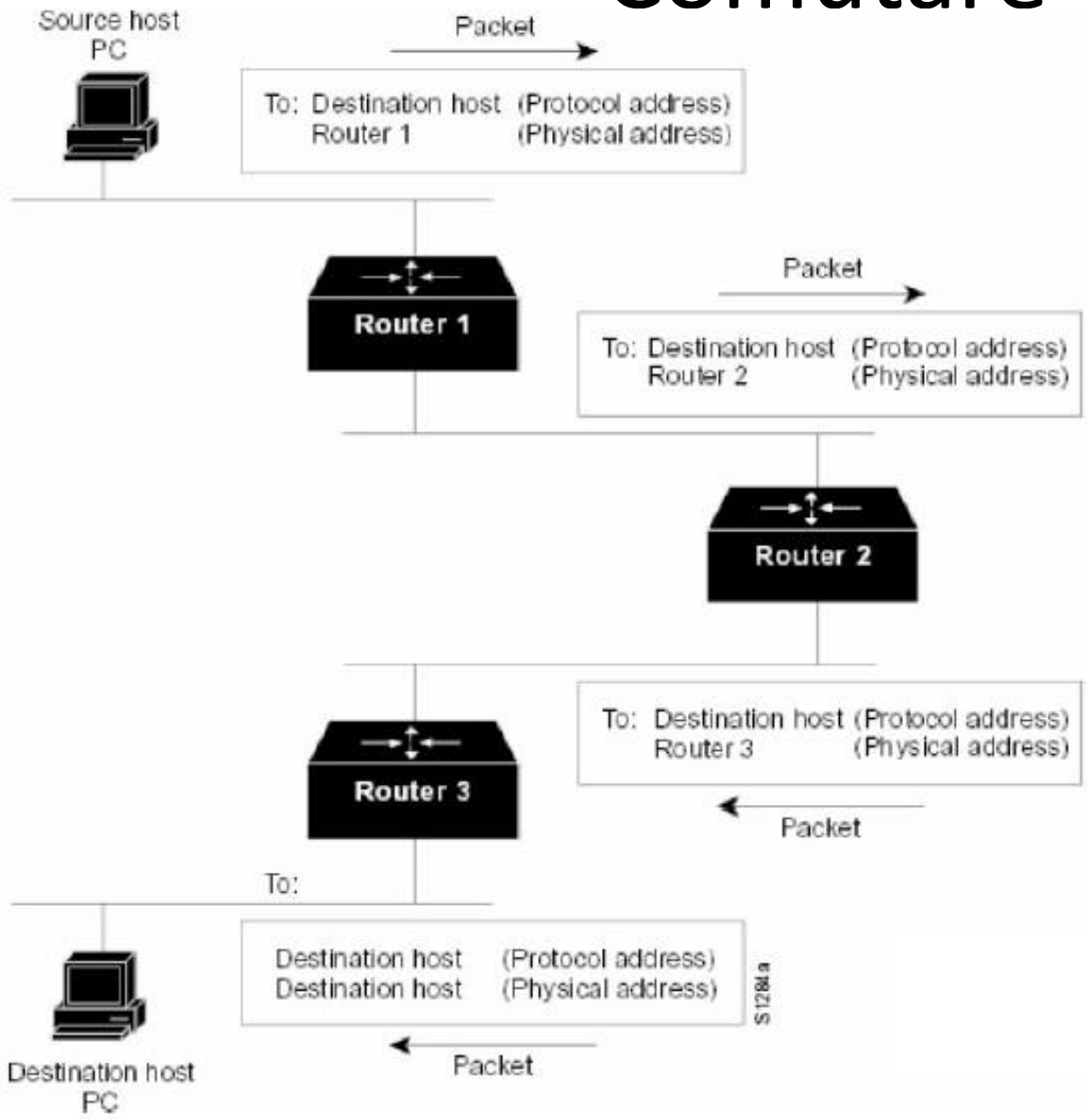
Comunicare *end-to-end* intre o gazda client si un server la nivelul retea

[Computer and Communication Networks, 2006, Nader F. Mir]

Comutare

- O gazda (eng. *host*) are de trimis un pachet la un alt *host*
- *Host*-ul sursa trimite pachetul la un router, folosind adresa hardware (MAC) a acestuia, un pachet continand adresa de retea a gazdei destinatie
- Routerul examineaza adresa de retea a destinatarului, iar daca nu cunoaste unde sa trimita pachetul, il va distruge
- Altfel, va modifica adresa continuta de pachet in adresa hardware a urmatorului *hop* (punct indermediar de transmitere – *Intermediate System*) si va trimite pachetul spre acesta
- Daca urmatorul *hop* nu este destinatia finala, atunci procesul se repeta pentru un alt router, s.a.m.d.

Comutare

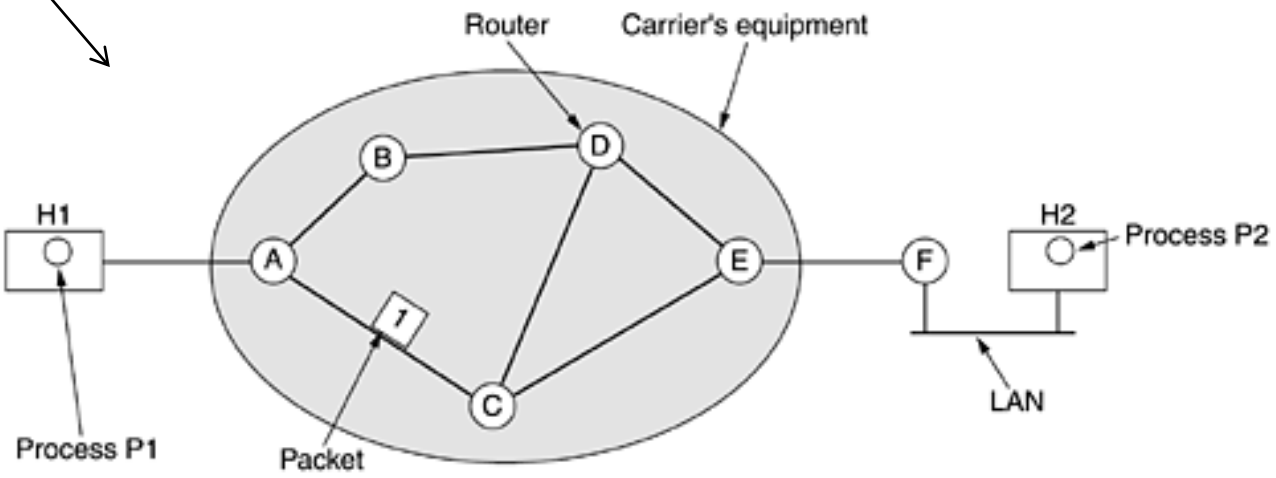


Procesul de comutare

[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Rutare

Multimea tuturor ruterelor (engl. *communication subnet*)



Context pentru protocoalele de la nivelul retea

Nivelul retea:

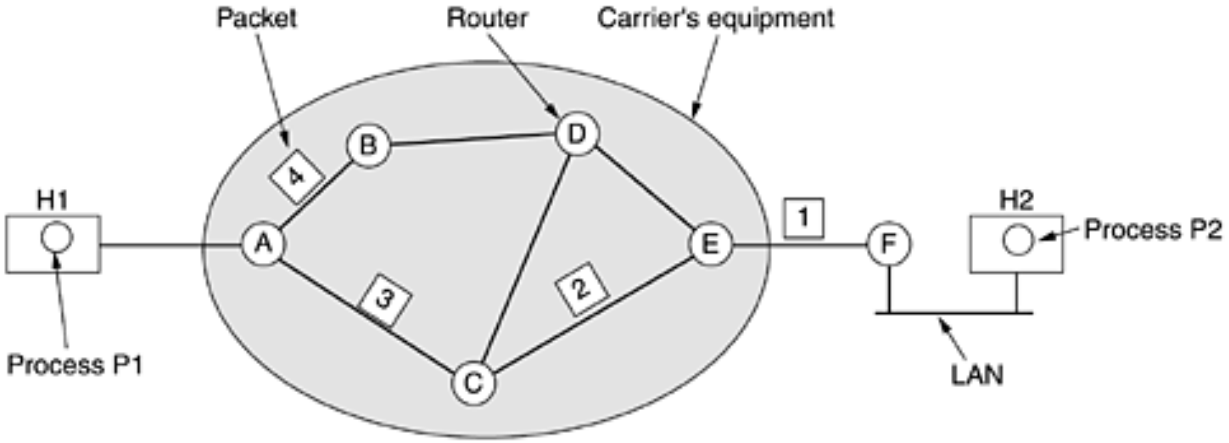
- trebuie sa cunoasca topologia ruterelor si sa aleaga calea pe care un pachet trebuie trimis spre destinatie
- trebuie sa faca alegerea astfel incat sa evite supraincarcarea unor linii de comunicatie si a unor rutere (vezi slide-urile urmatoare)

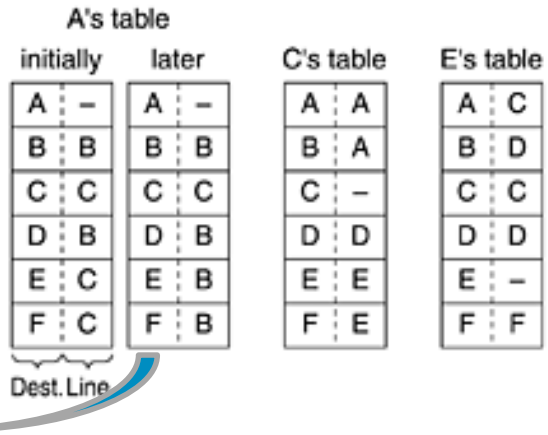
Determinarea caii optime de rutare

[Computer Networks, 2003 Andrew S. Tanenbaum]

Rutare

- In cazul in care la nivelul retea avem servicii neorientate conexiune, pachetele (numite si *datagrame*) sunt trimise individual si sunt rutate in mod independent una de alta





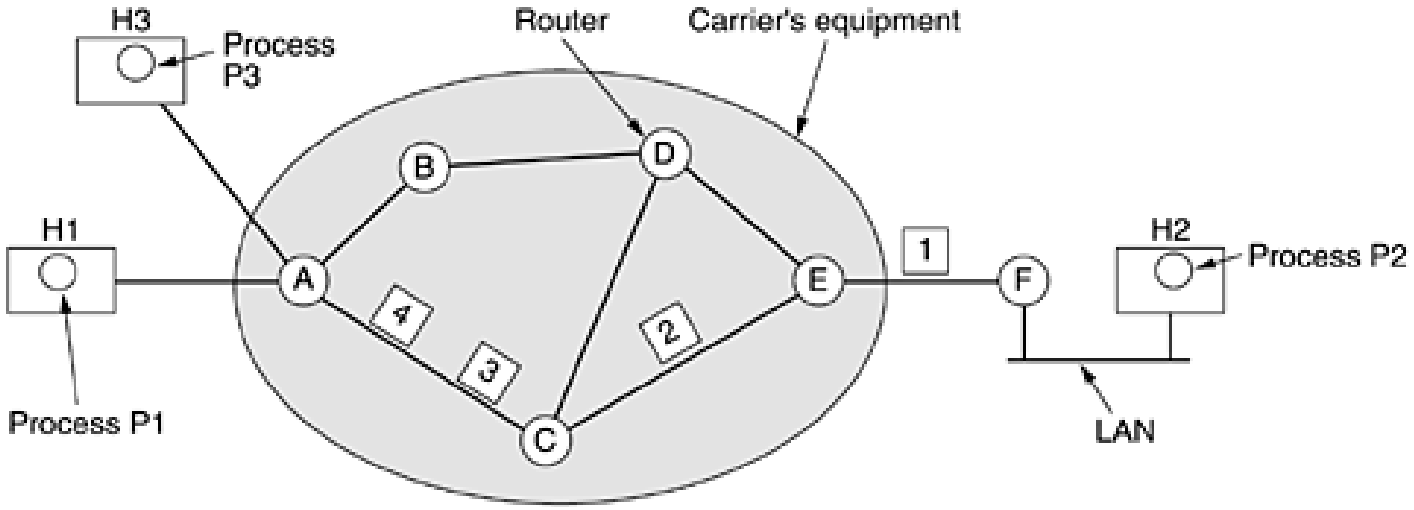
Algoritmii de rutare fac managementul tabelelor de rutare

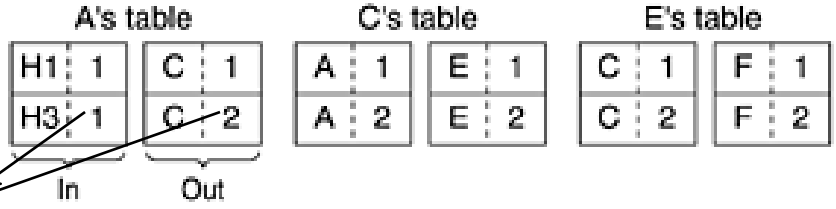
Rutare in interiorul unui *datagram subnet*

[Computer Networks, 2003 Andrew S. Tanenbaum]

Rutare

- In cazul in care la nivelul retea avem servicii orientate conexiune se folosesc circuite virtuale si decizia de rutare se ia la stabilirea unui nou circuit





label switching

Rutare in interiorul unui *virtual-circuit subnet (session routing)*

[Computer Networks, 2003 Andrew S. Tanenbaum]

Rutare

Comparatie intre *datagram subnet* si *virtual-circuit subnet*

Issue	Datagram subnet	Virtual-circuit subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC

[Computer Networks, 2003 Andrew S. Tanenbaum]

Rutare

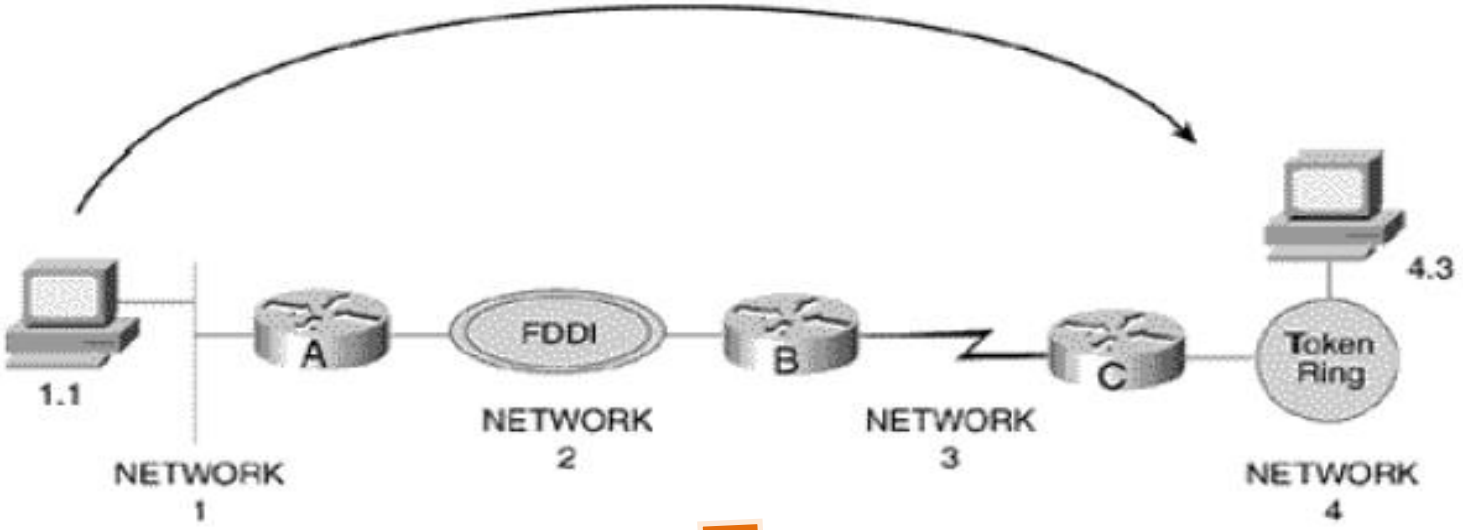
Determinarea caii de rutare

- Pentru fiecare cale de rutare se determina un cost (metrica)
  - Lungimea caii, siguranta, intarzierea, largimea de banda, incarcarea, costul comunicarii

- Algoritmii de rutare initializeaza si mentin (pentru fiecare gazda) tabele de rutare continand informatii de dirijare
  - Rute catre gazde specificate
  - Rute spre retele specificate
  - O ruta implicita

Rutare

Un router creeaza o cale logica intre subretele



O aplicatie ruland pe gazda 1.1 nu trebuie sa cunoasca drumul pentru a trimite date aplicatiei de pe gazda 4.3

[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Rutare

Algoritmii de rutare - caracteristici:

- **Acuratete** (engl. *Accuracy*) – un algoritm trebuie sa opereze in mod corect si rapid pentru gasirea destinatiei
- **Complexitate redusa** – important pentru rutere cu resurse fizice (soft) limitate
- **Optimalitate** – abilitatea de a gasi ruta optima
- **Robustete** – capacitatea de a functiona corect pentru o perioada lunga de timp, in circumstante diferite
- **Adaptabilitate** – la aparitia unei erori in retea, algoritmul trebuie sa se adapteze (de ex. caderea nodurilor sau coruperea tabelelor de rutare)
- **Convergenta** – algoritmii de rutare trebuie sa convearga rapid atunci cand sunt distribuite mesaje de rutare de actualizare
- ***Load balancing*** – un algoritm de rutare cantareste diferite posibilitati de rutare pentru evitarea legaturilor incete sau a congestiilor

Rutare

- Abstractizare
  - Retea = graf
  - Dirijarea= gasirea drumului de cost minim de la un nod sursa la un nod destinatie

  Tipuri de rutare:
  - **Centralizata** – drumul de cost minim poate fi determinat avand disponibile toate informatiile despre retea

    <– **algoritmi folosind starea legaturii**
  - **Descentralizata** – drumul de cost minim este determinat in mod iterativ, distribuit (nici un nod nu poseda informatii complete despre costurile legaturilor din retea) <- **algoritmi cu vectori distanta**

Rutare

- Rutare folosind **starea legaturii**
  - Topologia retelei & costurile tuturor legaturilor sunt cunoscute
  - Un nod trebuie sa cunoasca identitatile & costurile nodurilor vecine
  - Fiecare nod difuzeaza prin *broadcast* identitatile si costurile tuturor legaturilor de la acel nod la altele

Rutare

- Rutare **cu vectori distanta**
  - Fiecare nod primeste informatii de la nodurile vecine, realizeaza calcule si distribuie rezultatele inapoi la vecinii directi – algoritmul este distribuit si asincron
  - Fiecare nod mentine o tabela de distanta (*distance table*)
  - **X**: nodul dorind sa realizeze o rutare la nodul **Y** via nodul vecin **Z**
  - **$D^x(Y,Z)$**: suma costului legaturii directe intre **X** si **Z** (**c(X,Z)**) plus costul curent al drumului minim de la vecinii lui **Z** la **Y**:

    $$D^x(Y,Z)=c(X,Z) + \min_w\{D^z(Y,w)\}$$

  - Tabela de rutare a unui nod poate fi construita cunoscand tabela de distanta a nodului

Rutare

Algoritmii de rutare – clasificare:

- **Statici (neadaptivi)**
  - Topologia legaturilor se incarca pentru o perioada de timp in tabelele de rutare a fiecarui nod
  - Dezavantaje:
    - Reteaua trebuie sa aiba o dimensiune optima pentru a putea fi controlabila
    - Daca au loc esuari in retea, nu se poate reactiona imediat
- **Dinamici (adaptivi)**
  - Starea retelei este “invatata” din comunicarea ruterelor cu vecinii lor; starea fiecarei regiuni din retea este propagata in retea dupa ce toate nodurile isi actualizaza tabelele de rutare => fiecare ruter poate gasi calea cea mai buna pe baza informatiilor de la nodurile vecine

Rutare

Algoritmii de rutare – clasificare:

- Statici (neadaptivi)
  - Dirijare pe calea cea mai scurta
  - Inundare (eng. *flooding*)
  - *Deflecting routing (*sau *hot-potato routing)*
- Dinamici (adaptivi)
  - Cu vectori distanta
  - Folosind starea legaturilor
  - Dirijare ierarhica
  - Prin difuziune (*broadcast*)
  - Cu trimitere multipla (*multicast*)

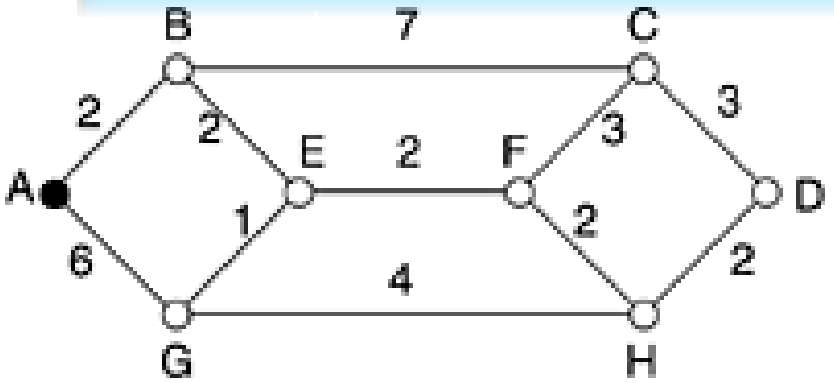
Rutare

Algoritmii de rutare – clasificare:

- **Statici** (neadaptivi)
  - **Dirijare pe calea cea mai scurta** (eng. *Shortest path routing*)

    Algoritmul lui Dijkstra (calculeaza drumul de cost minim)
    - Este folosit de protocolul OSPF

"shortest path": nr. de hopuri => ABC si ABE sunt egale

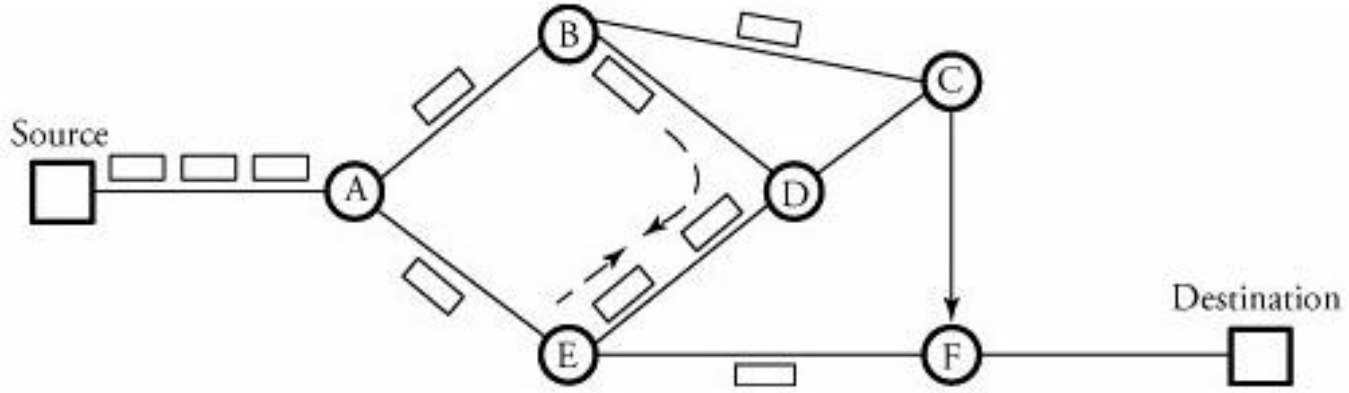


Alte metrici posibile: distanta geografica, largimea de banda, costuri de comunicare etc.

Rutare

Algoritmii de rutare – clasificare:

- Statici (neadaptivi)
  - **Inundare** (eng. *flooding*)
    - Un pachet primit este copiat si transmis prin toate legaturile de comunicare (exceptand cea pe unde a venit)
    - Problema: *packet reflection* (un nod poate primi o copie nedorita a unui pachet)
    - Utilizari ale algoritmilor de tip flooding: aplicatii militare, baze de date distribuite, etc.



Rutare

Algoritmii de rutare – clasificare:

- Statici (neadaptivi)
  - ***Deflection routing***
    - La fiecare pas un pachet este examinat in raport cu adresa destinatie; daca legatura ceruta este libera pachetul este trimis, altfel este deviat (*deflected*) catre o alta linie de comunicare aleasa aleator;
    - Un pachet are asociat un camp cu o valoare de prioritate care il poate ajuta pe viitor sa castige disputa cu alte pachete

Rutare

Algoritmii de rutare – clasificare:

- **Dinamici** (adaptivi)
  - **Cu vectori distanta**
    - Fiecare router mentine un tabel (vector) cu distanta si linia de comunicare catre destinatie; tabelele sunt actualizate cu informatiile de la vecini

    Algoritmul Bellman-Ford
    - Algoritm folosit de protocoalele RIP, BGP, IGRP

Exemplu:

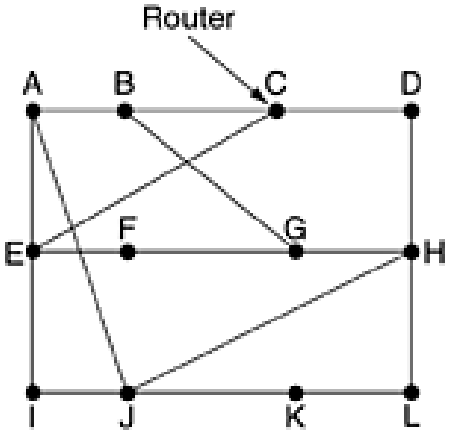
- Consideram ca metrica: intarzierea (*msec*);
- Routerele vor sti intarzierile asociate vecinilor sai

Rutare

Algoritmii de rutare – clasificare:

- Dinamici (adaptivi)
  - **Cu vectori distanta**

Exemplu:



J doreste sa calculeze ruta catre G

J->A->G = 26 (18+8) msec

.....

J->H->G 18 msec

New estimated delay from J

To	A	I	H	K	Line
A	0	24	20	21	8 A
B	12	36	31	28	20 A
C	25	18	19	36	28 I
D	40	27	8	24	20 H
E	14	7	30	22	17 I
F	23	20	19	40	30 I
G	18	31	6	31	18 H
H	17	20	0	19	12 H
I	21	0	14	22	10 I
J	9	11	7	10	0 –
K	24	22	22	0	6 K
L	29	33	9	9	15 K
	JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6	New routing table for J

Vectors received from J's four neighbors

[Computer Networks, 2003 Andrew S. Tanenbaum]

Rutare

Problema: conform algoritmului cu vectori distanta, la fiecare actualizare a rutelor, tabelele de rutare trebuie trimise fiecarui vecin; unele pachete cu informatii legate de dirijare trec pe ruta de pe care deja au venit (*reverse route*)

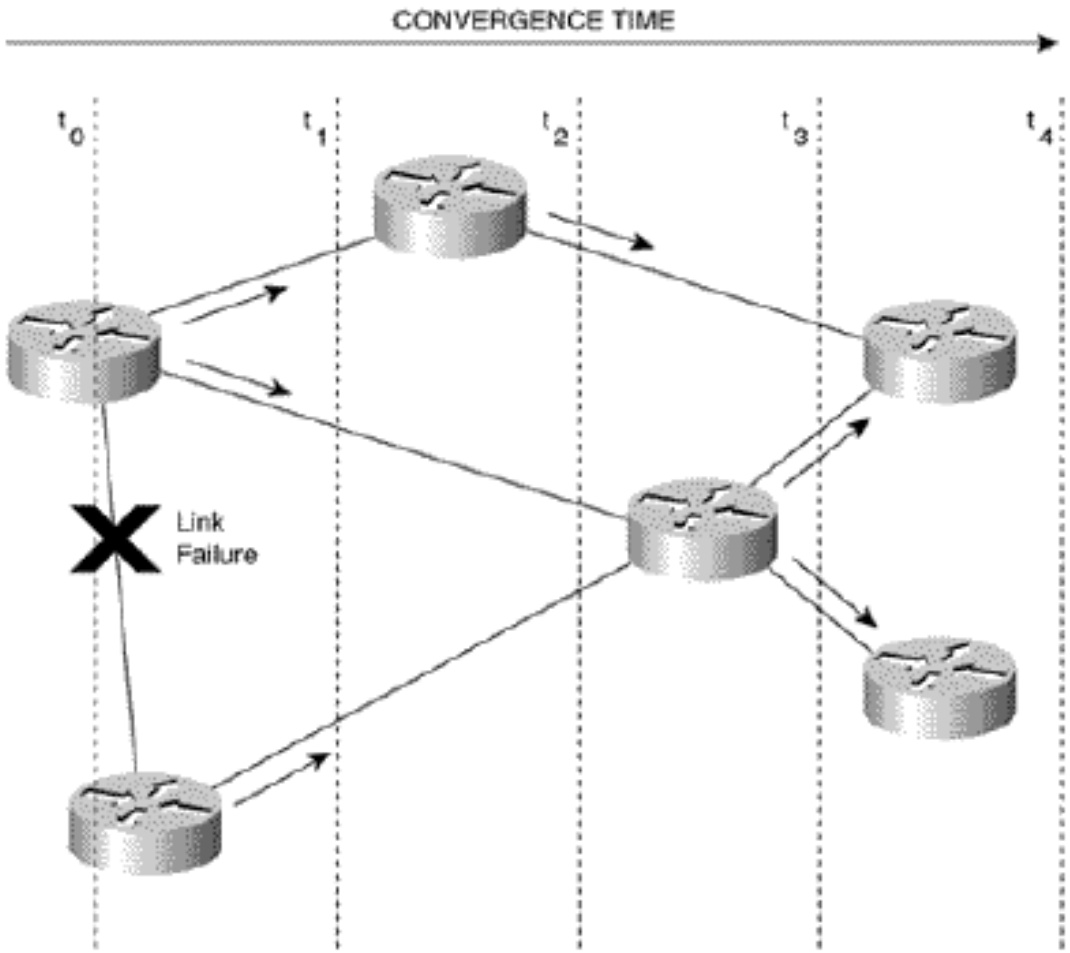
Intrebare: Pot fi evitate rutele de tip *reverse*?

Raspuns: utilizarea tehnicii ***split horizon***

- Cand *router*-ul trimite actualizari de rute folosind o anumita interfata de retea, ele nu vor fi expediate retelelor ale caror rute au fost invatate din actualizari primite via acea interfata

Rutare

Problema:
Modificarea topologiei (deprecierea convergentei algoritmilor de rutare)



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Rutare

Algoritmii de rutare – clasificare:

- Dinamici (adaptivi)
  - **Folosind starea legaturilor**

  Fiecare router trebuie sa:
  - Descopere vecinii si sa le “invete” adresele de retea
  - Masoare intarzierea sau costul asociat fiecarui vecin
  - Construiasca un pachet prin care anunta pe “toti” ceea ce a invatat

    Dilema: cand trebuie construite pachetele? (de ex. periodic sau cand apare un eveniment special)
  - Trimita pachetul
  - Calculeze cea mai scurta cale catre fiecare router

Rutare

Algoritmii de rutare – clasificare:

- Dinamici (adaptivi)
  - **Dirijare ierarhica**

Necesitate: in retele de mari dimensiuni nu este fezabil ca un router sa aiba cate o intrare despre fiecare alt router;

Mecanism: Ruterele stiu detalii asociate unei *regiuni ,* dar nu stiu detalii despre structura interna a altor regiuni

Obs.:

- Pentru retele de dimensiuni mari, ierarhia de nivel 2 nu este suficienta si atunci regiunile se grupeaza in clustere, clusterele in zone, zonele in grupuri, etc.
- Care este numarul optim de niveluri?

  Pentru un *subnet* cu N routere numarul optim ***ln N***
  [Kamoun&Kleinrock, 1979]

Rutare

Algoritmii de rutare – clasificare:

- Dinamici (adaptivi)
  - **Prin difuziune (*broadcast routing*)**

  Utilizare: actualizarea stocurilor (de la bursa de valori), *streaming multimedia,* serviciu de distribuire a rapoartelor despre vreme, etc.

  Modalitati:

  - **Sursa trimite cate un pachet distinct fiecarui destinatar**

    Obs.: Metoda ineficienta: neutilizarea latimii de banda; Sursa trebuie sa aiba adresele tuturor destinatarilor
  - ***Flooding* -** util cind alte metode nu pot fi aplicate

    Problema: Se genereaza prea multe pachete si se consuma multa latime de banda

Rutare

Algoritmii de rutare – clasificare:

- Dinamici (adaptivi)
  - **Cu trimitere multipla (*multicast routing*)**

  Exemplu de utilizare:

  Un proces doreste sa transmita un mesaj unui grup de procese implementand un sistem de baze de date distribuite

  Obs.: Se poate face *broadcast,* insa uneori informatia nu este destinata a fi vazuta de oricine

  Mecanism: *router*-ul va face periodic o interogare asupra *host*-urilor care apartin unui grup; apoi informatia este propagata catre routere

Rutare

Exemplu:

```
[adria@thor ~] $ netstat -r
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
webapps.vpn     *               255.255.255.255 UH        0 0          0 tun1
172.18.100.2    *               255.255.255.255 UH        0 0          0 tun0
85.122.23.0     *               255.255.255.128 U         0 0          0 eth2
172.18.100.0    172.18.100.2    255.255.255.0   UG        0 0          0 tun0
172.17.17.0     webapps.vpn     255.255.255.0   UG        0 0          0 tun1
default         gw-info-c.uaic. 0.0.0.0         UG        0 0          0 eth2
[adria@thor ~] $ netstat -rn
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
172.17.17.1     0.0.0.0         255.255.255.255 UH        0 0          0 tun1
172.18.100.2    0.0.0.0         255.255.255.255 UH        0 0          0 tun0
85.122.23.0     0.0.0.0         255.255.255.128 U         0 0          0 eth2
172.18.100.0    172.18.100.2    255.255.255.0   UG        0 0          0 tun0
172.17.17.0     172.17.17.1     255.255.255.0   UG        0 0          0 tun1
0.0.0.0         85.122.23.126   0.0.0.0         UG        0 0          0 eth2
```

[http://docstore.mik.ua/orelly/networking_2ndEd/tcp/ch02_04.htm]

Exemplu: Pentru destinatia 172.17.17.0 routerul (gateway-ul) folosit este 172.17.17.1 ; Gateway = 0.0.0.0 -> interfata de retea locala

Flag-uri: U(*up*) – ruta este operationala; H - indica o ruta catre o anumita gazda; G - ruta utilizeaza un gateway exterior

Crearea tabelelor de rutare

- Rute statice: comanda UNIX **route**
- Descoperirea unui router prin ICMP
  - Protocol de tip broadcast care descopera routerele unei retele locale

Rutare

Protocoale de rutare - clasificare

- ***Intradomain routing protocol*** – realizeaza rutarea pachetelor intr-un domeniu
  - RIP (Routing Information Protocol)
  - OSPF (Open Shortest Path First)
- ***Interdomain routing protocol*** – realizeaza rutarea pachetelor intre domenii
  - BGP (Border Gateway Protocol)
  - EGP (Exterior Gateway Protocol)
    - RFC 827, 904
    - nu mai este utilizat, fiind inlocuit de BGP

Rutare

RIP (Routing Information Protocol)

- RFC 1058, 1723
- Mecanismul de functionare:
  - Se aplica algoritmul Bellman-Ford (pentru *host*-uri si routere)
  - Pentru fiecare router, se creeaza un ***vector*** continand costul rutei si alte informatii
  - Daca survin modificari intr-un punct, acestea sunt propagate periodic la routerele si *hos*t-urile vecine cu acel punct

Rutare

RIP (Routing Information Protocol)

- Foloseste mesaje IP
- Fiecare *router* trimite un *broadcast* continand intreaga tabela de rutare a *router*-ului – la fiecare 30 sec.
- O intrare a tabelei de rutare RIP contine:
  - Adresa IP
  - Metrica (numarul de hop-uri: 1-15)
  - Timeout (in secunde)
- Retelele conectate direct au metrica =1 (un hop)

Tabela de rutare A: nodul B e la 1 *hop* distanta (conexiune directa), nodul C la 2 *hop*-uri

- Daca o ruta da timeout, metrica devine 16 (nu exista conexiune) si ruta e stearsa dupa 1 minut

Rutare

RIP (Routing Information Protocol)

- Daca o informatie de rutare se modifica (de ex. o legatura sau un *router* esueaza), propagarea acestei schimbari are loc foarte lent – RIP sufera de convergenta lenta
- RIP
  - Este un protocol matur, stabil, larg suportat si usor de implementat
  - Este indicat a fi folosit de sistemele autonome de dimensiuni reduse fara rute redundante
  - In practica este inlocuit in majoritatea situatiilor de OSPF

Rutare

OSPF (Open Shortest Path First)

- RFC 1247, 2328
- Fiecare router ce foloseste OSPF cunoaste starea intregii topologii de retea (algoritm folosind starea legaturii) si transmite actualizari la toate routerele

- Conduce la trafic aditional, care poate conduce la congestii

  OSPF permite ca traficul sa fie distribuit pe rute cu costuri similare (*load balancing*)

  OSPF suporta rutarea dupa tipul serviciilor (ToS)

  – protocolul IP contine campul ToS (in general neutilizat)
- Convergenta mai rapida
- Ofera suport pentru folosirea mai multor tipuri de metrici

Rutare

OSPF (Open Shortest Path First)

- Opereaza intr-o ierarhie de entitati de retea

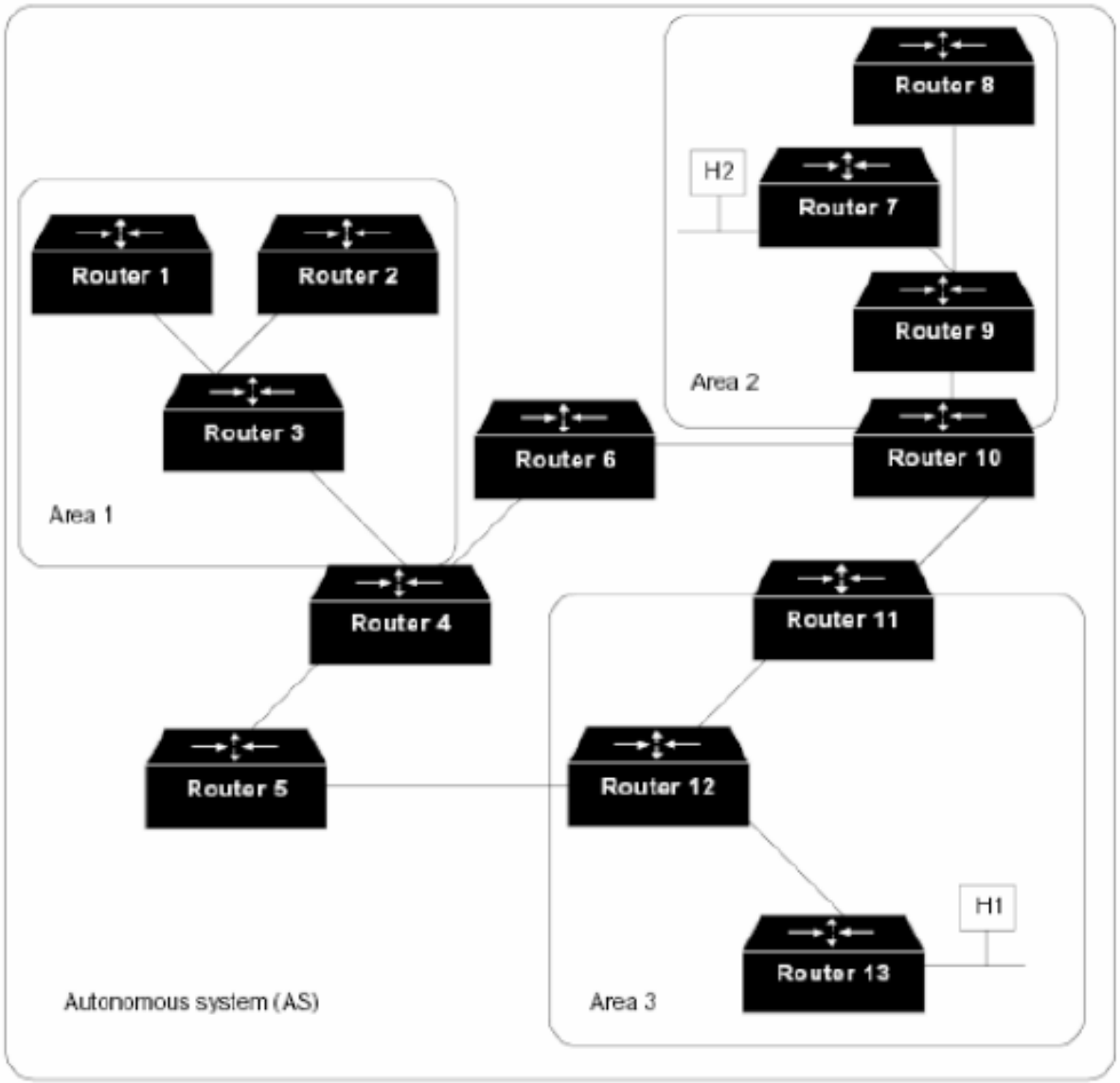
  Motivatie: retele de dimensiuni mari => un router nu poate cunoaste intreaga topologie

  - **Sistemul autonom** (AS) – colectie de retele care partajeaza aceeasi strategie de dirijare
  - Un AS este divizat in **domenii (engl. areas)** – grupuri contigue de retele si gazde; routerele au aceeasi informatie privitoare la topologie si ruleaza acelasi algoritm
  - **Coloana vertebrala** (*backbone* sau *area 0*) – responsabila cu distributia informatiilor de rutare intre domenii; orice router conectat la doua sau mai multe domenii face parte din *backbone* (aceste routere vor rula algoritmi corespunzatori pt fiecare domeniu)

Rutare

OSPF (Open Shortest Path First)

Un AS si domeniile sale conectate via routere



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Rutare

OSPF (Open Shortest Path First)

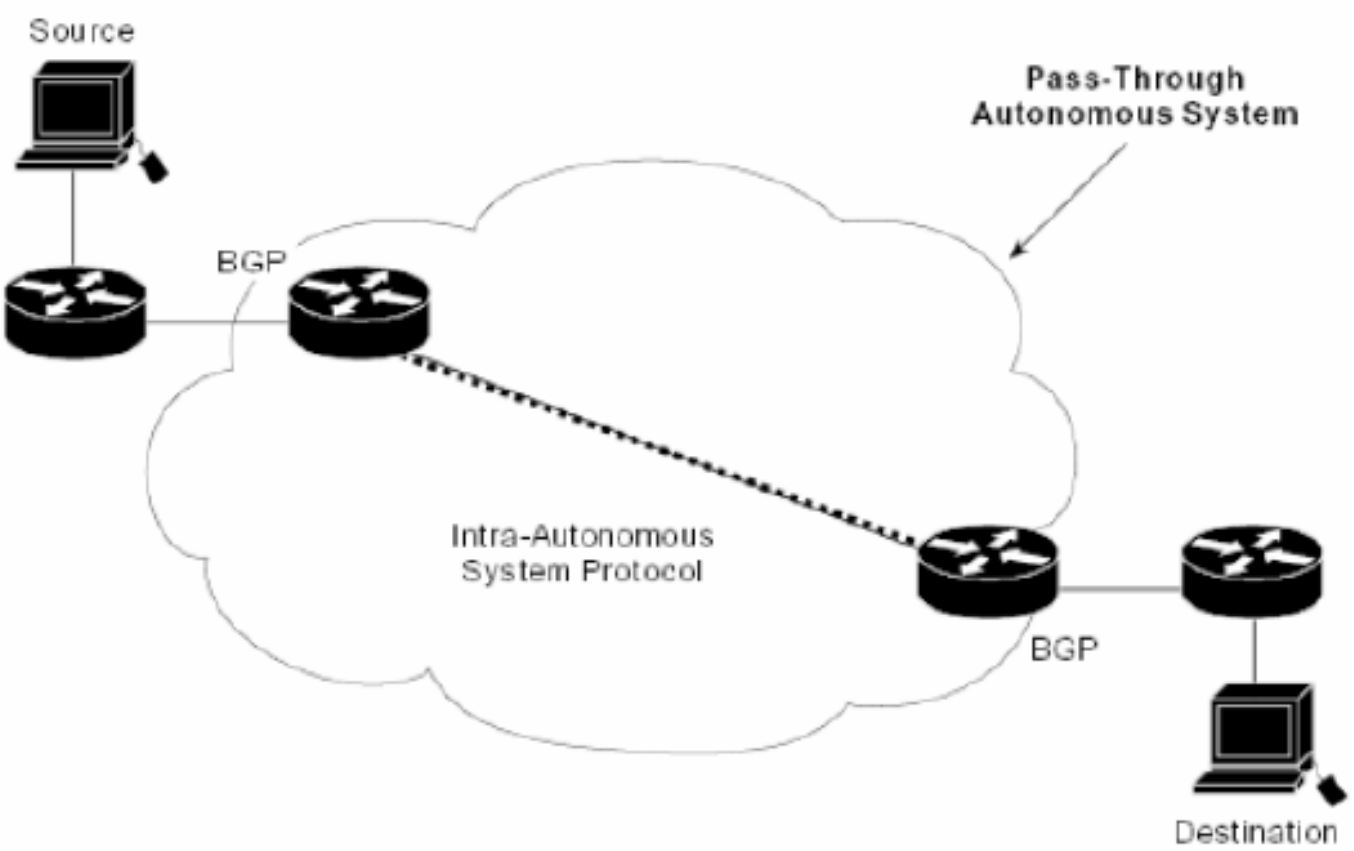
Tipuri de mesaje OSPF:

Message type	Description
Hello	Used to discover who the neighbors are
Link state update	Provides the sender's costs to its neighbors
Link state ack	Acknowledges link state update
Database description	Announces which updates the sender has
Link state request	Requests information from the partner

- Cu un mesaj "hello" un router isi afla vecinii (de ex. toate routerele din LAN)
- Fiecare router face *flood* periodic cu un mesaj (ce are asociat un numar de secventa) de tipul *Link state update*; la aceste mesaje se fac confirmari *Link state ack*
- *Database description* furnizeaza numerele de secventa asociate intrarilor legaturilor avute in evidenta de emitator

Rutare

BGP (Border Gateway Protocol)



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Rutare

BGP (Border Gateway Protocol)

- Utilizat pentru comunicarea intre routere aflate in sisteme autonome diferite
- Functii majore:
  - *Neighbor relationship* – se refera la acordul dintre routerele din doua sisteme autonome de a schimba informatii pe baza unor reguli (un router poate refuza stabilirea unei astfel de relatii in functie de: regulile domeniului, supraincarcare etc)
  - *Neighbor maintenance* – routerele isi vor trimite mesaje de tip *keep-alive*
  - *Network maintenance* – fiecare router tine o baza de date cu subretelele existente pentru o rutare eficienta in acea subretea

Rutare

BGP (**Border Gateway Protocol**)

- Exista patru tipuri de pachete BGP:
  - *Open:* folosit pentru stabilirea unei relatii dintre doua routere
  - *Update:* contine informatii actualizate despre rute
  - *Keep-alive:* folosit pentru confirmarea de relatii stabilite anterior
  - *Notification*: folosit atunci cand apar erori
- Perechile de routere BGP comunica intre ele folosind conexiuni TCP
- BGP este un protocol bazat pe vectori distanta cu urmatoarele diferente:
  - nu se pastreaza doar costul asociat unei destinatii, ci se mentine si calea catre acea destinatie
  - nu se furnizeaza vecinilor doar costul estimat, ci si calea exacta
- RFC 1771-1774, 4271

Rutare

Alte protocoale:

- Interior Gateway Routing Protocol (IGRP)
  - Imbunatatire CISCO a RIP
- Enhanced IGRP (EIGRP)
- Simple Multicast Routing Protocol (SMRP)
  - Rutare de fluxuri multimedia la Apple (via AppleTalk)

  Obs.: Din 2009 AppleTalk este nesuportat, se utilizeaza TCP/IP
- Resource Reservation Protocol (RSVP) (RFC 2205)
  - Nu este un protocol de rutare, dar ofera functionalitati similare
  - Asigura calitatea serviciilor IP

Rutare| privire de ansamblu

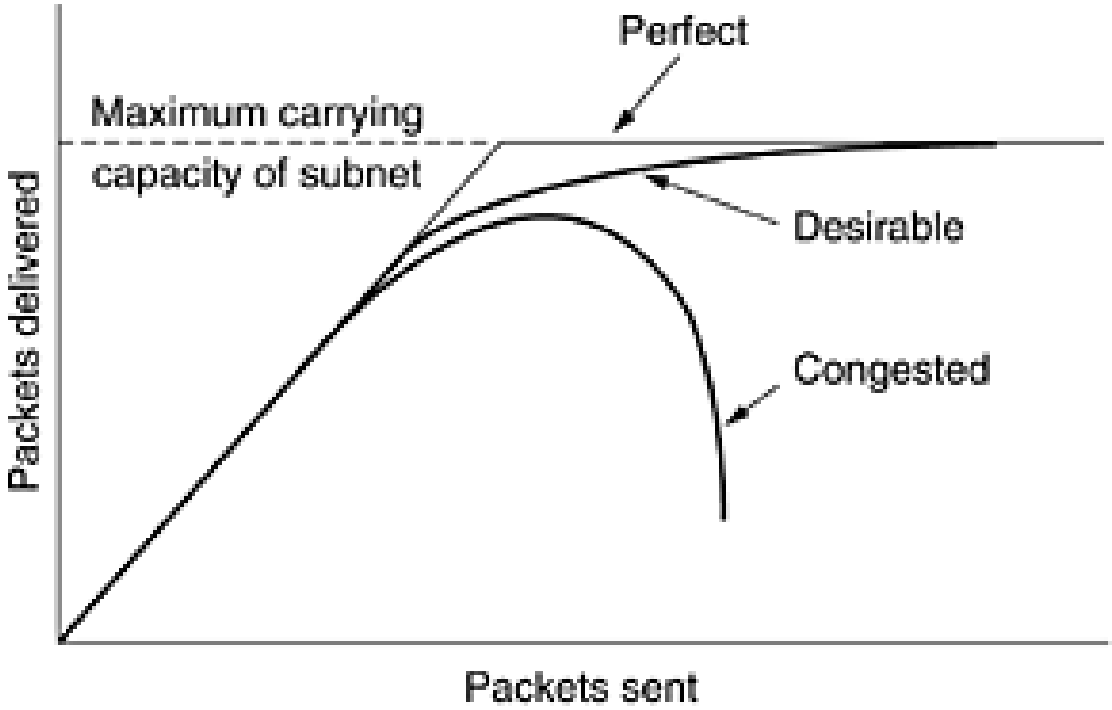
Rutare interna:

- RIP (Routing Information Protocol)
- IGRP (Interior Gateway Routing Protocol)
- EIGRP (Enhanced IGRP)
- OSPF (Open Shortest Path First)
- IS – IS (Intermediate System to Intermediate System) pentru ISO/OSI

- Rutare externa
  - BGP (Border Gateway Protocol)
  - EGP (Exterior Gateway protocol)

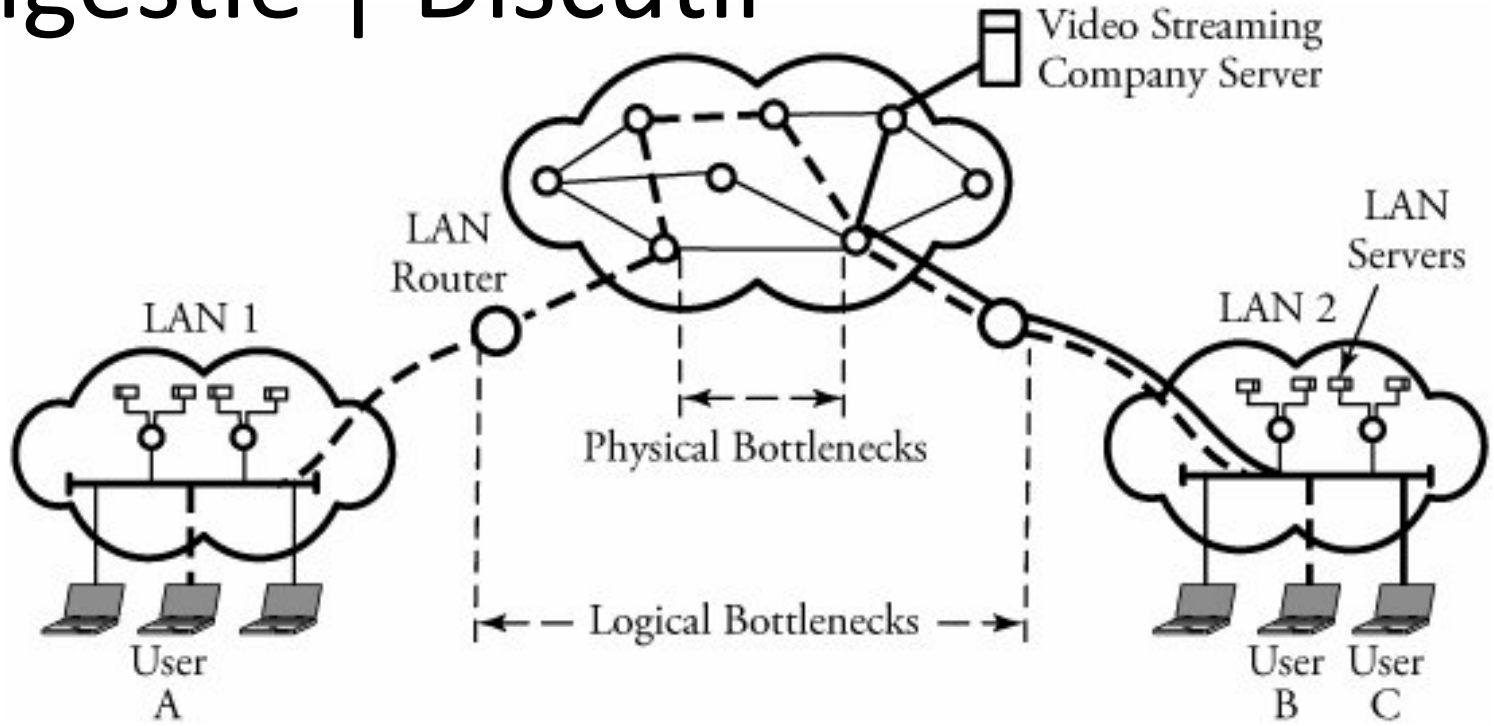
Congestie | Discutii

- Apare atunci cand se realizeaza supraincarcarea resurselor unei retea



In cazul unui trafic foarte mare, se poate instala congestia si performantele scad brusc

Congestie | Discutii



Congestia poate aparea:

- La nivelul legaturii de date: cand latimea de banda nu este suficienta
- La nivelul retea: cand coada de pachete de la noduri nu poate fi controlata
- La nivelul transport: cand legatura logica dintre doua rutere aflate intr-o sesiune de comunicare nu mai poate fi controlata

Congestie| Discutii

- Controlul congestiei – solutii
  - **Open-loop**: rezolvarea inseamna de fapt prevenirea aparitiei congestiilor printr-un design si decizii potrivite
  - **Close-loop**
    - Monitorizarea sistemului pentru detectarea congestiilor

      Metrici: procentul de pachete eliminate datorita lipsei spatiului in *buffer*, intarzierea pachetelor, etc.
    - Trimiterea acestei informatii la nodurile care pot lua decizii
    - Ajustarea operatiilor pentru corectarea problemei

Congestie| Discutii

Obs.:

controlul congestiei **!=** controlul fluxului

- Controlul congestiei asigura faptul ca reteaua are capacitatea de a transporta traficul oferit; implica actiunile tuturor *hos*t-urilor si a routerelor
- Controlul fluxului se ocupa de comunicarea *point-to-point* dintre un emitator si un receptor si se asigura faptul ca un emitator nu transmite date mai repede decat poate receptorul sa le proceseze

Rezumat

- **Nivelul retea**
  - **Activitatea de rutare (dirijare)**
    - **Preliminarii**
    - **Caracterizare**
    - **Rutare**
    - **Protocoale de rutare**
      - **RIP & OSPF**
      - **BGP & EGP**
- **Congestie – discutii generale**

Bibliografie

Content Networking Fundamentals, Silvano Da Ros, Publisher: Cisco Press Pub Date: March 30, 2006 Print ISBN-10: 1-58705-240-7 Print ISBN-13: 978-1-58705-240-8 Pages: 576

Computer Networks, Andrew S. Tanenbaum, Publisher : Prentice Hall

Computer and Communication Networks, Nader F. Mir, Publisher: Prentice Hall Pub Date: November 02, 2006 Print ISBN-10: 0-13-174799-1 Print ISBN-13: 978-0-13-174799-9 Pages: 656

http://www.tuxick.net/linux/ip6routing.html

http://www.6diss.org/workshops/see-2/routing-external.pdf

http://www.ip6.com/us/book/Chap7.pdf

http://www.nanog.org/meetings/nanog44/presentations/Monday/SmithBonica_IPv6_N44.pdf

Intrebari?

Retele wireless

Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- Preliminarii
- Componente
- Tipuri de retele *wireless*. Caracteristici.
- IP Mobil
- Aplicatii
- Wireless Sensor Networks (WSN)

Preliminarii

- Semnalele wireless sunt transportate prin aer de undele electromagnetice
- Spectrul de frecvenţe radio este un continuum al undelor electromagnetice folosite pentru date şi comunicaţii de voce
- Reţelele care transmit semnale prin atmosfera prin frecvenţă radio (RF) sunt cunoscute sub numele de reţele wireless sau reţele WLAN (wireless LAN)

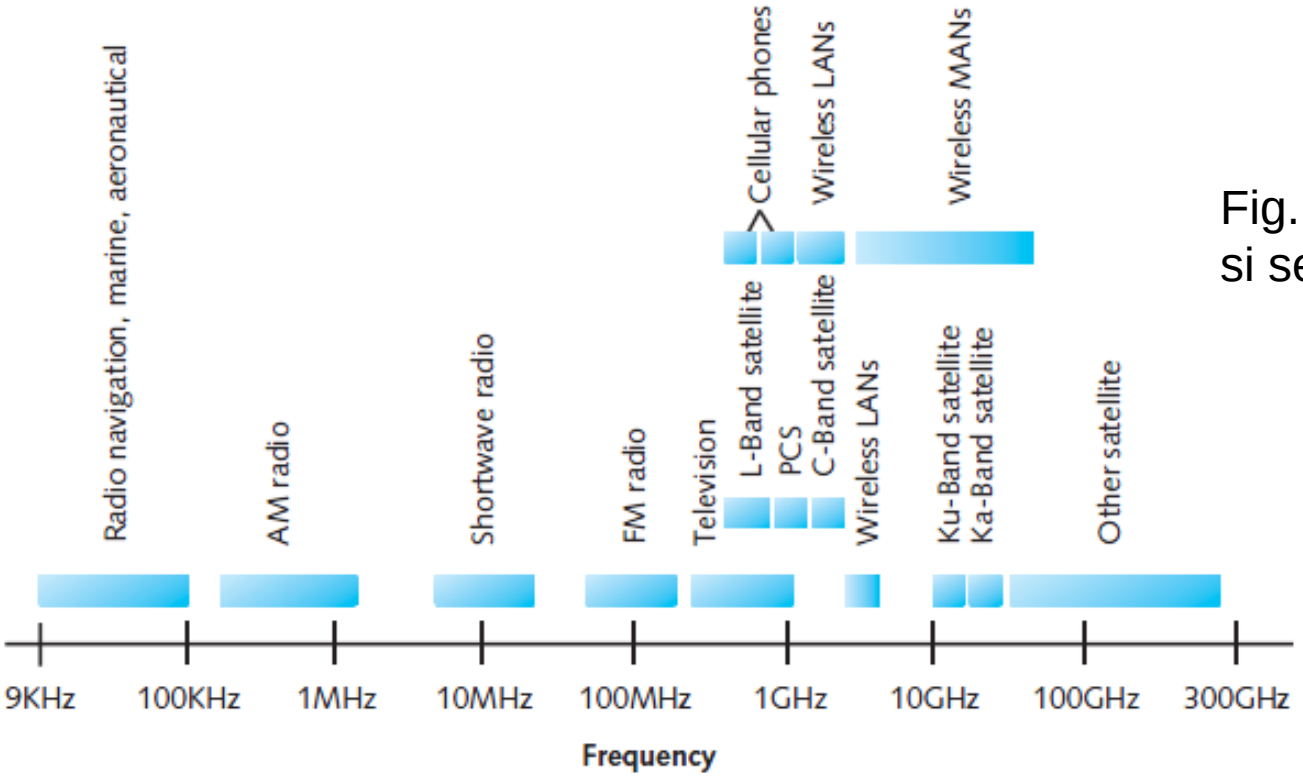


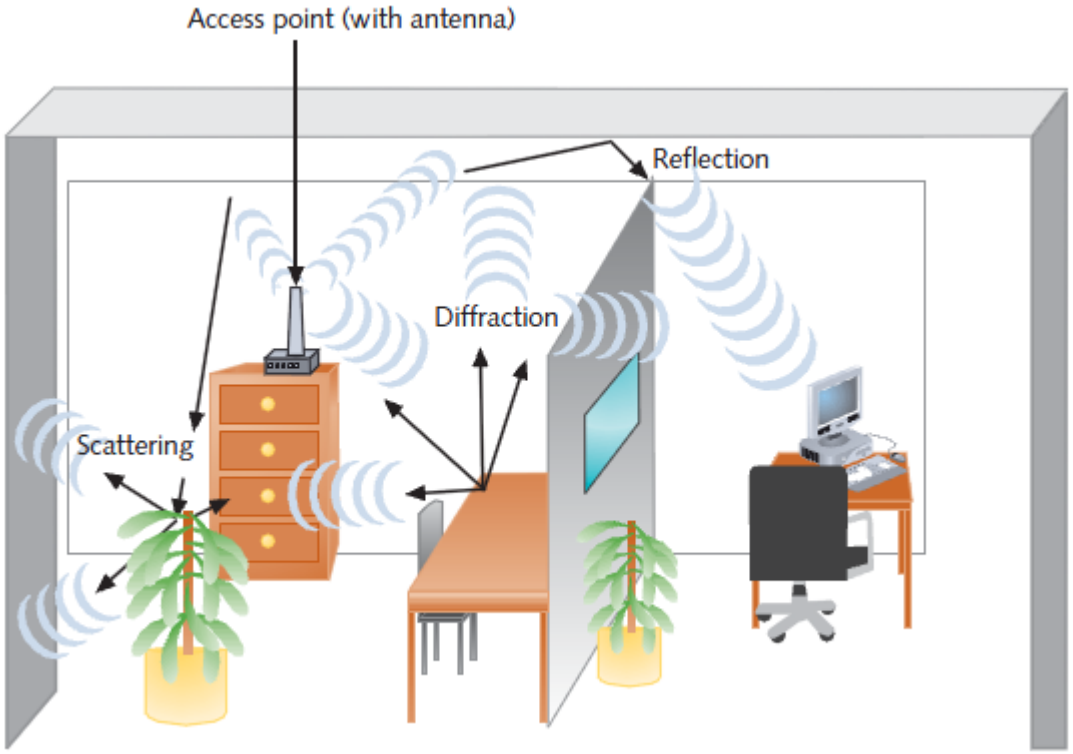
Fig. Spectrul *wireless* si serviciile Wireless

[Network+ Guide to Networks, Tamara Dean]

Preliminarii

Propagarea semnalului:

- ideal: in line dreapta de la emitator la destinatar - LOS (*line-of-sight*)
- *reflection*
- *diffraction* – cauzata de obstacole ca obiecte cu margini ascutite (e.g. colturile birourilor, peretilor)
- *scattering* – cauzata de intalnirea cu un obiect care are dimensiuni mici în comparaţie cu lungimea de undă a semnalului sau poate fi determinata de rugozitatea unei suprafete; pentru semnalele care sunt transmise in aer liber, ploaia, ceata, ninsoarea pot provoca acest efect



[Network+ Guide to Networks, Tamara Dean]

Preliminarii

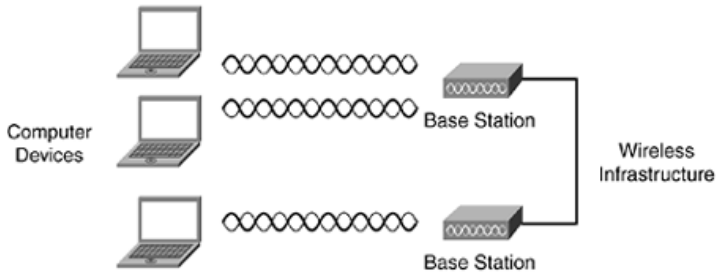
- *Fixed* versus *Mobile*
  - Sistemele wireless *fixed* – locatia emitatorului si receptorului nu se schimba (legatura *point-to-point*)
    - *Toata energia este folosita pentru transmiterea semnalului, si nu pentru transmiterea intr-o arie geografica mare*
  - Sisteme wireless *mobile* – receptorul poate fi localizat oriunde in aria emitatorului

Preliminarii

- Probleme:
  - Acoperirea si penetrarea
  - Latimea de banda
  - Latenta
  - Fiabilitatea transmisiei
  - Standardizarea
- Provocari:
  - Descoperirea locatiei
  - Detectarea mutarii si (re)stabilirea caii de comunicare

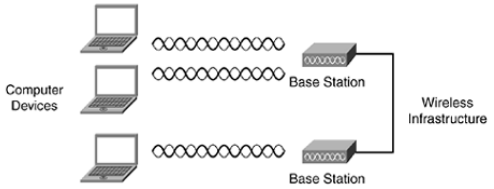
Preliminarii

- In retelele *wireless* comunicarea are loc intre dispozitive (*computer devices*)*: smartphone*-uri/PDAs (*Personal Digital Assistants*), laptopuri, tablete, servere, imprimante, senzori, …
  - Caracteristici ale acestor *dispozitive*
    - Procesor, memorie
    - Un mijloc de interfaţare cu un anumit tip de reţea
  - Obs.: In trecut, telefoanele traditionale nu intrau in aceasta categorie, dar majoritatea telefoanelor existente in acest moment incorporeaza caracteristicile de mai sus



[Wireless Networks first-step, Jim Geier]

Retele *wireless*|Componente



- **Dispozitive**

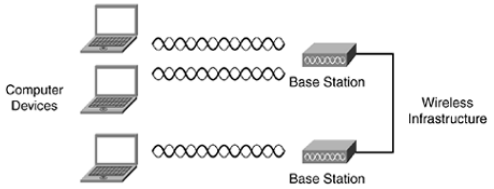
Aspecte de interes

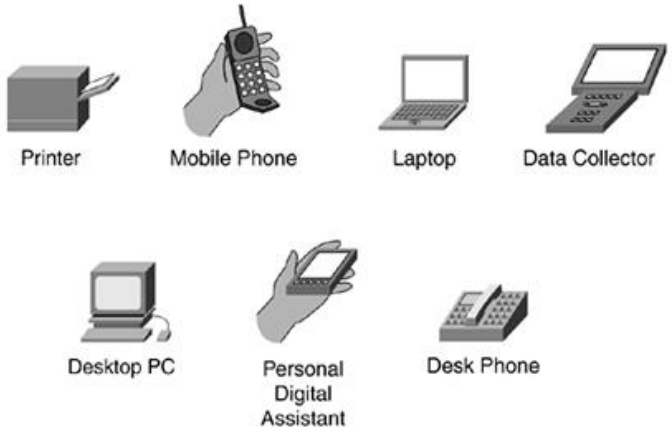
- Marime & Greutate
- Memorie disponibila

pentru aplicatii & date

- Viteza procesorului
- Caracteristicile ecranului (rezolutie, adincime de culoare, etc)
- Mecanismele de intrare (achizitie de date)
- Suport pentru mobilitate din partea sistemului de operare

Retele *wireless*|Componente





- **Dispozitive**

Aspecte de interes

- Slot-uri pentru extinderea

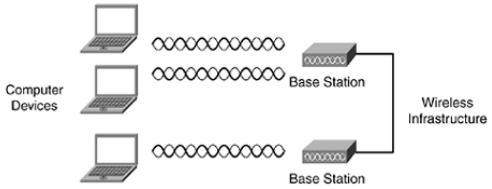
ulterioara (memorie suplimentara, conectivitatea cu alte dispozitive)

– Timpul de viata al bateriei

– Caracteristici integrate: camera digitala, tastatura, porturi cu infrarosu, Bluetooth,…

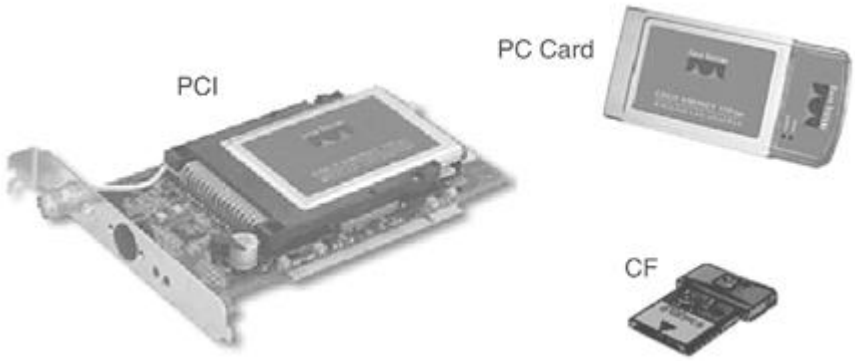
– Suport software: aplicatii, instrumente de dezvoltare, navigatoare mobile, drivere pentru hardware etc.

Retele *wireless*|Componente



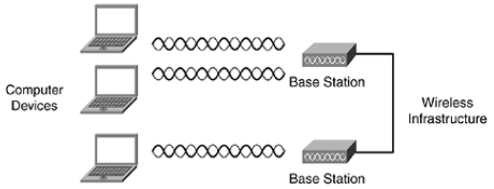
- **NIC (Network Interface Card)**

- Asigura interfata dintre dispozitive si infrastructura retelei wireless



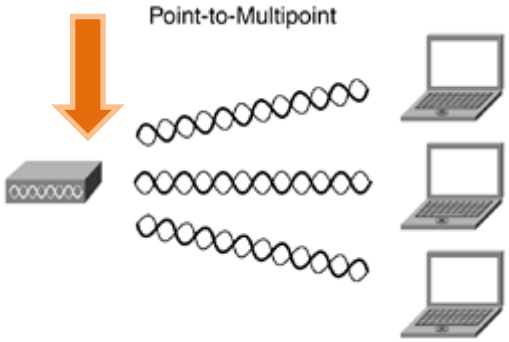
- Implementeaza un anumit standard (e.g. IEEE 802.11ad) => va permite interfatarea cu o retea wireless compatibila

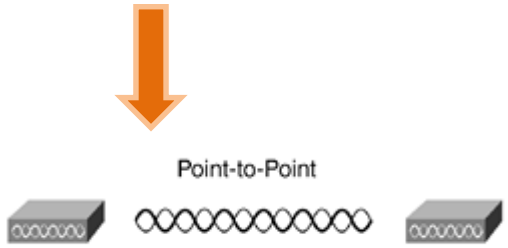
Retele *wireless*|Componente



In infrastructura unei retele wireless urmatoarele componente asigura comunicarea sau furnizeaza o serie de functionalitati:

- ***Base stations***
  - Componente care interfateaza comunicatiile *wireless* cu cele *wired*
  - Exemplu: puncte de acces (access point) sau routere
  - Poate suporta comunicatii *point-to-point* sau *point-to-multipoint*





Retele *wireless*|Componente

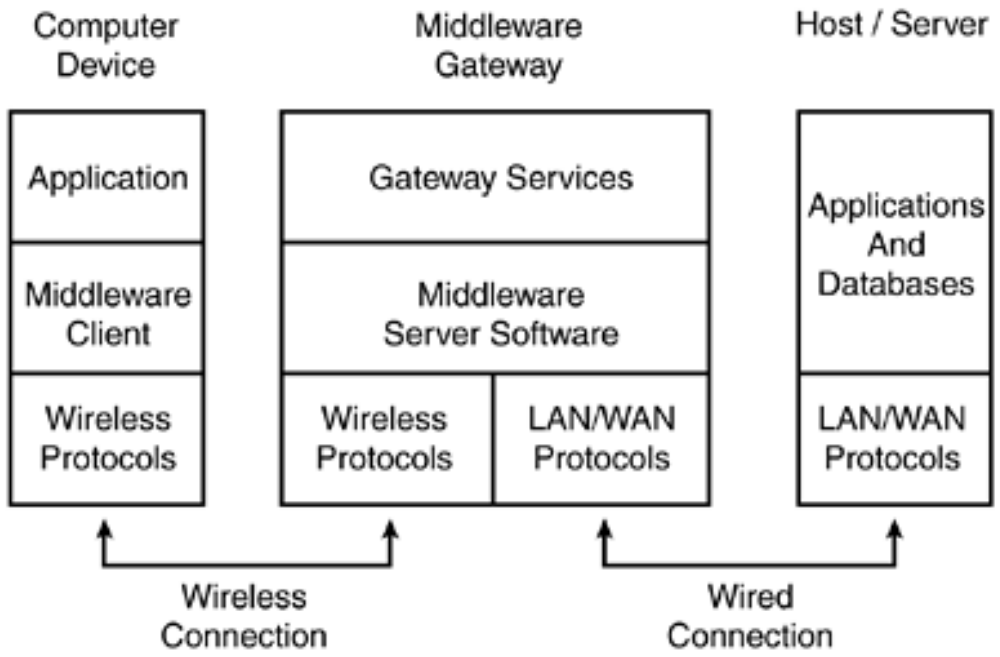
- ***Access controllers***
  - Este de obicei o componenta hardware care se afla intre punctul de access si partea de retea protejata
  - Functionalitati: autentificarea si autorizarea utilizatorilor, criptare, managementul latimii de banda
  - Exista si puncte de access care integreaza aceste functionalitati

Retele *wireless*|Componente

- **Soft de conectare**
  - Are ca scop asigurarea unei comunicatii eficiente si sigure in reteaua *wireless*

Exemplu: *wireless middleware software*

- optimizare
- repornire inteligenta
- (re)afisare eficienta
- etc.



Retele *wireless*

- Categorii – in functie de dimensiunea zonei pe care o acopera
  - **WPAN** *(Wireless Personal-Area Network)*
  - **WLAN** *(Wireless Local-Area Network)*
  - **WMAN** *(Wireless Metropolitan-Area Network)*
  - **WWAN** *(Wireless Wide-Area Network)*

Retele *wireless*|WPAN

WPAN *(Wireless Personal-Area Network)*

- Spatiu de operare ~ 10m
- Inlocuieste cablurile de interconectare la alte echipamente
- Performante: moderate
- Standarde:

  IrDa ; Bluetooth; (IEEE) 802.15, 802.11ah/Wi-Fi HaLow (IoT)

Retele *wireless*|WPAN

Standarde de conectivitate pentru **WPAN**

- **IrDa** (*Infrared Data Association*): comunicatie punct-la-punct bidirectionala via porturi cu infrarosu
  - Poate fi folosita intr-o arie limitata fara obstacole (*line-of-sight*)
  - Nu este afectata de interferente *RF (radio frequency*)

  Exemplu: sincronizare *smartphone* - PC

Retele *wireless*|WPAN

Standarde de conectivitate pentru **WPAN**

- **Bluetooth**
  - Introdus in 1998, asigura interconectivitatea intre dispozitive, folosind frecventa 2.4 GHz
  - Un dispozitiv poate fi setat ca putind fi descoperit in mod general, limitat sau deloc (descoperirea e automata)
  - 8 dispozitive formeaza un *piconet* (1 *master* si 7 dispozitive *slave*). Reteaua *ad-hoc* formata are suprafata de maxim 10m.
    - Piconet-urile se stabilesc dinamic in functie de modul cum dispozitivele intra sau ies dintr-o anumita zona

Retele *wireless*|WPAN

Standarde de conectivitate pentru **WPAN**

- **Bluetooth**
  - *Masterul piconetului* este un dispozitiv a carui caracteristici (ceas, adresa) definesc caracteristicile canalului fizic al *piconetului*
  - La un moment dat, data poate fi transferata intre master si un slave; dispozitivul master foloseste un mecanism de tip *round-robin* pentru comunicarea cu fiecare dispozitiv slave
  - Orice dispozitiv poate trece din starea slave in master si invers – vezi www.bluetooth.com
  - Mai multe *piconet*-uri = *scatternet*

Retele *wireless*|WPAN

Standarde de conectivitate pentru **WPAN**

IEEE 802.15: se bazeaza pe modelul Bluetooth, pentru a oferi standarde de comunicare wireless

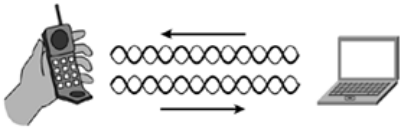
www.ieee802.org/15

- Ofera si interoperabilitate cu dispozitive suportind 802.11
- 802.15.1 – latime de banda: 1 Mbps
- 802.15.3 - latime de banda: 20 Mbps

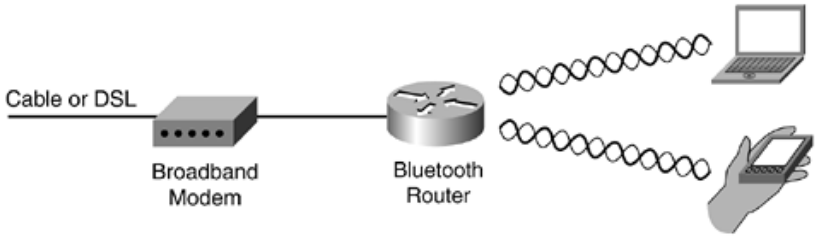
UWB (Ultra Wide Band) – tehnologie radio pentru comunicatii pe arii limitate, utilizind frecvente >5Ghz

Retele *wireless*|WPAN

Utilizari ale sistemelor wireless PAN:



- **Sincronizare**
  - Ex. Intre telefoane mobile cu un laptop, tableta sau PC
- **Streaming multimedia**
- **Control**
  - Ex: mouse wireless, tastatura wireless
- **Acces mai usor la dispozitive**
  - Ex: conexiunea wireless dintre un PC si o imprimanta
- **Conexiune Internet**



- **Enterprise**
  - Folosesc WPAN pentru sincronizari sau acces la dispozitive
  - Pentru conectivitatea la Internet se utilizeaza WLAN

Retele *wireless*|WPAN

Utilizari ale sistemelor wireless PAN

- **Bluetooth**

'Smart Hip' uses *Bluetooth* technology to monitor joint implants

When it comes to replacing wires with *Bluetooth* wireless technology, we can't think of any better place to do this than with medical devices surgically implanted in your body. That's why the new "Smart Hip" sounds so intriguing.

The Smart Hip monitors the performance of hip implants in real time, sending the information wirelessly from inside your body to a nearby *Bluetooth* enabled computer. The Smart Hip is actually a network of capsules, measuring sensors and actuators placed on the hip implant. Once activated by the doctor, these sensors send information to help prevent eventual problems after surgery. This is important because of the 1 million hip joint surgeries annually in Europe and the United States, an estimated that 5% to 10% of them eventually generate further health problems, which usually requires additional surgery.

The Smart Hip is being developed in Portugal by researchers at the University of Porto and other organizations.

http://www.bluetooth.com/English/Experience/Pages/cool-clever.aspx

Retele *wireless*|WPAN

Standarde de conectivitate pentru **WPAN**

- **Bluetooth**

GADGETS Bluetooth , CES ,

Bluetooth powers gizmos for health, wristwatches, and more

Yardena Arar @dennyarar Jan 8, 2013 12:52 PM

Bluetooth for all

LAS VEGAS— Forget mice and headsets: The latest Bluetooth gadgets can help monitor your health, keep track of personal belongings, and empower a wristwatch to show text messages and the identity of smartphone callers.

Most of the devices here at CES and on display at an event held for Bluetooth device makers use the low-voltage wireless technology to communicate with mobile apps to either record or deliver information and create peer-to-peer wireless connections.

At the Bluetooth SIG's CES event, vendors showed a slew of electronic nagging gadgets, including a toothbrush for monitoring dental hygiene and wristwatches that can deliver text messages and caller ID info from a paired cellphone.

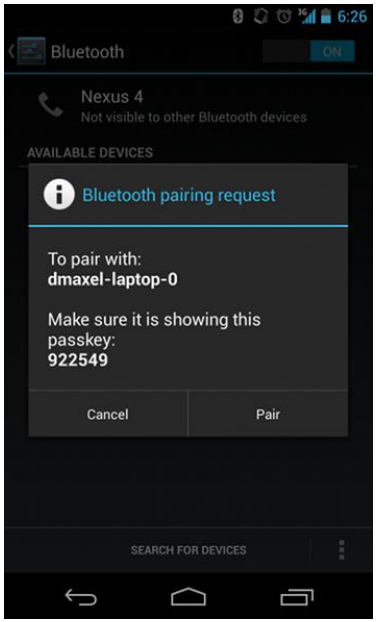
Tweet 114 Like 30 +1 3 Share

http://www.pcworld.com/article/2024238/bluetooth-powers-gizmos-for-health-wristwatches-and-more.html

Retele *wireless*|WPAN

Standarde de conectivitate pentru **WPAN**

- **Bluetooth**



http://www.makeuseof.com/tag/top-uses-for-bluetooth-on-your-android-phone/

Retele *wireless*|WLAN

WLAN *(Wireless Local-Area Network)*

- Spatiu de operare ~ 100m (cladiri, campusuri)
- Extensie/alternativa la LAN-urile conventionale
- Performante: ridicate
- Standarde pentru nivelul fizic & a legaturii de date :
  - 802.11b,a,g,n, WiGig (802.11ad)
    - (802.11k, r, y, n, w, p, z, v, u, s) => IEEE 802.11-2012
  - HiperLAN/2, 802.11ac

[Wireless Networks first-step, Jim Geier]

Retele *wireless*|WLAN

WLAN – Componente

- **Dispozitive utilizator**
  - PC-uri, laptopuri, PDA-uri, smarthpone-uri -> echipate corespunzator
- **Radio NIC** (*Network Interface Card*) sau *adaptere* sau *card radio*
  - Opereaza in cadrul dispozitivului si ofera conectivitatea *wireless*
  - Implementeaza si suporta unul sau mai multe versiuni al unui standard (e.g. 802.11a, 802.11b/g, ...)
- **Punct de acces** (*Access point*)
  - Contine un card radio care comunica cu un dispozitiv utilizator folosind o retea *wireless*

Retele *wireless*|WLAN

WLAN – Componente

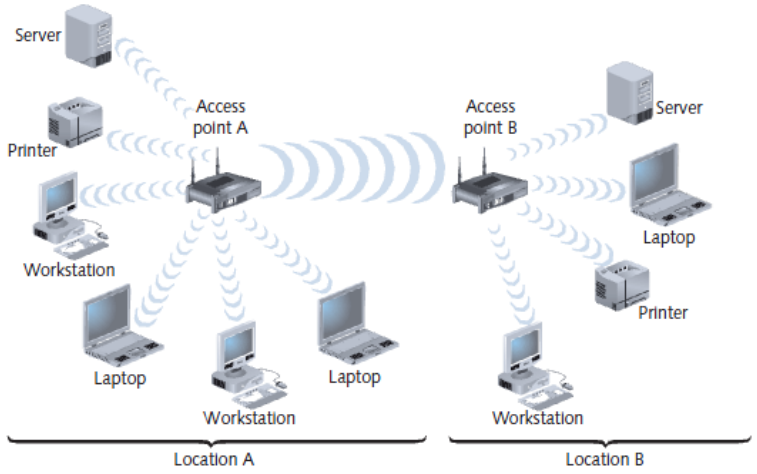
- Exemplu de **punct de access** ce ofera

posibilitatea de conectare a unui WLAN la un LAN :

- se asigura posibilitatea de configurare a

diferitelor functionalitati (eventual printr-o interfata http)

- ✓ SSID (Service Set Identifier)
- ✓ puterea de transmisie a punctului de acces
- ✓ un identificator pentru canalul de acces (de ex. 1, 6, 11)
- ✓ stabilirea unui nivel de securitate – activarea unui mecanism de criptare (de ex. WPA, ...)



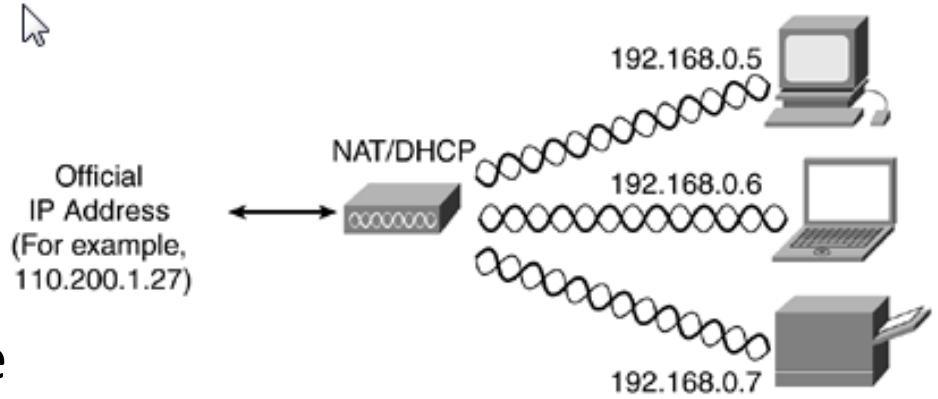
Interconectarea de retele LAN

[Network+ Guide to Networks, Tamara Dean]

Retele *wireless*|WLAN

WLAN – Componente

Routere



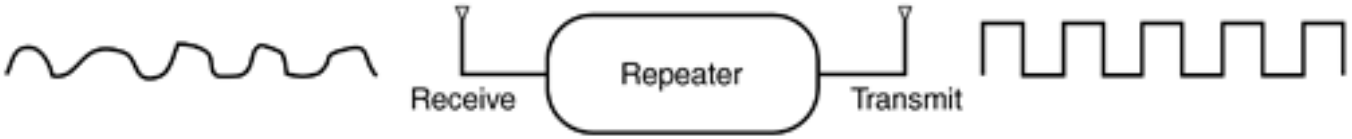
- Dispozitiv ce asigura rutarea corespunzatoare a pachetelor (folosind IP, tabele de rutare si alte protocoale interne ... vezi cursurile 2, 3, 12)
- Routerele wireless – adauga unui router Ethernet functionalitatea asigurata de un punct de acces (802.11)
- Routerele implementeaza NAT (Network Address Translation) si DHCP (Dynamic Host Configuration Protocol)

Retele *wireless*|WLAN

WLAN – Componente

Repetor (***Repeater***)

- Dispozitive care asigura regenerarea semnalului (=>largirea ariei retelei) fara adaugarea de noi puncte de acces



- Determina o reducere a performantei WLAN-urilor; repetorul primeste si retransmite fiecare *frame* pe acelasi canal radio, ceea ce duce la dublarea cantitatii de trafic in retea
- Obs. Repetoarele trebuie utilizate in mod optim pentru evitarea duplicarii datelor transmise

Retele *wireless*|WLAN

Sisteme WLAN

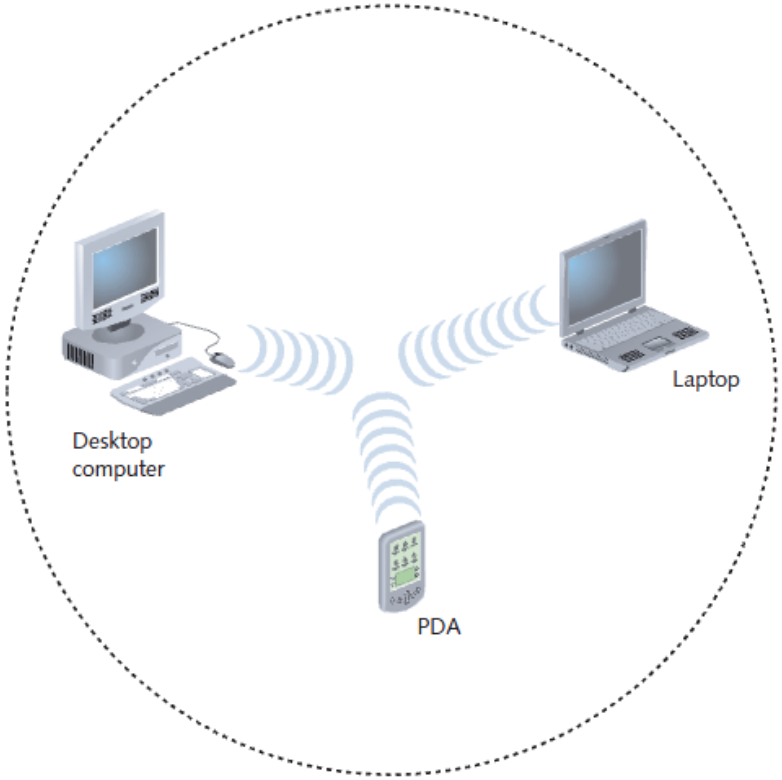


Fig. ***Ad-Hoc WLAN***

- Numar mic de noduri
- Transmisie directa intre dispozitive via wireless NIC
- Nu se utilizeaza puncte de acces
- Se mai numesc si retele *peer-to-peer*

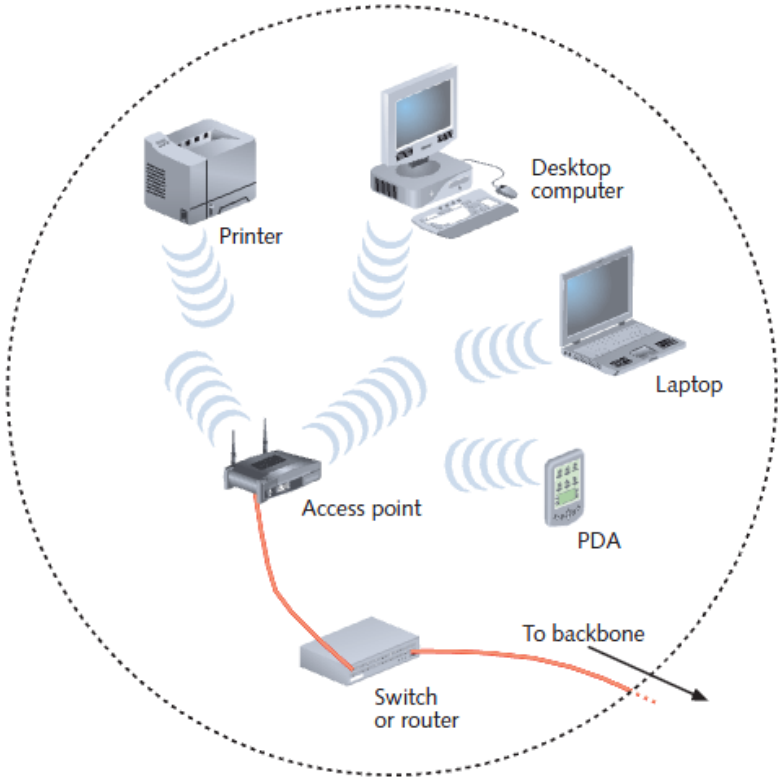


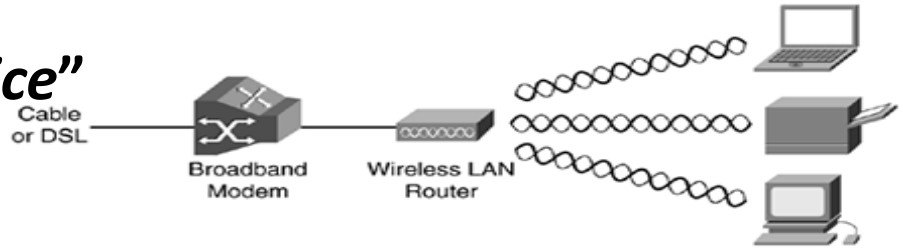
Fig. ***Infrastructure WLAN***

- Se folosesc AP-uri

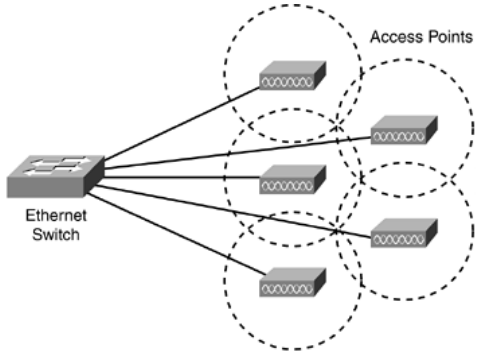
Retele *wireless*|WLAN

Sisteme WLAN

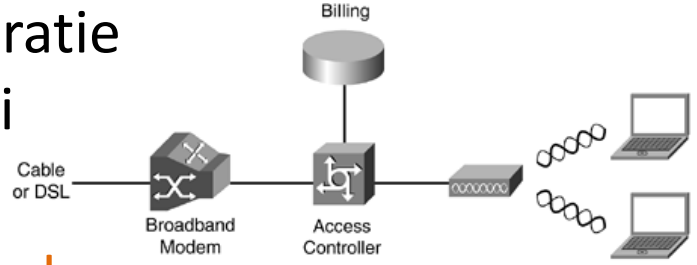
- **La nivel de *"Home&Small Office"***



- **La nivel de Enterprise**



- **WLAN-uri publice** – pot avea o configuratie simpla (e.g. un router wireless) sau mai complexa



http://compnetworking.about.com/od/wireless/tp/wifihotspotfind.htm

[Wireless Networks first-step, Jim Geier]

Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

- WI-FI Alliance – organizatie care a propus gruparea sub **WI-FI** a tuturor standardelor 802.11 existente si viitoare

  http://www.wi-fi.org

  Caracteristici: frecventa utilizata, aria acoperita, …

- *Use case*: aveti un laptop cu un wireless NIC care suporta unul/mai multe standarde 802.11 si mergeti in preajma unui *hot spot*

  –*Association* – actiune a subnivelului MAC ce implica schimb de pachete intre AP si dispozitiv, dupa care utilizatorul este notificat de existenta unei retele wireless

  • scanare activa – dispozitivul trimite un frame special (*probe*) pe toate canalele disponibile in aria sa de acoperire; un AP va putea raspunde cu un cod de stare si un ID pentru statia respectiva

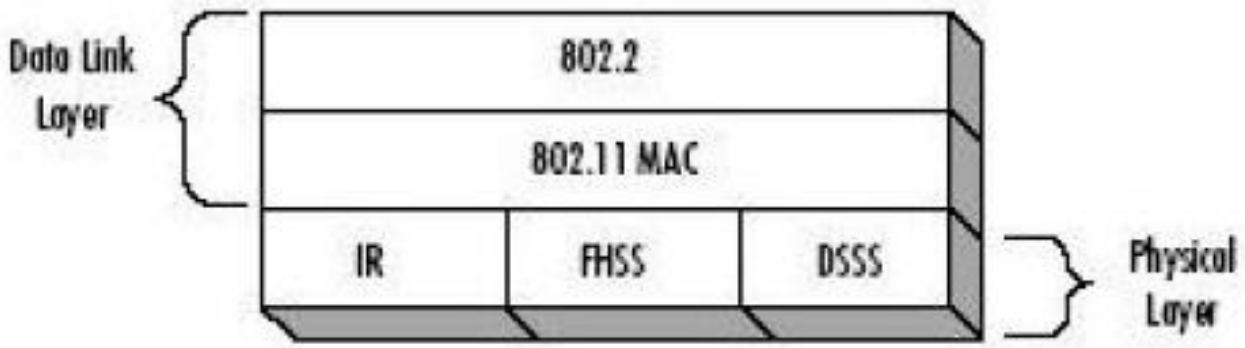
  • scanare pasiva – dispozitivul asculta toate canalele pentru identificarea unui *beacon frame* (care contine rata de transmisie, SSID – service set identifier) – (e.g. KisMAC - wireless network monitor disponibil pentru OS X)

  www.mhprofessional.com/downloads/products/0071666613/0071666613_chap01.pdf

Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

- Specificatiile 801.11 definesc modul de functionare al substratului MAC si protocoale la nivelul fizic (e.g. IR – InfraRed , DSSS - Direct Sequence Spread Spectrum, FHSS - Frequency Hopping Spread Spectrum)



Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

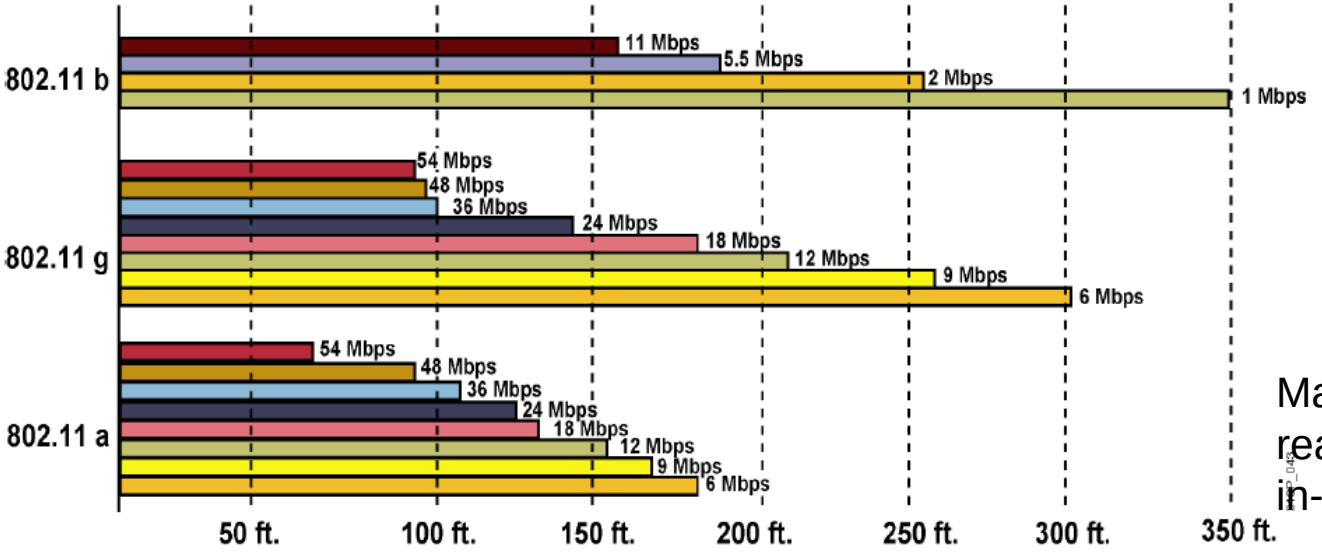
- **IEEE 802.11b** (din 1999)
  - Foloseste frecventa de 2.4 Ghz
  - Latimea de banda (*throughput*) poate fi de maxim 11 Mbps (efectiv: 5Mbps; DSSS si FHSS)
  - Aria de comunicare este ~100m. (viteza de transfer depinde de departarea dispozitivului) e.g. pentru o distanta de 65m intre doua dispozitive, transferul se efectueaza cu maxim 2 Mbps

- **IEEE 802.11a**
  - Foloseste frecventa de 5 Ghz (mai putine coliziuni)
  - Ofera pina la 54 Mbps (efectiv: 11-18 Mbps)
  - Necesita putere mai mare de transmisie si o arie de actiune mai mica fata de 802.11b (sunt necesare eventual mai multe puncte de access)
  - Nu este compatibil cu 802.11b/g

Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

- **IEEE 802.11g**
  - Standard din 2003, combina facilitatile de la precedentele
  - Este compatibil cu 802.11b
  - Foloseste frecventa de 2.4 Ghz
  - Cresterea latimii de banda la 54Mbps (efectiv: 20 - 25Mbps)



Masuratori realizate in-door

Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

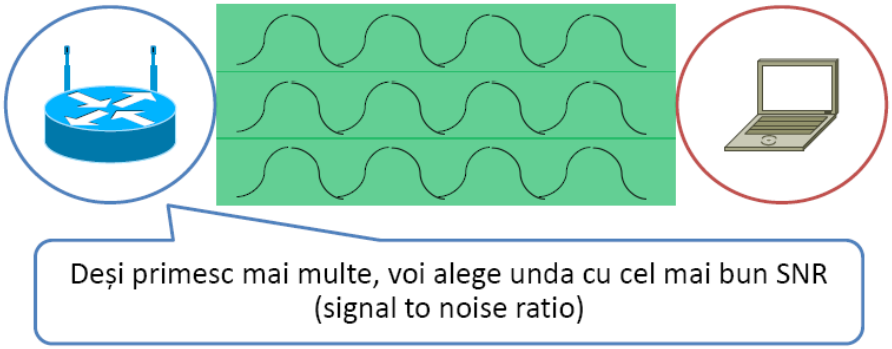
- **IEEE 802.11n-2009**
  - Foloseste frecventa de 2.4 Ghz sau 5Ghz
  - Imbunatateste semnificativ transferul de date de la (802.11a, 802.11g) - 54 Mbit/s pina la 600 Mbit/s => amenintare pentru FastEthernet si o platforma posibila pentru telefonie si semnale video
  - Compatibil cu standardele anterioare
  - Folosește multiple antene și tehnologia MIMO
  - Imunitate crescută la zgomot folosind modulări avansate a undei purtătoare
  - Suportă *packet aggregation* (un singur header pentru mai multe pachete de date)
  - http://standards.ieee.org/getieee802/download/802.11n-2009.pdf

Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

MIMO

- MIMO folosește procesoare avansate pentru multiplexare spațială a semnalului
- Procesorul poate multiplexa și demultiplexa datele pentru a obține throughput mai mare
- Efectul multipath = procesul prin care se primesc mai multe unde purtătoare a acelorași informații dar care s-au reflectat diferit și cu claritate diferită
- În 802.11g procesorul alegea cea mai clară undă



Retele *wireless*|WLAN

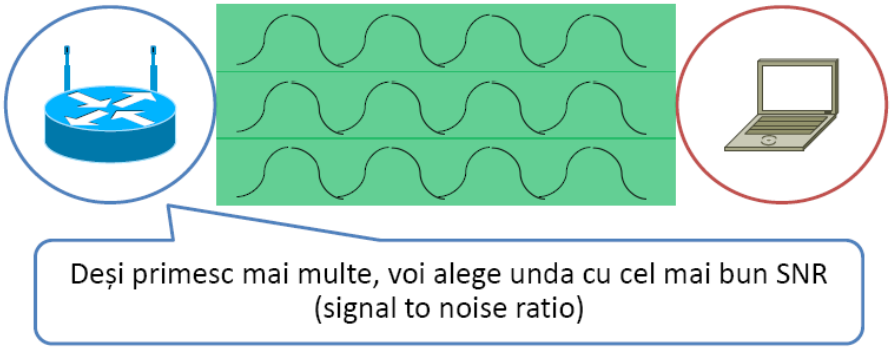
Standarde de conectivitate pentru **WLAN**

MIMO

•Problema: undele cu SNR mai slab sunt ignorate, chiar dacă este posibil ca acestea să conțină informații relevante

•În 802.11n, tehnologia MRC (Maxim Ratio Combining) implementată în procesorul plăcii de rețea compune toate undele pentru a obține claritate maximă a informației => throughput mai bun

•Exemplu: Dacă aveți o placă 802.11n într-o rețea 802.11g veți avea throughput mai bun decât clienții 802.11g



Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

- **IEEE 802.11ad - WiGig**
  - Standard anuntat din 2009
  - 2011 – versiunea 1.1
  - Foloseste benzile de 2.4, 5, 60 GHz
  - Rata de transfer de aprox. 7Gbit/s (de 10 ori mai rapid decat rata maxima 802.11n)
  - Mentine compatibilitatea cu standardele existente

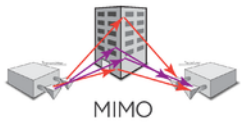
http://www.wigig.org/

Retele *wireless*|WLAN

Standarde de conectivitate pentru **WLAN**

- **IEEE 802.11ac**
  - Dezvoltat din 2008; Decembrie 2013 – standard
  - Foloseste banda de 5 GHz
  - **MIMO (multiple-input and multiple-output)** –> *Multi-user MIMO algorithms are developed to enhance MIMO systems when the number of users or connections is greater than one*
  - Throughput: multi-station – aprox. 1 gigabit per second ; single-link throughput – aprox. 500 Mbit/s

Standard Wi-Fi	802.11 b 802.11 a 802.11 g 802.11 n 802.11 ac
Tehnologie	MU-MIMO



Retele *wireless*|WLAN

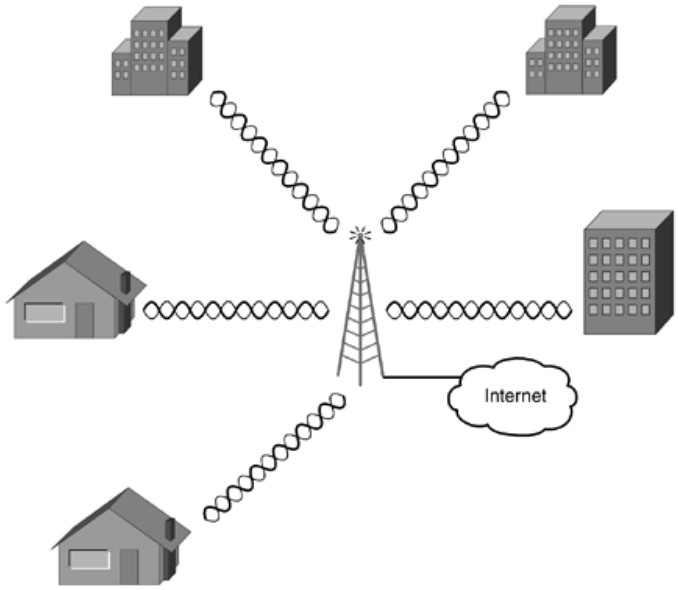
Standarde de conectivitate pentru **WLAN**

- **HomeRF (Radio Frequency)**
  - Din: 1998 -> 2003, initiat de Siemens, Motorola, Philips
  - Destinat comunicatiilor casnice
  - Suporta comunicatii de calitate prin voce
- **HiperLAN/2 (High Performance Radio LAN)**
  - Alternativa europeana la standardele IEEE 802.11 - European Telecommunications Standards Institute (ETSI)
  - Utilizeaza frecventa de 5Ghz
  - Transfer de maxim 54 Mbps, pe o raza de ~150m
  - Incorporeaza facilitati pentru asigurarea QoS (pentru transmisii multimedia in timp real)

Retele *wireless*|WMAN

WMAN *(Wireless Metropolitan-Area Network)*

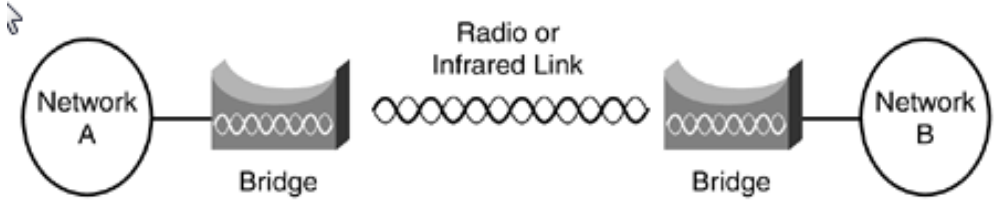
- Spatiu de operare: un oras
- In general, exista o dispozitie fixa a retelei
- Utile atunci cind metoda traditionala cu fir nu este fezabila
- Standarde:
  - 802.16
  - Se utilizeaza si 802.11



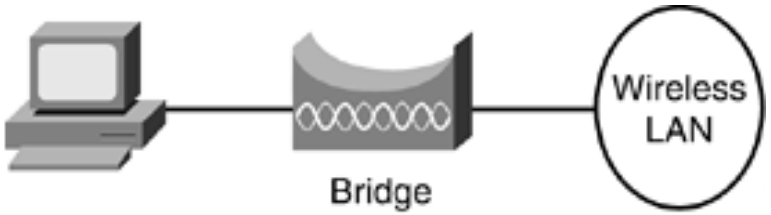
Retele *wireless*|WMAN

WMAN - componente

- ***Bridges:*** asigura conectivitatea a doua retele care utilizeaza protocoale similare sau diferite la nivelul legaturii de date



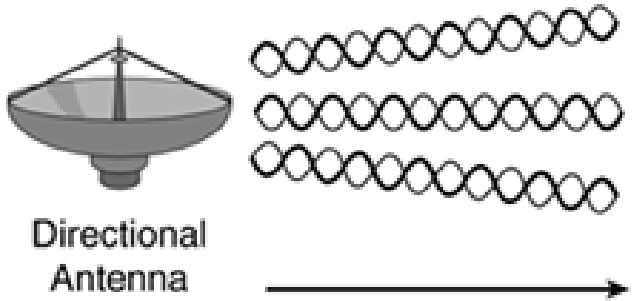
  - Difera de un punct de acces
  - Un *bridge Ethernet-wireless* poate fi utilizat pentru conectarea printr-un port Ethernet a unui dispozitiv la un punct de acces



Retele *wireless*|WMAN

WMAN - componente

- ***Antene:*** pentru retelele WMAN se folosesc in special antene directionale, pentru maximizarea intensitatii undelor radio intr-o directie



  - Se pot utiliza si alte tipuri de antene (e.g. antenele semidirectionale acopera o suprafata dubla fata de cele omnidirectionale)

Retele *wireless*|WMAN

Sisteme **WMAN –** asigura conectivitatea intre cladiri si utilizatori in cadrul unui oras folosind cateva configuratii

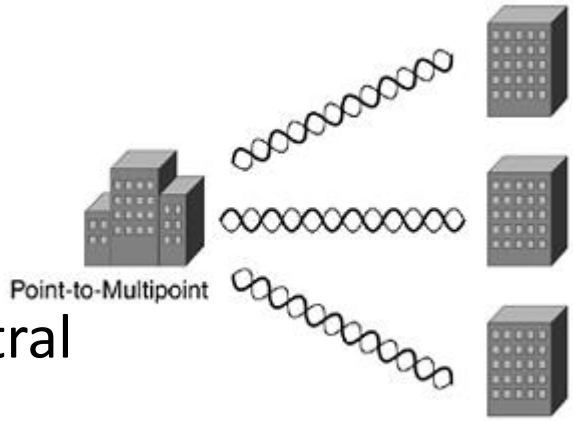
– *point-to-point*: utilizeaza RF sau transmisie prin infrarosu, folosind antene (semi)directionale care pot atinge arii de 30 mile pentru sistemele RF

Exemplu: un centru medical poate utiliza o comunicare *point-to-point* intre spitalul principal si o clinica din acelasi oras

Retele *wireless*|WMAN

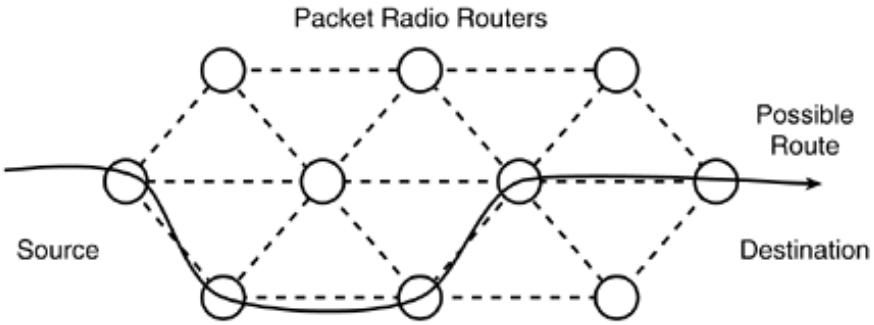
Sisteme **WMAN**

– *point-to-multipoint*: utilizeaza o antena omnidirectionala



Exemplu: o companie cu un sediu central si cu depozite|instalatii de fabricare in acelasi oras sau aceeasi zona rurala

– Sisteme *"packet radio"*: utilizeaza routere wireless speciale care trasmit pachetele intre ele



Retele *wireless*|WMAN

Standarde de conectivitate pentru **WMAN**

- **WI-FI/802.11**
  - Se folosesc antene directionale si *bridge*-uri *wireless*
  - Costuri scazute, dar performantele sunt limitate pentru un numar mare de utilizatori;
  - Interferentele RF apar daca exista mai multe retele 802.11
- **802.16** sau **WiMAX** (Worldwide Interoperability for Microwave Access)
  - Exemple:
    - 802.16-2009 Air Interface for Fixed and Mobile Broadband Wireless Access System (extinde 802.16-2004, 802.16-2004/Cor 1, 802.16e, 802.16f, 802.16g and P802.16i)

Retele *wireless*|WMAN

Standarde de conectivitate pentru **WMAN**

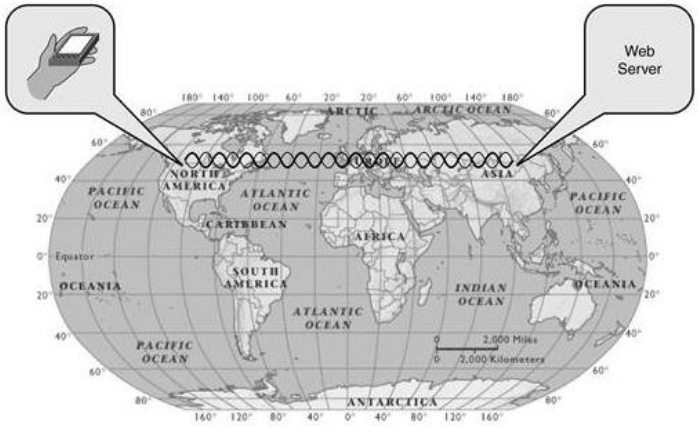
- **802.16** sau **WiMAX** (Worldwide Interoperability for Microwave Access)

802.16-2012	IEEE Standard for **Air Interface for Broadband Wireless Access Systems** It is a rollup of 802.16h, 802.16j and Std 802.16m (but excluding the WirelessMAN-Advanced radio interface, which was moved to IEEE Std 802.16.1). Released on 2012-August-17.	Current
802.16.1-2012	IEEE Standard for **WirelessMAN-Advanced Air Interface for Broadband Wireless Access Systems** Released on 2012-September-07.	Current
802.16p-2012	IEEE Standard for **Air Interface for Broadband Wireless Access Systems** **Amendment 1: Enhancements to Support Machine-to-Machine Applications** Released on 2012-October-08.	Current
802.16.1b-2012	IEEE Standard for **WirelessMAN-Advanced Air Interface for Broadband Wireless Access Systems** **Amendment 1: Enhancements to Support Machine-to-Machine Applications** Released on 2012-October-10.	Current
802.16n-2013	IEEE Standard for **Air Interface for Broadband Wireless Access Systems** **Amendment 2: Higher Reliability Networks** Approved on 2013-March-06.	Current
802.16.1a-2013	IEEE Standard for **WirelessMAN-Advanced Air Interface for Broadband Wireless Access Systems** **Amendment 2: Higher Reliability Networks** Approved on 2013-March-06.	Current

Retele *wireless*|WWAN

WWAN *(Wireless Wide-Area Network)*

- Spatiu de operare: global rezultat in urma cooperarii mai multor companii (AT&T, Verizon, Sprint, etc) de telecomunicatii
- Transferul de date se realizeaza prin comutare de pachete (*packet-switch*)
  - Contrast cu modul de comutare prin circuite virtuale
  - Nu necesita conexiuni dedicate
  - Permit mai multor utilizatori sa foloseasca o singura conexiune



[Wireless Networks first-step, Jim Geier]

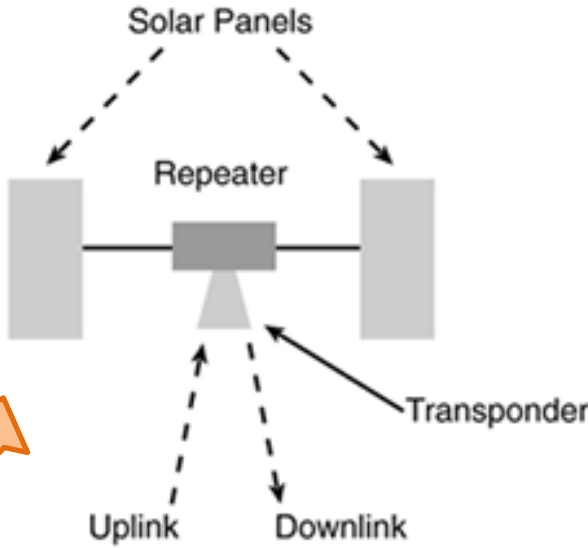
Retele *wireless*|WWAN

WWAN – componente

- **Radio NIC** – utile pentru a integra un laptop sau alte dispozitive intr-un WWAN; Unele telefoane mobile au integrate aceste dispozitive, insa apar probleme de integrare deoarece exista diferite tipuri de WWAN-uri

  (e.g. in functie de hardware-ul satelitului)

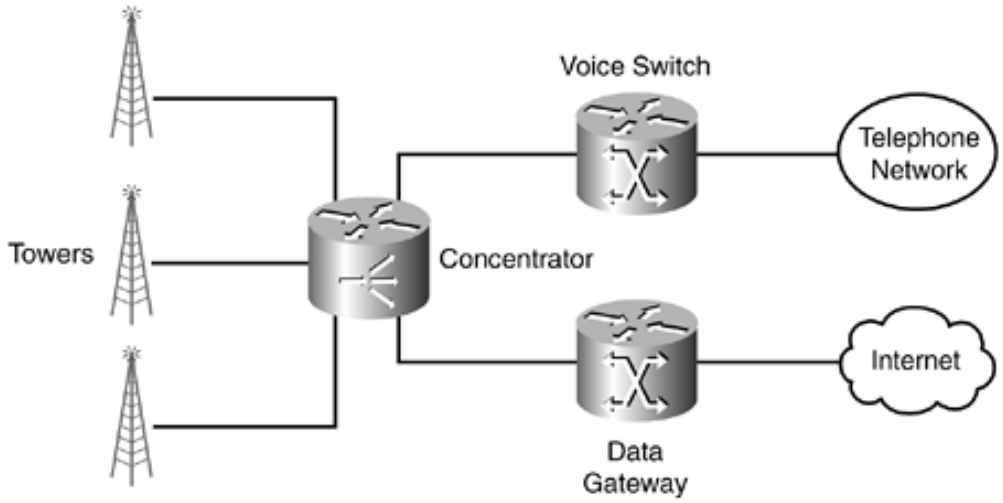
- **Base Stations**
  - ***Cell towers***
  - **Sateliti**
- **Antene**



Retele *wireless*|WWAN

Sisteme WWAN

WWAN bazate pe celule



- Format din:
  - *Cell towers* : recepteaza semnale de la utilizator si transmite informatie inapoi la utilizator
  - *Voice switches* – realizeaza conectarea dispozitivului utilizator la alt utilizator folosind sistemul de telefonie
  - *Gateway*-uri – transforma sistemul intr-un WWAN; face posibil accesul utilizatorilor la Internet

[Wireless Networks first-step, Jim Geier]

Retele *wireless*|WWAN

Sisteme WWAN

WWAN bazate pe celule

- Celula (cell): zona geografica avind o arie de acoperire a semnalului
  - Depinde de protocol, puterea semnalului, obstacole
  - Raza de actiune 1-40 km
  - Celulele sunt coordonate de un *cell system*
  - Ariile foarte populate contin *micro-celule* (diam. ~100m)
  - Conexiunea dintre utilizatori din celule diferite este pasata printr-un proces numit *handoff* sau *handover*
  - Plasarea elementelor de retea se realizeaza conform unor strategii de optimizare a acoperirii si maximizarii semnalului

Retele *wireless*|WWAN

WWAN

- **Multiplexarea:** semnalul este folosit (partajat) de mai multi utilizatori
  - **Frequency-division multiple access** (FDMA): fiecare semnal din cadrul canalului de comunicatie are o frecventa unica (modelul posturilor radio)
  - **Time-division multiple acess** (TDMA): se asigneaza fiecarui utilizator segmente de timp in care poate comunica
  - **Code-division multiple access** (CDMA): fiecare semnal are atasat un cod, toate semnalele fiind transmise pentru a “umple” intreaga latime de banda; receptorul va procesa doar semnalele avind codul “corect”

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 2 (2G)

- Imbunatatire a generatiei 1 (’70-’80)
- In afara transmisiei digitale de voce suporta transmisii de date (9.6 – 19.2 Kbps)
- Code Division Multiple Access (CDMA IS-95)
- **Global System for Mobile Communication** (**GSM**)
- Facilitati: mesaje scurte (SMS), aplicatii de tip calendar, managementul informatiilor personale (PIM), tonuri, jocuri, acces via WAP...

WAP

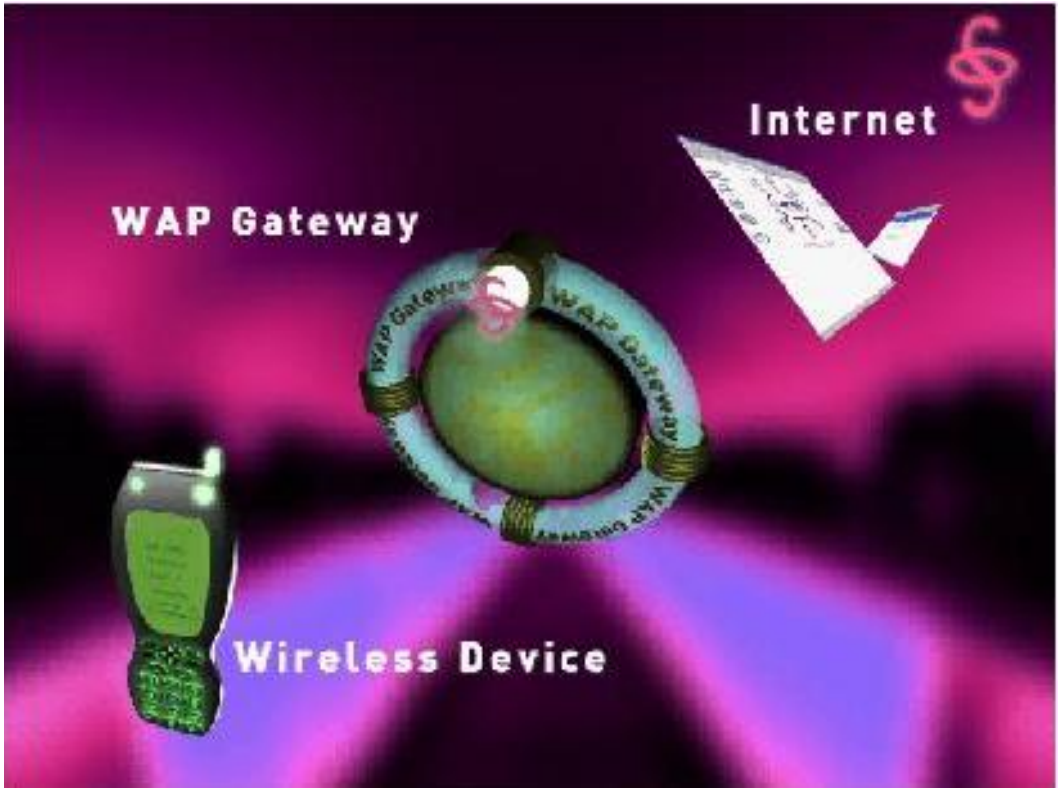
Protocolul WAP (Wireless Application Protocol)

- Standard permitind accesarea informatiilor si serviciilor oferite de Internet via un dispozitiv mobil
  - sub coordonarea consortiului Open Mobile Alliance (fost WAP Forum)

  www.wapforum.org
- Versiuni: WAP1.0, WAP 1.1, WAP 1.2, WAP 2.0

WAP

Protocolul WAP (Wireless Application Protocol)



-Poarta WAP: mediator intre Internet si dispozitiv folosind WAP

Pentru WAP 2 nu este neaparat necesara, folosindu-se direct HTTP/1.1

[International Engineering Consortium, http://www.iec.org]

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 2.5 (2.5G):

- In general - rata de transfer de ~115kbps
- **General Packet Radio Services** (**GPRS**) – max. 171.2 kbps -> **Enhanced Data for Global Evolution** (**EDGE**) – max. 474 kbps
- Code Division Multiple Access CDMA 2000 1x
- Retelele GPRS se comporta similar cu cele LAN clasice, aplicatiile fiind mai usor de implementat
- Un dispozitiv GPRS <-> terminal

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 3 (3G):

- Familia de standarde 3GPP (Third Generation Partenership Project) definita de International Telecommunication Union (www.3gpp.org, www.3gpp2.org)
- Se bazeaza pe Universal Mobile Telecomunications System (UMTS) (Wideband CDMA (W-CDMA), TD-CDMA (Time Division Code Division Multiple Access), …);
- Rate de transfer
  - 128 kbits/s pentru dispozitive in miscare rapida
  - 384 Kbits/s pentru dispozitivele in miscare lenta
  - 2.05Mbits/s pentru dispozitivele care stationeaza

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 3 (3G):

- Caracteristici ale serviciilor 3G:
  - Mobilitate sporita si conectivitate permanenta (retelele 3G folosesc IP)
  - Asigurarea de servicii multi-media (streaming audio si video)
  - Download rapid de fisiere de dimensiuni mari
  - Accesul la aplicatii de corporatie
  - asigurare QoS
- Discutii: 3G alternativa la WLAN
  - WLAN – ofera o rata de transfer mai mare
  - Dezvoltarea unui WLAN este mai putin costisitoare

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 4 (4G):

- Rate de transfer
  - 100 Mbit/s pentru dispozitive in miscare rapida
  - 1 Gbit/s pentru dispozitivele care stationeaza sau sunt in miscare lenta
- **Pre-4G** (**Extensii 3G**): 3GPP
  - 3 GPP Standardul LTE (Long Term Evolution)
    - Lansat in decembrie 2009, Telia Sonera – primele servicii "4G" in Suedia si Norvegia (terminale utilizator – Samsung)
  - IEEE 802.16m sau WirelessMAN-Advanced sau WiMAX
  - UMB (Ultra Mobile Broadband) sau 3GPP2
- 6 Decembrie2010 ITU (International Communication Union) -> versiunile curente de LTE, UMB, … nu respecta standardul 4G

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 4 (4G):

- Rate de transfer
  - 100 Mbit/s pentru dispozitive in miscare rapida
  - 1 Gbit/s pentru dispozitivele care stationeaza sau in miscare lenta

Exemple:

•USA: 5 servicii publice LTE: MetroPCS, Verizon Wireless, AT&T, US Cellular, Sprint Nextel

•Coreea de Sud, Mobile WiMAX - 2010

•Ungaria: T-Mobile Hungary ofera servicii 4G LTE din 1 Ianuarie 2012

•Romania: Orange (http://www.orange.ro/4g/index.html?gclid=CIjH6uSK7LQCFUId3godkWUA0w) , Vodafone (noiembrie 2012), Cosmote, RCS-RDS şi 2k Telecom au achiziţionat licenţe pentru lărgirea spectrului de frecvenţă

Dispozitive: Apple iPhone 5S, HTC One (M8), Samsung Galaxy S5,….

http://www.techradar.com/news/phone-and-communications/mobile-phones/best-4g-phones-2013-10-to-choose-from-1161825#null

Retele *wireless*|WWAN

Sisteme WWAN

WWAN de generatie 5 (5G) …..

- *"On January 22, 2016, the Swedish mobile network equipment maker Ericsson said it had partnered with TeliaSonera to develop 5G services based on TeliaSonera's network and Ericsson's 5G technology. The partnership aims to provide 5G services to TeliaSonera customers in Stockholm, Sweden and Tallinn, Estonia in 2018. Sweden has long been a pioneer ICT nation and notably Ericsson and TeliaSonera launched the world's first commercial 4G network in Sweden in 2009"*
- On January 29, 2016, Google revealed that they are developing a 5G network called SkyBender. They planned to distribute this connection through sun-powered drones
- *Also on February 22, 2016, Samsung and Verizon joined to begin trial for 5G.*
- *"In September 2016, China's Ministry of Industry and Information Technology announced that the government-led 5G Phase-1 tests of key wireless technologies for future 5G networks were completed with satisfactory results. The tests were carried out in 100 cities and involved seven companies – Datang Telecom, Ericsson, Huawei, Intel, Nokia Shanghai Bell, Samsung and ZTE. The next step in 5G technology development involving tech trials is under way, with planned commercial deployment in 2022 or 2023."*
- *On October 17, 2016, Qualcomm announced the first 5G modem, the Snapdragon X50, as the first commercial 5G mobile chipset*

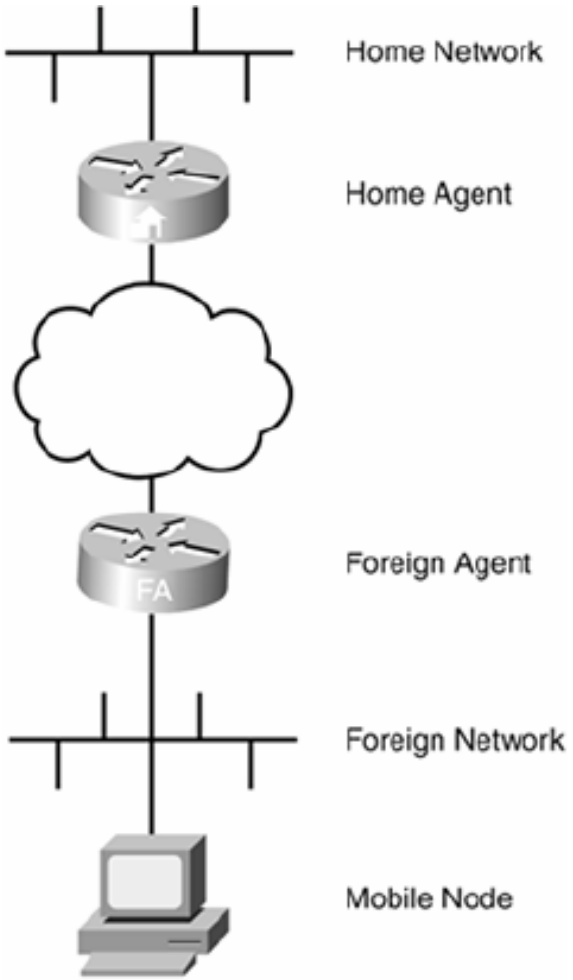
5G Overview: Key Technologies – Meisam Khalil Arjmandi

Wireless Internet

- **Mobile IP**
  - Protocol care permite unui dispozitiv mobil deplasarea dintr-o retea in alta si mentinerea adresei IP
  - Mobile IPv4 – RFC 3344, RFC 4721
  - Mobile IPv6 – RFC 3775, RFC 6275 (Proxy Mobile IPv6 – PMIPv6)
  - Protocol de rutare in care dispozitivele terminale (*end devices*) isi semnalizeaza propriile actualizari de rute si tunelele dinamice de date elimina necesitatea propagarii informatiilor privitoare la rute
  - Un utilizator poate folosi (*roam*) diverse sub-retele IP si legaturi de acces, mentinindu-se o comunicare continua
- Implementari:
  - Interactive Protocol for Mobile Networking - http://medianet.kent.edu/ipmn/main.html
  - Network Mobility (NEMO)

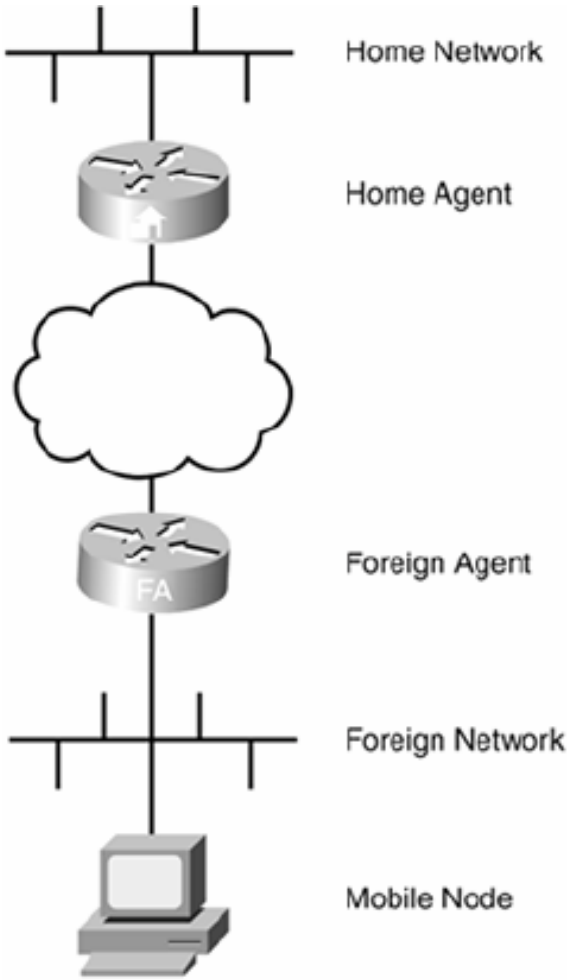
Wireless Internet: IP mobil

- Componente:
  - **Nod mobil**: dispozitiv folosind IP
  - **Home agent (HA)**: nod responsabil cu redirectarea datelor spre locatia curenta a nodului mobil
    - Proceseaza actualizari de rutari IP (*registrations*)
    - Expediaza date via tuneluri dinamice
  - **Forign Agent(FA)**: un router atasat la o legatura de acces, aflat la celalt capat al tunelului stabilit cu un nod mobil
    - Ofera (*advertises*) una sau mai multe adrese IP referite drept **CoA** (*care of address*)
    - Cind un nod se inregistreaza la un Home Agent, o va face via un FA
    - FA trebuie sa fie conectat direct cu nodul mobil



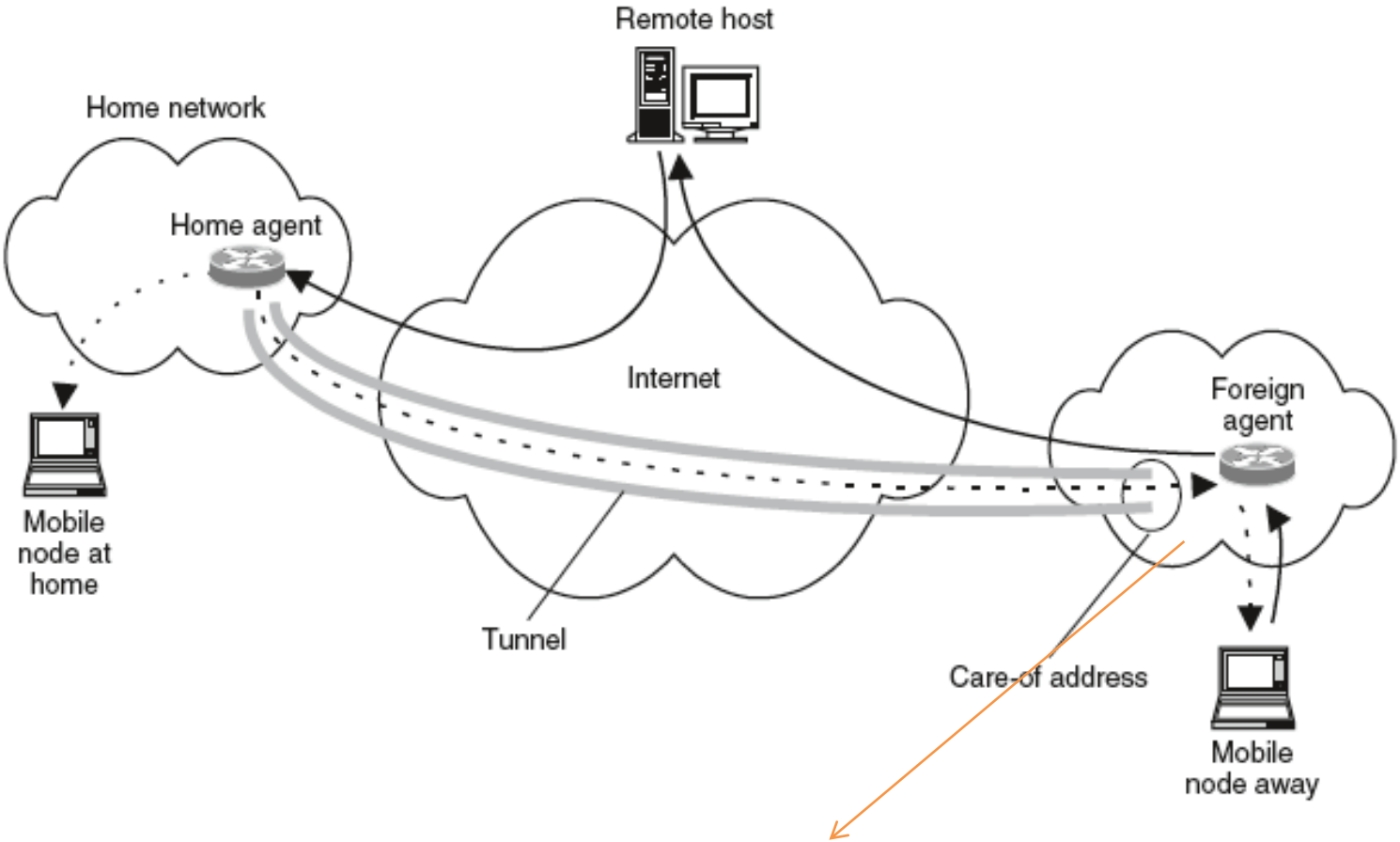
Wireless Internet: IP mobil

- Nodul mobil are doua adrese:
  - **Home Address**
    - Adresa IP a nodului mobil;
    - poate fi alocata static ori dinamic, in timpul procesului de inregistrare
  - **CoA** (***Care-of Adress***)
    - adresa IP valida si rutabila
    - Desemneaza marginea retelei ce poate fi accesata prin rutari obisnuite
    - Reprezinta punctul terminal al unui tunel



[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Wireless Internet: IP mobil



Daca **nodul mobil** este in *Foreign Network*, mesajele sunt trimise la *Home agent*, apoi sunt trimise prin tunel la *Foreign agent* care le va trimite la **nodul mobil**

[Wireless Networking Complete, Morgan Kaufmann]

Wireless Internet: IP mobil

Mecanismul general:

Un nod care doreste sa comunice cu nodul mobil va utiliza permanent *home adress* a acestuia

- Daca nodul mobil se gaseste in *HomeNetwork*, atunci daca cineva ii trimite un pachet se foloseste *IP forwarding*
- Daca nodul mobil se afla intr-o retea straina se foloseste CoA
  - Nodul mobil isi inregistreaza locatia reala la *home agent*
  - Pachetele sunt trimise printr-un tunel de la *home agent* la CoA (capatul tunelului)

  Obs. Daca nodul nu primeste mesaje de tip *agent advertisment* atunci incearca sa obtina o adresa prin tehnici precum DHCP pentru a-si cunoaste locatia curenta

Wireless Internet: IP mobil

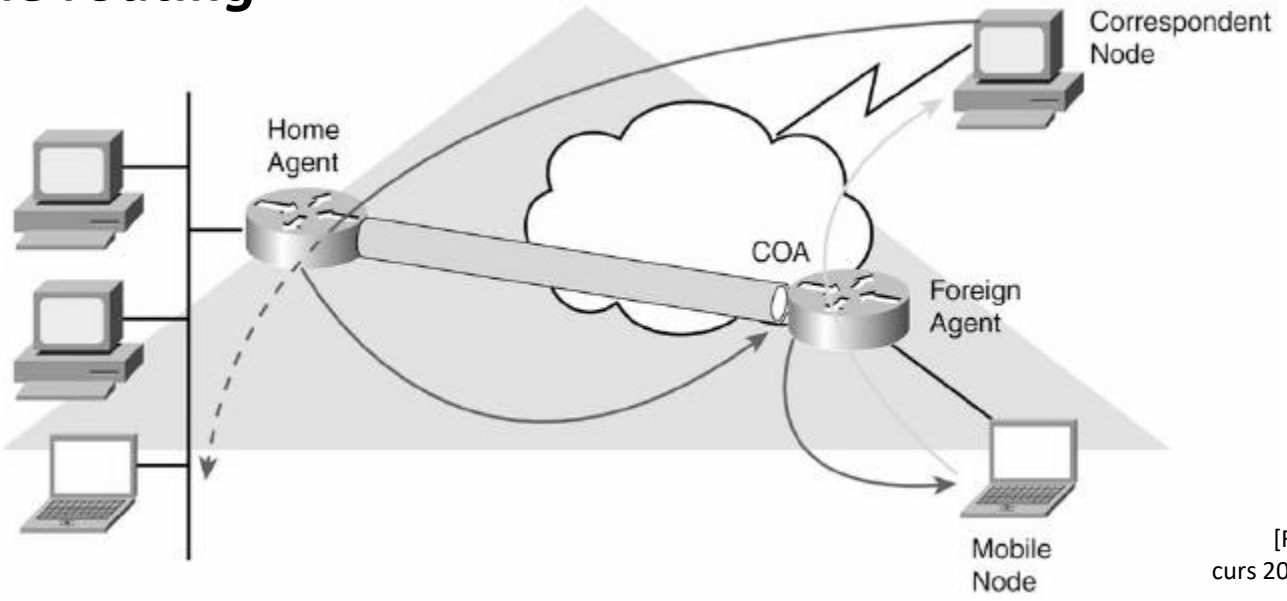
Tuneluri: legaturi logice la distanta de 1 hop, aflate la marginile Foreign Network la care sunt atasate nodurile mobile

- Pot transporta orice pachete IP intre punctele finale ale comunicatiei
- Incapsularea datelor se face via IP-in-IP – RFC 2003 (20 de bytes suplimentari)

Wireless Internet: IP mobil

Rutarea:

- Cind **nodul mobil** doreste sa trimita pachete, el nu le va trimite la *home agent*, ci le va trimite direct la nodul cu care doreste sa comunice, folosind *home adress* ca valoare a cimpului *source adress* a pachetului IP (folosind desigur *foreign agent*) – ***triangle routing***



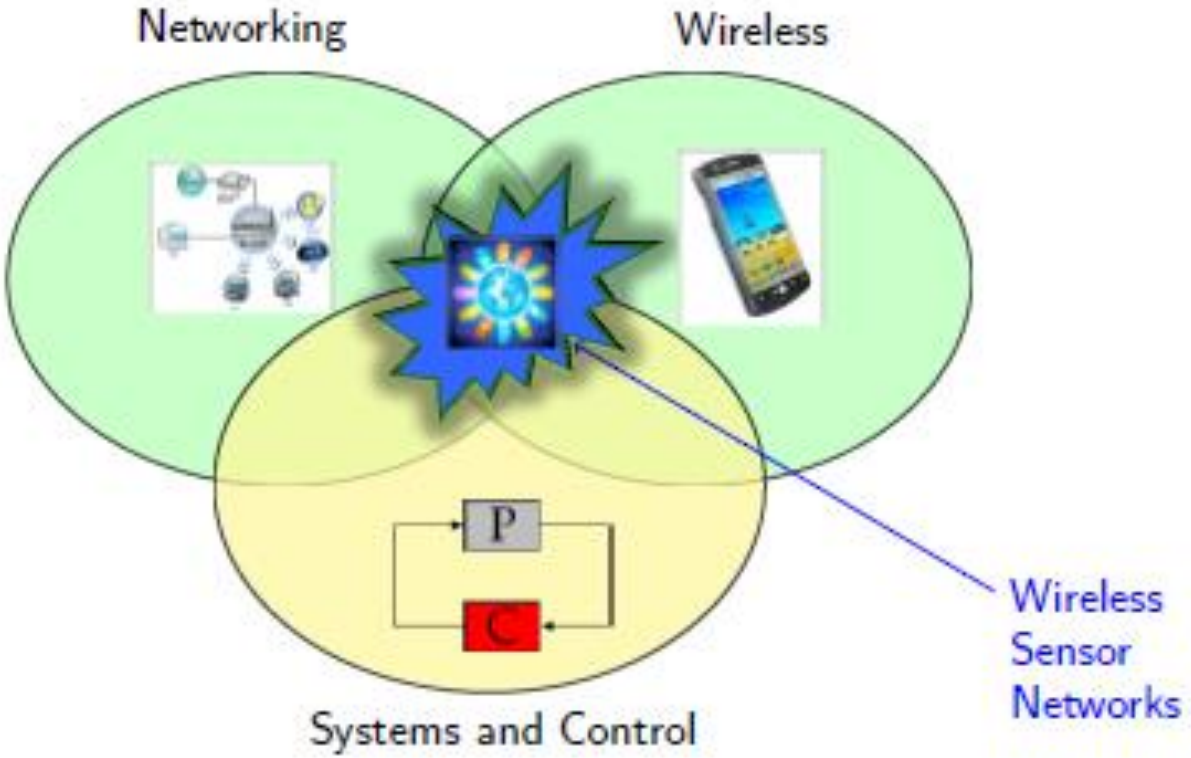
[Retele de calculatoare – curs 2007-2008, Sabin Buraga]

Aplicatii *Wireless*

- Retelele wireless ofera suport in multe domenii
  - Access la Internet
  - *Voice over Wireless*
  - Sanatate
  - Educatie
  - Servicii
  - Vinzari
  - …

Aplicatii *Wireless*

Wireless Sensor Network



Aplicatii *Wireless*

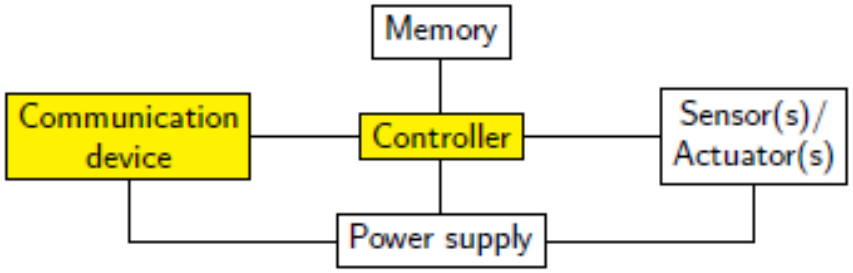
- *"Wearable sensors"*

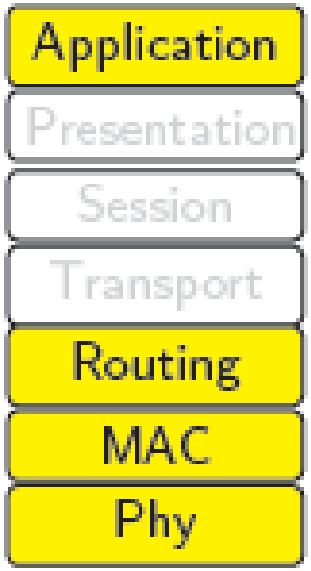
- *"Cladiri Inteligente"*

- *"Monitorizarea mediului"*: gradul de poluare al apei

WSN| Componentele unui node wireless

Comportamentul unui node este specificat de un set de protocoale





[Cosmin Varlan …Curs…Arduino…]

Aplicatii *Wireless*

Provocari:

- Asigurarea conectivitatii continue
  - Calitatea continutului
  - Concurenta (partajarea/managementul resurselor)
  - Agilitatea (viteza/flexibilitatea adaptarii la schimbari)
- Reducerea puterii consumate
- Asigurarea independentei de dispozitiv a aplicatiilor wireless

Bibliografie

- Network+ Guide to Networks, Fifth Edition, Tamara Dean, Network +, ISBN-13: 978-1-423-90245-4, 2009
- Wireless Networks first-step, Jim Geier, Publisher : Cisco Press, 2004
- Cisco: Mobile IP traffic will triple fixed data traffic by 2016

http://www.networkworld.com/newsletters/converg/2012/022012convergence2.html

- http://www.amazon.com/Building-Wireless-Sensor-Networks-Processing/dp/0596807732
- http://smallbusiness.chron.com/bluetooth-mouse-vs-wireless-mouse-55689.html
- https://www.bluetooth.com/
- http://www.makeuseof.com/tag/top-uses-for-bluetooth-on-your-android-phone/
- Is Bluetooth Radiation Really Dangerous? https://www.rfsafe.com/bluetooth-radiation-dangerous-cell-phone-radiation/
- http://andrei.clubcisco.ro/cursuri/f/f-sym/3rl/cursuri2012/RL_curs_11.pdf
- https://www.cnet.com/news/att-jumps-into-fast-lane-with-ultraspeedy-5g-field-trial/
- https://books.google.ro/books?id=UcL1CwAAQBAJ&pg=PA25&lpg=PA25&dq=wwan+5G&source=bl&ots=9ENJYCAu-R&sig=uyXVSg9rDy63clQBSeVETGSEdSk&hl=en&sa=X&ved=0ahUKEwi575up-bXRAhXCJSwKHSYsDcwQ6AEINjAE#v=onepage&q&f=false
- http://www.revsys.com/writings/quicktips/ssh-tunnel.html
- http://www.makeuseof.com/tag/vpn-tunnel-set/

Rezumat

- Preliminarii
- Componente
- Tipuri de retele *wireless*. Caracteristici.
- IP Mobil
- Aplicatii
- Wireless Sensor Networks (WSN)

Intrebari?

Retele de calculatoare
Introducere in securitate

Lenuta Alboaie (adria@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- Preliminarii
- Aspecte importante
- Vulnerabilitati
- Atacuri
- Prevenirea si supravietuirea
- Monitorizarea
- Testarea
- Raspunsul la incidente
- Protocoale
- Probleme specifice
- Statistici 2015
- Previziuni 2016

*Multumiri:

Sabin Corneliu Buraga
Dragos Acostachioaie

Preliminarii

- Asigurarea calitatii aplicatiilor (Internet)
  - Corectitudine si robustete (*reliability*)
  - Extindere & reutilizare (*modularitate*)
  - Compatibilitate
  - Eficienta
  - Portabilitate
  - Usurinta in utilizare (*usability*)
  - Functionalitate
  - Relevanta momentului lansarii (*timeliness*)
  - Mentenabilitate
  - Reparabilitate, economie
  - **Securitate**

Preliminarii

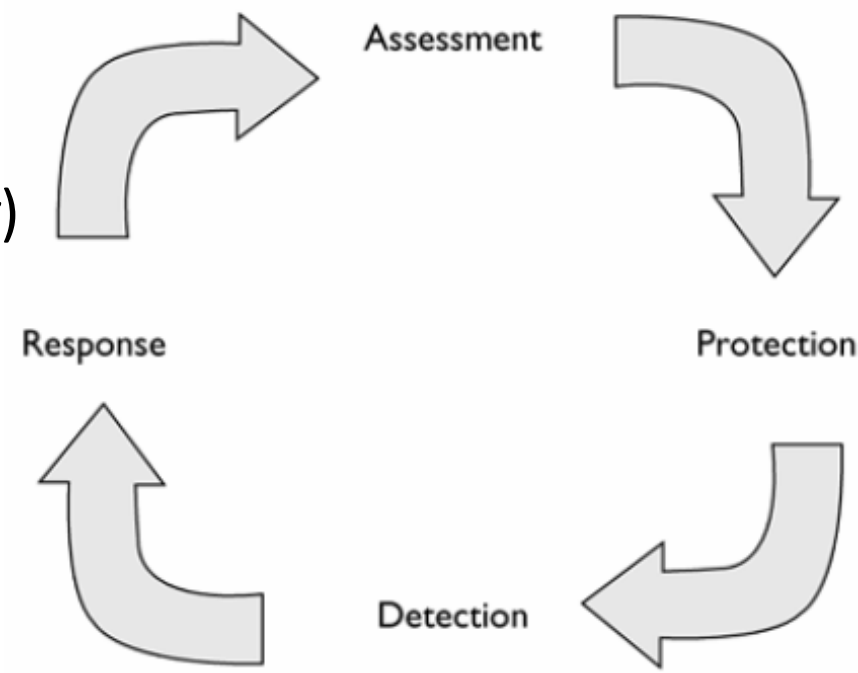
- **Incident de securitate** = eveniment aparut in cadrul retelei, cu implicatii asupra securitatii unui calculator sau a retelei
  - Sursa: interiorul ori exteriorul retelei
- **Securitatea este procesul de mentinere a unui nivel acceptabil de risc perceptibil**
  - "*Security is a process, not an end state*." (Mitch Kabay, 1998)
- **Cracker** *versus* **hacker**
- **Realitatea:**
  - Peste 70% din organizatii sufera de pierderi financiare datorate incidentelor de securitate
  - Cauze:
    - Virusi informatici: > 75%, Acte malitioase interne: > 40%, Actiuni malitioase externe: 25%, Erori software: 70%, Spionaj industrial: 10%

Preliminarii

- **Mituri:**
  - Securitatea prin obscuritate (*security throught obscurity - STO*)
    - *"bunk mentality" security*
    - Ignorarea problemelor
    - Nedocumentarea erorilor cunoscute, algoritmilor de criptare folositi
  - *Cracke*r-ii "*ascunsi*" nu pot fi detectati
  - Organizarea in grupuri malefice a *crackeri*-lor
    - Deseori nu (exceptii: Cult of Dead Cow, …)
  - Software-ul de scanare de virusi ofera protectie totala
  - Conexiunile internet nu pot fi detectate
  - Din moment ce un fisier este sters, el se pierde pentru totdeauna

Preliminarii

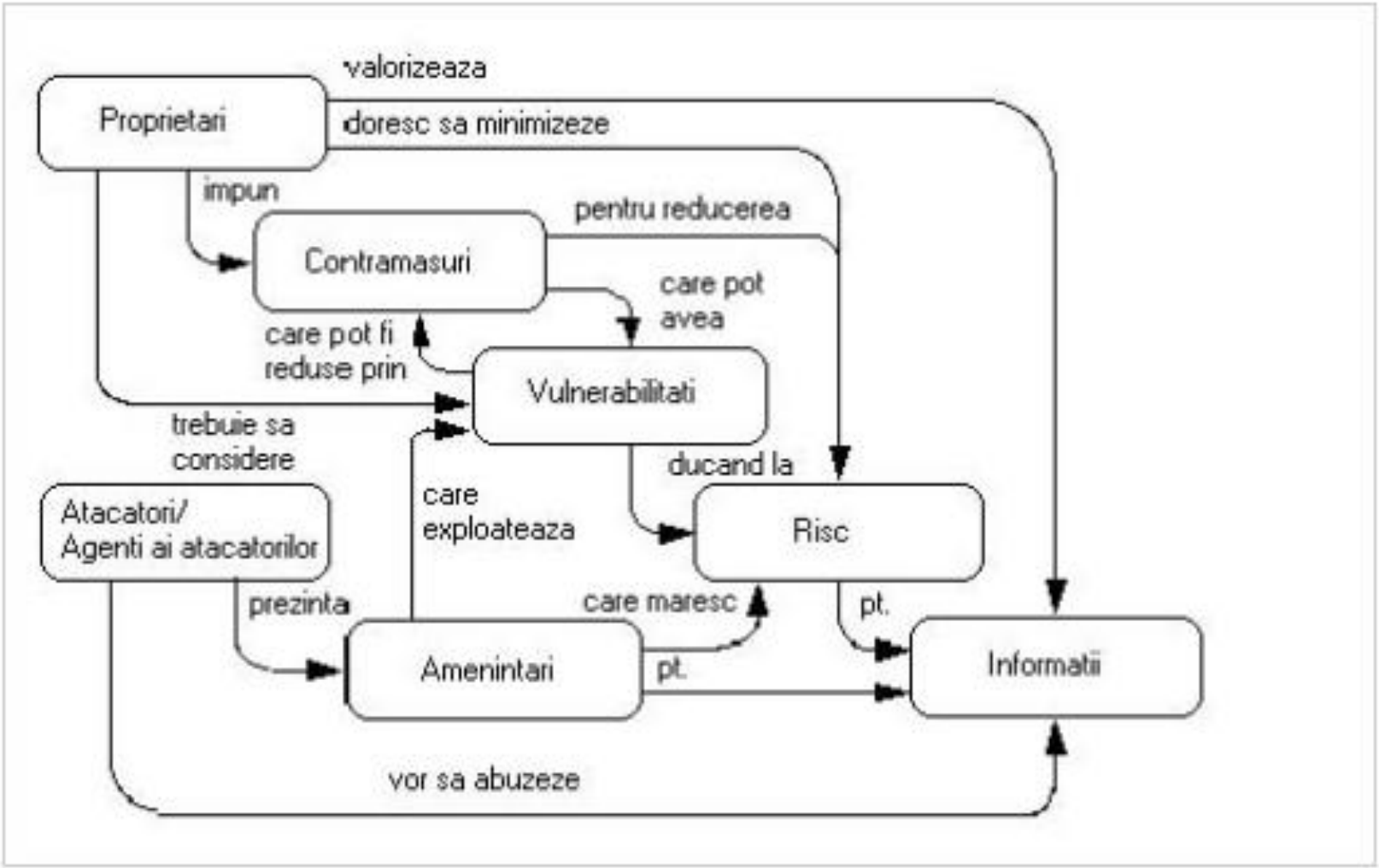
- Faze ale procesului de securizare:



security audit

  - Estimare a riscurilor (*assessment*)
    - Activitati manageriale + actiuni tehnice
  - Protejare (*protection*)
    - Prevenire
  - Detectare (*detection*)
    - Identificarea incidentelor (intrusions)
  - Raspuns la atacuri (*response*)
    - Restaurarea functionalitatii (*patch & proceed*)
    - Alegerea remediilor legale (*pursue & prosecute*)

Preliminarii



http://www.securitatea-informatica.ro/securitatea-informatica/riscurile-de-atac-asupra-securitatii-sistemelor-informationale/

Forme de protectie

- Controlul accesului
  - Identificarea
  - Autentificarea
  - Autorizarea
  - Acces
- Confidentialitatea
- Intimitatea (*privacy*)
- Integritatea
- Disponibilitatea
- Nerepudierea

Forme de protectie

- **Controlul accesului**
  - Proces prin care se ofera sau nu acces la resursa/serviciu
  - Terminologie
    - Identificare
      - Exemplu: introducerea *username*
    - Autentificare – “***verify that someone is who they claim they are”***
      - Exemplu: introducerea parolei
      - Serverul ofera suport pentru autentificari de baza sau bazata pe algoritmi de tip *digest* (e.g. MD5)
      - Ex. mecanisme dedicate: Kerberos, RADIUS, TACACS+,…
    - Autorizare
      - ***“determines what a user is and is not allowed to do”***
      - Specifica actiunile (rolurile) pe care un utilizator le poate realiza
      - Exemplu: utilizatorul autorizat are dreptul sa fie logat?
    - Acces
      - Politici care definesc efectiv permisiuni sau privilegii
      - Exemplu: accesul utilizatorului la anumite date (intr-un interval orar sau doar de la un anumit IP, setari facebook – cine poate vedea o resursa?

Eavesdropping …. RFC1510.

Forme de protectie

Controlul accesului - Modele

- Mandatory Access Control (MAC)
  - *End-user*-ul nu poate modifica/transfera controlul asupra resurselor
- Discretionary Access Control (DAC)
  - Detinatorul resursei poate acorda drepturi acesteia si altor utilizatori
  - Ex. Apple Macintosh, UNIX, Windows (User Account Control (UAC)) cer aceasta permisiune cand un soft este instalat
- Role Based Access Control (RoBAC)
  - Asigneaza permisiuni unui rol in organizatie, apoi unui utilizator i se asociaza acest rol
- Rule Based Access Control (RuBAC)
  - Asigneaza controlul in mod dinamic unui utilizator pe baza unui set de reguli. Fiecare resursa contine un set de proprietati de acces bazate pe aceste reguli.
  - Exemplu: Cineva din reteaua A doreste sa acceseze o resursa din reteaua B; RBAC include regula ca daca cineva are adresa din reteaua A poate accesa resursele din B;

Forme de protectie

- **Controlul accesului –** Implementari
  - Sisteme de control - la nivel hardware
    - Accesul la terminal (e.g. verificarea amprentelor, senzori *real-time anti-break)*
    - Visual event monitoring
    - Carduri de identificare
    - Identificare biometrica (e.g. recunoastere - fingerprint, iris & voice recognition)
  - Sisteme de control - la nivel software
    - Drepturi de acces (permisiuni) + liste de control al accesului (ACL – *Access Control List*)
    - Tehnici de tip SSO (*Single Sign-On*)

Forme de protectie

- **Confidentialitatea**
  - Imposibilitatea unei terte entitati sa aiba acces la datele vehiculate intre doi receptori
  - Solutii:
    - Conexiuni private intre cele 2 puncte terminale ale canalului de comunicatie; datele circula printr-un tunel oferit de o retea privata virtuala (VPN – *Virtual Private Network*)
    - Criptarea datelor via diverse tehnici (biblioteci specializate si/sau oferite de mediile de dezvoltare)
      - Emitatorul cripteaza mesajele
      - Receptorul decripteaza mesajele

Forme de protectie

- **Intimitatea** (*privacy*)
  - Confundata, deseori cu confidentialitatea care se aplica datelor
  - Vizeaza drepturile ce trebuie respectate privind caracterul datelor vehiculate
  - Brese:
    - Stocarea necorespunzatoare a datelor la nivel de server (*information disclosure*)
    - Atacuri de tip *phishing*
    - Configurarea necorespunzatoare a sistemelor

Forme de protectie

- **Integritatea**
  - Implica detectarea incercarilor de modificare neautorizata a datelor transmise
  - Solutii:
    - Algoritmi de tip *digest*
    - Semnaturi digitale
- **Disponibilitatea**
  - O anumita resursa poate fi accesata la momentul oportun
  - Cauze ale indisponibilitatii
    - Atacuri de refuz al serviciilor DoS (*Denial Of Service*), Atacuri distribuite de tip DDoS (*Distributed DoS*) – (vezi slide 22)

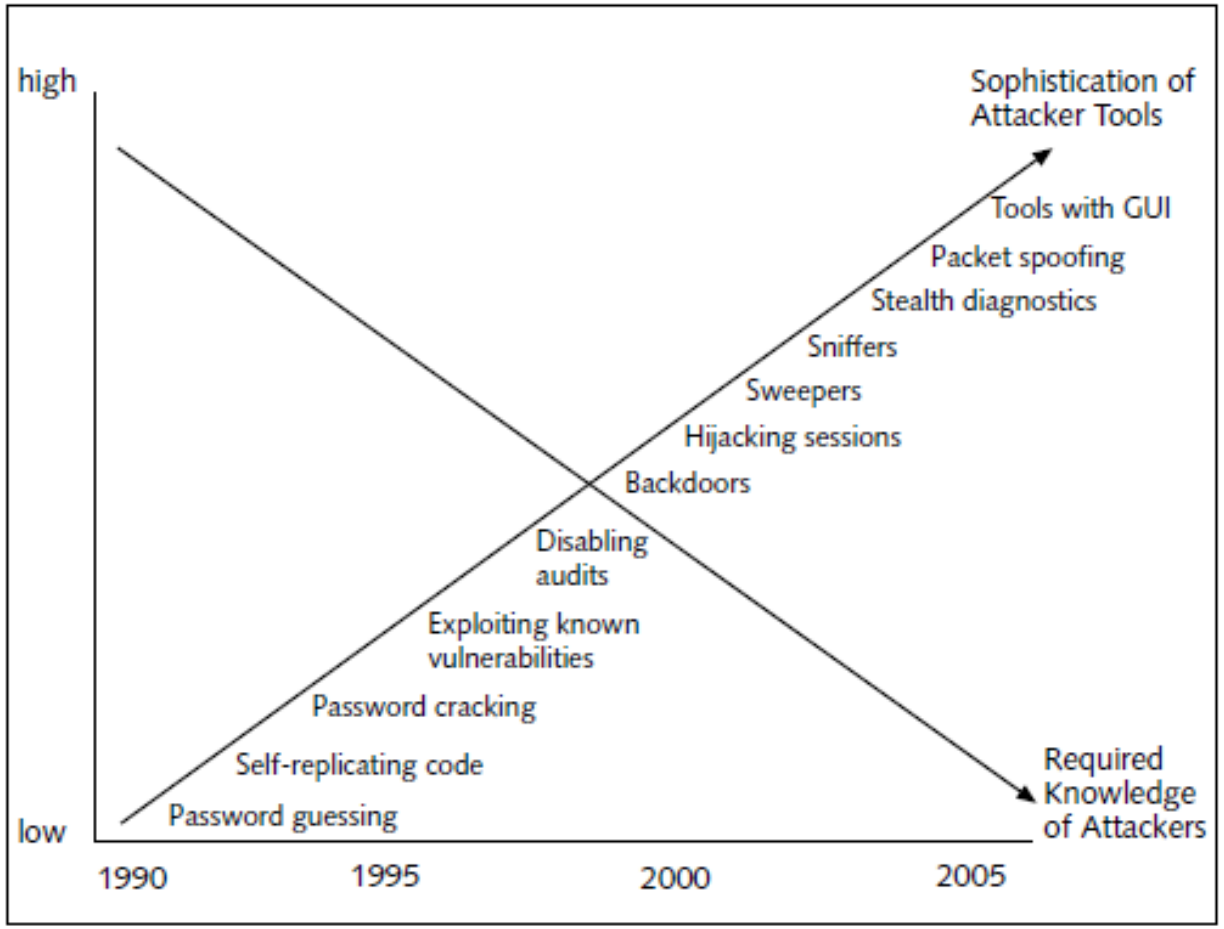
Forme de protectie

- **Nerepudierea**
  - Expeditorul mesajului nu poate afirma ca nu l-a trimis
  - Solutie: certificate digitale
    - Stocheaza datele privind identitatea unei entitati detinatoare a unui secret (parola, serie a cartii de credit, certif. digital, …)
    - Emise de o autoritate de certificare (CA – *Certification Authority*)
    - Verificate de o autoritate de inregistrare (RA – *Registration Authority*)
    - Serviciile PKI puse la dispozitie de sistem

Infrastructura cu chei publice (*PKI - Public Key Infrastructure*)

Dificultati in apararea contra unui atac

- Viteza de atac
  - "Slammer worm infected 75,000 computers in the first 11 minutes after it was released and the number of infections doubled every 8.5 seconds"
- Complexitatea atacului
- Disponibilitatea instrumentelor de atac
- Detectarea rapida a vulnerabilitatilor
- *Delay patching*
- Atacuri distribuite
- Confuzia utilizatorilor



[Security guide to network security fundamentals, Mark Ciampa]

Vulnerabilitati

- Pentru Internet, securitatea trebuie sa ia in considerare:
  - Clientul: interactiune, date personale,…
  - Datele de tranzit: securitatea retelei, schimb de mesaje, ne-repudiere
  - Serverul: securitatea serverului (serverelor), securitatea aplicatiilor, disponibilitatea serviciilor
- **Vulnerabilitate** = slabiciune a unui sistem hardware/software care permite utilizatorilor neautorizati sa aiba acces asupra lui
  - Nici un sistem nu este 100% sigur
  - Vulnerabilitati apar si datorita proastei administrari

Vulnerabilitati

- **Riscuri asociate oamenilor**
  - Depasesc jumatate din tipul de atacuri in retea
  - Exemplu:
    - Atacatorul foloseste *social engineering* pentru obtinerea parolei utilizator
    - Administatorul creaza sau configureaza incorect grupurile de utilizatori si drepturile lor de acces
    - Configurarea necorespunzatoare a programelor, serverelor si retelelor
    - *Bug*-uri existente in programe (introduse neintentionat deseori)
    - Ignorarea/nedocumentarea *bug*-urilor cunoscute
    - Lipsa suportului din partea producatorilor
    - Comoditatea sau necunoasterea problemelor de securitate de catre administrator ori de conducerea organizatiei
    - Angajati necinstiti care abuzeaza de politicile de acces
    - Pastrarea drepturilor pentru angajati care nu mai fac parte din organizatie
    - ….

Vulnerabilitati

- **Riscuri asociate transmisiilor si nivelului hardware**
  - Transmisia poate fi interceptata
    - *Man-in-middle attack*
      - Exemplu: un cracker capata acces aupra unui AP care ofera acces liber la WI-FI
  - Brodacast-ul realizat de un hub intr-un segment de retea poate fi vulnerabil la *sniffing*
  - Porturile serverelor neutilizate pot fi exploatate (solutie: p*ort scanner*)
  - Neconfigurarea corespunzatoare a routerelor poate permite utilizatorilor externi vizualizarea adreselor private
  - Neschimbarea suficient de des a parolelelor pentru routere si alte dispozitive
  - Accesul fizic la echipamentele retelei (servere, routere, sisteme intermediare,…)

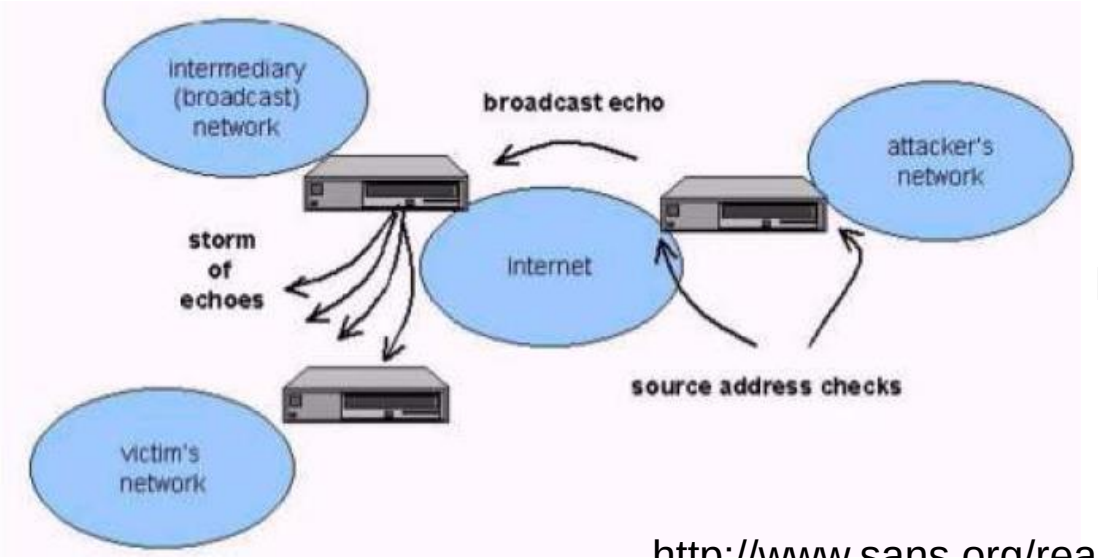
Vulnerabilitati

Riscuri asociate cu protocoalele si software-ul utilizat

- **Obs. Distinctia hardware vs. software este greu de facut deoarece protocoalele si nivelul hardware opereaza in tandem**
- Implementari DNS eronate
  - Vulerabilitati BIND
- Servicii “antice” operationale
  - E.g., telnet, TFTP, …
- Servicii/protocoale oferind date necriptate
    - FTP, SMTP, POP
- “Gauri” prezente in aplicatii (*application holes*)
  - Exemplificari: Apache, IIS, MSIE, Outlook, phpBB, …
- Script-urile CGI (Common Gateway Interface)
- Existenta conturilor/configuratiilor implicite
- Permisiuni inadecvate pentru fisiere, servicii, conturi-utilizator
- Lipsa mecanismelor de monitorizare & detectare a intrusilor
- Existenta exemplelor de configuratii, programe demonstrative ce pot fi exploatate

Vulnerabilitati

- **Riscuri asociate accesului la internet**
  - Un firewall poate sa nu fie corespunzator configurat => obtinerea de adrese IP, ce vor fi utilizate pentru *IP Spoofing*
  - Atac de tip *denial-of-service*: sistemul a devenit nefunctionabil deoarece este invadat cu transmisii de date
    - Atac de tip *smurf: "flood of broadcast ping messages"*



Exemplu: Atac de tip *smurf*

http://www.sans.org/reading_room/whitepapers/threats/icmp_attacks_illustrated_477?show=477.php&cat=threats

Atacuri

- Cunoasterea profilului atacatorului
- Atribute ce trebuie considerate
  - Resurse disponibile

    (financiare, tehnice, pregatirea in domeniu,…)
  - Timpul alocat

    (atacatorii rabdatori vor avea mai mult succes)
  - Riscul asumat – depinde de obiective

    (atacul ar putea fi revendicat sau nu de *cracker*)
  - Accesul la Internet & calitatea acestuia: tip (*wireless*, conexiune satelit,…), mod de alocare al IP-ului
  - Obiectivele urmarite (recunoastere mondiala, denigrarea tintei, furt de informatii, furt de bani, …)

Atacuri

- Niveluri de atac
  - **Oportunist** (*script kid*)
    - Scop “recreational”
    - Fara obiective/tinte clar definite
    - Se utilizeaza programe disponibile liber pentru a scana sau testa vulnerabilitati uzuale (e.g., software de scanare, *rootkits*,…)
    - Nu necesita acces in interiorul sistemului
    - Cunostinte vagi despre sistemul/organizatia tinta
    - Masuri de precautie:
      - Ziduri de protectie (*firewall*-uri)
      - Actualizarea versiunilor de programe

Atacuri

- Niveluri de atac
  - **Intermediar**
    - Obiectiv conturat, la nivelul organizatiei
    - Se vor efectua aceleasi actiuni ca la atacul “recreational”, dar se incearca ascunderea lor
    - Atacatorul are mai multa rabdare
    - Cunostinte tehnice mai profunde (uzual, la nivelul unui administrator de retea)
    - Probabilitate mai mare de succes, posibil efecte mai mari
  - **Sofisticat**
    - Obiectiv foarte bine conturat; Tinta este de cele mai multe ori o organizatie
    - Atacurile pot trece peste masurile de prevedere
    - Atacatorul va avea multa rabdare; Se investeste timp pentru colectarea de informatii despre sistemul/organizatia tinta
    - Foarte bune abilitati tehnice; Probabilitate mare de succes

Atacuri

Tip atacator	Resurse	Timp	Instrumente	Risc	Acces	Obiective
Atacator recrea-țional	Cunoştinte tehnice în general limitate	De obicei oportunist	Utilizează instrumente liber disponibile	Posibil să nu înțeleagă/ aprecieze riscul	Extern	Recunoaştere personală, să-şi dezvolte abilitățile de cracker
Angajat sau fost angajat	Depinde de abilitățile personale	Poate fi răbdător şi aştepta apariția unei oportunități	Utilizează instrum. liber disponibile. Dacă a fost admin., ar putea dezv. singur instrumente	Înțelegere a riscului, mai ales dacă este încă angajat	Intern sau extern	Avantaje personale. Denigrarea organizației

Atacuri

Tip atacator	Resurse	Timp	Instrumente	Risc	Acces	Obiective
Activist cu motivație etică sau politică	Posibil să lucreze în echipă	Posibil răbdător, un evenim. poate însă determ. o acțiune rapidă	Utilizează instrumente liber disponibile	Nu este conştient de riscuri	Extern	Denigrarea organizației. Impresionarea opiniei publice. Impresionarea instituțiilor guv.
Spion industrial	Cunoştinte avansate	Răbdător. Va încerca probabil ascunderea identității proprii	Poate modifica sau crea instrumente noi	Întelegere medie a riscului	Extern	Vânzarea inform. proprietare. Aflarea de informații despre concurență sau determinarea strategiilor organizației țintă

Atacuri

Tip atacator	Resurse	Timp	Instrumente	Risc	Acces	Obiective
Atacator la nivel național (nation-state)	Poate angaja resurse importante	Răbdător, însă informațiile dorite pot fi necesare într-un timp scurt	Ar putea dezvolta instrumente specifice dacă este mare câştigul	Înțelegere medie a riscului	Extern	Accesarea de informații guvernamentale sau informațiile proprietare ale unei organizații

Atacuri

- **Tipuri de atac**
  - **Accesul la nivel de utilizator**
    - Atac prin acces via un cont de utilizator obisnuit sau cu privilegii superioare
    - Etape:
      - Colectarea de informatii (utilizatori, vulnerabilitati notorii, configuratii de sisteme tipice,…)
      - Exploatarea
      - Deteriorarea: acces la date importante, alterarea informatiilor, asigurarea accesului ulterior la sistem, modificarea jurnalelor de sistem
    - Solutii: eliminarea programelor, modulelor & serviciilor care nu sunt neaparat necesare, analizarea fisierelor de jurnalizare

Atacuri

- **Tipuri de atac**
  - **Accesul de la distanta**
    - Nu necesita acces-utilizator la sistem
    - Creaza refuzuri de servicii prin cereri incorecte, eventual cu “caderea” serviciilor prost proiectate
    - Etape:
      - Colectarea de informatii – identificarea de servicii
      - Exploatarea – trimiterea de pachete la portul gasit
      - Deteriorarea: distrugerea unui serviciu de retea, defectarea/incetinirea (temporara) a unui serviciu sa a sistemului

Atacuri

- **Tipuri de atac**
  - **Accesul de la distanta la diverse aplicatii**
    - Trimiterea de date invalide aplicatiilor, nu serviciilor de retea (traficul nu este afectat)
      - Exemple: SQL injection
    - Nu necesita obtinerea unui cont de utilizator
    - Etape:
      - Colectarea de informatii – identificarea aplicatiei (e.g. server sau client Web, aplicatie de birou, sistem de stocare, solutie de mesagerie, …)
      - Exploatarea – trimiterea continutului, direct sau indirect (e.g., via e-mail sau FTP), spre aplicatie
      - Deteriorarea
        - » Stergerea/copierea fisierelor utilizatorilor
        - » Modificarea fisierelor de configuratie

Atacuri

- **Tipuri de atac**
  - **Inocularea de programe pe calculatorul utilizatorului**
    - Plasarea de programe *malware (virusi,* spioni, cai troieni, bombe, s*careware*…) – via script-uri, plugin-uri, componente ActiveX etc. Efecte:
      - Apelarea neautorizata de programe
      - Colectarea/distrugerea de resurse
      - Lansarea de atacuri spre alte sisteme
      - Crearea de usi ascunse (*traps/backdoors*)
      - Furtul identitatii utilizatorului
      - ….

http://lifehacker.com/5560443/whats-the-difference-between-viruses-trojans-worms-and-other-malware

Atacuri

- **Tinta**
  - Organizatii publice sau guvernamentale
    - Recunoastere in rindul cracker-ilor
    - Captarea atentiei mass-mediei
    - Revendicari etice, politice,…
  - Furnizori de Internet
    - Sabotarea activitatii
  - Companii Private
    - Discreditare
    - Furt de informatii
    - Razbunare din partea fostilor angajati
  - Persoane Fizice
    - Cu scop “recreational”

Atacuri

- **Moduri de atac**
  - Spargerea sau penetrarea (*cracking*)
    - Actiunea de descoperire a unor vulnerabilitati si de profitare de pe urma acestora
    - Acces neautorizat la sistem efectuat de *cracker*
      - Accesare, fara alta actiune – rol pasiv
      - Accesare cu alterare/distrugere a informatiilor – activ
      - Accesare cu control asupra sistemului; uneori cu creare de “usi din spate” (backdoors) – rol activ
      - Nu se acceseaza sistemul, ci se realizeaza actiuni distructive de refuz al serviciilor

Atacuri

- **Moduri de atac**
  - ***E-mail bombing***
    - Trimiterea repetata a unui mesaj (de dimensiuni mari) spre o adresa e-mail a unui utilizator
    - Incetineste traficul, umple discul
    - Unele atacuri pot folosi adrese e-mail multiple existente pe serverul tinta
    - Se poate combina cu falsificarea adresei (*e-mail spoofing*)
  - ***E-mail spamming***
    - Trimiterea de mesaje nesolicitate (reclame)
    - Adresa expeditorului este falsa
    - Efectul atacului este accentuat daca mesajul va fi trimis pe o lista de discutii

Atacuri

- **Moduri de atac**
  - **Abonarea la liste de discutii**
    - “Atac” ce determina enervarea victimei, facilitat de diverse programe disponibile in Internet
    - Cauzeaza trafic inutil in retea
  - **Flasificarea adresei expeditorului (*e-mail spoofing*)**
    - Folosita pentru ascunderea identitatii expeditorului sau pentru determinarea utilizatorului sa raspunda la atac ori sa divulge informatii (e.g. parole)
    - Slabiciune datorata protocolului SMTP
    - Utilizatorii trebuie educati sa nu raspunda expeditorilor necunoscuti si sa nu divulge informatii confidentiale

Atacuri

- **Moduri de atac**
  - ***Social engineering***
    - Manipularea utilizatorilor de catre un *craker – phishing (preluarea identitatii)*
    - Tipuri: intimidare, santaj, presiune, autoritate, flatare, substitutie de persoana, vanitate etc.
    - Atacatorul colecteaza date privitoare la persoana si/sau organizatia vizata si aplica principii de persuasiune
    - http://www.securityfocus.com/infocus/1527

Atacuri

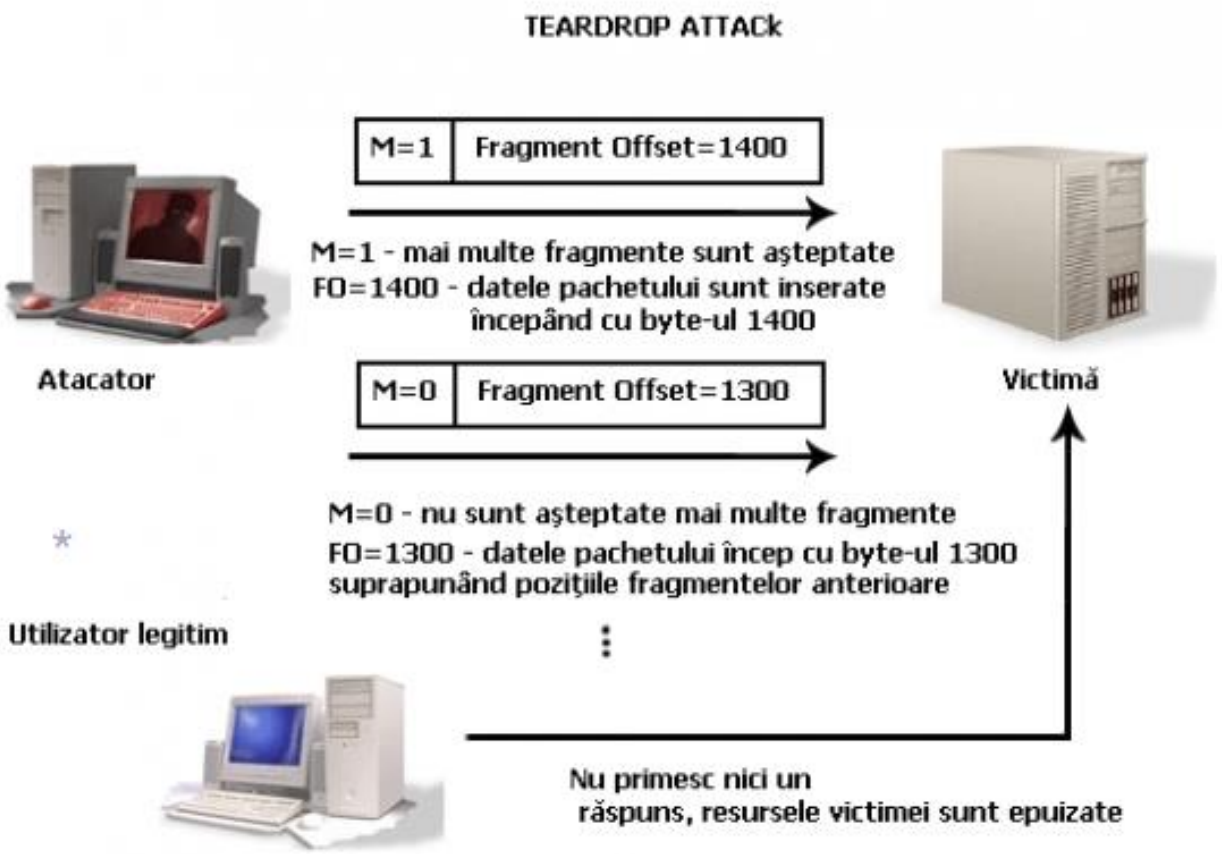
- **Moduri de atac**
  - **Refuzul serviciilor *(Denial Of Service)***
    - Obiectiv: Degradarea calitatatii functionarii unor servicii sau dezafectarea lor
    - Modalitate: supraincarcarea serverului sau a retelei
    - Consumarea resurselor *host*-ului
      - *flood*-uri TCP SYN
      - *flood* ICMP ECHO (ping)
    - Consumarea latimii de banda
      - *flood* UDP
      - *flood* ICMP

Atacuri

- **Moduri de atac**
  - **Refuzul serviciilor *(Denial Of Service)***
    - De obicei, atacatorul isi falsifica adresa sursa (*IP spoofing*)
    - Se pot modifica porturile sursa/destinatie (pentru a trece de *firewall*-uri)
    - Exemple:
      - SYN flood – cereri multiple de realizarea a conexiunii
      - *Ping of death* – atac cu pachete ICMP mari
      - *Teardrop* – exploatarea in general a implementarilor TCP/IP care nu gestioneaza corect pachetele IP (versiunile de Windows 3.1x, Windows 95, windows NT si versiuni Linux (inainte de 2.0.32))
      - *Smurf* – atac ICMP asupra adresei de broadcast

Atacuri

- **Moduri de atac**
  - **Teardrop**



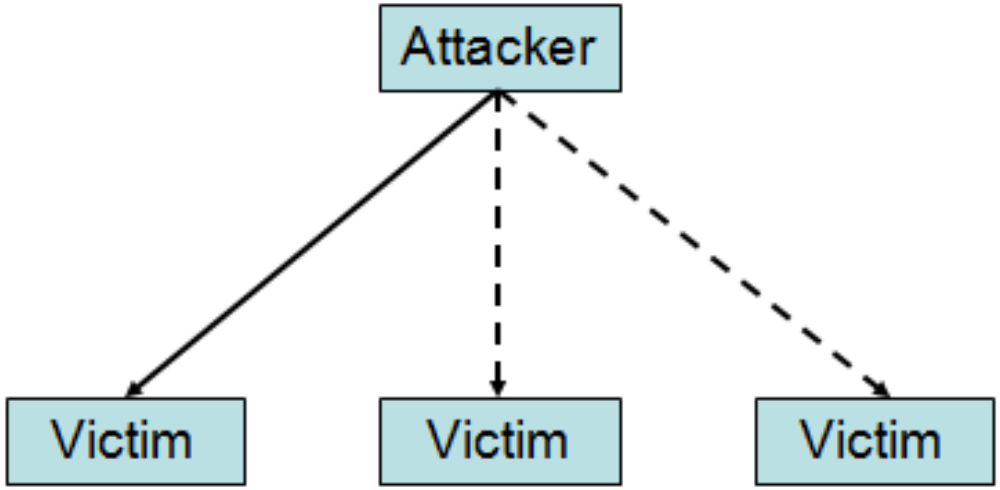
http://www.dlsit.ro/articole/106_Atacul-Teardrop

Atacuri

- **Moduri de atac**
  - **Refuzul serviciilor *(Denial Of Service)***

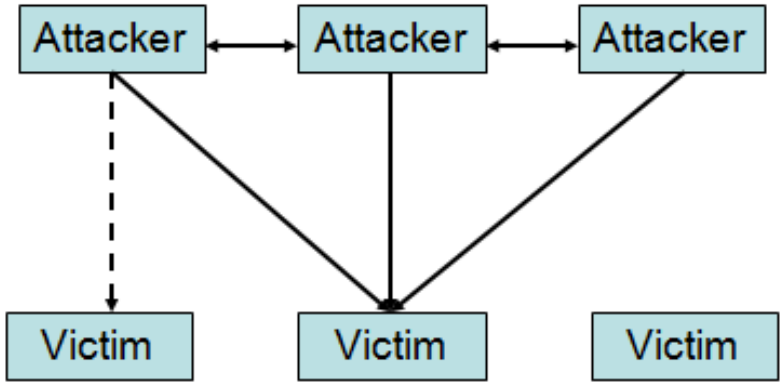
    DoS simplu

    - De obicei, atacatorul isi falsifica adresa sursa (*IP spoofing*)
    - Usor de rezolvat



Atacuri

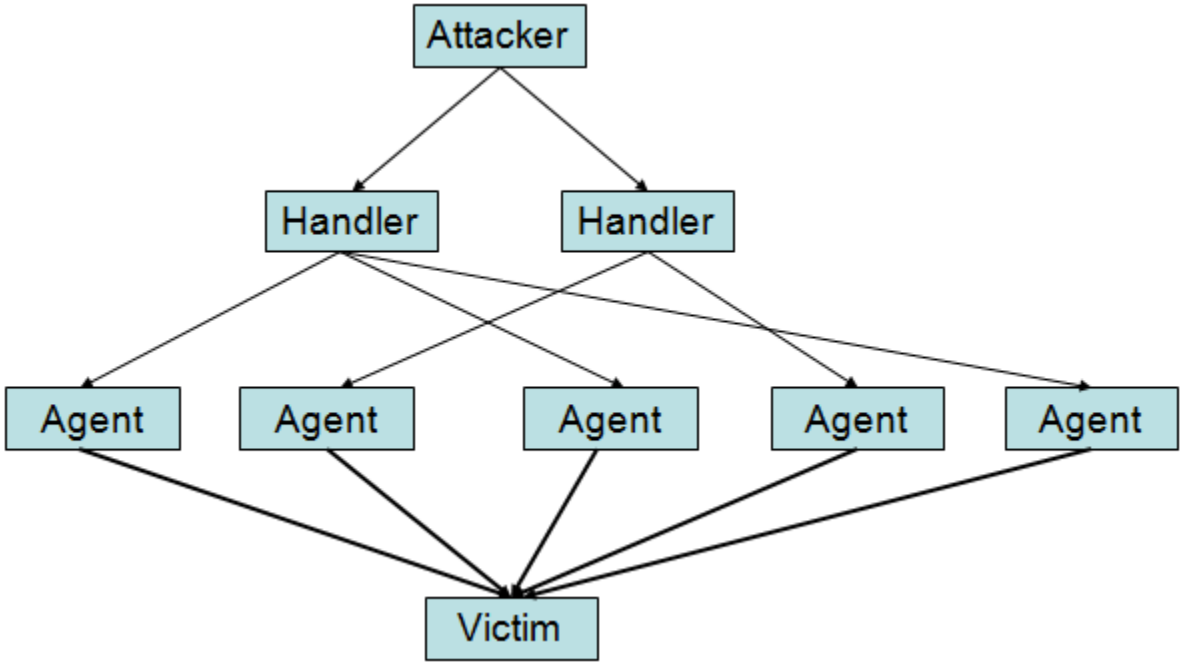
- **Moduri de atac**
  - **Refuzul serviciilor *(Denial Of Service)***
    - **DoS coordonat**
      - La primul pas, este atacata o alta victima pentru a se ascunde adevaratul atac
      - Atacatorul isi ascunde de obicei adresa de origine
      - Greu de rezolvat



Atacuri

- **Moduri de atac**
  - **Refuzul serviciilor *(Denial Of Service)***

    DDoS (*Distributed DoS*)



Atacuri

- **Moduri de atac**
  - **Refuzul serviciilor *(Denial Of Service)***
    - **DDoS (*Distributed DoS*)**
      - *Handlers* – sunt de obicei serverele puternice (care ascund usor pachetele de atac)
      - *Agents* – sunt de obicei utilizatori ce au computerele infectate
      - Foarte dificil de depistat atacatorul
      - Este diferit de *FlashCrowd*
        - » Slashdot Effect, Victoria Secret Webcast

(http://www2.research.att.com/~bala/papers/www02-fc.html)

Atacuri

- **Moduri de atac**
  - **Depasirea capacitatii buffer-elor (*buffer overflow*)**
    - Unele programe pot aloca spatiu insuficient pentru unele date, depasirile survenite pot produce executarea de comenzi ca utilizator privilegiat (root)
    - Unele functii C – precum gets(), getwd(), strcpy(), strcat() – ofera premisele aparitiei de *buffer overflow*- uri
    - Exemple: suprascriere de cod, alterarea stivei de pointeri
    - Uzual atacul provine din interior, dar poate fi si din exterior (via un cal troian)

Atacuri

- **Moduri de atac**
  - **Interceptarea retelei (*IP sniffing*)**
    - Monitorizarea datelor care circula printr-o interfata de retea
      - Se pot detecta parole transmise necriptate
    - Atacul provine din interior
    - Pentru retele de mare viteza (peste 100M/s) unele pachete nu pot fi captate de *sniffer*
    - Software-ul interceptor trebuie supravegheat
    - Exemple: tcpdump, Wireshark (Ethreal)

Atacuri

- **Moduri de atac**
  - **Virusi**
    - Programe ce efectueaza operatii nedorite (distructive), cu capacitati de “multiplicare”
      - Infectarea altor programe (uzual, executabile)
    - Mai putin raspinditi in Unix/Linux, de obicei avind efect doar daca se executa sub auspicii de *root*
    - Pot genera si *e-mail bombing*
    - Remedii: utilizarea de antivirusi si porti de *e-mail*

Atacuri

- **Moduri de atac**
  - **Cai troieni (trojan horses)**
    - Programe rau intentionate “deghizate sub forma unor executabile “utile”
    - Apeleaza programe neautorizate sau sunt modificate, incluzind cod nelegitim
    - Actiuni: colectarea de informatii, distrugerea de informatii, lansarea de atacuri spre alte sisteme
    - Exemple: “vaduva neagra” (blocheaza sau corupe browsere Web)

Atacuri

- **Moduri de atac**
  - **Usi ascunse (back doors/ traps)**
    - Caz particular de cai troieni
    - Creaza o “poarta” (e.g. utilizator, port,…) care permite accesul ulterior la calculator si/sau castigarea de privilegii
  - **Viermi (worms)**
    - Programe care se multiplica, transferandu-se pe alte gazde si efectuand (eventual) distrugeri
    - Exemplu celebru: Internet Worm (Morris Worm) (1988)
      - *fork bomb effect*

Atacuri

- **Moduri de atac**
  - **Ghicirea parolelor (*password guessing*)**
    - Majoritatea proceselor de autentificare folosesc parole
    - Cu cat utilizatorul trebuie sa retina mai multe parole, cu atit sistemul de protectie via parole este predispus la brese in securitate:
      - Alegerea unor parole slabe
      - Partajarea parolelor (colegi, prieteni,…)
      - Scrierea parolelor pe hirtie
      - Folosirea aceleiasi parole timp indelungat, pentru mai multe aplicatii/sisteme
    - Folosirea unui program ce determina parolele prost alese (prea simple, prea scurte, cuvinte din dictionar,..)
    - Protectie prin /etc/shadow, reguli stricte de schimbarea parolelor, educarea utilizatorilor
    - Alte solutii: SSO (Single Sign On), identificare biometrica, etc.

Atacuri

- **Moduri de atac**
  - **Utilizarea tehnicilor de *reverse code-engineering***
    - Analiza aplicatiilor binare fara cod-sursa accesibil (*closed –source*), pentru a se observa modul de executie la nivel scazut
    - Folosita si pentru a studia codul *malware*
    - Instrumente: editoare hexa, dezasambloare, depanatoare, monitoare de sistem,…
    - Apar probleme de legalitate

Prevenirea

- La ce nivel trebuie luate masuri de securitate?
  - Nivel fizic: inhibarea ascultarii mediilor de transmisie, interzicerea accesului fizic la server, …
  - Nivelul legatura de date: criptarea legaturii
  - Nivelul retea: ziduri de protectie (*firewall*-uri)
  - Nivelul transport: criptarea conexiunilor (SSL – *Secure Socket Layer*, TLS – *Transport Layer Security*)
  - Nivelul aplicatiei: monitorizare si actualizarea software-ului, jurnalizare, educarea utilizatorilor, politici generale adoptate,…

Prevenirea

- Elaborarea de politici de securitate
  - Planificarea cerintelor de securitate (confidentialitate, integritate, disponibilitate,..)
  - Evidentierea riscurilor
    - Scenarii de risc
  - Analiza raportului cost-beneficii
    - Costurile prevenirii, refacerii dupa dezastru etc.
  - Stabilirea politicilor de securitate
    - Politica generala (nationala, organizationala,…)
    - Politici separate pentru diverse domenii protejate
    - Standarde & reglemetari (recomandari)

Prevenirea

- Elaborarea de politici de securitate - exemplu
  - Gestionarea accesului (nume de cont, modul de schimbare a parolei, politica de acces din exterior,…)
  - Clasificarea utilizatorilor (grupuri, permisiuni, utilizatori speciali, utilizatori administratori,…): ACL (Access Control List)
  - Accesul la resurse (drepturi de acces la fisiere, directoare, criptarea fisierelor importante,…)
  - Monitorizarea activitatii (fisiere de jurnalizare)
  - Administrarea copiilor de siguranta (tipuri de salvari, medii de stocare, durata pastrarii, …)

Prevenirea

- Principii de baza
  - Simplificare – configurarea sistemului astfel incat sa acorde vizitatorilor cele mai scazute privilegii
  - Reducere – minimizarea ariei de actiune
  - Intarire – “never trust user input” + securizarea accesului la fisiere/aplicatii externe
  - Diversificare – utilizarea mai multor niveluri de protectie (fara *security through obscurity*)
  - Documentarea – memorarea setarilor, strategiilor si masurilor adoptate pentru securitate
- Obs. Siguranta sistemului depinde de cea mai vulnerabila componenta a acestuia

Supravietuirea

- **Supravietuirea** = capacitatea unui sistem (calculator/retea) de a-si indeplini misiunea, in timp util, in prezenta atacurilor, defectelor sau accidentelor
- **Atac** = eveniment potential distrugator provocat intentionat de persoane rau-voitoare
- **Defect** = eveniment potential distrugator cauzat de deficiente ale sistemului sau ale unui factor de care depinde sistemul (e.g., defecte hardware, bug-uri software, erori ale utilizatorilor)
- **Accident** = evenimente neprevazute (e.g. dezastre naturale, caderi de tensiune,…)
- Sistemul trebuie sa sustina macar indeplinirea functiilor vitale (*mission-critical*)
  - Identificarea servicilor esentiale,
  - Identificarea perimetrilor de securitate majora

Supravietuirea

- **Proprietati ale sistemului:**
  - Rezistenta la atacuri = strategii de respingere a atacului (e.g. autentificarea utilizatorilor, *firewall*-uri, validarea obligatorie a datelor de intrare)
  - Recunoasterea atacurilor si efectelor lor = strategii pentru restaurarea informatiilor, limitarea efectelor, mentinerea/restaurarea serviciilor compromise

    RAID (Redundant Array of Independent Disks),

    SAN (Storage Area Network), backup-uri, cluster-e,…)
  - Adaptarea la atacuri = strategii pentru imbunatatirea nivelului de supravietuire -> invatarea din greseli

Monitorizarea

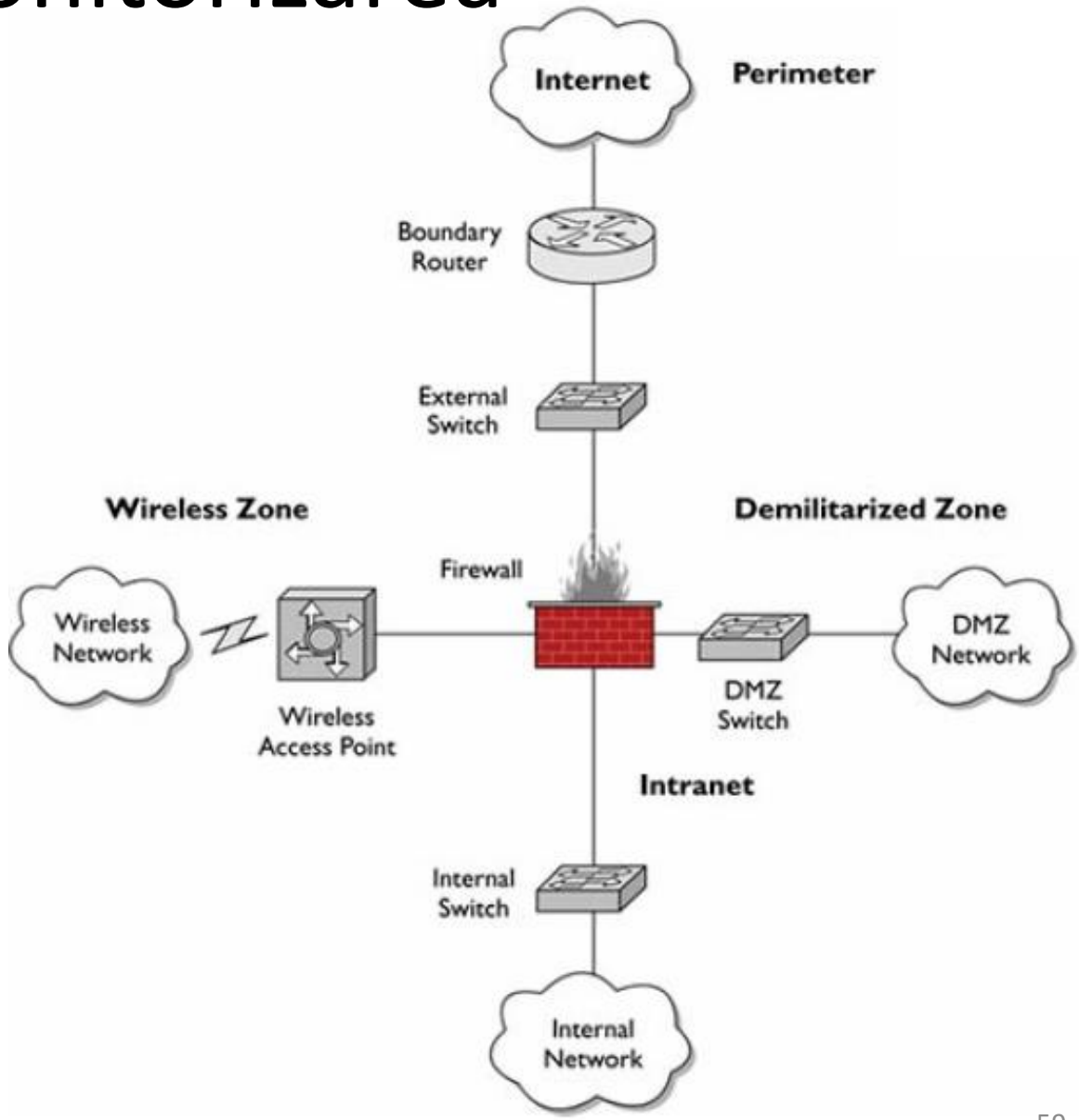
- **Monitorizarea securitatii retelei (NSM – Network Security Monitoring) =** colectarea, analiza si aprecierea indicatorilor si avertismentelor privind **detectarea** si **raspunsul** la incidente de securitate
  - **Indicator**: actiune observabila care confirma intentiile sau capacitatile de atac
  - Indicatorii generati de sistemele de detectie a intrusilor se mai numesc si **alerte** (vizind un anumit **context**)
- Observatii:
  - Detectarea este realizata (automat) de produse software
  - Analizarea implica factori umani
  - Aprecierea incidentului reprezinta un proces de luare a deciziilor

Monitorizarea: detectarea

- **Principii:**
  - Intrusii care comunica (direct/indirect) cu victimele pot fi detectati
  - Detectia prin luare de probe (*sampling*) este superioara lipsei de detectie (Nu pot fi monitorizate toate datele)
  - Detectia pe baza analizei traficului este superioara lipsei detectiei
- Obs.
  - Colectarea tuturor datelor este problematica
  - Colectarea datelor poate fi efectiva daca se bazeaza pe aparitia unor evenimente
  - Instrumentele de detectie trebuie sa fie optimizate si trebuie sa asigure ergonomia utilizatorului

Monitorizarea

- Divizarea retelei in zone de interes
  - Fiecare zona poate fi tinta unor atacuri
  - DMZ separa datele sensibile de cele disponibil public



Monitorizarea

- Pot fi colectate date provenite de la:
  - *Hub*-uri, porturile *switch*-urilor (via SPAN – Switched Port Analyzer), *tap*-uri (*test access port* – dispozitiv de retea proiectat special pentru monitorizare), portile de filtrare (*filtering bridges*) – pentru retele cu fir
  - Senzori intre *firewall-ul* organizatiei si punctul de acces *wireless*, o platforma *wireless* – pentru retele *wireless*
- Realizarea de statistici
  - La nivel de router (e.g. *CISCO accounting*)
  - La nivel de sistem de operare
    - Instrumente: lpcad, ifstat, tcpdstat, MRTG (Multi Router Traffic Grapher) etc.

Monitorizarea: identificarea

- Date (trafic de retea)
  - Normale
    - Privind HTTP, FTP, SMTP, POP3, DNS, IP, SSL/TLS etc
  - Suspicioase
    - Apar dubioase la prima vedere, dar nu cauzeaza probleme pentru corporatie, ci eventual doar utilizatorului
  - Malitioase
    - Au impact negativ asupra securitatii organizatiei

Monitorizarea: validarea

- Validarea asociaza un incident preliminar unei categorii de evenimente:
  - Acces neautorizat ca *root* (administrator)
  - Acces neautorizat la nivel de utilizator
  - Incercare de accesare neautorizata
  - Atac (D)Dos soldat cu succes
  - Violare a politicii de securitate
  - Scanare, probare, detectie
  - Infectie cu virusi

Monitorizarea: reactia

- Dupa aparitia unui incident de securitate, trebuie demarata o reactie:
  - Pe termen scurt – STIC (*SHORT-Term Incident Containment*)
    - Exemplu: inchiderea portului *switch*-ului prin care se realizeaz atacul, deconectarea fizica, introducerea unei reguli noi de filtrare a datelor etc.
  - Intrarea in stare de urgenta
  - O importanta majora o are analiza (*analyst feedback*) ->Implica personal specializat

Monitorizarea

- Se poate recurge si la capcane pentru *craker*-i: ***honeypots***
  - Masini-tinta special configurate pentru a observa atacurile *cracker-ilor*
  - Mai multe *honeypots* formeaza un *honeynet* (http://www.honeynet.org)
  - Pentru a detecta & studia noi tehnici de atac si pentru a contracara diverse incidente de securitate
  - Folosind un daemon (*honeyd*), se pot imita servicii de retea, rulind intr-un mediu virtual

Testarea

- Teste de verificare a:
  - Capacitatii de deservire a clientilor
  - Robustetei
  - Rularii in situatii extreme
- Teste referitoare la performante
- Teste specifice legate de exploatare
  - Pregatirea adecvata a exploatarii in practica (*deployment*)
  - Teste de incarcare (*load testing*)
- Teste privind opacizarea datelor (*obfuscation*)
  - Datele nu trebuie stocate in locatii predictibile
- Teste privitoare la integrarea componentelor
- Teste specifice legate de programare (ex. Lungimea parametrilor trimisi de client, a interogarilor SQL, etc)

Testarea

- Instrumentele de stresare (*stressing tools*) pot da informatii privitoare la:
  - Performanta (timp de raspuns, timp de generare a continutului etc.)
  - Scalabilitate (memoria ocupata, utilizarea discului, numarul inregistrarilor inserate, accesarea altor tipuri de resurse, …)
  - Corectitudine (functionarea eronata a unor componente)
  - Lacune de securitate
- Metodologii de analiza a riscurilor: **DREAD** (Damage potential, Reproductibility, Exploitability, Affected users, Discoverability), **OCTAVE** (Operationally Critical Threat Asset and Vulnerability Evaluation), **STRIDE** (Spoofing identity, Tempering with data, Repudiation, Information disclosure, Denial of service, Elevation of privilege), **OSSTMM** (Open Source Security Testing Methodology Manual) – www.ostmm.org

Testarea

- **SANS** (System Administration, Networking, and Security) :
  - Pregatire
  - Identificare
  - Controlul efectelor
  - Eradicare
  - Recuperare
  - Continuare (*follow-up*)
- Raspunsurile agresive sunt prohibite! (*hack back*)
- *Forensics* = procesul de “prindere” a cracker-ilor
  - Uzual, are loc dupa un incident de securitate
  - Implica: analiza hardware-ului (discuri, RAM,…), log-urilor etc.
  - Instrumente: WinHex, FIRE (forinsec and Incident Response Environment), ForensiX

Protocoale – exemple

- **Nivelul retea**

IPSec – RFC 2401, 2402, 2406, 2408

– Servicii oferite: controlul accesului, integritatea datelor, autentificare, confidentialitate

– Poate fi implementat in cadrul unui *router* sau *firewall*

– Nu necesita modificarea software-ului la nivel de transport/aplicatie

– Autentificarea &integritatea se precizeaza intr-un antet special: Authentication Header

– Confidentialitatea este asigurata de algoritmi de criptare via date suplimentare

ESP (*Encapsulating Security Payload*)

Protocoale – exemple

- **Nivelul transport**

TLS (**Transport Layer Security**) – RFC5246 (TLS 1.2)

– Imbunatatire a SSL (Secure Socket Layer) creat de Netscape

SSL(Secure Sockets Layer)

- Metoda de criptare a transmisiilor TCP/IP (e.g. pagini Web, date introduse in *form-uri* web) - HTTP over Secure Sockets Layer or HTTP Secure

– Ofera servicii de securitate de baza pentru TCP

– Fiecare conexiune dintre un client si server reprezinta o sesiune (*session*)

– Starea unei sesiuni = identificator unic al sesiunii, certificat digital, metoda de compresie, metoda de cifrare (algoritm de criptare sau de tip hash), cod secret partajat de client & server

– Un mesaj are asociat un cod de autentificare: MAC (Message Authentication Code)

Protocoale – exemple

- **Nivelul transport**

  TLS (**Transport Layer Security**) – RFC5246 (TLS 1.2)

  – **Alert Protocol –** mecanism care permite managementul alertelor provenite de la un punct terminal: mesaj neasteptat receptionat, eroare de decompresie, MAC incorect, certificat eronat

  – **Handsake Protocol –** permite autentificarea serverului la client si vice-versa, plus negocierea algoritmilor de criptare si a cheilor; se realizeaza inainte de transmiterea efectiva a datelor

Protocoale – exemple

- **Nivelul aplicatie**

SSH (Secure Shell)

– negociaza si stabileste o conexiune criptata intre un server si un client SSH via metode diverse de autentificare

– Implementari: ssh, PuTTY, SCP (*Secure CoPy*),…

PCP (Pretty Good Privacy) – RFC 3156

– Ofera confidentialitate & autentificarea mesajelor e-mail si a fisierelor transmise prin retea

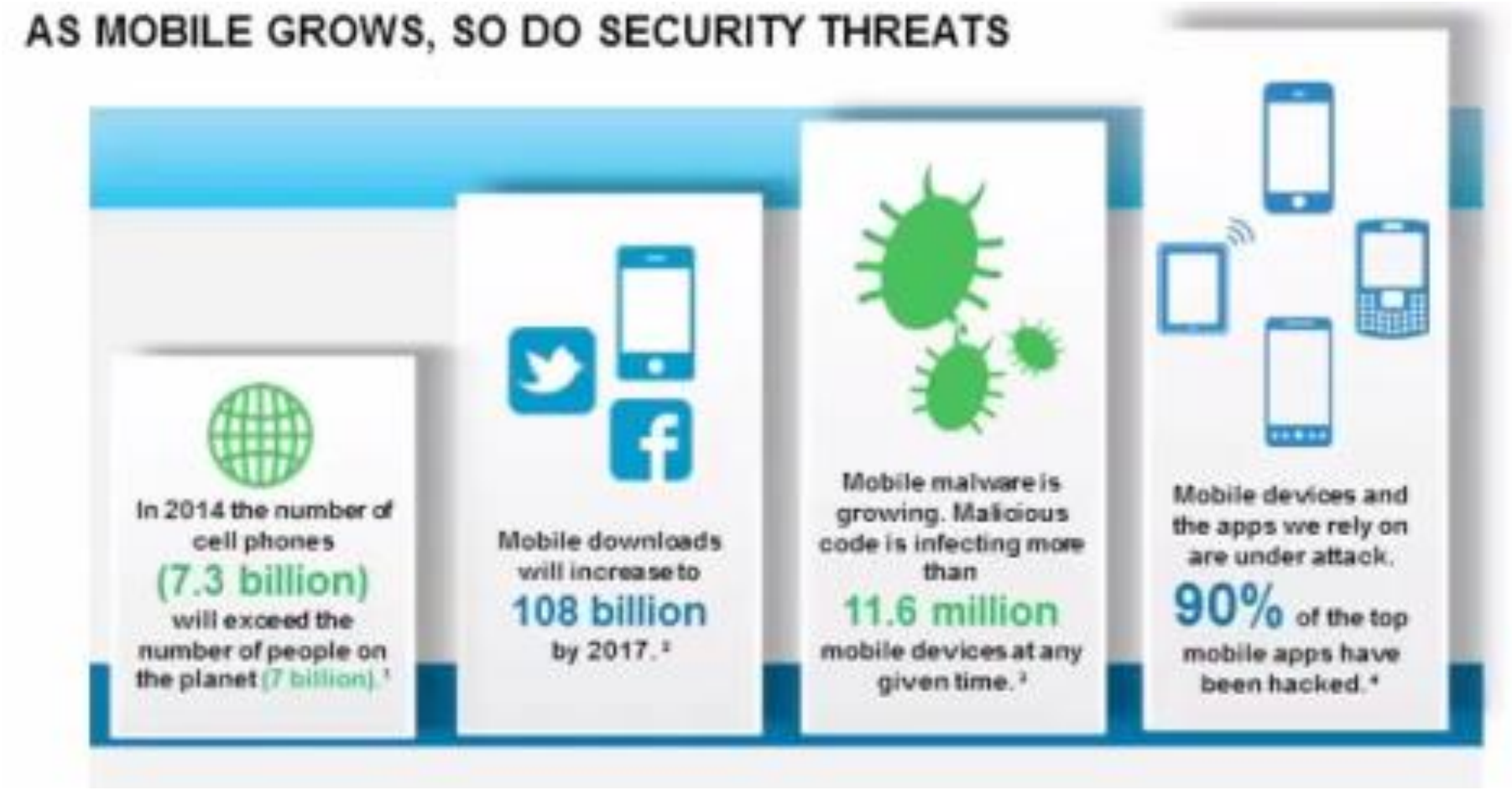
– Foloseste o pleiada de algoritmi de criptare

– Implementari: GPG (GNU Privacy Guard)

S/MIME – RFC 3369, 3370, 3850, 3851

– Pune la dispozitie extensii de securitate pentru MIME (Multipurpose Internet Mail Extension)

Atacuri


AS MOBILE GROWS, SO DO SECURITY THREATS
In 2014 the number of cell phones
(7.3 billion)
will exceed the number of people on the planet (7 billion).[1]
Mobile downloads will increase to
108 billion
by 2017.[2]
Mobile malware is growing. Malicious code is infecting more than
11.6 million
mobile devices at any given time.[3]
Mobile devices and the apps we rely on are under attack.
90% of the top mobile apps have been hacked.[4]

Atacuri

- Atacuri la nivel *wireless*
  - Diminuarea semnalului
  - Capturarea pachetelor de date (*wireless sniffing*)
  - Atacuri asupra WEP (Wired Equivalent Privacy)
  - Crearea de virusi/ cod malitios
  - Folosirea resurselor retelelor wireless publice sau ale unor companii
    - *Snooping* (accesarea datelor private)
    - *Masquerading* (furt de identitate al unui dispozitiv)
    - DoS (refuz al serviciilor)

Probleme specifice

- **Sistemele wireless**
  - Necesitate: un mediu sigur (autentificare, integritate a datelor, confidentialitate, autorizare, nerepudiere)
  - Pericole – tipuri de atacuri:
    - Falsificarea identitatii (*spoofing*)
    - Interceptarea (*sniffing*)
    - Alterarea datelor (*tampering*)
    - Interferenta (*jamming*) – e.g. la Bluetooth
    - Furtul (*device theft*)

Probleme specifice

- **Sistemele wireless**
  - Solutii:
    - WEP (Wired Equivalent Privacy) – standard vechi inlocuit in 2003 de WPA
    - WPA (WI-FI Protected Access) – subset al 802.11i – foloseste metoda de criptare diferita (RC4 fata de AES), WPA2
    - Protocoale de securitate WTLS (Wireless Transport Layer Security)
    - Securitatea la nivel IP: IPSec
    - Extensii de securitate in cadrul IP-ului mobil
    - Firewall-uri
    - …

Statistici

Top atacuri in 2016: https://securelist.com/analysis/kaspersky-security-bulletin/76858/kaspersky-security-bulletin-2016-executive-summary/

- **xDedic – the shady marketplace – "***facilitated the buying and selling of hacked server credentials. Around 70,000 compromised servers were on offer – although later evidence suggests that there could have been as many as 176,000 – located in organisations around the world"*
- In February 2016, hackers used the **SWIFT** credentials of Bangladesh Central Bank employees to send fraudulent transaction requests to the Federal Reserve Bank of New York, asking it to transfer millions of dollars to various bank accounts in Asia. The hackers were able to get $81 million transferred to the Rizal Commercial Banking Corporation in the Philippines and an additional $20 million to Pan Asia Banking.
  - Society for Worldwide Interbank Financial Telecommunication (SWIFT) provides a network that enables financial institutions worldwide to send and receive information about financial transactions in a secure, standardized and reliable environment. SWIFT also sells software and services to financial institutions, much of it for use on the SWIFTNet Network
- …

http://securelist.com/analysis/kaspersky-security-bulletin/68010/kaspersky-security-bulletin-2014-overall-statistics-for-2014/

Statistici

Top atacuri in 2016: https://securelist.com/analysis/kaspersky-security-bulletin/76858/kaspersky-security-bulletin-2016-executive-summary/

- In October, attackers used a botnet of over half a million internet-connected home devices to launch a DDoS attack against Dyn – a company that provides DNS services to Twitter, Amazon, PayPal, Netflix and others. The world was shocked, but warnings about unstable IoT security have been around for a long time....

http://securelist.com/analysis/kaspersky-security-bulletin/68010/kaspersky-security-bulletin-2014-overall-statistics-for-2014/

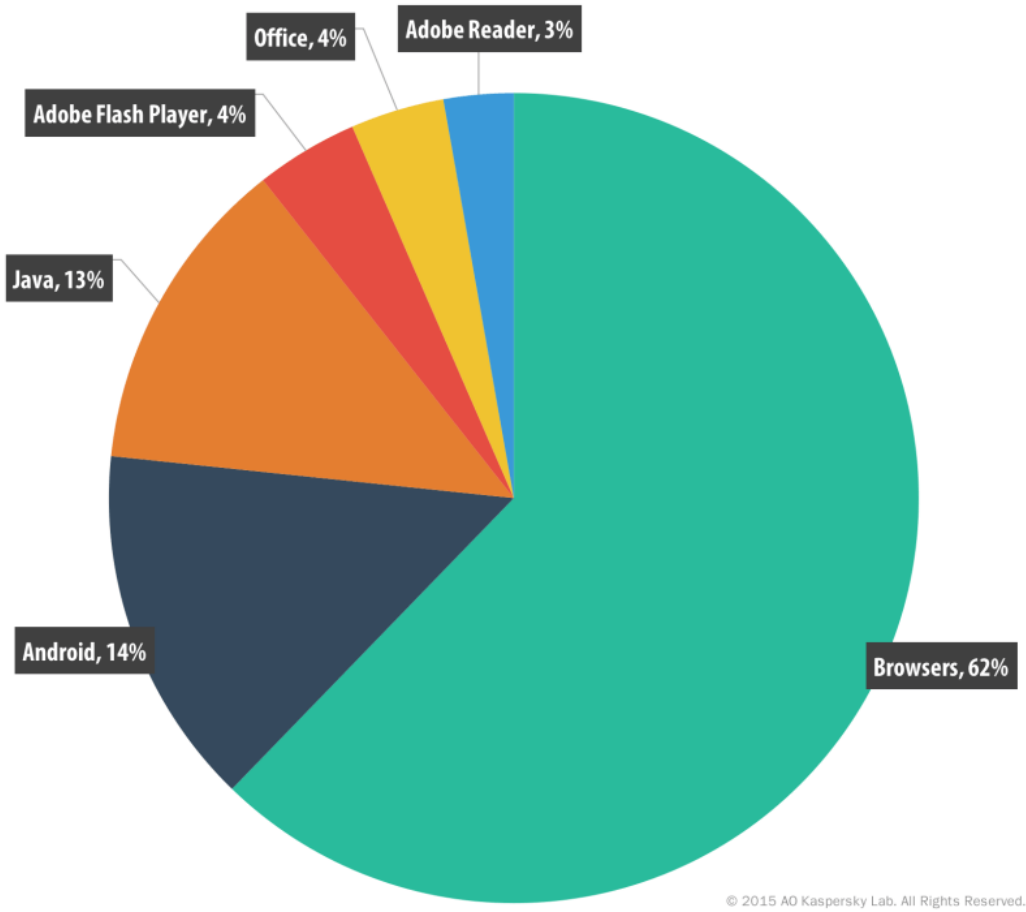
Statistici

Perioada 2015

Top aplicatii vizate de

Atacatori (inclusiv pentru

dispozitivele mobile)



http://securelist.com/analysis/kaspersky-security-bulletin/68010/kaspersky-security-bulletin-2014-overall-statistics-for-2014/

Statistici

	Name	Verdicts*	percentage of users**
1	CTB-Locker	Trojan-Ransom.Win32.Onion / Trojan-Ransom.NSIS.Onion	25.32
2	Locky	Trojan-Ransom.Win32.Locky / Trojan-Dropper.JS.Locky	7.07
3	TeslaCrypt (active till May 2016)	Trojan-Ransom.Win32.Bitman	6.54
4	Scatter	Trojan-Ransom.Win32.Scatter / Trojan-Ransom.BAT.Scatter / Trojan-Downloader.JS.Scatter / Trojan-Dropper.JS.Scatter	2.85
5	Cryakl	Trojan-Ransom.Win32.Cryakl	2.79
6	CryptoWall	Trojan-Ransom.Win32.Cryptodef	2.36
7	Shade	Trojan-Ransom.Win32.Shade	1.73
8	(generic verdict)	Trojan-Ransom.Win32.Snocry	1.26
9	Crysis	Trojan-Ransom.Win32.Crusis	1.15
10	Cryrar/ACCDFISA	Trojan-Ransom.Win32.Cryrar	0.90

Top 10 programe malitioase

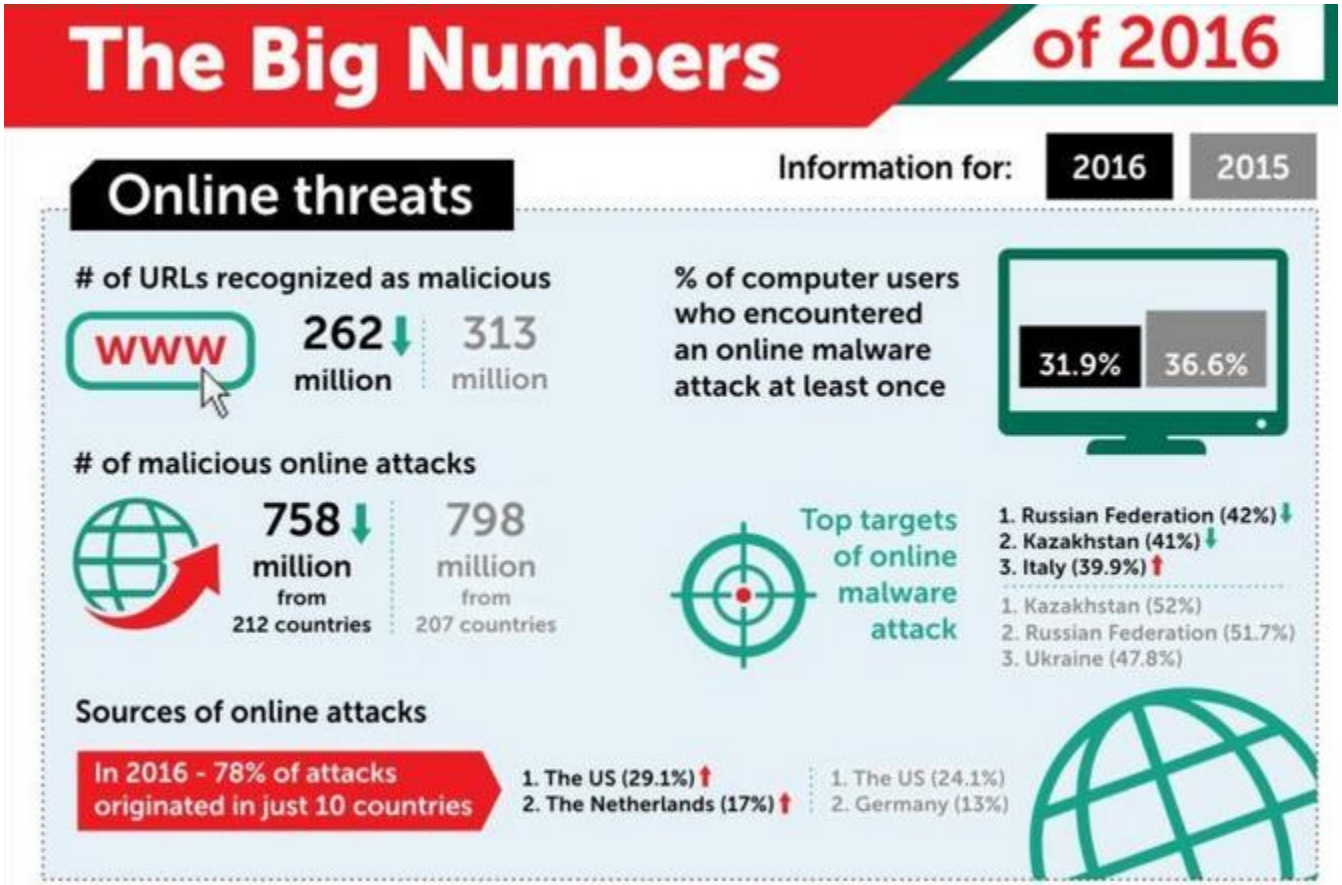
Statistici

According to Kaspersky Lab research, in 2016, one in every five businesses worldwide suffered an IT security incident as a result of a ransomware attack.

- 42% of small and medium-sized businesses were hit by ransomware in the last 12 months.
- 32% of them paid the ransom.
- One in five never got their files back, even after paying.
- 67% of those affected by ransomware lost part or all of their corporate data – and one- in-four spent several weeks trying to restore access.

	Industry sector	% attacked with ransomware
1	Education	23
2	IT/Telecoms	22
3	Entertainment/Media	21
4	Financial Services	21
5	Construction	19
6	Government/ public sector/defence	18
7	Manufacturing	18
8	Transport	17
9	Healthcare	16
10	Retail/wholesale/leisure	16

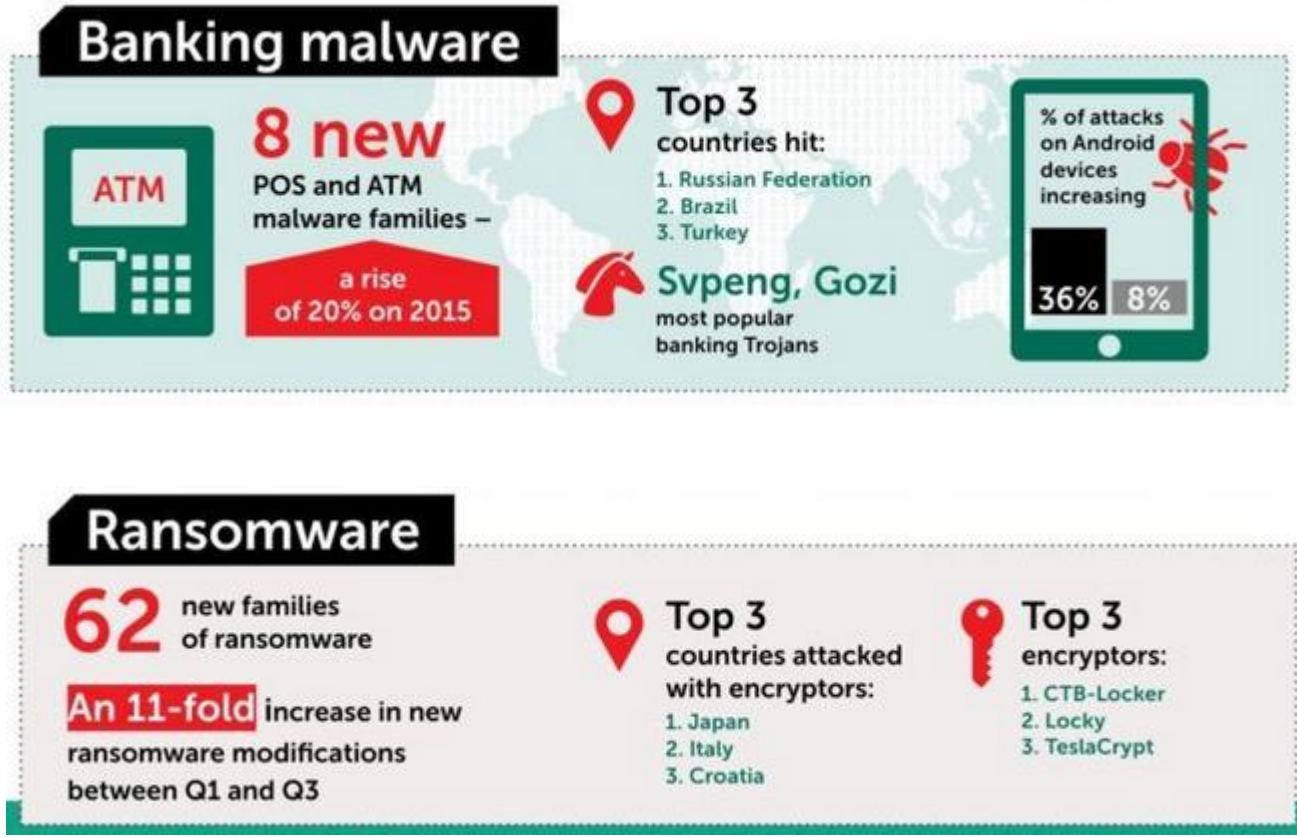
Statistici



https://securelist.com/analysis/kaspersky-security-bulletin/76858/kaspersky-security-bulletin-2016-executive-summary/

Statistici

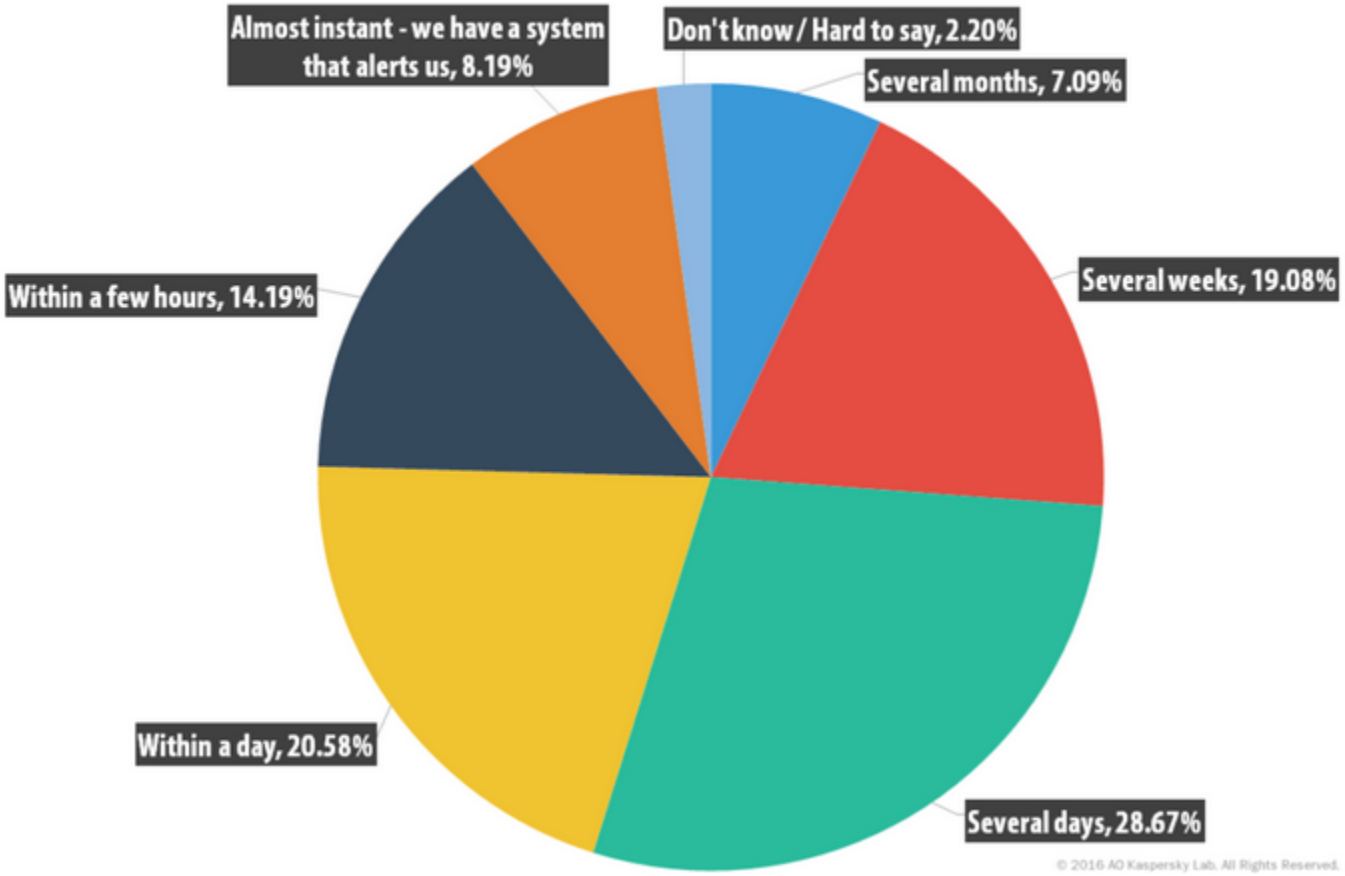
Perioada 2016



https://securelist.com/analysis/kaspersky-security-bulletin/76858/kaspersky-security-bulletin-2016-executive-summary/

Statistici

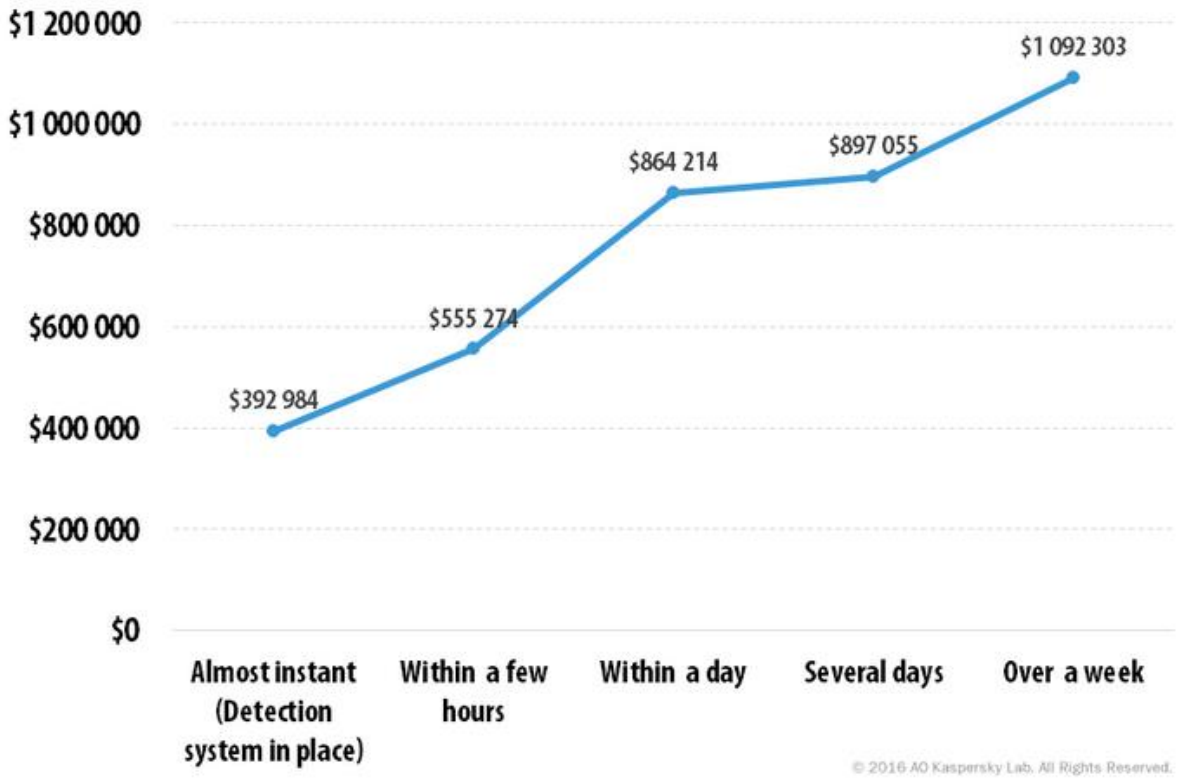
Timpul de detectare al incidentelor de securitate - Statistici



https://securelist.com/analysis/kaspersky-security-bulletin/76858/kaspersky-security-bulletin-2016-executive-summary/

Statistici

Timpul de detectare al incidentelor de securitate – Statistici&Costuri



Cost of recovery vs. time needed to discover a security breach for enterprises

https://securelist.com/analysis/kaspersky-security-bulletin/76858/kaspersky-security-bulletin-2016-executive-summary/

Previziuni -2017

- *1. Ransomware attacks will decrease in volume and effectiveness in the second half of 2017.*
- *2. Windows vulnerability exploits will continue to decline, while those targeting infrastructure software and virtualization software will increase.*
- *3. Hardware and firmware will be increasingly targeted by sophisticated attackers.*
- *4. Hackers using software running on laptops will attempt "dronejackings" for a variety of criminal or hacktivist purposes.*
- *5. Mobile attacks will combine mobile device locks with credential theft, allowing cyber thieves to access such things as banks accounts and credit cards.*
- *6. IoT malware will open backdoors into the connected home that could go undetected for years.*
- *7. Machine learning will accelerate the proliferation of and increase the sophistication of social engineering attacks.*
- *8. Fake ads and purchased "likes" will continue to proliferate and erode trust.*
- *9. Ad wars will escalate and new techniques used by advertisers to deliver ads will be copied by attackers to boost malware delivery capabilities.*
- *10. Hacktivists will play an important role in exposing privacy issues.*
- *11. Leveraging increased cooperation between law enforcement and industry, law enforcement takedown operations will put a dent in cybercrime.*
- *12. Threat intelligence sharing will make great developmental strides in 2017.*
- *13. Cyber espionage will become as common in the private sector and criminal underworld as it is among nation-states.*
- *14. Physical and cybersecurity industry players will collaborate to harden products against digital threats.*

http://www.securitymagazine.com/articles/87628-threat-trends-to-watch-in-2017

Rezumat

- Preliminarii
- Aspecte importante
- Vulnerabilitati
- Atacuri
- Prevenirea si supravietuirea
- Monitorizarea
- Testarea
- Raspunsul la incidente
- Protocoale
- Probleme specifice
- Statistici & Previziuni

Bibliografie

- Security guide to network security fundamentals, Mark Ciampa, 2009
- Network+ Guide to Networks, Fifth Edition, Tamara Dean, Network +, ISBN-13: 978-1-423-90245-4, 2009
- http://www.scribd.com/doc/55573866/57/Func%C5%A3iile-Hash
- http://www.lsec.be/upload_directories/documents/2011/sophos-security-threat-report-2011-wpna.pdf
- http://securelist.com/statistics/
- https://securelist.com/analysis/kaspersky-security-bulletin/73038/kaspersky-security-bulletin-2015-overall-statistics-for-2015/
- http://www.dlsit.ro/articole/106_Atacul-Teardrop
- https://brighttalkservices.app.box.com/s/ym2v7t9bbuchnj53gmrdt6g4uinbb59u

Intrebari?

“Little by little, one travels far.”
(J. R. R. Tolkien)